



UNIVERSIDAD NACIONAL DE EDUCACIÓN A DISTANCIA
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Proyecto de fin de Grado en Ingeniería Informática

ENS2025. Simulador de ensamblador IEEE 694

José María Pérez Redondo

Dirigido por: Álvaro Rodrigo Yuste

Curso 2024/2025, convocatoria junio



ENS2025. Simulador de ensamblador IEEE 694

**Proyecto de fin de Grado en Ingeniería Informática
de modalidad específica**

Realizado por: José María Pérez Redondo

Dirigido por: Álvaro Rodrigo Yuste

Fecha de lectura y defensa: 10 de julio de 2025

Agradecimientos

A mi familia, sin cuya ayuda no habría podido hacer tanto en tan poco tiempo.

A mi director, Álvaro Rodrigo Yuste, por confiar en que sería capaz de acometer un trabajo de esta envergadura y llevarlo a término.

A mis amigos, por su apoyo y porque tras meses desaparecido me acogerán como si el tiempo no hubiera pasado.

Y, por último, a Lola. Por ser, en general, la mejor perrita del mundo, y por recordarme dos lecciones muy importantes: que el mundo sigue girando por muy ocupado que esté y que es necesario salir a tomar el aire de vez en cuando.

Resumen

ENS2025 es un simulador y ensamblador para el estándar de lenguaje ensamblador IEEE 694, concebido como una herramienta de apoyo docente en asignaturas relacionadas con la arquitectura de computadores y la teoría de compiladores. Este proyecto nace de la necesidad de modernización de su predecesor, ENS2001; una herramienta que, pese a su uso extendido, presentaba importantes limitaciones debido a su antigüedad, tales como una interfaz gráfica desfasada, comportamientos inesperados, problemas de portabilidad y una arquitectura con un diseño e implementación difíciles de mantener.

La nueva herramienta está desarrollada en Java, lo que garantiza la total portabilidad a cualquier sistema operativo capaz de ejecutar una máquina virtual de dicho lenguaje con una versión mínima. Además, no solamente posee compatibilidad funcional con los programas ensamblados por la herramienta original, sino que mejora dicho simulador empleando una arquitectura de software modular y mantenible, que facilita (y, en ocasiones, trivializa) la modificación de sus componentes o del juego de instrucciones. Entre sus funcionalidades más destacadas se encuentra una interfaz gráfica de usuario completamente renovada mediante JavaFX y la corrección de errores lógicos del sistema anterior. Además, el simulador permite a los usuarios visualizar en todo momento el estado del computador. Es decir, el valor de los registros y la memoria, facilitando la depuración de código.

Gracias a todas estas características, ENS2025 busca convertirse en una herramienta de nueva generación. Con este simulador se persigue facilitar significativamente la enseñanza y el aprendizaje del lenguaje ensamblador, proporcionando una herramienta robusta y eficiente que sirva de apoyo para futuras generaciones tanto de profesores como de alumnos.

Abstract

ENS2025 is a simulator and assembler for the IEEE 694 assembly language standard, conceived as a teaching support tool for subjects related to computer architecture and compiler theory. This project was born from the need to modernize its predecessor, ENS2001; a tool that, despite its continued use, presented significant limitations due to its age, such as an outdated graphical interface, unexpected behavior, portability issues, and an architecture with a design and implementation that were difficult to maintain.

The new tool is developed in Java, which guarantees full portability to any operating system capable of running a virtual machine in that language, provided they support a minimum version. Furthermore, it not only offers functional compatibility with programs assembled by the original tool, but also improves on the simulator itself by using a modular and maintainable software architecture, which facilitates (and sometimes even trivializes) further modification of its components or the instruction set. Among its most notable features are a completely revamped graphical user interface based on JavaFX and the correction of logical errors from the previous system. In addition, the simulator allows users to view the computer's status at all times, i.e., the values of registers and memory, facilitating code debugging.

Thanks to all these features, ENS2025 aims to become a next-generation tool. This simulator aims to significantly facilitate the teaching and learning of assembly language, providing with a robust and efficient tool that will serve as a support for future generations of both teachers and students.

Palabras clave

Simulador, ensamblador, IEEE 694, arquitectura de computadores, teoría de compiladores, herramienta docente, Java, JavaFX.

Keywords

Simulator, assembler, assembly, IEEE 694, computer architecture, compiler theory, educational tool, Java, JavaFX.

Índice

1. Introducción	1
2. Análisis de soluciones previas	3
2.1. Fundamentos teóricos	3
2.2. Otras herramientas de simulación	5
2.3. Los predecesores de ENS2025	8
3. Propuesta	17
3.1. Introducción	17
3.2. Visión general del producto	18
3.3. Requisitos específicos	23
4. Planificación y costes	33
4.1. Planificación	33
4.2. Costes	35
4.3. Cálculo de costes sin <i>crunching</i>	36
5. Solución	39
5.1. Paradigma de diseño	39
5.2. Diseño heredado de ENS2001	40
5.3. Diseño e implementación de ENS2025	59
5.4. Diseño e implementación de las interfaces de nivel de usuario	131
5.5. Decisiones de implementación	148
6. Pruebas	151
6.1. Pruebas de operandos y direccionamientos	151
6.2. Pruebas de dos operandos en una única palabra	153

6.3. Pruebas de instrucciones aritmético-lógicas	154
6.4. Pruebas de instrucciones de control de subrutinas	154
6.5. Pruebas de instrucciones de salto	156
6.6. Pruebas de instrucciones entrada/salida	156
6.7. Pruebas de excepciones en expresiones	157
6.8. Pruebas de etiquetas	158
6.9. Pruebas de procesamiento de texto	158
6.10. Pruebas de estrés	159
6.11. Pruebas de excepciones por división entre cero	160
6.12. Pruebas de directivas del ensamblador	160
6.13. Pruebas de restricciones de valores	162
6.14. Programas de ejemplo	164
6.15. Pruebas de despliegue	165
7. Conclusiones y trabajos futuros	169
7.1. Consecución de objetivos propuestos	169
7.2. Líneas de trabajo futuras	170
Bibliografía	173
A. Formato del fichero de configuración	175
B. Manual de usuario	177
B.1. Introducción	177
B.2. Arquitectura de la máquina virtual	177
B.3. Configuración del sistema	179
B.4. Estructura del código ensamblador	181
B.5. Modos de direccionamiento	183
B.6. Juego de instrucciones	186

B.7. Etiquetas	195
B.8. Directivas del ensamblador	195
B.9. Detección de errores en el código fuente	196
B.10. Interfaz de línea de comandos	196
B.11. Interfaz Gráfica de Usuario	201
B.12. Tablas de referencia	210

C. Guía para la inclusión de nuevas instrucciones	213
--	------------

Índice de tablas

4.1. Desglose del coste estimado del proyecto.	36
4.2. Comparativa de costes con y sin <i>crunching</i>	37
5.1. Estructura de una instrucción según sus operandos	42
5.2. Tabla de condiciones de activación de los biestables de estado.	46
5.3. Tabla de códigos de direccionamiento.	56
5.4. Tabla de códigos de registro.	56
5.5. Tabla de códigos de instrucción.	57
5.6. Tabla de posicionamiento de biestables en el registro SR.	57
5.7. Estructura de la primera palabra.	57
5.8. Operando de 16 bits.	58
5.9. Operandos de 8 bits.	58
5.10. Operandos de 4 bits.	58
5.11. Algunos ejemplos de una palabra con dos operandos.	58
5.12. Tipo enumerado Flag.	68
5.13. Tipo enumerado RegisterID.	69
5.14. Tipo enumerado AddressingMode.	69
5.15. Tipo enumerado InstructionType.	70
5.16. Interfaz IOInterface.	73
5.17. Tipo enumerado StackGrowth.	74
5.18. Tipo enumerado NumericalRepresentation.	74
5.19. Clase Settings (dividida por motivos de espacio).	75
5.20. Clase CustomException.	78
5.21. Clase ISACodex.	81
5.22. Clase VerificationCodex.	82

5.23. Clase Instruction.	83
5.24. Clase Operand.	85
5.25. Clase InstructionData.	86
5.26. Clase NoOpInstruction.	86
5.27. Clase OneOpInstruction.	87
5.28. Clase OneOpIOInstruction.	87
5.29. Clase TwoOpInstruction.	87
5.30. Clase RegisterBank.	94
5.31. Clase Memory.	95
5.32. Clase BreakpointManager.	95
5.33. Clase Architecture.	97
5.34. Clase Decoder.	102
5.35. Clase Simulator.	104
5.36. Clase DisassembledInstruction.	107
5.37. Clase Disassembler.	108
5.38. Clase Reference.	112
5.39. Clase OperandBlueprint.	113
5.40. Clase InstructionBlueprint.	113
5.41. Clase FileData.	115
5.42. Clase InstructionParser.	116
5.43. Acciones a emprender por el procesador de expresiones en función del token actual y el último token leído.	119
5.44. Clases Token y ValueToken.	120
5.45. Clase ExpressionParser.	120
5.46. Clase FileParser.	121
5.47. Clase MemoryPointer.	123
5.48. Clase EncodedInstruction.	123

5.49. Clase Encoder.	124
5.50. Clase Writer.	126
5.51. Clase MemoryWriter.	127
5.52. Tipo enumerado DirectiveType.	127
5.53. Clase Assembler	128
5.54. Clase ControlUnit.	131
 A.1. Tabla de parámetros de configuración	 175
 B.1. Tabla de códigos de registro.	 179
B.2. Tabla de posicionamiento de biestables en el registro SR.	179
B.3. Estructura de una instrucción según sus operandos	182
B.4. Tabla de códigos de direccionamiento.	186
B.5. Formato de instrucciones.	210
B.6. Tabla de códigos de direccionamiento.	211
B.7. Tabla de códigos de registro.	211
B.8. Tabla de posicionamiento de biestables en el registro SR.	211
B.9. Tabla de modos de direccionamiento aceptados en cada operando por cada ins- trucción.	212

Índice de figuras

2.1. Ventana principal de QtSpim.	6
2.2. Simulador de Higginson en plena ejecución.	7
2.3. Interfaz del simulador WebMIPS.	8
4.1. Diagrama de Gantt con la planificación original de tareas y sus respectivas estimaciones de tiempo.	34
5.1. Interacción de una interfaz de usuario genérica con la unidad de control.	60
5.2. Vista interna simplificada de la unidad de control.	60
5.3. Diagrama de casos de uso.	62
5.4. Diagrama de secuencia del caso de uso “Consultar banco de registros”.	62
5.5. Diagrama de secuencia del caso de uso “Ejecutar instrucción”.	63
5.6. Plantilla de implementación del método getMessage() en una subclase de CustomException	79
5.7. Jerarquía de herencia de las clases abstractas de instrucciones	84
5.8. Vista de la unidad de control con la arquitectura expandida.	93
5.9. Vista de la unidad de control con el simulador expandido.	104
5.10. Vista de la unidad de control con el ensamblador expandido	109
5.11. Vista de la unidad de control con el procesador de texto expandido dentro del ensamblador.	114
5.12. Vista de la unidad de control con el codificador-escritor expandido dentro del ensamblador.	122
5.13. Vista general de la ventana de interfaz gráfica.	138
5.14. Divisiones de la barra de herramientas.	139
5.15. Banco de registros y biestables. El registro A tiene el foco.	140
5.16. Cuadros de diálogo de edición de valores.	140
5.17. Visor de memoria con su barra de herramientas.	141

5.18. Los visores de pila y código de ENS2001.	143
5.19. Visor de código con su barra de herramientas.	144
6.1. Contenido del fichero de pruebas de validez de operandos de la instrucción ADD.	152
6.2. Contenido de un fichero de prueba de invalidez de modo de direccionamiento para la instrucción ADD.	152
6.3. Contenido de un fichero de prueba de invalidez de número de operandos para la instrucción ADD.	153
6.4. Contenido del fichero de pruebas de validez/invalidez de operandos de la instrucción HALT.	153
6.5. Contenido del fichero de pruebas de instrucciones de dos operandos que comparten una misma palabra.	153
6.6. Extracto del fichero de pruebas de instrucciones aritméticas.	154
6.7. Extracto del fichero de pruebas de instrucciones lógicas.	154
6.8. Extracto del fichero de pruebas de biestables.	155
6.9. Extracto del fichero de pruebas de instrucciones de control de subrutinas	155
6.10. Contenido del fichero de pruebas de instrucciones de salto incondicional.	156
6.11. Extracto del fichero de pruebas de instrucciones de salto condicional	157
6.12. Extracto del fichero de pruebas de instrucciones de entrada/salida con direccionamiento a memoria	157
6.13. Extracto del fichero de pruebas de instrucciones de entrada/salida con otros direccionamientos	158
6.14. Contenido de los distintos ficheros de pruebas de excepciones en expresiones.	158
6.15. Contenido de los distintos ficheros de pruebas de etiquetas.	159
6.16. Contenido de los distintos ficheros de pruebas de procesamiento de texto.	159
6.17. Código de la prueba de estrés de la instrucción WRCHAR.	160
6.18. Contenido de los distintos ficheros de pruebas de división entre cero.	160
6.19. Extracto del fichero de pruebas de validación de la directiva EQU.	161
6.20. Contenido del fichero de pruebas de validación de la directiva END.	161
6.21. Contenido del fichero de pruebas de validación de la directiva RES.	162

6.22. Extracto del fichero de pruebas de validación de la directiva ORG.	163
6.23. Contenido del fichero de pruebas de validación de la directiva DATA.	164
6.24. Contenido de los distintos ficheros de pruebas de restricciones de valores.	165
6.25. Contenido del fichero de pruebas de validación de restricciones de valores.	166
A.1. Contenidos del fichero de configuración.	175
B.1. Ejemplos válidos de instrucciones y comentarios.	182
B.2. Ejemplos inválidos de instrucciones y comentarios.	182
B.3. Ejemplo de directiva ORG provocando sobrescritura de código.	196
B.4. Vista general de la ventana de interfaz gráfica.	202
B.5. Divisiones de la barra de herramientas.	204
B.6. Banco de registros y biestables. El registro A tiene el foco.	204
B.7. Cuadros de diálogo de edición de valores.	205
B.8. Visor de memoria con su barra de herramientas.	206
B.9. Visor de código con su barra de herramientas.	207
C.1. Código completo de la clase que implementa la instrucción SHL.	214
C.2. Contenido del conjunto de instrucciones de dos operandos tras la adición de SHL.	214
C.3. Contenido del conjunto de asociación de los modos de direccionamiento del primer operando de la nueva instrucción SHL.	214
C.4. Código abreviado del mapa de conjuntos de asociación de modos de direccionamiento para el primer operando de cada instrucción, con la adición de SHL.	215

Capítulo 1

Introducción

Uno de los pilares de la Ingeniería Informática es la programación: la implementación de un algoritmo para su uso requiere de su codificación en algún lenguaje con un conjunto de reglas, que permita la reproducción fidedigna de los pasos que se deben tomar para recrear dicho algoritmo. La proliferación de lenguajes de alto nivel orientados a la facilidad de uso, combinada con los avances en hardware que absorben la pérdida de eficiencia derivada de la abstracción y estandarización de ciertos procesos, permiten el desarrollo de productos software más complejos con un tiempo de aprendizaje menor. El precio a pagar por herramientas más rápidas, seguras y sencillas de manejar conlleva una pérdida de flexibilidad al no poder gestionar los niveles más bajos de la implementación. Paradójicamente, el auge de estas herramientas hace cada vez más importante comprender cómo interactúan a bajo nivel con los sistemas en los que operan, tanto para optimizar las existentes como para desarrollar las siguientes generaciones. Esta comprensión profunda de la interacción entre lenguaje y máquina es una parte esencial del proceso formativo de los ingenieros informáticos, y se aborda en asignaturas clave que requieren herramientas didácticas especializadas.

Una de esas asignaturas es Procesadores del Lenguaje II, impartida en el Grado en Ingeniería Informática de la Universidad Nacional de Educación a Distancia. Durante las prácticas de esta asignatura, los alumnos deben construir un compilador de un lenguaje dado, que transforma el código en dicho lenguaje en instrucciones en ensamblador. Para comprobar la correcta implementación del compilador, se usa una herramienta de ensamblado y simulado de instrucciones en el estándar IEEE 694 llamada ENS2001 [1].

Con cerca de un cuarto de siglo como herramienta vigente, ENS2001 ha cumplido con su propósito de manera adecuada y eficiente, y podría seguir cumpliéndolo durante otro cuarto de siglo más. Sin embargo, su considerable antigüedad ha comenzado a mostrar ciertas limitaciones. ENS2001 presenta una interfaz gráfica de usuario con una estética desfasada y problemas de portabilidad, restringida principalmente a entornos Windows, lo que dificulta su uso en sistemas operativos más extendidos en la actualidad, como Linux y MacOS. Más allá de lo estético y operativo, su diseño interno sufre ciertas deficiencias arquitectónicas que comprometen su mantenibilidad y la capacidad de introducir nuevas funcionalidades de forma ágil y segura. Aunque su funcionamiento ha sido fiable, ciertos errores lógicos y comportamientos inesperados han persistido a lo largo de los años, aunque son errores lo bastante poco comunes como para que no hayan impactado de forma significativa en su valor didáctico.

Es en este contexto donde surge la motivación para el desarrollo de este Proyecto de Fin de Grado, el ensamblador y simulador de IEEE 694 ENS2025 (en adelante, simplemente ENS2025). Este proyecto aborda la creación de una nueva versión de la herramienta, manteniendo la compatibilidad exacta con el comportamiento funcional de la misma, pero corrigiendo los errores identificados y renovando por completo su arquitectura, que se ve rediseñada y reimplementada en el lenguaje Java. ENS2025 busca modernizar la interfaz gráfica mediante JavaFX, garantizar

una portabilidad completa a diversos sistemas operativos, e implementar un código más modular y mantenible. Asimismo, se incorporan nuevas características que enriquecen la experiencia del usuario y facilitan la depuración, posicionando a ENS2025 como un sucesor directo capaz relevar a ENS2001 como herramienta a usar en las prácticas de Procesadores del Lenguaje II, así como en otras asignaturas de otras universidades que dependieran hasta ahora de ENS2001.

Con el fin de alcanzar esa meta, se establecen de manera formal los siguientes objetivos a cumplir:

- 1. Réplica del comportamiento del simulador de ENS2001:** Dada la necesidad de continuidad, el objetivo más importante es que ENS2025 actúe del mismo modo que ENS2001, interpretando las mismas instrucciones de la misma manera.
- 2. Identificación y corrección de errores:** En el caso de los errores, no se procede a su replicación, sino a su corrección; de tal forma que el resultado final funcione como *pretendía* funcionar ENS2001 bajo esas circunstancias.
- 3. Aportación de nuevas características:** Para evitar que ENS2025 sea una “simple” reescritura de ENS2001, se deben buscar características adicionales que aportar.
- 4. Portabilidad:** Se debe procurar que esta nueva versión sea compatible con el mayor número posible de sistemas operativos.
- 5. Creación de un código más mantenible:** Para facilitar cambios futuros, tanto para versiones posteriores de ENS2025 como para el aprovechamiento de partes del mismo para otros proyectos de software, la implementación se debe adherir a paradigmas de diseño que favorezcan la modularidad y mantenibilidad. A fin de garantizar la mantenibilidad, se emplearán únicamente librerías estándar, salvo que lo contrario sea indispensable para la implementación de una función crítica.
- 6. Mejora de la interfaz gráfica:** A fin de mejorar la experiencia de usuario, se debe proporcionar una interfaz gráfica modernizada.

Además, en el apéndice B se suministra un manual de usuario simplificado, diseñado para ser distribuido de forma independiente junto al software. Por cuestiones evidentes de similitud en estructura y funcionalidades, dicho manual está estructurado en base al manual de ENS2001 suministrado junto a su memoria y distribuido con su aplicación [1]. Dado que el manual existe para explicar en pocas palabras al usuario los conceptos básicos de la arquitectura y el funcionamiento de las interfaces, su contenido deriva en gran medida de extractos de esta misma memoria, abreviados, simplificados y complementados con tablas replicadas, según sea necesario.

Finalmente, en el último apartado del manual se incluyen tablas de consulta rápida sobre la estructura del lenguaje ensamblador.

Capítulo 2

Análisis de soluciones previas

Aunque ya se ha establecido que el presente Proyecto de Final de Grado se basará en la mejora directa del simulador ENS2001, no deja de ser necesario analizar los fundamentos tras dicha herramienta, tanto a nivel de hardware como de software, así como el recorrido de herramientas previas que han inspirado la solución actual y de otras herramientas existentes en la actualidad con propósito similar.

2.1. Fundamentos teóricos

Antes de adentrarse en el análisis de soluciones ya existentes, es imprescindible establecer una base mínima sobre el marco teórico sobre el que se asienta este tipo de software. No es propósito de esta sección hacer un informe exhaustivo sobre teoría de compiladores o arquitectura de computadores, sino de desglosar los conceptos básicos que rigen el diseño y operación de las herramientas que se detallarán más adelante.

2.1.1. El proceso de compilación

En el ámbito de la informática, un compilador es una herramienta que lleva a cabo la tarea de transformar un programa escrito en un lenguaje determinado (lenguaje fuente) a una versión equivalente en un lenguaje distinto, conocido como lenguaje objeto [2], generalmente de nivel más bajo. Este proceso es fundamental para el desarrollo de software, ya que permite la ejecución de código escrito en lenguajes de alto nivel (más inteligibles para humanos) en arquitecturas de máquina específicas.

El proceso de compilación se divide en las etapas de análisis y síntesis. En el análisis se descompone el código fuente en sus unidades mínimas constituyentes, generando una representación intermedia del código original. Después, en la síntesis, se toma esta representación intermedia como base para la construcción del programa objeto. Cada una de estas etapas, a su vez, está dividida en distintas fases.

En el caso de la etapa de análisis, está compuesta por estas fases:

1. **Análisis léxico:** En esta fase, se procesan secuencialmente los caracteres del código fuente con el objetivo de formar agrupaciones con significado denominadas tokens, que representan las unidades más básicas de significado del lenguaje.
2. **Análisis sintáctico:** Una vez generada la secuencia de tokens mediante análisis léxico, se verifica que la estructura que forman se ajusta a las reglas gramaticales del lenguaje.

En esta fase, los tokens se agrupan en árboles de análisis sintácticos o árboles de sintaxis abstracta, que representan la estructura jerárquica del conjunto de tokens.

- 3. Análisis semántico:** Finalmente, en esta fase se verifica el significado del programa, asegurándose de que el código escrito, más allá de ceñirse a las reglas de la gramática del lenguaje, tiene sentido y es lógicamente coherente.

Por otro lado, la etapa de síntesis se compone de las siguientes fases:

- 1. Generación de código intermedio:** En esta fase se transforma el árbol de sintaxis final generado durante la etapa de análisis y lo transforma en un código fácilmente manipulable y optimizable por el compilador. Generalmente esto se traduce en la creación de código de tres direcciones o de cuádruplas.
- 2. Optimización de código:** Tras generar el código intermedio, el compilador puede transformarlo en una versión más eficiente sin alterar la semántica del programa. Para ello, el compilador identifica ineficiencias, como código inalcanzable o repetición de expresiones comunes, y las optimiza según proceda.
- 3. Generación de código final:** En esta última fase, se traduce el código intermedio optimizado en el código objeto específico de la arquitectura de destino.

En el caso del lenguaje ensamblador, las dos primeras fases de la etapa de síntesis no tienen lugar, generándose directamente el código final.

2.1.2. La arquitectura Von Neumann

La arquitectura Von Neumann (también llamada arquitectura Princeton), establecida por John von Neumann en 1945 [3], es la base arquitectónica de la computación moderna, utilizándose hasta nuestros días como base de la mayoría de procesadores de uso general actuales. Dicha arquitectura contiene los siguientes elementos:

- Unidad de procesamiento, que a su vez contiene la unidad aritmético-lógica y los registros.
 - La unidad aritmético-lógica realiza, como su nombre indica, operaciones básicas tanto de carácter aritmético (suma, resta...) como de carácter lógico (and, or...).
 - Los registros son un sistema de almacenamiento integrado que permite acceso rápido a la información con la que el procesador está trabajando en el momento.
- Unidad de control para la decodificación de las instrucciones y coordinación del proceso de ejecución de las mismas, así como para mantener el registro del contador de programa, un puntero que dirige a la instrucción a ejecutar.
- Memoria, que consiste en un conjunto de celdas de un ancho de bits determinado, que pueden contener datos e instrucciones.

- Unidades¹ de entrada y salida, que conectan con los dispositivos periféricos para gestionar la recepción y envío de información de y hacia el usuario.

Al tratarse de un simulador de esta arquitectura, estos elementos no tienen por qué verse reflejados necesariamente en la estructura interna²; pero es importante conocerlos para sentar las bases en las que se asientan paradigmas arquitectónicos más modernos como RISC³ o CISC⁴.

2.1.3. El estándar IEEE 694

El estándar IEEE 694 constituye una especificación para el lenguaje ensamblador que define una serie de requisitos esenciales para la creación de código fuente [4]. Su relevancia radica en la estandarización de elementos críticos que aseguran la consistencia y portabilidad del código ensamblador entre diferentes implementaciones o sistemas. Este estándar aborda, entre otros, los siguientes aspectos clave:

- **Formato de instrucción:** Define la estructura sintáctica de las instrucciones.
- **Juego de instrucciones:** Especifica el conjunto completo de operaciones que el procesador es capaz de ejecutar, categorizadas por su función (aritméticas, lógicas, de transferencia de datos, de salto...).
- **Modos de direccionamiento:** Describe las diversas maneras en que las instrucciones pueden especificar la ubicación de sus operandos (inmediato, directo a memoria o registro, indirecto...), impactando directamente en la flexibilidad y eficiencia del acceso a memoria.
- **Directivas del ensamblador:** Define comandos especiales para el ensamblador que controlan el proceso de traducción (asignación de memoria, definición de constantes...).

Para el presente Proyecto de Fin de Grado, el estándar IEEE 694 es de capital importancia, ya que el simulador ENS2001, sobre el cual se fundamenta ENS2025, se adhiere a este estándar. De nuevo, el propósito de ENS2025 es replicar el comportamiento de ENS2001, sin adiciones ni sustracciones en materia arquitectónica, juego de instrucciones o modos de direccionamiento. Puesto que dicho comportamiento se modelará más adelante en este capítulo de forma resumida, así como en capítulos posteriores de manera exhaustiva, no se ofrecerá una transcripción exhaustiva de la especificación aquí para evitar redundancias con la documentación original y las descripciones detalladas de la arquitectura de ENS2001.

2.2. Otras herramientas de simulación

En esta sección se detallarán brevemente algunas herramientas existentes con un propósito similar a ENS2001 y ENS2025 (esto es, simulación de ensamblador con propósitos educativos),

¹Referidas como “órganos” por el propio von Neumann.

²Por ejemplo, como se verá más adelante, ENS2025 no incorpora un módulo de ALU como tal.

³Computador con conjunto de instrucciones reducido (*Reduced Instruction Set Computer*), como MIPS.

⁴Computador con conjunto de instrucciones complejas, (*Complex Instruction Set Computer*), como x86.

pero que no terminan de amoldarse a las funcionalidades que busca esta línea de producto por diversos motivos (principalmente porque no implementan el estándar IEEE 694, que es el que se busca seguir en este Proyecto Final de Grado).

2.2.1. QtSpim

QtSpim [5] es un simulador que compila y ejecuta programas en ensamblador MIPS32 (esto es, arquitectura MIPS de 32 bits), implementando casi por completo el juego de instrucciones para dicha arquitectura. QtSpim es, en realidad, el nombre de la última versión del software, que ha sido mantenido por su autor durante más de tres décadas, y que funciona en los tres sistemas operativos principales.

Como se puede ver en la figura 2.1, el simulador integra un visor del código fuente del fichero cargado, si bien no permite la edición del mismo de manera directa, así como de parte de la memoria. Aunque es una aproximación comprensible (en un procesador real el usuario tampoco tiene facilidades para manipular estos contenidos a su antojo), dar al usuario la libertad de explorar y hacer cambios bajo su propia responsabilidad expedita el proceso de aprendizaje en usuarios noveles, y facilita la depuración en usuarios ya familiarizados con la herramienta.

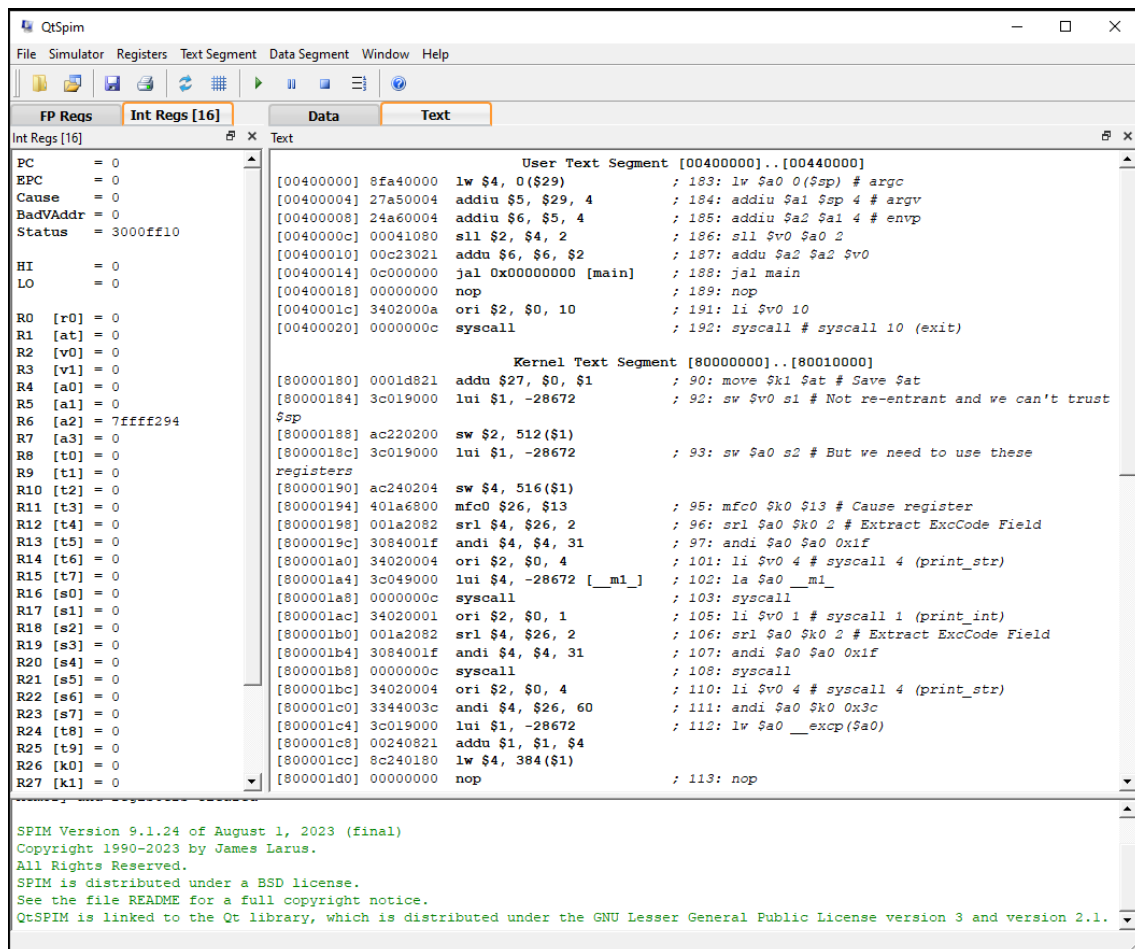


Figura 2.1: Ventana principal de QtSpim.

2.2.2. AQA Assembly Language Simulator

Este simulador online de nombre genérico, desarrollado por Peter Higginson [6], está diseñado para funcionar con AQA [7], un lenguaje ensamblador derivado de ARM creado específicamente con fines educativos y como parte del plan de estudios de la junta examinadora británica del mismo nombre [8]. Si bien el juego de instrucciones es limitado en su cantidad, no lo es en su variedad: contiene instrucciones de transferencia de datos, aritméticas, lógicas y de salto, aunque con modos de direccionamiento limitados a registro e inmediato.

El simulador implementado por Higginson no solamente ejecuta las instrucciones del lenguaje, sino que, además, muestra de forma animada el flujo que siguen los datos dentro del procesador, indicando qué componente gestiona cada parte del proceso, como se puede ver en la figura 2.2.

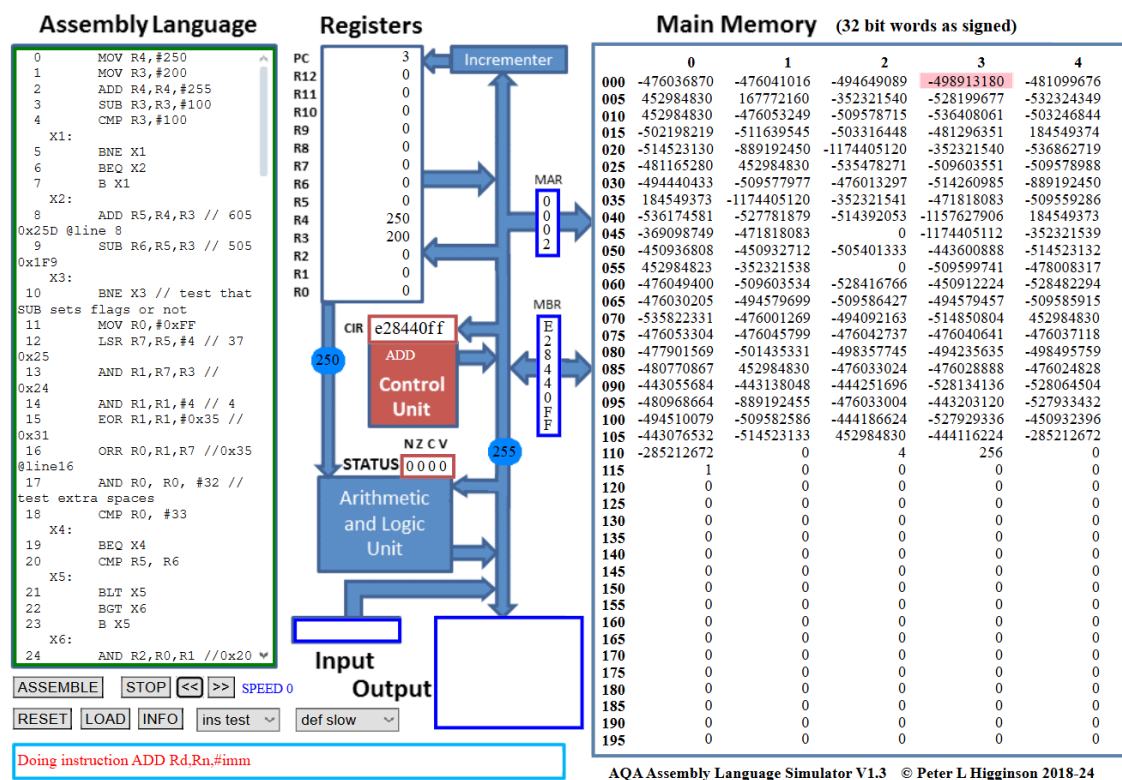


Figura 2.2: Simulador de Higginson en plena ejecución.

2.2.3. WebMIPS

WebMIPS es un simulador online creado por la Universidad de Siena para la asignatura de Arquitectura de Computadores [9], con capacidad para simular un conjunto limitado de instrucciones de la arquitectura. Al igual que el simulador de AQA visto con anterioridad, WebMIPS muestra de manera gráfica el simulador. Sin embargo, en vez de una representación conceptual animada, el nivel de detalle del procesador es mucho mayor, y cada "paso" de la ejecución avanza un ciclo de reloj, mostrando cómo las instrucciones van atravesando las diversas etapas del *pipeline* de MIPS: Obtención de la instrucción, decodificación, ejecución, acceso a memoria y escritura del resultado, como se puede ver en la figura 2.3. Al finalizar la simulación, se muestra también el total de ciclos de reloj empleados en ejecutar todo el programa.

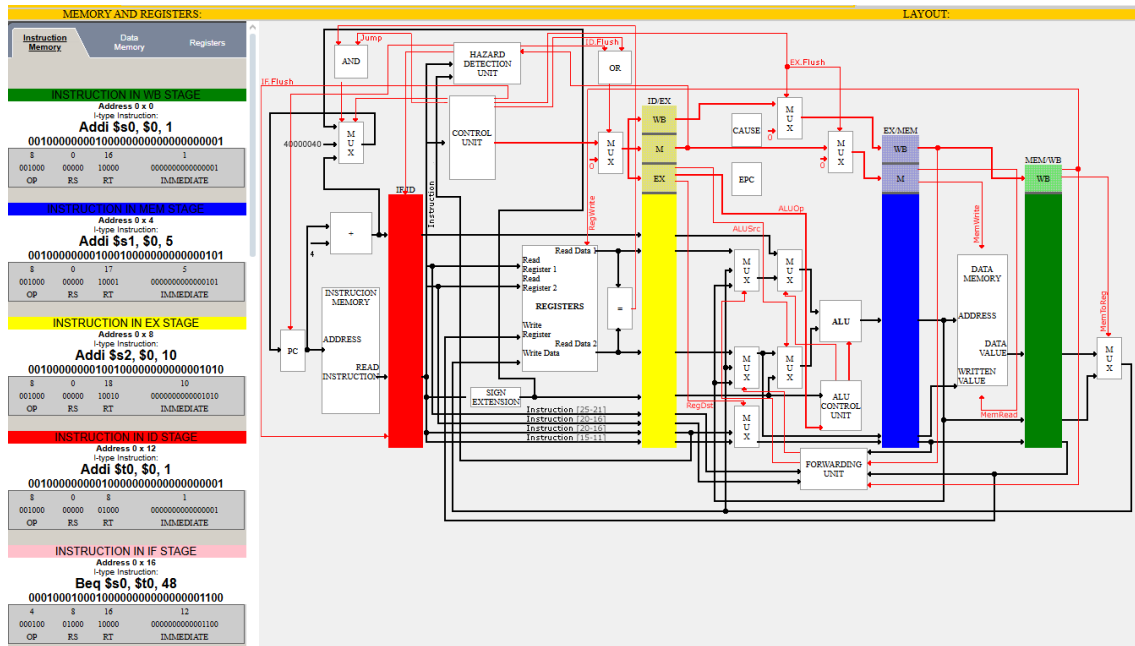


Figura 2.3: Interfaz del simulador WebMIPS.

2.3. Los predecesores de ENS2025

En esta sección se detallarán aquellas herramientas que tienen relación directa con la línea de producto de ENS2025 (es decir, simuladores del estándar IEEE 694). Cada una de estas herramientas es una evolución de la anterior, por lo que se puede interpretar esta sección como una línea temporal en el desarrollo de herramientas de simulación de ensamblador en el estándar IEEE 694.

2.3.1. ASS

ASS es, esencialmente, el predecesor de los simuladores *ENS*. Es una primera aproximación sencilla y efectiva (está contenida en un único fichero en lenguaje C de menos de 2000 líneas), aunque bastante limitada, como se verá en algunas de las características de su arquitectura. La información sobre dichas características, mostradas a continuación, se ha extraído del manual de la aplicación [10]:

- Tamaño de palabra de 16 bits, almacenando enteros con signo (con valores de -32768 a 32767).
- Memoria de 16384 palabras.
- Código almacenado en memoria a partir de la primera posición.
- Pila con crecimiento en dirección negativa, comenzando en la última posición de memoria.
- Cinco registros, cada uno con un propósito:
 - **Acumulador (Acum):** Almacena resultados de las operaciones.

- **Puntero de pila (SP).**
 - **Registro especial (R):** Sirve de extensión al registro acumulador para resultados de operaciones aritméticas, almacenando los 16 bits más significativos de la operación si los hubiera (indicando, por tanto, acarreo); salvo en el caso de la división, donde sirve para almacenar el resto de la misma.
 - **Contador de operación (CO):** Indica la posición de memoria donde está almacenada la instrucción a ejecutar (es decir, un contador de programa).
 - **Registro índice (IX):** Para direccionamientos relativos.
- Cuatro modos de direccionamiento: inmediato, directo a memoria, indirecto y relativo.
 - Instrucciones sin operandos o con un operando solamente (que puede ser omitido, en cuyo caso se usa de manera implícita el registro acumulador), divididas en varios tipos:
 - **Instrucciones de no operación:** NOP. No emplea operandos.
 - **Instrucciones de transferencia de datos:** LDA, STA, LDSP, STSP, LDR, STR, LDIX, STIX. Emplean un operando. Las operaciones LD- almacenan en el registro de su sufijo (A, SP, R, IX) el contenido del operando, mientras que las operaciones ST- almacenan el contenido del registro sufijo en el operando.
 - **Instrucciones aritméticas:** ADD, SUB, MUL, DIV, INC, DEC, NEG. Emplean un operando. Las cuatro primeras (suma, resta, multiplicación, división) aplican su operación pertinente usando de primer operando el registro acumulador y de segundo operando el operando suministrado, almacenando el resultado de vuelta en el acumulador. La división entre cero produce un error de ejecución, al igual que una división cuyo cociente exceda el rango de representación de 16 bits. Las instrucciones INC y DEC incrementan y decrementan el contenido del operando suministrado en una unidad, y NEG cambia el signo del operando.
 - **Instrucciones lógicas:** AND, OR, NOT. Emplean un operando. Las dos primeras ejecutan su operación lógica usando el acumulador y el operando suministrado, almacenando el resultado en el acumulador, y la tercera realiza una inversión bit a bit del operando.
 - **Instrucciones de manejo de pila:** PUSH, POP. Emplean un operando. La primera decrementa el valor del SP y almacena el valor del operando en la nueva dirección apuntada por el SP, y la segunda realiza el proceso opuesto.
 - **Instrucciones de salto:** J, JZ, JNZ, JP, JNP, JM, JNM. Emplean un operando. La primera es salto incondicional al valor del operando, y las dos siguientes son saltos si el acumulador es igual a 0 y distinto de 0, respectivamente. Las instrucciones JP y JNP saltan al operando si el acumulador es mayor que 0 o menor o igual que 0, respectivamente, y las instrucciones JM y JNM saltan al operando si el acumulador es menor que 0 o mayor o igual que 0, respectivamente.
 - **Instrucciones de llamada y retorno:**
 - CALL: Emplea un operando. Inserta en la pila el valor del registro CO e introduce en CO el valor del operando.
 - RET: No emplea operandos. Extrae de la pila un valor y lo introduce en el registro CO.
 - STOP: No emplea operandos. Retorna al sistema operativo.

- **Instrucciones de entrada y salida:** INPUT, WRITE, ININT, WRINT, WRSTR. Emplean un operando. Las dos primeras reciben y muestran un carácter, las dos siguientes hacen lo mismo pero con un valor entero, y la última muestra una cadena empezando por la dirección suministrada mediante el operando.
- Directivas del ensamblador:
 - **L: EQU Expresión:** Recibe de argumento una expresión. Asocia a la etiqueta L el valor de dicha expresión en la tabla de símbolos.
 - **C: DC N:** Define una constante C con valor N.
 - **DS N:** Reserva N posiciones de memoria y las inicializa a 0.
 - **DFSTR S:** Inserta la cadena de caracteres S en la memoria, finalizándola con un valor 0.
 - **END:** Señala el final del programa.
- Ensamblado en un número variable de pasadas, en función de las etiquetas definidas y su posición respecto a las referencias.
- Interfaz de línea de comandos, disponible para MS-DOS y Linux. Notablemente limitada, la interfaz solamente permitía cargar el archivo para su ejecución, junto con la opción de guardar el código máquina generado en un archivo de nombre fijo y la opción de ejecución “paso a paso”, que más bien era una ejecución verbosa que mostraba en pantalla el contenido de los registros tras cada instrucción.
- Tamaños máximos impuestos a la longitud de un mnemónico (24 caracteres), longitud de una línea (80 caracteres) y máximo de etiquetas y pasadas realizables (256).

Como se puede observar tras esta lista de especificaciones, ASS es un simulador bastante rudimentario. De hecho, diverge del estándar IEEE 694 en algunas cuestiones como la sintaxis de los modos de direccionamiento o el número de operandos, aunque esto último puede explicarse por su arquitectura reducida. La razón por la que se incluye en esta sección en vez de en la relativa a otras herramientas es por su cualidad de “ancestro” de las herramientas posteriores.

2.3.2. ENS96

ENS96 surge como versión mejorada de ASS⁵, construyendo sus capacidades sobre las de la herramienta anterior. La primera diferencia notable de ENS96 frente a ASS es su declaración de adhesión al estándar IEEE 694, aplicando el estándar de manera correcta a la sintaxis de los modos de direccionamiento. Las mejoras introducidas por ENS96 afectan tanto al aspecto arquitectónico como a la “calidad de vida” de las interacciones con el usuario, facilitando las mismas y dotando a la interfaz de usuario de gran cantidad de funcionalidades. La información que se muestra, al igual que en el apartado de ASS, se extrae directamente del manual de la aplicación [?]:

- Tamaño de palabra de 16 bits, almacenando enteros con signo (con valores de -32768 a 32767).

⁵Iniciando así de manera formal la línea de software *ENS* y aportando, de paso, el nombre de la misma.

- Memoria de 32768 palabras.
- Código almacenado en memoria a partir de la primera posición.
- Banco de registros con 14 registros:
 - **Acumulador (A):** Mismo comportamiento que en ASS.
 - **Contador de programa (PC):** Mismo comportamiento que el OC de ASS.
 - **Puntero de pila (SP):** Mismo comportamiento que en ASS.
 - **Ocho registros de propósito general (R1-R8):** Almacenan valores bajo demanda.
 - **Registro de estado (SR):** Almacena los biestables de positivo, negativo, cero y overflow.
 - **Dos registros de índice (IX, IY):** Misma finalidad que en ASS.
- Desaparecen las limitaciones de longitud de línea y de número de etiquetas declarables, pero se mantiene la de longitud de mnemónico (de hecho, se reduce a 20 caracteres).
- Cinco modos de direccionamiento: inmediato, directo a registro, directo a memoria, indirecto y relativo (este último con notación distinta para valores de desplazamiento decimales y hexadecimales).
- Instrucciones con cero, uno o dos operandos, divididas en distintos tipos⁶:
 - **Instrucciones de no operación:** NOP. No emplea operandos.
 - **Instrucciones de transferencia de datos:** LD, ST, MOVE. Emplean dos operandos. LD usa de primer operando exclusivamente un registro, y carga en el mismo el valor del segundo operando. ST guarda el valor del primer operando en el segundo, y MOVE hace lo mismo que ST, pero exclusivamente entre dos registros.
 - **Instrucciones aritméticas:** ADD, SUB, MUL, DIV, MOD, INC, DEC, NEG. Las cinco primeras emplean dos operandos, el resto solamente uno. Todas ellas modifican el registro SR en función de sus resultados.
 - **Instrucciones lógicas:** AND, OR, XOR, NOT. Las tres primeras emplean dos operandos, y NOT solamente uno. También modifican el registro SR.
 - **Instrucciones de comparación:** CMP. Emplea dos operandos. Resta el primero menos el segundo sin almacenar el resultado en el acumulador y modifica el registro SR.
 - **Instrucciones de manejo de pila:** PUSH, POP. Ambas emplean un operando.
 - **Instrucciones de salto:** J, JZ, JNZ, JP, JNP, JM, JNM, JOV, JNOV. Emplean un solo operando. La primera es la instrucción de salto incondicional, y cada pareja de las otras 8 está ligada al estado activo/inactivo de los distintos biestables: cero (-Z y -NZ), positivo (-P y -NP), negativo (-M y -NM) y overflow (-OV y -NOV).
 - **Instrucciones de llamada y retorno:** CALL, RET. Al igual que en ASS, CALL tiene un operando y RET no tiene ninguno.

⁶Muchas de ellas tienen una descripción de funcionamiento idéntica a la de su homónima de ASS, por lo que no se repetirán descripciones si no existen diferencias en ese aspecto.

- **Instrucciones de fin de ejecución:** STOP. No emplea operandos.
- **Instrucciones de entrada y salida:** INPUT, WRITE, ININT, WRINT, WRSTR. Emplean un operando.
- Implementa las mismas directivas de ensamblador que ASS, con idéntico funcionamiento.
- Ensamblado en dos pasadas, usando la segunda para resolver las referencias adelantadas.
- Interfaz de usuario, disponible para MS-DOS. La experiencia de usuario respecto a ASS mejora notablemente, incluyendo las siguientes funcionalidades:
 - Desplazamiento del puntero de lectura de la propia interfaz (separado de los punteros del simulador en sí).
 - Desensamblado de instrucciones desde la posición actual, desde una posición concreta o entre dos posiciones.
 - Visualización total o parcial de la memoria en formato hexadecimal y ASCII.
 - Visualización y modificación de registros.
 - Modificación del comportamiento de la pila, pudiendo elegir entre que el SP apunte al primer espacio libre sobre la pila (comportamiento por defecto) o que apunte al elemento en la cima de la pila.
 - Inserción de un punto de ruptura (solamente se permite uno).
 - Ejecución del código.
 - Activación/desactivación de la ejecución paso a paso.
 - Activación/desactivación del modo verboso (muestra la instrucción desensamblada que se va a ejecutar).
 - Visualización y modificación de valores dentro de la memoria.
 - Ensamblado de un nuevo fichero y posterior carga en memoria del código generado.
 - Visualización de la tabla de etiquetas.
 - Visualización del contenido de un fichero.
 - Carga y guardado de los valores de memoria desde un fichero.
 - Consulta del fichero de ayuda.
 - Salida a un shell de DOS.
 - Salida del programa.

2.3.3. ENS2001

Dado que el proceso de mejora sobre las capacidades de ENS2001 es el motivo de este Trabajo, las funcionalidades de dicho simulador serán analizadas en profundidad en los distintos capítulos del mismo. Sin embargo, sí es conveniente hacer un pequeño resumen de las mismas, extraídas directamente de la documentación [1]:

- Tamaño de palabra de 16 bits, almacenando enteros con signo (con valores de -32768 a 32767).

- Memoria de 65536 palabras.
- Código almacenado en memoria a partir de la primera posición, salvo que se especifique lo contrario mediante directiva **ORG**.
- Banco de registros con 16 registros:
 - Acumulador (A): Misma funcionalidad que en ENS96.
 - Contador de programa (PC): Misma funcionalidad que en ENS96.
 - Registro de estado (SR): Almacena en los 6 bits menos significativos los biestables de cero, desbordamiento, acarreo, signo (negativo), paridad y parada de ejecución.
 - Puntero de pila (SP): Misma funcionalidad que en ENS96.
 - Registros índice (IX e IY): Para direccionamiento relativo.
 - Diez registros de propósito general (R0-R9): Misma funcionalidad que en ENS96.
- Se mantiene la ausencia de restricciones en longitud de línea y número de etiquetas declarables, y desaparece también la restricción de tamaño máximo de mnemónico.
- Seis modos de direccionamiento: inmediato, directo a registro, directo a memoria, indirecto, relativo a contador de programa y relativo a índice.
- Instrucciones de cero, uno y dos operandos. Éstas serán ampliamente detalladas tanto en el capítulo de solución de esta memoria como en el Apéndice B de la misma, al final del cual se podrá encontrar una tabla de referencia de las instrucciones y los tipos de direccionamiento que admite cada una.
- Directivas del ensamblador:
 - [etiqueta] **EQU Expresión**: Resuelve la expresión y la asigna a la etiqueta que preceda a la directiva, si existe. En caso contrario, no se genera error pero el valor generado se pierde.
 - **DATA DATO[, DATO]***: Inserta en la memoria un conjunto de valores enteros y cadenas de caracteres, separados por comas.
 - **RES Expresión**: Resuelve la expresión y reserva un número de posiciones de memoria igual al resultado, sin inicializarlas⁷.
 - **ORG Expresión**: Resuelve la expresión y escribe el código subsiguiente a partir de la dirección de memoria obtenida como resultado.
 - **END**: Finaliza el ensamblado de código, ignorando cualquier texto que pueda haber en el documento a partir de la directiva.
- Ensamblado en dos pasadas, usando la segunda para resolver las referencias adelantadas.
- Establecimiento de zonas de código y pila, que son acotaciones de la memoria que permiten la depuración del código mediante la opción de generar errores si son invadidas por punteros.

⁷Al menos, ése es el funcionamiento buscado. Como se verá más adelante, la directiva **RES** sufre algunos problemas de inicialización.

- Establecimiento de direcciones de crecimiento de la pila; puede crecer de forma negativa (como en las herramientas anteriores) o positiva.
- Interfaz de usuario por línea de comandos, disponible para Windows, MacOS y Linux, que incluye las siguientes funcionalidades:
 - Activación y desactivación del modo de ejecución paso a paso.
 - Activación y desactivación de puntos de ruptura en diferentes direcciones (a diferencia de ENS96, ENS2001 sí permite varios puntos de ruptura en la memoria).
 - Activación y desactivación del modo de ejecución con depuración (es decir, activar o desactivar la parada de ejecución al encontrar un punto de ruptura).
 - Ensamblado de fichero y carga de sus instrucciones en la memoria.
 - Desensamblado de un número determinado de instrucciones a partir de una dirección determinada.
 - Ejecución del código.
 - Activación y desactivación de la comprobación de invasión de la zona de código por parte del SP.
 - Activación y desactivación de la comprobación de invasión de la zona de pila por parte del PC.
 - Consulta de la ayuda de los comandos.
 - Cambio de dirección de crecimiento de la pila.
 - Volcado y carga del contenido de la memoria mediante ficheros binarios.
 - Cambio de la base de representación numérica (decimal con signo o hexadecimal).
 - Consulta de todos los parámetros de configuración.
 - Visualización de una cantidad determinada de posiciones de la pila.
 - Visualización del contenido del banco de registros.
 - Visualización y modificación del contenido de un registro.
 - Activación y desactivación del reinicio del banco de registros cuando se solicite una ejecución mientras el biestable de parada está en alto.
 - Salida del programa.
- Se introduce por primera vez una interfaz gráfica de usuario, exclusiva para Windows. Dicha interfaz está formada por seis ventanas separadas, que pueden ser abiertas, cerradas y desplazadas de manera independiente a voluntad del usuario:
 - **Ventana principal:** Contiene la barra de herramientas (con botones para ensamblar un fichero, ejecutarlo, cargar o guardar un fichero binario con el contenido de la memoria, guardar un fichero con el contenido de las zonas de texto y abrir la ventana con las opciones de configuración del sistema), los menús (que contienen las mismas opciones que la barra de herramientas, junto con un menú para volver a abrir ventanas cerradas) y una zona de texto que muestra los mensajes del sistema (ficheros cargados, mensajes de error de ensamblado o en tiempo de ejecución...). Su cierre causa la salida del programa.

- **Ventana del visor de memoria:** Contiene una tabla que muestra el contenido de la memoria dirección a dirección, indicando además si dichas direcciones están dentro de la zona de código y/o de la de pila. Permite saltar a una dirección específica y reiniciar el contenido de la memoria.
- **Ventana del visor de memoria:** Contiene una tabla que muestra el contenido de la memoria dirección a dirección, indicando además cuál de esas direcciones está apuntada por el SP. Permite saltar a una dirección específica o al SP.
- **Ventana del visor de código fuente:** Contiene una tabla que muestra el contenido de la memoria desensamblado en forma de instrucciones legibles por el usuario, indicando su dirección, la posición de memoria de la primera palabra de cada una, la posición ocupada por el PC y si alguna de ellas tiene o no puntos de ruptura. La navegación vertical a través de la misma regenera la lista de instrucciones desensambladas. Permite desensamblar a partir del PC o de una dirección especificada.
- **Ventana del banco de registros:** Contiene los valores almacenados por los registros del simulador, así como los de los seis biestables. Muestra también el código en ensamblador de la siguiente instrucción a ejecutar, así como las delimitaciones de las zonas de código y pila. Por último, permite reiniciar los valores de los registros.
- **Ventana de consola:** Contiene una zona de texto donde se muestran los mensajes generados por el simulador mediante las instrucciones de salida, y donde se habilita temporalmente la escritura para el usuario a petición de las instrucciones de entrada.

2.3.4. Comentario sobre los predecesores de ENS2025

Como se puede observar, el denominador común de los cambios de versión no es tanto los avances en materia de amplitud del juego de instrucciones o de modos de direccionamiento disponibles (ya que son aspectos dictados por las necesidades del plan de estudios del momento), sino mejoras progresivas de la experiencia de usuario. Sin embargo, el aspecto del diseño interno no se ha cuidado de la misma manera, resultando en software poco orientado a la mantenibilidad y los cambios futuros. Además, aunque la interfaz gráfica es un gran avance respecto a la versión anterior, el hecho de que ésta solamente funcione en Windows es un factor bastante limitante. Estos dos aspectos, como se verá a lo largo de los capítulos siguientes, son dos de los objetos principales de ENS2025.

Capítulo 3

Propuesta

El propósito de este capítulo es especificar las características y funcionalidades del sistema a construir. Para ello, se utilizará una adaptación del documento de Especificación de Requisitos Software (ERS) detallado en el estándar IEEE 29148-2011 [11].

3.1. Introducción

Para comenzar con dicha especificación, es fundamental delimitar en primera instancia el proyecto. La siguiente sección define, por tanto, la naturaleza de la aplicación a desarrollar, su propósito y el contexto para el que ha sido concebida.

3.1.1. Ámbito

La aplicación a desarrollar es un ensamblador-simulador del estándar IEEE 694, escrito en Java como adaptación del proyecto ENS2001 [1], desarrollado en C++ por Federico J. Álvarez como su Trabajo Final de Carrera para la Universidad Politécnica de Madrid en 2003. La aplicación recibirá el nombre ENS2025, siguiendo con la nomenclatura establecida por ENS96 [12].

ENS2025 es, al igual que sus predecesores, un generador de código máquina mediante ficheros escritos en ensamblador siguiendo el estándar IEEE 694. Incluye también un simulador de ejecución del código ensamblado.

El ámbito de aplicación es principalmente académico y, puesto que aspira a reemplazar a su predecesor como la herramienta de uso en las asignaturas en las que éste se usa hasta la fecha, está construido con la idea de replicar sus requisitos y comportamiento dentro de lo posible.

3.1.2. Términos y abreviaturas

Los términos, acrónimos y abreviaturas usados en este documento de requisitos son los siguientes:

- **IEEE:** *Institute of Electrical and Electronic Engineers* (Instituto de Ingenieros Eléctricos y Electrónicos). Asociación de ingenieros de carácter global, dedicada a la normalización y el establecimiento de estándares en los ámbitos de la ingeniería eléctrica, electrónica e informática.

- **JVM:** *Java Virtual Machine* (Máquina Virtual de Java). Entorno de ejecución del lenguaje Java. Actúa como capa entre el sistema operativo y los programas en Java, permitiendo que cualquier sistema operativo capaz de ejecutar una JVM pueda ejecutar los programas escritos en Java, sin necesidad de adaptarlos¹.
- **UTF-8:** *8-bit Unicode Transformation Format* (Formato de Transformación de Unicode de 8 bits). Formato de codificación que permite la representación de la mayoría de los caracteres del mundo. Es el formato más extendido.
- **BOM:** *Byte Order Mark* (Marca de Orden de Bytes). Secuencia opcional de bytes que identifica una codificación específica de Unicode.
- **SO:** Sistema Operativo.
- **CLI:** *Command Line Interface* (Interfaz de Línea de Comandos). Programa software cuya interacción con el usuario tiene lugar empleando una consola de comandos como medio.
- **GUI:** *Graphical User Interface* (Interfaz Gráfica de Usuario). Programa software cuya interacción con el usuario tiene lugar por medio de elementos gráficos como imágenes, botones, ventanas...
- **API:** *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Código que permite a dos aplicaciones comunicarse entre sí para compartir datos mediante el establecimiento de una serie de métodos que tienen como objeto dicha comunicación.
- **UNED:** Universidad Española de Educación a Distancia.
- **E/S:** Entrada/Salida.

3.2. Visión general del producto

A continuación, se ofrece una descripción global de la aplicación desde diferentes puntos de vista. Se comienza por enmarcar el producto en su contexto para establecer una perspectiva general del mismo, detallando después sus interfaces, funciones y las características de sus usuarios potenciales.

3.2.1. Perspectiva del producto

El propósito de este apartado es la descripción de las relaciones fundamentales del software con otros sistemas, sus usuarios y los entornos en los que interactúa.

En el caso de ENS2025, esta perspectiva se simplifica notablemente, puesto que no forma parte de un sistema mayor ni se relaciona con otros similares, operando como una aplicación independiente y autocontenida. Su única dependencia directa es la JVM, sobre la que se ejecuta de manera autónoma respecto al SO subyacente.

¹Esto se cumple siempre y cuando dichos programas no utilicen librerías que hagan uso de recursos específicos del sistema operativo, claro está.

A continuación, se trata con más detalle cada uno de los elementos que definen esta perspectiva, abarcando sus interfaces con el entorno, sus características operacionales y sus restricciones.

3.2.1.1. Interfaces del sistema

La única interfaz de sistema es la propia JVM, que actúa como capa de abstracción entre el programa y el SO subyacente. Para garantizar su compatibilidad con los principales SO, no se han empleado librerías específicas de ninguno de ellos. Si bien la versión GUI del programa emplea JavaFX, y éste utiliza servicios específicos de cada SO para el renderizado de los elementos gráficos, este proceso es transparente para el usuario, requiriendo del mismo únicamente que utilice la versión generada específicamente para su SO.

Ambas versiones se compilarán usando Java 11; pero la versión gráfica no requiere necesariamente una JVM instalada, ya que se facilitará un directorio portable. Si el usuario decide volver a compilar el código, se requerirá como mínimo Java 11 para el compilado (aunque se ha compilado usando compatibilidad con compilador de Java 8 y es altamente probable que funcione en sistemas con Java 8 instalado, se recomienda tener al menos Java 11).

Por otro lado, ENS2025 procesa ficheros de texto plano no generados por el mismo codificados en el estándar UTF-8 sin BOM. Queda a elección del usuario la elección del software utilizado para generar estos ficheros.

3.2.1.2. Interfaces de usuario

ENS2025 tendrá dos versiones ejecutables diferentes, una de ellas mediante CLI y otra mediante GUI. Al margen de las diferencias en experiencia de usuario, ambas versiones poseen el mismo núcleo funcional de ensamblado y simulación de ficheros de código.

3.2.1.3. Interfaces de hardware

No existen interacciones con hardware especializado, ni interacciones fuera de lo común con los dispositivos estándar (ratón, teclado, monitor).

3.2.1.4. Interfaces de software

Al no tratarse de un sistema de software con varios componentes significativos, no son destacables interfaces del mismo. Tampoco realiza llamadas a ninguna API externa, ni posee una para recibir llamadas.

3.2.1.5. Interfaces de comunicación

Este apartado no aplica para este software.

3.2.1.6. Restricciones de memoria

El principal consumo de memoria de ENS2025 viene definido, precisamente, por el banco de memoria del procesador, consistente en 216 palabras de 16 bits, o 128 KB en total. Incluso con el coste de memoria adicional derivado de los propios elementos, la cantidad de memoria utilizada directamente por las estructuras internas de la aplicación es insignificante en comparación con la memoria consumida por la propia JVM. Por lo tanto, cualquier sistema capaz de ejecutar fluidamente la JVM no tendrá problemas de rendimiento derivados del uso de memoria adicional.

Aunque ENS2025 está desarrollado con vistas a facilitar en cierta medida la refactorización a tamaños de palabra y de memoria mayores (dando paso a la posibilidad de que se pudiera emplear en el futuro para simular procesadores de 32 bits, por ejemplo), dicha refactorización sí tendría un impacto considerable en el consumo de memoria. Por ejemplo, un simulador de arquitectura de 32 bits con un banco de memoria completo contendría 2^{32} palabras de 32 bits (4 bytes), por lo que solamente la estructura interna del banco de memoria ya requeriría 16 GB.

3.2.1.7. Operaciones

En este apartado se detallan las distintas categorías de operaciones que ENS2025 es capaz de realizar, clasificadas según su naturaleza y el nivel de interacción que requieren.

■ Operaciones interactivas:

- Carga de ficheros de ensamblador.
- Inicio e interrupción de la simulación en sus distintos modos (normal, paso a paso y depuración).
- Modificación de la configuración de la aplicación.
- Inserción y eliminación de puntos de ruptura.
- Inserción de datos por consola cuando la simulación lo requiera.
- Modificación de datos de la simulación (banco de registros, biestables, posiciones de memoria).
- Guardado y carga de imagen de estado del procesador.
- Guardado de los mensajes generados por la aplicación (solamente en aplicación GUI).

■ Operaciones desatendidas:

- Proceso de codificación del fichero de ensamblador en instrucciones.
- Bucle de simulación de la aplicación.
- Decodificación de las instrucciones.
- Ejecución de las instrucciones.

■ Funciones de soporte de procesamiento de datos:

- Importación de los ficheros de código ensamblador. La aplicación acepta cualquier tipo de fichero en texto plano, aunque por convención se les ha asignado la extensión `.ens`. La versión GUI de la aplicación mostrará por defecto únicamente los ficheros con esa extensión.
- Codificación de los datos obtenidos de un fichero ensamblador en instrucciones interpretables por el simulador.

■ **Operaciones de copia de seguridad y recuperación:**

- No hay operaciones programadas de forma regular y automática.
- El usuario tiene la opción de guardar en un fichero una imagen del estado actual del simulador para su posterior carga. Dicho fichero es un volcado binario del contenido de la memoria, el banco de registros y las delimitaciones de las zonas de código y pila.
- En la versión CLI de la aplicación, ante la eventualidad de un bucle infinito, el usuario tiene la posibilidad de interrumpir la ejecución del programa. En ese caso, el programa generará antes de cerrarse un fichero de imagen de memoria como los descritos en el punto anterior, que puede ser cargado manualmente durante la ejecución o mediante un parámetro específico en la invocación por línea de comandos.

3.2.1.8. Requisitos de adaptación del sitio.

Este apartado no aplica para este software.

3.2.1.9. Interfaz con servicios

Este apartado no aplica para este software.

3.2.2. Funciones del producto

La aplicación tiene dos funciones principales:

- Por un lado, el ensamblador, que procesa ficheros de texto y genera código máquina interpretable por el simulador, o el primer error con el que se encuentre (todos los errores son fatales).
- Por otro lado, el simulador, que interpreta el código máquina generado y lo convierte en instrucciones que ejecuta.

3.2.3. Características del usuario

Debido al carácter específico y hecho a medida de ENS2025, el perfil de usuario principal es tanto el Equipo Docente como los alumnos de la asignatura de Procesadores del Lenguaje II

de la UNED. Por extensión, cualquier docente o alumno de una carrera o curso que emplee ENS2001 con fines similares entra en este perfil.

En un ámbito más general, se pueden distinguir dos categorías de usuario:

- **Estudiantes de Informática y aficionados al campo del estudio:** esta categoría engloba a usuarios que buscan comprender los principios fundamentales de la arquitectura de computadores y del lenguaje ensamblador. Esta aplicación les permite experimentar con la ejecución de código ensamblador de manera visual e interactiva.
- **Personal docente:** esta categoría engloba a usuarios que, como su nombre indica, son profesores y tutores que necesitan una herramienta para mostrar conceptos básicos de arquitectura de computadores y lenguaje ensamblador en un entorno práctico, fácil de usar y que permita la creación de multitud de ejemplos y ejercicios.

Puesto que ENS2025 simula una versión simplificada de un procesador (no se tienen en cuenta cuestiones como los ciclos empleados por las instrucciones o las diferencias entre tiempos de acceso al banco de registros y la memoria), perfiles de usuario más avanzados que buscan profundizar su aprendizaje sobre la arquitectura de computadores encontrarán un uso limitado para esta aplicación.

3.2.4. Supuestos y dependencias

Se parte del supuesto de que las necesidades de las prácticas de la asignatura Procesadores del Lenguaje II no sufrirán cambios significativos en materia de contenido y evaluación en un futuro previsible (principalmente, el estándar de ensamblador utilizado), por lo que los requisitos se mantendrán en el tiempo de cara a futuras versiones.

3.2.5. Distribución de requisitos

Se identifica un módulo principal, que contiene toda la lógica del programa (unidad de control), y los módulos de las dos interfaces que conectan dicho módulo con el usuario (interfaz de consola y gráfica) y permiten la gestión de la configuración de la aplicación y de la entrada y salida de datos y mensajes, sirviendo como punto de entrada y salida de la misma.

Dentro del módulo principal, se identifican los siguientes submódulos, con sus funcionalidades asignadas:

- **Arquitectura:** Almacenamiento y gestión de la memoria y del banco de registros y biestables.
- **Ensamblador:** Carga de ficheros de texto y codificación del contenido a código máquina.
- **Desensamblador:** Conversión del código máquina a código ensamblador legible por humanos.

- **Decodificador:** Conversión del código máquina a instrucciones ejecutables.
- **Simulador:** Gestión del bucle de ejecución de instrucciones.

3.2.6. Requisitos especificados

Este apartado proporciona una visión general de alto nivel de las funcionalidades y capacidades principales de la aplicación. Los detalles específicos de cada requisito se presentarán en secciones posteriores. Dichas funcionalidades y capacidades son las siguientes:

- **Funcionalidades:**
 - La aplicación debe permitir la carga y ejecución de instrucciones de un procesador de 16 bits.
 - La aplicación debe mantener el comportamiento (instrucciones permitidas, comportamiento de las instrucciones) de la versión ENS2001, corrigiendo únicamente los errores identificados en dicha versión.
 - La aplicación debe permitir la carga de programas desde ficheros externos.
 - La aplicación debe permitir la interacción con el usuario para visualizar e introducir datos.
- **Capacidades:**
 - Entradas:
 - Ficheros de programas en ensamblador.
 - Datos introducidos por el usuario a través de la interfaz E/S.
 - Salidas:
 - Visualización del estado de la memoria y los registros.
 - Visualización de instrucciones desensambladas.
 - Salidas generadas por la simulación.
 - Mensajes del sistema.
 - Funciones:
 - Carga de programas.
 - Ejecución de instrucciones.
 - Manipulación de la arquitectura de memoria.
 - Interacción con el usuario.

3.3. Requisitos específicos

Esta sección detalla los requisitos que debe cumplir la aplicación para considerar satisfactorio su desarrollo. Al tratarse de una refactorización de una versión anterior de la aplicación, y al buscar mantenerse lo más fiel posible a las especificaciones de la misma, la lista de requisitos específicos de ENS2025 coincide en gran medida con la de ENS2001, con algunos requisitos originales modificados y otros adicionales introducidos en esta versión.

3.3.1. Interfaces externas

REQ-01: Interfaces de usuario.

Se construirá la misma aplicación con dos interfaces distintas. La primera es una interfaz de línea de comandos (CLI), ejecutable mediante consola. La segunda es una interfaz gráfica (GUI). Ambas serán ejecutables en cualquier sistema operativo que pueda instalar y ejecutar la versión mínima o posterior de la JVM requerida para cada una.

3.3.2. Funciones

REQ-02: Arquitectura del sistema.

El sistema simula una arquitectura basada en el procesador Z80 [13] y estructurada de la siguiente forma:

- Procesador con tamaño de palabra de 16 bits.
- Memoria de 2^{16} (65536) palabras de 16 bits.
- Banco de 16 registros:
 - **R0 a R9:** Registros de propósito general.
 - **A:** Registro acumulador.
 - **SR:** Registro de estado.
 - **IX, IY:** Índices.
 - **SP:** Puntero de pila.
 - **PC:** Contador de programa.
- Biestables de estado contenidos en los 6 bits menos significativos del registro de estado:
 - **Z:** Cero.
 - **C:** Acarreo.
 - **V:** Desbordamiento.
 - **P:** Paridad.
 - **S:** Signo.
 - **H:** Fin de ejecución.
- Entrada y salida por consola de números enteros de 16 bits y texto, tanto caracteres individuales como cadenas de ellos.
- Capacidad de operar sobre enteros con signo de 16 bits.
- Generación de excepciones durante la ejecución:
 - Instrucción no implementada.

- División entre cero.
- Final de memoria alcanzado.
- Punto de ruptura encontrado.
- Contador de programa invade la zona de pila (opcional).
- Puntero de pila invade la zona de código (opcional).
- Desbordamiento de pila (opcional).
- Subdesbordamiento de pila (opcional).

REQ-03: Instrucciones.

El lenguaje ensamblador que se defina debe ser idéntico al empleado por la aplicación anterior, ENS2001, tanto en estructura gramatical como en las instrucciones implementadas y su comportamiento. En el Anexo I de esta memoria se especificará el juego de instrucciones completo.

REQ-04: Operandos.

Al igual que en el requisito anterior, el número de modos de direccionamiento disponibles para cada operación, así como la combinación y orden de los mismos, queda dictado por los disponibles en ENS2001. En el Anexo I, en la misma tabla de instrucciones, se detallará el tipo de direccionamiento disponible para cada operando.

REQ-05: Ficheros fuente.

El código fuente de los programas a ensamblar y simular se introducirá en el sistema mediante un fichero de texto plano. El nombre y ubicación de este fichero no tienen relevancia para la ejecución, y serán proporcionados al sistema cuando éste lo requiera. En la versión de consola de la aplicación, este fichero podrá ser proporcionado durante el lanzamiento como parámetro.

REQ-06: Ensamblado.

El código fuente se ensamblará de acuerdo al lenguaje descrito. El resultado del ensamblado se cargará en memoria, siempre que no hayan surgido errores durante el proceso, dejando el simulador preparado para comenzar a ejecutar las instrucciones a conveniencia del usuario.

REQ-07: Visualización de errores de ensamblado.

El sistema debe informar de los errores (si los hubiera) en el proceso de ensamblado, aportando toda la información posible sobre el error y deteniendo el proceso de ensamblado. Aparte de errores relativos al procesamiento y codificación del código del fichero, también se informará de errores como la imposibilidad de encontrar o abrir el fichero especificado, o incompatibilidad en la codificación del texto del fichero.

REQ-08: Designación de zona de código.

Tras el proceso de ensamblado, el sistema designará automáticamente las direcciones de inicio y fin de la zona de código. La dirección de inicio será la dirección más cercana al comienzo de la memoria en la que se haya reservado un espacio, insertado datos o escrito una instrucción. Análogamente, la dirección de fin será la dirección más cercana al final de la memoria en la que se haya reservado un espacio, insertado datos o escrito una instrucción.

REQ-09: Designación de zona de pila.

Tras el proceso de ensamblado, el sistema designará automáticamente las direcciones de inicio y fin de la zona de pila. Para ello, tras designar la zona de código, determinará cuál es la zona de memoria más grande de entre las zonas existentes antes y después de la zona de código. La dirección de inicio dependerá del modo de crecimiento de la pila: una pila con crecimiento positivo tendrá su dirección de inicio situada al principio de la zona de memoria seleccionada, y una pila con crecimiento negativo tendrá su dirección de inicio situada al final de la zona de memoria seleccionada.

La dirección de fin de la pila se asignará inicialmente al mismo valor que la de inicio, y su valor cambiará de forma dinámica a lo largo de una simulación, tomando como valor el valor que tenga en ese momento el puntero de pila.

Se considerará “zona de pila” el conjunto de direcciones de memoria comprendido entre esas dos direcciones, independientemente de la dirección de crecimiento de la pila.

REQ-10: Volcado de memoria a fichero.

El sistema proporcionará la opción de crear una imagen con el estado actual del simulador (contenido de la memoria, del banco de registros y de las zonas de código y pila) en un fichero binario a petición del usuario.

REQ-11: Recuperación de estado desde fichero.

El sistema proporcionará la opción de recuperar el estado del simulador desde un fichero creado previamente por el sistema (REQ-10) a petición del usuario.

REQ-12: Retrocompatibilidad de los ficheros de volcado.

En la medida de lo posible, los ficheros de volcado generados por la versión ENS2001 y los generados por ENS2025 deberán ser compatibles en ambas direcciones: cada versión deberá poder cargar una imagen de estado del simulador sin problemas usando un fichero de volcado generado por la otra.

REQ-13: Base de representación numérica.

El usuario podrá elegir el formato de representación para los números enteros, que podrá ser hexadecimal, decimal con signo o decimal sin signo. Cuando la representación sea decimal, algunos valores mantendrán su representación con signo o sin signo de manera independiente a la modalidad decimal seleccionada:

- Las direcciones de memoria siempre se representarán sin signo.
- Los modos de direccionamiento relacionados con direcciones de memoria siempre se representarán sin signo.
- Los modos de direccionamiento que representen desplazamientos respecto a un índice siempre se representarán con signo.
- Los bancos de registros IX, IY, SP y PC, que generalmente se usan para almacenar

direcciones de memoria, siempre se representarán sin signo.

REQ-14: Consulta de una posición de memoria.

El sistema permitirá visualizar el contenido de una posición de memoria.

REQ-15: Consulta de una zona de memoria.

El sistema permitirá visualizar el contenido de una zona de memoria.

REQ-16: Edición de una posición de memoria.

El sistema permitirá editar el contenido de una zona de memoria.

REQ-17: Reinicio del contenido de la memoria.

El sistema permitirá reiniciar el contenido de toda la memoria.

REQ-18: Consulta de la pila del sistema.

El sistema permitirá visualizar el contenido de una zona de la pila a partir de la posición del puntero de pila.

REQ-19: Consulta del valor de un registro.

El sistema permitirá visualizar el contenido de un registro.

REQ-20: Edición del valor de un registro.

El sistema permitirá editar el contenido de un registro.

REQ-21: Reinicio del banco de registros.

El sistema permitirá reiniciar el contenido del banco de registros, lo que asignará valor 0 a todos ellos salvo al registro del puntero de pila. Este último actualizará su contenido al valor de la dirección designada como inicio de la zona de pila.

REQ-22: Desensamblado de una zona de memoria.

El sistema permitirá desensamblar el contenido de la memoria a partir de una posición indicada por el usuario, convirtiendo a lenguaje ensamblador los datos extraídos.

REQ-23: Configuración de crecimiento de la pila.

El usuario podrá determinar si la pila crece de forma positiva o negativa.

REQ-24: Modificación de la zona de código.

El sistema permitirá modificar las direcciones de inicio y fin del código.

REQ-25: Modificación de la zona de pila.

El sistema permitirá modificar la dirección de inicio de la zona de pila. No permitirá modificar de forma directa el valor de la zona de fin, ya que éste se encuentra ligado al valor del puntero de pila.

REQ-26: Definición de puntos de ruptura.

El sistema permitirá establecer y eliminar puntos de ruptura, que detendrán la ejecución de instrucciones por parte del simulador.

REQ-27: Ejecución paso a paso.

El usuario podrá optar por ejecutar el código instrucción a instrucción para visualizar con mayor granularidad los cambios producidos en el estado del simulador.

REQ-28: Ejecución hasta un punto de ruptura (depuración).

El usuario podrá optar por ejecutar el código hasta que se encuentre con una instrucción marcada con un punto de ruptura, en cuyo caso abandonará su decodificación y detendrá la simulación sin ejecutarla.

REQ-29: Ejecución estándar del código.

El usuario podrá ejecutar el código con normalidad, ignorando puntos de ruptura y recorriendo de forma secuencial el flujo de instrucciones hasta que encuentre una instrucción de fin de ejecución o se produzca un error.

REQ-30: Detención de la simulación.

El simulador detendrá la ejecución del código si se cumple una de las siguientes condiciones:

- El biestable H está en alto (valor 1).
- Se produce una excepción durante la ejecución.
- Se está ejecutando el código en modo depuración y se encuentra un punto de ruptura.
- El usuario elige la ejecución paso a paso y el simulador se detiene tras ejecutar una única instrucción.

REQ-31: Consola de entrada/salida.

El sistema mostrará una consola para las operaciones de entrada/salida. Dicha consola será la propia consola del sistema cuando se esté trabajando en la versión CLI de la aplicación, y un área de texto dedicada cuando se trate de la versión GUI.

REQ-32: Comprobar si el contador de programa invade la zona de pila.

Se dará al usuario la posibilidad de indicar al sistema que debe comprobar durante la simulación si el contador de programa toma en algún momento un valor comprendido entre las direcciones de inicio y fin de la pila, generando una excepción y deteniendo la simulación.

REQ-33: Comprobar si el puntero de pila invade la zona de código.

Se dará al usuario la posibilidad de indicar al sistema que debe comprobar durante la simulación si el puntero de pila toma en algún momento un valor comprendido entre las direcciones de inicio y fin de la zona de código, generando una excepción y deteniendo la simulación.

REQ-34: Comprobación de desbordamiento (overflow).

Se dará al usuario la posibilidad de indicar al sistema que debe comprobar durante la simulación si el puntero de pila incurre en overflow, generando una excepción y deteniendo la simulación. Se considerará overflow si, como resultado de una operación de inserción en la pila que genere un desplazamiento del puntero, éste rebasa el límite de memoria (en caso de crecimiento positivo, pasaría de la última dirección disponible a 0, y viceversa en caso de crecimiento negativo).

REQ-35: Comprobación de subdesbordamiento (underflow).

Se dará al usuario la posibilidad de indicar al sistema que debe comprobar durante la simulación si el puntero de pila incurre en underflow, generando una excepción y deteniendo la simulación. Se considerará underflow si, como resultado de cualquier operación, el puntero de pila termina en una dirección “anterior” al inicio de pila (en caso de crecimiento positivo, una posición anterior es una posición con dirección de memoria menor que la del inicio de pila, y al revés para crecimiento negativo). La comprobación de underflow tendrá menor prioridad que la de overflow si ambas están activadas.

REQ-36: Detención arbitraria de la simulación.

El sistema permitirá al usuario detener la simulación en cualquier momento, independientemente del estado en que se encuentre la misma.

REQ-37: Reinicio de registros al ejecutar.

Se dará al usuario la posibilidad de indicar al sistema que, si recibe una orden de iniciar simulación mientras el biestable H está en alto (valor 1), deberá primero reiniciar los valores del banco de registros, reiniciando de forma efectiva la ejecución desde el principio.

REQ-38: Consulta de la configuración.

El usuario tendrá la posibilidad de consultar en cualquier momento los valores actuales de configuración del sistema.

REQ-39: Consulta de puntos de ruptura en el código desensamblado.

El usuario podrá ver junto al código fuente los puntos de ruptura incondicionales definidos. No existirá la posibilidad de definir puntos de ruptura condicionales.

REQ-40: Almacenamiento de la salida generada durante la sesión.

La versión GUI de la aplicación permitirá al usuario en cualquier momento guardar en un fichero de texto el contenido de la sesión (mensajes del sistema y entrada/salida de la consola de simulación).

REQ-41: Exclusividad de la simulación.

La ejecución de la simulación será bloqueante para el resto de componentes del sistema. El usuario no tendrá acceso ni capacidad de modificar ningún elemento del sistema mientras la simulación esté ejecutándose, pudiendo optar solamente a detener prematuramente la misma.

REQ-42: Fin de ejecución de la aplicación.

El usuario podrá terminar la ejecución de la aplicación en cualquier momento.

REQ-43: Invocación con parámetros.

La versión CLI permitirá su invocación con parámetros que permitirán:

- Cargar y ensamblar un fichero de código al inicio del programa.
- Cargar una imagen de memoria al inicio del programa.
- Iniciar un proceso de carga de fichero de código, ensamblado, simulación y salida.
- Iniciar un proceso de carga de fichero de memoria, simulación y salida.
- Generar un fichero de memoria como volcado como complemento a un parámetro de simulación.
- Cargar un fichero de configuración temporal para el ciclo de ejecución presente.

3.3.3. Requisitos de usabilidad

Puesto que la versión ENS2001 se ha mantenido vigente durante más de dos décadas, se asume que su usabilidad es aceptable, si bien no necesariamente óptima. Respecto a esa versión, se buscan los siguientes requisitos de usabilidad:

REQ-44: Unificación de la GUI.

La versión ENS2001 generaba una ventana de GUI para cada elemento (banco de registros, memoria, visor de código, consolas. . .). La nueva versión del sistema deberá incorporarlos todos en una sola ventana.

REQ-45: Traslado de las opciones de ejecución:

En lugar de definirse en una ventana de configuración si la simulación debe ejecutarse con normalidad, paso a paso o en modo depuración, se añadirán las tres opciones de ejecución para que el usuario decida en todo momento usar la que más le convenga sin necesidad de navegar menús.

REQ-46: Traslado de las opciones de configuración:

En la versión GUI de la aplicación, se suprimirá la ventana de configuración, trasladando los elementos de la misma a una barra de herramientas que haga visibles todas las opciones para facilitar su acceso.

3.3.4. Requisitos de rendimiento

Se trata de una aplicación monousuario y, teniendo en cuenta la potencia de cálculo de los ordenadores personales actuales y que no se va a simular la arquitectura en tiempo real, no se plantea ninguna restricción en cuanto al rendimiento que se espera obtener.

3.3.5. Requisitos lógicos de la base de datos

Dado que ENS2025 opera con datos en memoria y ficheros planos para su estado y configuración, no se requiere una base de datos persistente para su funcionamiento. Por lo tanto, esta sección no aplica.

3.3.6. Restricciones de diseño

El diseño de la herramienta se efectuará empleando el lenguaje UML para modelar el diseño orientado a objetos, siguiendo los principios SOLID [14]

3.3.7. Cumplimiento de estándares

Aunque no se trate de un estándar general y debidamente tipificado, se hará lo posible por seguir la convención de nombres en Java [15].

3.3.8. Atributos del sistema de software

El sistema deberá ser portable a cualquier máquina con el mínimo coste posible. En principio, cualquier SO capaz de instalar y ejecutar la JVM con la versión mínima indicada debe poder ejecutar la herramienta.

3.3.9. Verificación

Las pruebas se detallan en el capítulo pertinente de esta memoria.

Capítulo 4

Planificación y costes

Antes de empezar a diseñar cualquier solución para el problema, es necesario considerar dos aspectos cruciales del proyecto. Por un lado, es necesario dividirlo en fases y tareas, y asignar una estimación de tiempo para cada una. Por otro lado, es necesario determinar los costes asociados al proyecto, tanto en capital económico como humano. En este capítulo se elaborará sobre ambos aspectos.

4.1. Planificación

El mayor desafío para la realización de ENS2025 es el tiempo. Su predecesor, ENS2001, fue desarrollado en aproximadamente dos años; el objetivo de este proyecto es condensar el desarrollo de un proyecto notablemente más amplio en cuatro meses. Por lo tanto, se asume de partida que será necesario un *crunch* considerable para alcanzar los objetivos fijados en el tiempo previsto. Aunque se prevé la reutilización de ciertos aspectos del diseño debido a que la plantilla de requisitos es mayormente idéntica, el cómputo de horas estimadas seguirá siendo elevado.

Se puede ver en el diagrama de Gantt de la figura 4.1 que la planificación del proyecto se divide en cinco fases:

- **Diseño y planificación:** En esta fase se lleva a cabo el proceso de análisis de los requisitos del proyecto, así como un diseño y modelado de alto nivel, que consiste en detallar la estructura general del sistema y las responsabilidades, funcionalidades y relaciones de los distintos elementos que lo componen. Tiene una duración estimada de 2 semanas (14 días).
- **Desarrollo de la unidad de control:** Por “unidad de control”, como se verá más adelante, se entiende el núcleo del software: la lógica interna que rige el funcionamiento del mismo. Puesto que uno de los objetivos es la creación de un código más mantenible mediante la modularización, el desarrollo de ENS2025 se divide en varias etapas de diseño y desarrollo relativas a cada elemento, profundizando así en lo detallado en la fase de diseño y planificación. De esta manera, si el desarrollo se llevase a cabo por un equipo en lugar de por una sola persona, estas tareas se podrían realizar de forma concurrente. Tiene una duración estimada de 6 semanas (42 días).
- **Interfaces de usuario:** En esta fase se desarrollan las dos versiones de la herramienta que interactúan con la unidad de control. Tiene una duración estimada de 3 semanas (21 días).
- **Pruebas y corrección de errores:** En esta fase se diseña un juego de pruebas con la información recabada durante los procesos de diseño e implementación de los distintos

módulos, y se corrigen posibles errores detectados durante la ejecución del mismo. Tiene una duración estimada de 15 días.

- **Documentación:** Aunque es de esperar que durante las fases anteriores se haya generado documentación y se haya recogido información sobre el proyecto, es en esta fase donde se lleva a cabo un proceso documentación formal, que culmina con la realización de esta memoria de trabajo. Tiene una duración estimada de un mes (30 días).

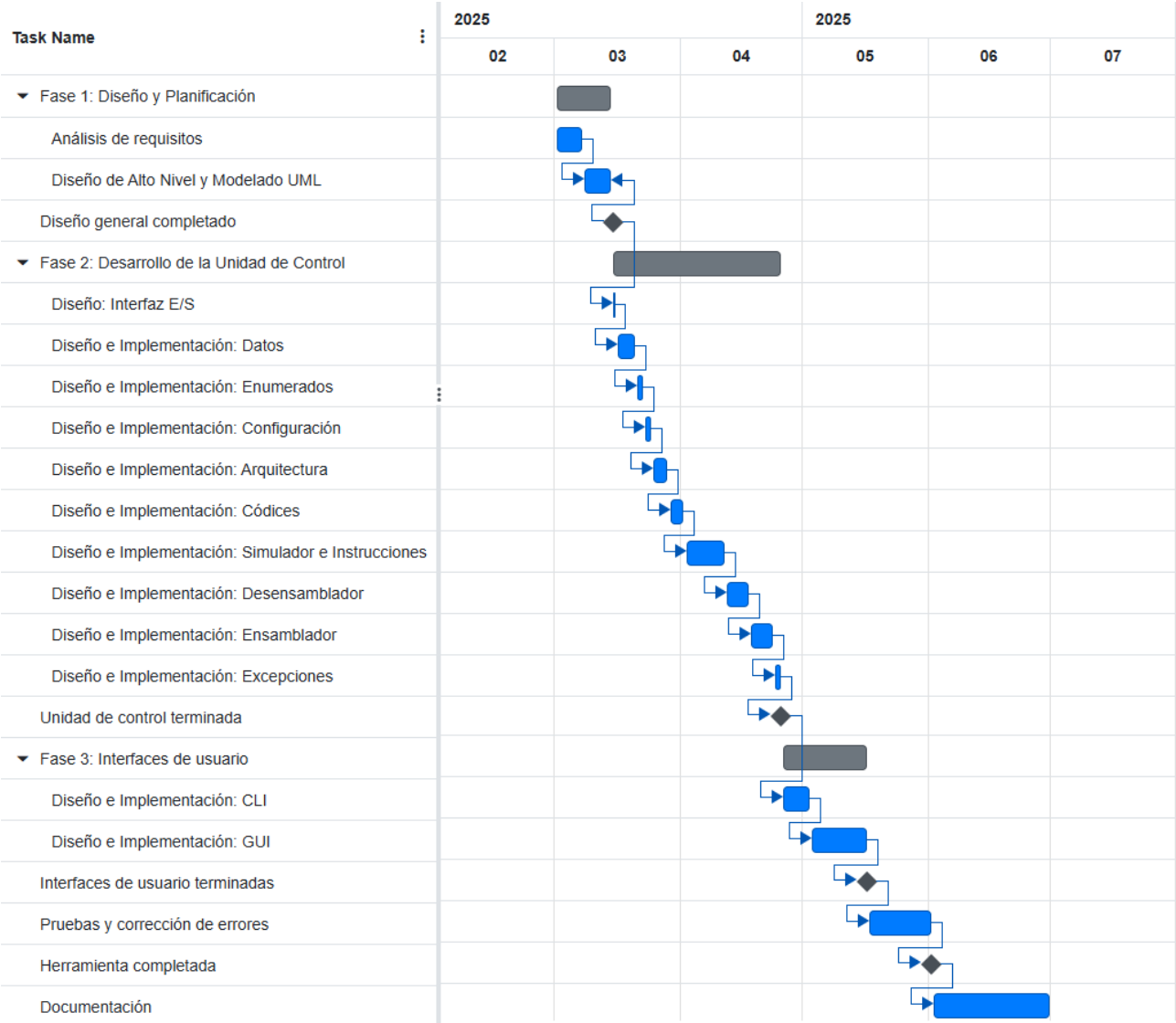


Figura 4.1: Diagrama de Gantt con la planificación original de tareas y sus respectivas estimaciones de tiempo.

Como ya se ha mencionado, para cumplir las estimaciones de cada fase y subfase será necesario invertir una gran cantidad de horas. Se estima que se requerirá un ritmo de trabajo constante de 10 horas diarias, incluyendo los fines de semana. Puesto que la duración conjunta de las cuatro fases es de 122 días, **se estima que se requerirán 1220 horas** para llevar a cabo el desarrollo del proyecto de principio a fin.

4.2. Costes

En esta sección se toma como supuesto el caso de una empresa que pretende externalizar el desarrollo de la herramienta producto de este proyecto, con el fin de estimar el valor económico del mismo.

Puesto que las distintas fases requieren distintos perfiles, se ha recurrido al convenio colectivo de empresas de consultoría y tecnologías de la información publicado en el BOE de 2025 [16]. Se han consultado en dicho documento las distintas áreas y grupos para asignarlos a cada tarea de la manera más ajustada, así como las tablas salariales de los mismos para tener un cálculo aproximado del coste bruto *mínimo* para la empresa, sin tener en cuenta impuestos derivados como el IVA.

En consecuencia, se ha definido el siguiente reparto de perfiles profesionales para cada fase del proyecto:

- **Diseño y planificación:** Para el diseño y coordinación general del proyecto, así como la elaboración de las responsabilidades de cada elemento (y equipos, si los hubiera), se estima que el rol se corresponde con el grupo A (coordinación y planificación de recursos y definición de objetivos y directrices) del área 4 (consultoría de negocio y tecnológica).
- **Desarrollo de la unidad de control:** En esta fase, donde se requiere un perfil capaz de interpretar las directrices generales del proyecto, pero con capacidad de toma de decisiones con respecto a cómo implementarlas, se estima que el rol se corresponde con el grupo B (coordinación y planificación, a un nivel menos amplio que el grupo A) del área 3 (programación).
- **Interfaces de usuario:** Se aplica el mismo criterio que en la fase anterior (grupo B, área 3).
- **Pruebas y corrección de errores:** Con la base de código ya desarrollada, se estima que la tarea de diseñar en base a la misma una batería de pruebas y corregir los errores evidenciados por ella se ajusta a un rol del grupo C (actividades de tipo técnico) del área 3.
- **Documentación:** Dada la magnitud y complejidad de la tarea, se estima que su desempeño corresponde a un perfil del grupo D (tareas administrativas con un grado de complejidad medio) del área 4.

Por otro lado, no se debe olvidar que, debido a la línea temporal acelerada del proyecto, se está trabajando un gran número de horas extraordinarias: dos horas de más cada día de lunes a viernes, y diez horas sábados y domingos. Por lo tanto, es necesario revisar el calendario para determinar las fechas en las que los diferentes perfiles realizaron horas de trabajo en fines de semana para contabilizar la totalidad de esas horas como extraordinarias, que reciben una bonificación del 75 % sobre la hora ordinaria¹.

¹El salario/hora ordinario de un perfil se obtiene dividiendo el salario anual total para dicho perfil entre 1800 (la jornada ordinaria máxima anual en horas).

Finalmente, es prudente añadir un fondo de contingencia sobre el coste total calculado. Este fondo está destinado a cubrir riesgos no planificados, como la necesidad de refactorizar un módulo debido a una modificación en los requisitos, la aparición de errores complejos durante la fase de pruebas o posibles retrasos en la toma de decisiones. Aunque el alcance del proyecto está bien definido y hay poca incertidumbre, el tiempo asignado para el proyecto es una limitación importante. Dado que casi la mitad del esfuerzo planificado (43,61 %) ya corresponde a horas extraordinarias, cualquier contingencia tendrá un impacto económico, haciendo indispensable esta reserva. Por todo ello, se considera que el fondo de contingencia debe ser un 20 % adicional del coste calculado. El resultado final se puede observar en la tabla 4.1.

Perfil	Salario/hora	Horas ordinarias	Horas extraordinarias	Total
Área 4 - A	16,732 €/h	80	60	3.095,43 €
Área 3 - B	16,169 €/h	360	270	13.461,09 €
Área 3 - C	15,331 €/h	80	70	3.104,49 €
Área 4 - D	11,158 €/h	168	132	4.451,98 €
Total				24.082,99 €
Total + 20 % de contingencia				28.899,59 €

Tabla 4.1: Desglose del coste estimado del proyecto.

4.3. Cálculo de costes sin *crunching*

Puede ser un ejercicio interesante calcular el coste, tanto económico como temporal, de cumplir los tiempos asignados a cada fase sin incurrir en horas extraordinarias (es decir, usando 40 horas semanales).

Por un lado, se puede calcular con facilidad el coste asociado a cada perfil al no tener que diferenciar entre horas ordinarias y extraordinarias:

- **Grupo A, área 4:** $140 \text{ horas} \times 16,732 \text{ €/h} = 2.342,49 \text{ €}$.
- **Grupo B, área 3:** $630 \text{ horas} \times 16,169 \text{ €/h} = 10.186,77 \text{ €}$.
- **Grupo C, área 3:** $150 \text{ horas} \times 15,331 \text{ €/h} = 2.299,62 \text{ €}$.
- **Grupo D, área 4:** $300 \text{ horas} \times 11,158 \text{ €/h} = 3.347,35 \text{ €}$.
- **Total:** $18.176,23 \text{ €}$.
- **Total + 20 % de contingencia:** $18.176,23 + 20 \% = \underline{\underline{21.811,76 \text{ €}}}$.

Por otro lado, el coste temporal se halla dividiendo el esfuerzo total estimado (1220 horas) entre una jornada estándar de 40 horas semanales, y multiplicando por 7 para obtener el número de

días naturales transcurridos desde el inicio del proyecto. Esto resulta en 30,5 semanas de trabajo que, redondeando, equivalen a 214 días naturales, lo que supone un aumento del 75.71 % del total de días requeridos y ahorrando un 24,43 % del coste inicial proyectado.

La comparación de ambas aproximaciones se puede observar en la tabla 4.2.

Métrica	Con <i>crunching</i>	Sin <i>crunching</i>
Coste económico	28.899,59 €	21.811,76 €
Coste de tiempo	122 días	214 días
Fecha de inicio	01/03/2025	01/03/2025
Fecha de finalización	30/06/2025	30/09/2025

Tabla 4.2: Comparativa de costes con y sin *crunching*.

Capítulo 5

Solución

Como ya se ha mencionado en el capítulo de planificación y costes, el desarrollo de ENS2025 se ha dividido en una fase inicial de planificación general de alto nivel, seguida de una fase compuesta de varios ciclos individuales de diseño e implementación para cada módulo del sistema, convirtiendo el proyecto en un conjunto de subproyectos.

Además, la implementación de cada módulo ha tenido un efecto de retroalimentación en el propio diseño. Encontrar un desafío al implementar un módulo, lógicamente, implica rediseñarlo; pero de ese rediseño se han desprendido en ocasiones mejoras y cambios que han tenido un efecto retroactivo en módulos ya implementados.

Por estos motivos, se ha optado por fusionar los capítulos de diseño e implementación en el capítulo presente, estructurado de tal forma que se incida sobre los distintos módulos y submódulos del sistema en forma de estructuras autocontenidas con sus apartados de diseño e implementación. Cuando proceda, se indicará también cómo se ha divergido de ENS2001, tanto en cuestiones de diseño e implementación como en corrección de errores.

5.1. Paradigma de diseño

El paradigma seguido durante el proceso de ENS2025 se cimenta en los principios SOLID descritos por Robert C. Martin [14]. La esencia de estos principios es la siguiente:

- **Principio de responsabilidad única:** Una clase debería tener una sola razón para cambiar.
- **Principio de abierto-cerrado:** Una clase debe estar abierta a su extensión y cerrada a su modificación.
- **Principio de sustitución de Liskov:** Las clases deberían ser reemplazables por instancias de sus subtipos sin alterar el funcionamiento del programa.
- **Principio de segregación de interfaces:** Muchas interfaces cliente específicas son mejores que una interfaz de propósito general.
- **Principio de inversión de la dependencia:** Se debe depender de abstracciones, no de implementaciones.

De estos cinco principios, los que más marcan de forma activa el diseño de ENS2025 son los dos primeros y el último. No hay muchas instancias de herencia en el sistema (aunque sea

extensa las pocas veces que se usa, como es el caso de las instrucciones y las excepciones); pero, cuando se da, las subclases no alteran la funcionalidad esperada de la superclase. De manera similar, las interfaces no abundan: la mayoría de componentes del sistema no interactúan con un número elevado de otros componentes; y aquellos con mayor uso están encapsulados en una fachada que actúa de punto de entrada y salida, así que las pocas interfaces que existen lo hacen directamente por necesidad. Por lo tanto, se puede decir que el principio de sustitución de Liskov y el de segregación de interfaces se cumplen principalmente de forma pasiva, por falta de oportunidades para incumplirlos.

Con respecto a los principios que sí se ha buscado cumplir activamente se puede decir lo siguiente:

- **Principio de responsabilidad única:** Se ha buscado implementar un mayor número de clases más modulares y dedicadas a tareas específicas que clases monolíticas. Con casi 150 ficheros en el proyecto, se puede considerar como cumplido.
- **Principio de abierto-cerrado:** Se ha buscado implementar métodos más cortos y específicos dentro de lo posible, así como estructuras de datos que favorezcan la expansión mediante adición en vez de modificación. Un buen ejemplo de ello es el proceso para añadir una nueva instrucción a la base de código.
- **Principio de inversión de la dependencia:** ENS2025 hace un uso extensivo del patrón fachada (Facade). Cada módulo del sistema está encapsulado en una clase envoltura que constituye una capa de abstracción de su contenido; y varias de estas fachadas pueden estar encapsuladas en otra fachada. De este modo, cada capa de abstracción solamente interactúa con las que tiene inmediatamente por encima (de la que recibe llamadas y a la que devuelve datos) y por debajo (a la que envía llamadas y de la que recibe datos). Aunque esto no constituye una aplicación automática del principio de inversión de dependencia, se considera un compromiso aceptable. Dado el tamaño y alcance del sistema, se considera innecesario implementar interfaces para cada componente de cada módulo y patrones como Service Locator para instanciar los distintos componentes.

En última instancia, los principios SOLID (y cualquier otro paradigma) son líneas de guía y no leyes inmutables. Por lo tanto, habrá ocasiones en las que su aplicación estricta no se considere conveniente, o se vea necesario llegar a un compromiso. Dichas decisiones deben ser tomadas conscientemente y estar justificadas. Incluso si la decisión no es acertada, es preferible actuar desde el desconocimiento o la inercia; una mala decisión sopesada y documentada es más fácil de revertir.

5.2. Diseño heredado de ENS2001

Debido a que el requisito principal es modelar el comportamiento de ENS2025 para que sea idéntico al de ENS2001 (correcciones de errores aparte), se pueden y deben reutilizar ciertas decisiones de diseño del sistema original para garantizar ese paralelismo. Si bien gran parte del “cómo” se va a descartar, el “qué” sigue siendo relevante y necesario.

5.2.1. Diseño del lenguaje ensamblador

Para emular el comportamiento de ENS2001, es necesario en primer lugar aceptar el mismo lenguaje e interpretarlo de la misma forma. Para ello, hay que analizar cinco aspectos clave del mismo: la estructura del código, los modos de direccionamiento, el juego de instrucciones y las directivas del ensamblador.

5.2.1.1. Estructura del código fuente

El código fuente es el texto plano que será interpretado por el sistema para su posterior ensamblado en instrucciones ejecutables. Los ficheros de código fuente siguen una estructura sencilla compuesta por una sucesión de líneas, con una estructura general que sigue el siguiente formato:

```
[[Etiqueta ':' ] Instrucción|Directiva] [';' Comentario]
```

No existe un número límite de caracteres por línea, por lo que la única limitación para la longitud de las etiquetas y los comentarios dependerá del sistema operativo o la memoria disponible.

Por otro lado, se permite la inserción de tantas líneas en blanco y/o con comentarios entre la declaración de una etiqueta y la instrucción o directiva como se desee, lo que deja una estructura general con un formato más similar al siguiente:

```
[[Etiqueta ':' ] [\n [';' Comentario]]* Instrucción|Directiva] [';' Comentario]
```

Como se puede observar, los requisitos del lenguaje son relativamente laxos, existiendo solamente estas reglas “duras”:

- La declaración de una etiqueta siempre debe ir seguida de ':', y siempre en la misma línea que la etiqueta.
- La declaración de una instrucción o directiva debe suceder por completo dentro de la misma línea.
- Una etiqueta solamente puede ir seguida de una instrucción o directiva, nunca de otra etiqueta.
- Se ignorará todo texto situado a la derecha de ';', incluido dicho carácter, hasta el final de línea.

El formato que comparten las instrucciones es el siguiente:

```
Mnemónico [Operando[',' Operando]]
```

Sin embargo, como se describirá más adelante, las operaciones están segmentadas según su número de operandos, por lo que el formato mencionado es una manera general de establecer su estructura común. En realidad, la estructura es fija para cada tipo, como se puede ver en la tabla 5.1.

N. Operandos	Estructura
0	Mnemónico
1	Mnemónico Operando
2	Mnemónico Operando ‘,’ Operando

Tabla 5.1: Estructura de una instrucción según sus operandos

La estructura de las directivas es algo más variable y, aunque varias comparten la misma estructura, ésta va inherente al mnemónico. Al haber solamente cinco de ellas, es más conveniente estudiarlas individualmente en su propio apartado.

Por otro lado, el lenguaje distingue entre mayúsculas y minúsculas. Se reconocerán los mnemónicos de instrucciones y directivas como tales siempre que estén escritos enteramente en mayúscula o en minúscula, pero nunca como una combinación de ambas. Dichas combinaciones se interpretarán como etiquetas si se encuentran en la posición adecuada para ser consideradas como tal (por ejemplo, se podría usar `Halt: HALT` como etiqueta de salto a la instrucción de fin de ejecución¹).

5.2.1.2. Modos de direccionamiento

Como se indica en la especificación de requisitos, el número de modos de direccionamiento disponibles para cada operación, así como la combinación y orden de los mismos, queda dictado por los disponibles en ENS2001 [1]. A su vez, ENS2001 escoge dicha lista de modos y combinaciones de los mismos a través de De Miguel [17]. Por imposición de requisitos, no se revisará la adición o combinación de otros modos. Los modos de direccionamiento empleados por ENS2025 son los siguientes:

- **Direccionamiento inmediato.**

En este modo de direccionamiento, el valor final del operando está explícito en el mismo. En la arquitectura de este sistema, eso significa que se emplean los 16 bits de palabra completos para definir dicho valor en complemento a 2.

Este modo de direccionamiento va indicado por un carácter de almohadilla (‘#’) precediendo al valor del operando.

Los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos `#100` y `#0x64` son equivalentes entre sí, y del mismo modo lo son entre sí `#-1`, `#65535` y `#0xFFFF`.

Por ejemplo, la instrucción `CMP #23, .R1` compara el valor 23 con el que esté almacenado en el registro R1.

- **Direccionamiento directo a registro.**

En este modo, el operando contiene un mnemónico con el nombre del registro donde se almacena el valor. Dado que el banco de registros del sistema propuesto contiene 16 registros, serán necesarios 4 bits para representar el identificador del registro escogido.

¹Por cuestionable que pueda resultar esta práctica, el propósito de ENS2025 es dar facilidades al usuario para que actúe con libertad, incluso si esa libertad implica la toma de decisiones poco adecuadas.

Este modo de direccionamiento va indicado por un punto (‘.’) precediendo al mnemónico del registro. Al igual que sucede con los mnemónicos de instrucción, se puede representar el mnemónico del registro mediante mayúsculas o minúsculas, siendo equivalentes los operandos `.r0` y `.R0`.

Por ejemplo, la instrucción `INC .R0` incrementa en 1 el valor almacenado en el registro `R0`.

■ Direccionamiento directo a memoria.

En este modo de direccionamiento, el operando expresa la dirección absoluta donde se encuentra el valor del mismo. Al igual que sucede con el direccionamiento inmediato, se requiere una palabra completa de 16 bits para expresar este direccionamiento para poder representar todas las posiciones de memoria disponibles.

Este modo de direccionamiento va indicado por un carácter de barra inclinada (‘/’) precediendo al valor del operando.

Al igual que en el direccionamiento inmediato, los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos `/100` y `/0x64` son equivalentes entre sí (representando la dirección de memoria 100), y del mismo modo lo son entre sí `/-1`, `/65535` y `/0xFFFF` (representando la dirección de memoria 65535).

Por ejemplo, la instrucción `INC /0x0500` incrementa en 1 el valor almacenado en la posición de memoria `0x0500`.

■ Direccionamiento indirecto.

En este modo de direccionamiento, el operando contiene un mnemónico con el nombre de un registro, y en dicho registro se almacena la dirección donde se encuentra el valor. Al igual que en el modo directo a registro, se necesitan 4 bits para representar el identificador del registro escogido.

Este modo de direccionamiento se representa de manera similar a la del directo a registro (el mnemónico precedido de un punto), pero insertando dicha forma de representación entre corchetes (por ejemplo, `[.R0]`).

Por ejemplo, sea `R2` con un valor almacenado de 100. La instrucción `DEC [.R2]` decrementará en 1 el valor almacenado en la posición de memoria 100 (es decir, equivale a la instrucción `DEC /100`).

■ Direccionamiento relativo a registro índice.

En este modo de direccionamiento, el operando expresa un desplazamiento sobre la dirección marcada por un puntero. El rango de desplazamiento respecto al puntero es de 8 bits en complemento a 2, lo que otorga un rango de desplazamiento de -128 a 127 posiciones de memoria respecto a la posición actual del puntero.

En esta implementación del sistema, los únicos punteros permitidos son los registros de índice `IX` e `IY` (por lo tanto, para acortar, es preferible referirse a este direccionamiento como *relativo a IX* o *relativo a IY*). A cada uno de ellos les corresponde un código de direccionamiento distinto, por lo que solamente son necesarios los 8 bits del desplazamiento para representar cada direccionamiento.

Este modo de direccionamiento se representa mediante una combinación del direccionamiento inmediato y el indirecto: el valor numérico va precedido por la almohadilla (‘#’)

y seguido por el registro en cuestión (IX o IY) en notación indirecta (precedido por un punto y entre corchetes).

Al margen de la diferencia de tamaño, se aplican las mismas reglas de interpretación numérica que con los direccionamientos inmediato y directo a memoria: las instrucciones `INC #64[.IX]` y `#0x40[.IX]` incrementan en 1 el valor almacenado en la dirección situada 40 posiciones *después* de la almacenada en IX, mientras que las instrucciones `INC #FF[.IY]`, `INC #-1[.IY]` y `INC #255[.IY]` incrementan en 1 el valor almacenado en la dirección situada 1 posición *antes* de la almacenada en IY.

Por ejemplo, sea IX con un valor almacenado de 100. La instrucción `MOVE #200, #25[.IX]` almacenará el valor 200 en la posición de memoria 125 (es decir, sería lo mismo que la instrucción `MOVE #200, /125`), y la instrucción `MOVE #200, #-10[.IX]` almacenará el valor 200 en la posición de memoria 90.

■ Direccionamiento relativo a contador de programa.

Similar al relativo a índice, este modo de direccionamiento requiere 8 bits en complemento a 2 para representar un desplazamiento de -128 a 127 posiciones respecto al puntero. La diferencia aquí es que el puntero es el valor del contador de programa *después* de ejecutar la instrucción.

Este modo de direccionamiento se representa mediante el carácter del símbolo de dólar ('\$'), seguido del valor del desplazamiento.

Al igual que en el relativo a índice, en este direccionamiento también se aplican las mismas reglas de representación numérica: las instrucciones `BR $0xFC`, `BR $-4` y `BR $251` son equivalentes.

Por ejemplo, sea la instrucción `BR $20` situada en la dirección `0x0000`. Cuando termine de ser leída, el valor del PC antes de ejecutar la instrucción será de `0x0002`, ya que es una instrucción que ocupa dos palabras. Por lo tanto, el valor final del PC una vez se ejecute la instrucción será de `0x0022`.

■ Direccionamiento implícito.

Este modo de direccionamiento no contiene información sobre la ubicación del objeto porque está predeterminado. Por ejemplo, las instrucciones aritméticas y lógicas como `ADD` o `XOR` almacenan el resultado generado en el registro A, y las instrucciones `CALL` y `RET` modifican automáticamente el contador de programa. Debido a ello, este direccionamiento es transparente para el proceso de ensamblado, ya que no requiere insertar información adicional.

Puesto que solamente existe un puntero de pila, almacenado en el registro SP, y las instrucciones `PUSH` y `POP` lo utilizan de manera automática sin requerir información de direccionamiento adicional, en este caso se puede considerar el direccionamiento relativo a pila como otra forma de direccionamiento implícito.

Sin embargo, este comportamiento no es universal. ENS2001 implementa ciertas diferencias de comportamiento para algunos modos de direccionamiento en función del tipo de instrucción, existiendo variaciones para las instrucciones de gestión de cadenas de caracteres (`INSTR` y `WRSTR`) y las instrucciones de salto. Esto dificulta la definición de un comportamiento homogéneo, e impide una correcta mantenibilidad si se pretendiera añadir nuevos modos de direccionamiento a dichas instrucciones (ya que habría que definir los comportamientos en las versiones específicas

de los métodos de extracción de datos de un operando, como se verá más adelante). Sin embargo, como la directiva principal es replicar el comportamiento de ENS2001, en este caso se ha optado por imitar esta divergencia, documentándola debidamente para dejar en manos de futuras versiones la decisión de unificar estos comportamientos o mantenerlos separados. Por lo tanto, aunque es en el apartado siguiente donde se clasifican con propiedad los tipos de instrucciones y se define la función a realizar por cada una, así como los modos de direccionamiento que acepta, es necesario profundizar en esta situación.

En primer lugar, se detalla el comportamiento de los modos de direccionamiento para las instrucciones de salto:

- **Direccionamiento directo a memoria.**

Se trata como un direccionamiento inmediato, ya que este último no se emplea en este tipo de instrucciones. Se usa el valor literal de la dirección, no el contenido de la misma. Por ejemplo, la instrucción `BR /1000` establece para el PC el valor 1000, no el valor contenido en la dirección 1000. Es decir, equivale a `MOVE #1000, .PC`, no a `MOVE /1000, .PC`.

- **Direccionamiento indirecto.**

El direccionamiento indirecto funciona de manera idéntica a la estándar.

- **Direccionamiento relativo a contador de programa.**

El direccionamiento relativo a contador de programa funciona de manera idéntica a la estándar.

Finalmente, se detalla el comportamiento de los modos de direccionamiento empleados por `INSTR` y `WRSTR` para determinar la posición donde se va a comenzar a leer o escribir la cadena:

- **Direccionamiento directo a memoria.**

Funciona igual que con las instrucciones de salto: se interpreta el operando como si fuera un direccionamiento inmediato. Es decir, `WRSTR /100` equivaldría a `WRSTR #100` si se tuviera que definir su comportamiento en base a las reglas generales de direccionamiento.

- **Direccionamiento indirecto.**

Se trata como un direccionamiento a registro. Es decir, en vez de usar el contenido del registro como dirección de memoria que apunta al primer carácter de la cadena, el valor contenido en el registro *es* la dirección del primer carácter de la cadena. Por lo tanto, `WRSTR [.R0]` equivaldría a `WRSTR .R0` si su comportamiento se tuviera que definir usando las reglas para los casos generales.

- **Direccionamiento relativo a registro índice.**

En este caso, el desplazamiento sobre el registro IX o IY apunta al primer carácter de la cadena, no al valor de la dirección de dicho carácter. Así, para un valor de 100 almacenado en IX, si la instrucción `INC #25[.IX]` equivale a `INC /125`, bajo el comportamiento general la instrucción `WRSTR #25[.IX]` tal y como funciona actualmente equivaldría en realidad a `WRSTR #125`.

5.2.1.3. Juego de instrucciones

En esta sección se detalla el juego de instrucciones completo de ENS2025. Consta de 37 instrucciones, agrupadas conceptualmente en distintos tipos según su funcionalidad:

- **Instrucciones de transferencia de datos:** MOVE, PUSH, POP.
- **Instrucciones aritméticas:** ADD, SUB, MUL, DIV, MOD, INC, DEC, NEG.
- **Instrucciones de comparación:** CMP.
- **Instrucciones lógicas:** AND, OR, XOR, NOT.
- **Instrucciones de salto (bifurcación):** BR, BZ, BNZ, BP, BN, BV, BNV, BC, BNC, BE, BO.
- **Instrucciones de manejo de subrutinas:** CALL, RET.
- **Instrucciones de entrada/salida:** INCHAR, ININT, INSTR, WRCHAR, WRINT, WRSTR.
- **Otras instrucciones:** NOP, HALT.

Las instrucciones aritméticas y de comparación, además, son las únicas que modifican los biestables de estado (a excepción del biestable H, modificado exclusivamente por la instrucción HALT). Los biestables y sus condiciones de activación se pueden ver en la tabla 5.2.

Biestable	Condiciones de activación
Z	El resultado de la operación es 0.
C	El resultado es superior a 65535.
S	El bit más significativo del resultado es 1.
V	El resultado es inferior a -32768 o superior a 32767.
P	El número de bits a 1 del resultado es impar.
H	Instrucción HALT leída.

Tabla 5.2: Tabla de condiciones de activación de los biestables de estado.

A continuación, se lista el juego de instrucciones completo junto con sus propiedades principales:

- **Instrucción NOP:**
 - **Descripción:** No operación.
 - **Número de operandos:** 0.
 - **Direccionamientos:** Ninguno.
 - **Comportamiento:** Ninguno.
 - **Modifica biestables:** No.
- **Instrucción HALT:**
 - **Descripción:** Detener ejecución.

- **Número de operandos:** 0.
- **Direccionamientos:** Ninguno.
- **Comportamiento:** Activa el biestable H.
- **Modifica biestables:** No (el resto de biestables mantienen su último valor, no se reinician).

■ **Instrucción MOVE:**

- **Descripción:** Copia de valores entre operandos.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Copia el contenido de OP1 en OP2.
- **Modifica biestables:** No

■ **Instrucción PUSH:**

- **Descripción:** Inserción en pila.
- **Número de operandos:** 1.
- **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
- **Comportamiento:** Almacena el valor contenido en el operador en la dirección de memoria apuntada por el puntero de pila (SP) y luego desplaza el puntero de pila una posición en la dirección de crecimiento de la misma.
- **Modifica biestables:** No.

■ **Instrucción POP:**

- **Descripción:** Extracción de la pila.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Desplaza el puntero de pila (SP) una posición en dirección opuesta al crecimiento de la misma y luego almacena el valor contenido en la nueva dirección apuntada por el SP en el operando.
- **Modifica biestables:** No.

■ **Instrucción ADD:**

- **Descripción:** Suma de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **Comportamiento:** Suma los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí
- **Instrucción SUB:**
 - **Descripción:** Resta de enteros.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Resta los contenidos de OP1 y OP2 (OP1-OP2) y almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** Sí.
- **Instrucción MUL:**
 - **Descripción:** Multiplicación de enteros.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Multiplica los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** Sí.
- **Instrucción DIV:**
 - **Descripción:** División de enteros.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Divide OP1 entre OP2 y almacena el resultado en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.
 - **Modifica biestables:** Sí.
- **Instrucción MOD:**
 - **Descripción:** Resto de la división de enteros.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **Comportamiento:** Resta los contenidos de OP1 y OP2 y almacena el resto de dicha división en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.
- **Modifica biestables:** Sí.

■ Instrucción INC:

- **Descripción:** Incremento unitario.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Incrementa en 1 el contenido de OP1.
- **Modifica biestables:** Sí.

■ Instrucción DEC:

- **Descripción:** Decremento unitario.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Decrementa en 1 el contenido de OP1.
- **Modifica biestables:** Sí.

■ Instrucción NEG:

- **Descripción:** Cambio de signo.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Cambia de signo el operando mediante la operación $0-OP1$.
- **Modifica biestables:** Sí.

■ Instrucción CMP:

- **Descripción:** Comparación.
- **Número de operandos:** 2
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** El mismo que en la instrucción SUB, pero sin almacenar el resultado de la operación en ningún sitio.
- **Modifica biestables:** Sí.

■ Instrucción AND:

- **Descripción:** AND lógico.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Efectúa la operación lógica AND entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** No.
- **Instrucción OR:**
 - **Descripción:** OR lógico.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Efectúa la operación lógica OR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si el bit de uno o ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** No.
- **Instrucción XOR:**
 - **Descripción:** XOR lógico.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Efectúa la operación lógica XOR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen distintos valores (uno 0 y el otro 1). Almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** No.
- **Instrucción NOT:**
 - **Descripción:** Negación lógica.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Invierte bit a bit el valor del operando (cambia los 0 por 1 y viceversa).
 - **Modifica biestables:** No.
- **Instrucción BR:**
 - **Descripción:** Bifurcación incondicional.

- **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 .
 - **Modifica biestables:** No.
- **Instrucción BZ:**
- **Descripción:** Bifurcación si $Z=1$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 1.
 - **Modifica biestables:** No.
- **Instrucción BNZ:**
- **Descripción:** Bifurcación si $Z=0$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 0.
 - **Modifica biestables:** No.
- **Instrucción BP:**
- **Descripción:** Bifurcación si $S=0$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 0.
 - **Modifica biestables:** No.
- **Instrucción BN:**
- **Descripción:** Bifurcación si $S=1$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 1.
 - **Modifica biestables:** No.
- **Instrucción BV:**
- **Descripción:** Bifurcación si $V=1$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 1.

- **Modifica biestables:** No.

■ **Instrucción BNV:**

- **Descripción:** Bifurcación si $V=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 0.
- **Modifica biestables:** No.

■ **Instrucción BC:**

- **Descripción:** Bifurcación si $C=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 1.
- **Modifica biestables:** No.

■ **Instrucción BNC:**

- **Descripción:** Bifurcación si $C=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 0.
- **Modifica biestables:** No.

■ **Instrucción BE:**

- **Descripción:** Bifurcación si $P=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 0.
- **Modifica biestables:** No.

■ **Instrucción B0:**

- **Descripción:** Bifurcación si $P=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 1.

- **Modifica biestables:** No.

■ **Instrucción CALL:**

- **Descripción:** Llamada a subrutina.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Almacena el valor actual del PC en la dirección de memoria apuntada por el SP, desplaza el SP una posición en la dirección de crecimiento de la pila y carga en el PC el valor almacenado en el operando (es decir, **CALL <OP>** equivale a **PUSH .PC** seguido de **BR <OP>**, siendo <OP> un operando genérico con direccionamiento permitido).
- **Modifica biestables:** No.

■ **Instrucción RET:**

- **Descripción:** Retorno de subrutina.
- **Número de operandos:** 0.
- **Direccionamientos:** Ninguno.
- **Comportamiento:** Desplaza el SP una posición en la dirección opuesta al crecimiento de la pila y carga en el PC el valor almacenado en la nueva dirección del SP (es decir, **RET** equivale a **POP .PC**)
- **Modifica biestables:** No.

■ **Instrucción INCHAR:**

- **Descripción:** Lectura de carácter.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Lee un carácter de la consola y lo codifica en ASCII, dejando a cero los 8 bits superiores de la palabra de 16 bits y almacenando el carácter leído en el operando.
- **Modifica biestables:** No.

■ **Instrucción ININT:**

- **Descripción:** Lectura de entero.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Lee un entero de la consola. Si la entrada no puede ser convertida en un entero (formato incorrecto, valor fuera de rango, entrada vacía), se considerará que se ha introducido un 0. Los enteros se pueden escribir en formato decimal (con y sin signo) y hexadecimal (sin signo y precedido por "0x").
- **Modifica biestables:** No.

■ **Instrucción INSTR:**

- **Descripción:** Lectura de cadena.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, indirecto, relativo.
 - **Comportamiento:** Lee una cadena de caracteres de la consola y la almacena en posiciones consecutivas de memoria a partir de la dirección indicada por el operando, acabada con el carácter ‘\0’. No se comprueba previamente si hay espacio para almacenar la cadena, por lo que se produce un error si se sobrepasa el límite superior de la memoria.
 - **Modifica biestables:** No.
- **Instrucción WRCHAR:**
- **Descripción:** Escritura de carácter.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
 - **Comportamiento:** Escribe en la consola el carácter cuyo valor ASCII se corresponde con los 8 bits inferiores del valor del operando.
 - **Modifica biestables:** No.
- **Instrucción WRINT:**
- **Descripción:** Escritura de entero.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
 - **Comportamiento:** Escribe en la consola el valor del operando. El formato de representación numérica (hexadecimal, numeral con/sin signo) dependerá de la configuración de ese parámetro en el sistema.
 - **Modifica biestables:** No.
- **Instrucción WRSTR:**
- **Descripción:** Escritura de cadena.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, indirecto, relativo.
 - **Comportamiento:** Escribe en la consola una cadena de caracteres, que estará almacenada en memoria a partir de la dirección indicada por el operando. Se escribirán caracteres hasta que se llegue al carácter ‘\0’. Por tanto, puede ocurrir que durante este proceso se genere un error si no se encuentra el carácter de fin de cadena antes de llegar al límite superior de la memoria.
 - **Modifica biestables:** No.

5.2.1.4. Etiquetas

Las etiquetas son mnemónicos definidos por el usuario que sirven como nombres simbólicos. Comienzan por un carácter alfabético, seguido o no de cualquier combinación de caracteres alfanuméricos y de barra baja ('_'). Solamente pueden ser definidos precediendo a una instrucción o directiva, y siempre irán finalizados por un carácter de dos puntos (':') a modo de separador (es decir, no es parte del mnemónico como tal). Toman el valor que tenga el puntero de escritura cuando son declaradas, antes del procesamiento de una instrucción o la ejecución de una directiva, salvo si se trata de la directiva EQU, en cuyo caso toman como valor el resultado de la expresión aritmética asociada a la directiva.

Las etiquetas pueden usarse como sustitución de cualquier literal entero que forme parte de un operando, como por ejemplo WRCHAR #VALOR, BR \$ini, WRSTR /cadena o ADD #BASE, #DESP[.IX]. La etiqueta se sustituye por su valor pertinente durante el ensamblado.

5.2.1.5. Directivas

Las directivas del ensamblador son órdenes dirigidas al programa ensamblador con el fin de organizar el proceso de ensamblado; pero no forman parte del código y, por tanto, no se codifican en lenguaje máquina para su posterior simulación.

Las directivas definidas para este sistema (así como sus comportamientos), son las siguientes:

- **Directiva ORG:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará la dirección de memoria a la que se moverá el puntero de escritura.
- **Directiva EQU:** Tiene como argumento una expresión aritmética. Asigna el resultado de dicha expresión a la etiqueta asociada. Si no se indica ninguna etiqueta, la expresión será evaluada igualmente, pero no se asignará su resultado a ningún elemento.
- **Directiva END:** No tiene argumentos. Señala el final del código fuente. No es obligatoria, ya que el proceso de ensamblado finaliza sin errores igualmente una vez que se termina de leer el fichero de código. El ensamblador ignorará todo lo que haya escrito a partir de esta directiva.
- **Directiva RES:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará cuántas posiciones de memoria se reservan (sin inicializar su valor). En caso de ir precedida de una etiqueta, ésta indicará la primera posición reservada.
- **Directiva DATA:** Tiene como argumento una sucesión tan larga como desee el usuario (siempre que la memoria lo permita) de valores numéricos y cadenas de caracteres (que deberán ir entrecomilladas) separados por comas. Las cadenas se almacenarán en memoria indicando su final mediante el carácter '\0' como añadido final. Pueden estar compuestas de cualquier carácter imprimible, y pueden contener los caracteres especiales '\0', '\n' y '\t', así como las propias comillas y la barra invertida usada como escapador de caracteres, siempre que vayan ambos precedidos de otra barra invertida ('\" y '\\').

5.2.2. Diseño del sistema

Del mismo modo que se heredan las decisiones sobre qué modos de direccionamiento, instrucciones y directivas se implementarán, también se heredan (tanto por conveniencia como por retrocompatibilidad) decisiones de diseño como los identificadores asociados a los distintos componentes de una instrucción, así como la arquitectura de instrucciones empleada.

5.2.2.1. Identificadores

Cada elemento de la arquitectura, sea un modo de direccionamiento, un registro, una instrucción o un biestable tiene un identificador. Estos identificadores permiten codificar el elemento en cuestión dentro de las instrucciones. A continuación, se describen mediante tablas los códigos identificadores de cada elemento, así como representación en bits y el tamaño requerido.

- **Modos de direccionamiento:** Representados en la tabla 5.3.

Código	Bits	Modo de direccionamiento	Ancho de representación
0	000	Sin operandos	N/A
1	001	Inmediato	16 bits
2	010	Registro	4 bits
3	011	Memoria	16 bits
4	100	Indirecto	4 bits
5	101	Relativo a IX	8 bits
6	110	Relativo a IY	8 bits
7	111	Relativo a PC	8 bits

Tabla 5.3: Tabla de códigos de direccionamiento.

- **Registros:** Representados en la tabla 5.4

Registro	Código	Bits
R0	0	0000
R1	1	0001
R2	2	0010
R3	3	0011
R4	4	0100
R5	5	0101
R6	6	0110
R7	7	0111

Registro	Código	Bits
R8	8	1000
R9	9	1001
A	10	1010
SR	11	1011
IX	12	1100
IY	13	1101
SP	14	1110
PC	15	1111

Tabla 5.4: Tabla de códigos de registro.

- **Instrucciones:** Representadas en la tabla 5.5

Instrucción	Código	Bits	Instrucción	Código	Bits	Instrucción	Código	Bits
NOP	0	000000	CMP	13	001101	BNC	26	011010
HALT	1	000001	AND	14	001110	BE	27	011011
MOVE	2	000010	OR	15	001111	BO	28	011100
PUSH	3	000011	XOR	16	010000	CALL	29	011101
POP	4	000100	NOT	17	010001	RET	30	011110
ADD	5	000101	BR	18	010010	INCHAR	31	011111
SUB	6	000110	BZ	19	010011	ININT	32	100000
MUL	7	000111	BNZ	20	010100	INSTR	33	100001
DIV	8	001000	BP	21	010101	WRCHAR	34	100010
MOD	9	001001	BN	22	010110	WRINT	35	100011
INC	10	001010	BV	23	010111	WRSTR	36	100100
DEC	11	001011	BNV	24	011000			
NEG	12	001100	BC	25	011001			

Tabla 5.5: Tabla de códigos de instrucción.

- **Biestables:** Aunque los biestables no usan un identificador como tal porque no se codifican dentro de una instrucción, cada uno ocupa una posición dentro de los LSB del registro SR, como se muestra en la tabla 5.6.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	H	S	P	V	C	Z

Tabla 5.6: Tabla de posicionamiento de biestables en el registro SR.

5.2.2.2. Formatos de instrucción

Con los diferentes identificadores ya definidos, es momento de definir las posibles estructuras que tendrán las instrucciones. Éstas estarán compuestas por bloques con tamaño comprendido entre una y tres palabras, dependiendo del número de operandos y los modos de direccionamiento de cada uno, si procede.

En primer lugar, se define la primera palabra utilizando el código de instrucción seguido de los códigos de direccionamiento de los operandos, como se ve en la tabla 5.7. Se puede apreciar que, aunque solamente son necesarios 6 bits para codificar una instrucción, se asignan 10 debido al espacio sobrante, lo que permite la adición de nuevas instrucciones en el futuro. De hecho, se podría destinar un bit más a cada modo de direccionamiento para añadir hasta ocho direccionamientos nuevos, y aún habría espacio para definir casi el cuádruple de la cantidad existente de instrucciones.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Código de operación										Dir. OP1		Dir. OP2			

Tabla 5.7: Estructura de la primera palabra.

El siguiente punto a tener en cuenta es el número de operandos. En el caso de las instrucciones sin operandos, por ejemplo, los modos de direccionamiento son 0 en ambos casos, dejando los seis bits menos significativos (tres de cada operando) a cero y no se añadirán más palabras.

Para codificar un operando, los tres bits menos significativos valen cero, y la segunda palabra contendrá el operando en función del modo de direccionamiento. Para el caso de direccionamientos de 16 bits, como son el inmediato y el directo a memoria, el valor del operando ocupa la palabra completa (tabla 5.8). Por otro lado, para los direccionamientos de 8 bits (relativos a índices y a PC), se utiliza la mitad de la palabra correspondiente a su número de operando (tabla 5.9). Finalmente, para los direccionamientos de 4 bits (directo a registro e indirecto), los datos se codifican en los 4 bits menos significativos de su mitad de palabra correspondiente (tabla 5.10). Además, existe la posibilidad de que dos operandos compartan una misma palabra si ninguno de los dos requiere una palabra completa (tabla 5.11).

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor del operando															

Tabla 5.8: Operando de 16 bits.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	Valor OP2							

Tabla 5.9: Operandos de 8 bits.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	-	-	Valor OP2			

Tabla 5.10: Operandos de 4 bits.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	Valor OP2			
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	Valor OP2			

Tabla 5.11: Algunos ejemplos de una palabra con dos operandos.

5.2.2.3. Las zonas de código y pila

Al cargar en la memoria del sistema un nuevo programa ensamblado, se calculan las direcciones de inicio y fin de estas dos zonas. La dirección inicial de la zona de código queda definida como la dirección de menor valor en la que se haya codificado una instrucción, insertado un dato o reservado memoria, al igual que la dirección más alta en la que sucede esto queda definida como la dirección final. Por otra parte, la zona inicial de la pila será asignada, en función de su

dirección de crecimiento, a la menor o mayor posición del bloque de memoria vacío más grande disponible, mientras que la posición final cambiará dinámicamente con el valor del SP.

El propósito de la existencia de ambas zonas es facilitar la depuración mediante la activación de los parámetros de comprobación de invasión. Estos parámetros (desactivados por defecto) permiten que se genere un error si en algún momento el PC entra dentro de la zona de pila o si el SP entra dentro de la zona de código, deteniendo la ejecución.

5.3. Diseño e implementación de ENS2025

A los elementos de diseño ya existentes se ha añadido otro parámetro de configuración: la comprobación de desbordamiento/subdesbordamiento (términos intercambiables por *overflow/underflow*) de la pila. Estos parámetros opcionales permiten detener la ejecución si el puntero de la pila excede la memoria asignada por el sistema (por ejemplo, una pila con crecimiento positivo desbordando más allá de la posición final de memoria y empezando a ocupar las posiciones iniciales) o su dirección inicial (por ejemplo, realizar una extracción de una pila vacía).

Además, aunque ya se detallarán más detenidamente en sus secciones respectivas, las interfaces de usuario han recibido funcionalidades adicionales. Entre ellas, destacan las siguientes:

- La interfaz de línea de comandos ahora acepta una serie de parámetros durante su invocación desde la consola del sistema, flexibilizando el sistema existente de carga de ficheros.
- La interfaz gráfica de usuario ahora también muestra en el visor de memoria las posiciones de los índices IX e IY, e incorpora botones para centrar la vista en cualquiera de ellos (y del PC y el SP).

La estructura general de ENS2025, como ya se ha mencionado, está compuesta de varias capas de abstracción. Al nivel más elevado se tiene la interacción de la aplicación de la interfaz de nivel de usuario, ya sea por línea de comandos o mediante interfaz gráfica, con el componente principal de ENS2025, la unidad de control (más concretamente, su clase fachada `ControlUnit`), así como su implementación particular de la interfaz entrada/salida `IOInterface`. De manera muy simplificada, el diagrama de la figura 5.1 representa esta interacción.

Por otro lado, la unidad de control se divide en otros cuatro componentes principales: el ensamblador, el desensamblador, el simulador y la arquitectura de memoria, relacionados de la forma mostrada en la figura 5.2.

Es importante señalar que el término “módulo” es usado de manera bastante amplia en las secciones posteriores para referirse no solamente a los componentes en sí, sino a las unidades conceptuales con sentido propio dentro del sistema (por ejemplo, los módulos de excepciones, tipos enumerados o instrucciones). Estas unidades conceptuales, en la práctica, constituyen principalmente atributos de los propios componentes, así como argumentos y retornos de métodos de los mismos. Por tanto, estos “módulos” se omiten en los diagramas de alto nivel para facilitar la comprensión de la relación entre componentes, eliminando relaciones que solamente aumentan la complejidad de dichos diagramas. Cada módulo contiene las tablas UML de

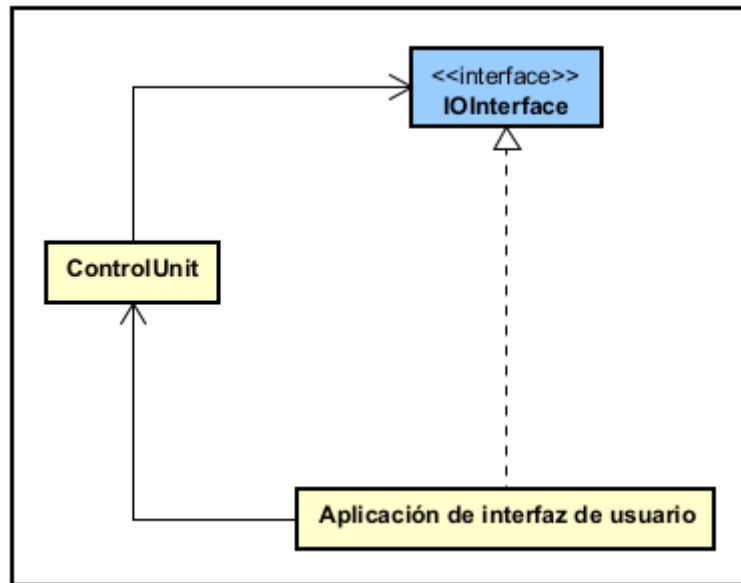


Figura 5.1: Interacción de una interfaz de usuario genérica con la unidad de control.

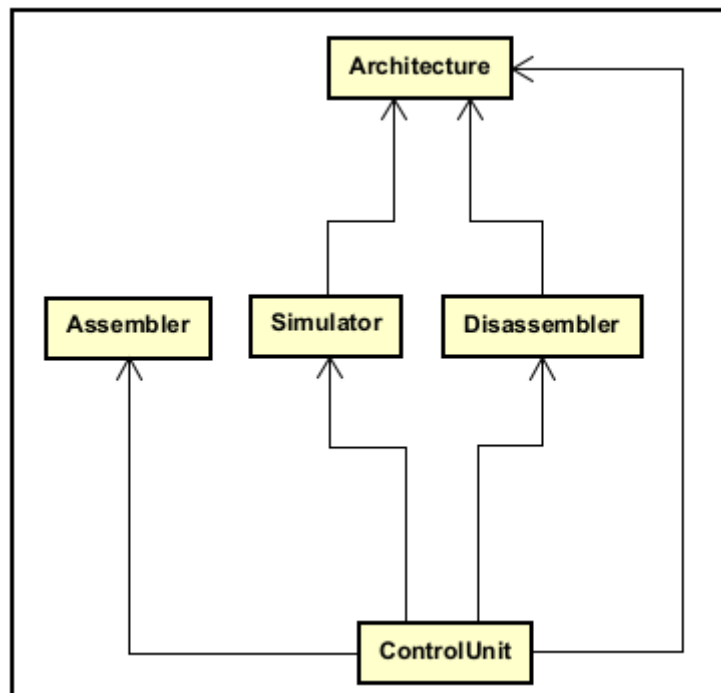


Figura 5.2: Vista interna simplificada de la unidad de control.

las clases que lo forman, facilitando así información sobre las relaciones entre clases con una granularidad más fina.

El esquema completo de ENS2025 es imposible de reproducir de forma legible en un documento de este formato. Por ello, se ha optado por una aproximación mixta: además de los diagramas mostrados en esta sección, se procederá a describir de forma más detallada cada módulo en el que proceda siguiendo un esquema *bottom-up* en las secciones siguientes; es decir, detallando primero los algoritmos y tareas que lleva a cabo y los elementos “atómicos” que lo conforman, para finalmente definir la clase envoltura del módulo si la hubiera y establecer las diferencias

clave de diseño e implementación respecto a ENS2001 en ese aspecto si procede. De esta forma, cada módulo se puede ver como una sección autocontenida de diseño e implementación.

En los módulos de las interfaces de usuario, la aproximación cambia: al estar totalmente desacoplados de la lógica del sistema, interesa más conocer sus funciones y cómo las realizan que elaborar una lista detallada de sus entidades componentes (esto es especialmente cierto para la GUI, donde la mayoría de clases son contenedores de otros contenedores o botones a los que se ha asociado una función especial para reducir boilerplate), sin perjuicio de detallar algunos elementos clave de las mismas (como la implementación que hace cada una de la interfaz de entrada/salida para poner al usuario en comunicación con la lógica del sistema, o el adaptador principal de la GUI).

5.3.1. Casos de uso

En esta sección se muestran los posibles casos de uso de ENS2025, y se detallarán algunos de ellos mediante diagramas de secuencia de forma ilustrativa.

Los posibles casos de uso vienen detallados por el diagrama de la figura 5.3. Nótese que no hay relaciones de inclusión ni extensión, ya que el propósito del mismo es establecer simplemente los casos de alto nivel iniciados por el usuario. De esta forma, se mantiene un enfoque en los requisitos funcionales de alto nivel, priorizando la claridad y la visión general del alcance del sistema.

Un caso de uso que ilustra a la perfección el uso extensivo del patrón fachada en ENS2025 es el de la lectura de un registro, ilustrado por la figura 5.4. Al igual que este caso, el resto casos sobre consultas y modificaciones funcionan de manera análoga.

Esto se puede observar también en el diagrama de secuencia de la figura 5.5, relativo al caso de uso de la ejecución de una instrucción (es decir, la ejecución paso a paso). Si bien se omiten comprobaciones internas dentro de un mismo módulo, control de errores y el bucle de obtener palabras adicionales y analizarlas (ya que es un procedimiento parecido al de la primera palabra), el concepto subyacente perdura. Se puede observar cómo tanto el simulador como el decodificador obtienen los datos que necesitan del banco de registros y de la memoria solicitándolos a la arquitectura, y cómo el decodificador encarga a los códigos (mediante una serie de llamadas simplificadas en el pseudométodo “DisecionarPalabra”) la obtención de los datos de instrucción almacenados en una palabra (o conjunto de ellas) y la validación semántica de las mismas, devolviendo al simulador la instrucción que se ejecuta.

5.3.2. Módulo del tipo de datos

Dados los requisitos de la arquitectura (enteros de 16 bits con signo), podría parecer lógico asumir que basta con asignar un dato de tipo `short` para la tarea de representar las palabras usadas por el simulador. Sin embargo, Java presenta una serie de desafíos que hacen esa decisión desaconsejable:

- **Imposibilidad para definir tipos primitivo:** A diferencia de los lenguajes derivados

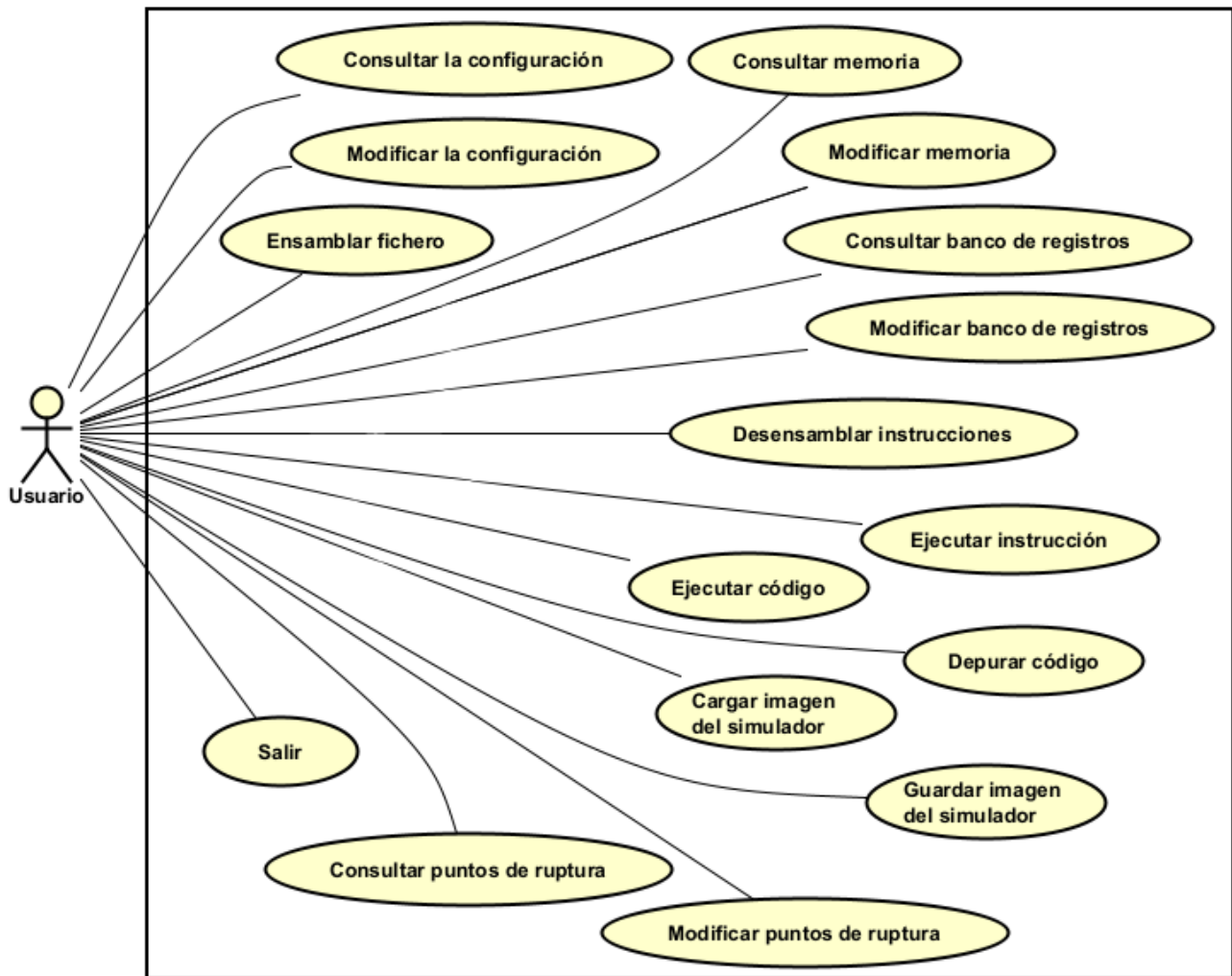


Figura 5.3: Diagrama de casos de uso.

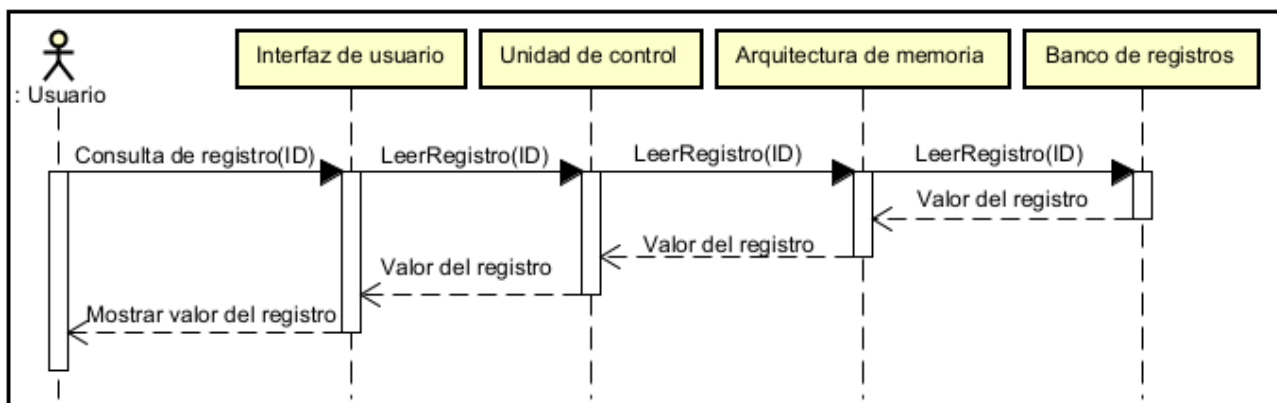


Figura 5.4: Diagrama de secuencia del caso de uso "Consultar banco de registros".

de C, Java no posee mecanismos directos de *aliasing* de primitivos como `typedef`, que permitirían definir un tipo sustitutivo fácilmente intercambiable en el futuro por otros de distinto tamaño, dotando al sistema de una alta mantenibilidad y facilidad de refactorización. Por lo tanto, para mantener estas cualidades se hace necesaria la creación de una clase envoltura del tipo de dato en cuestión.

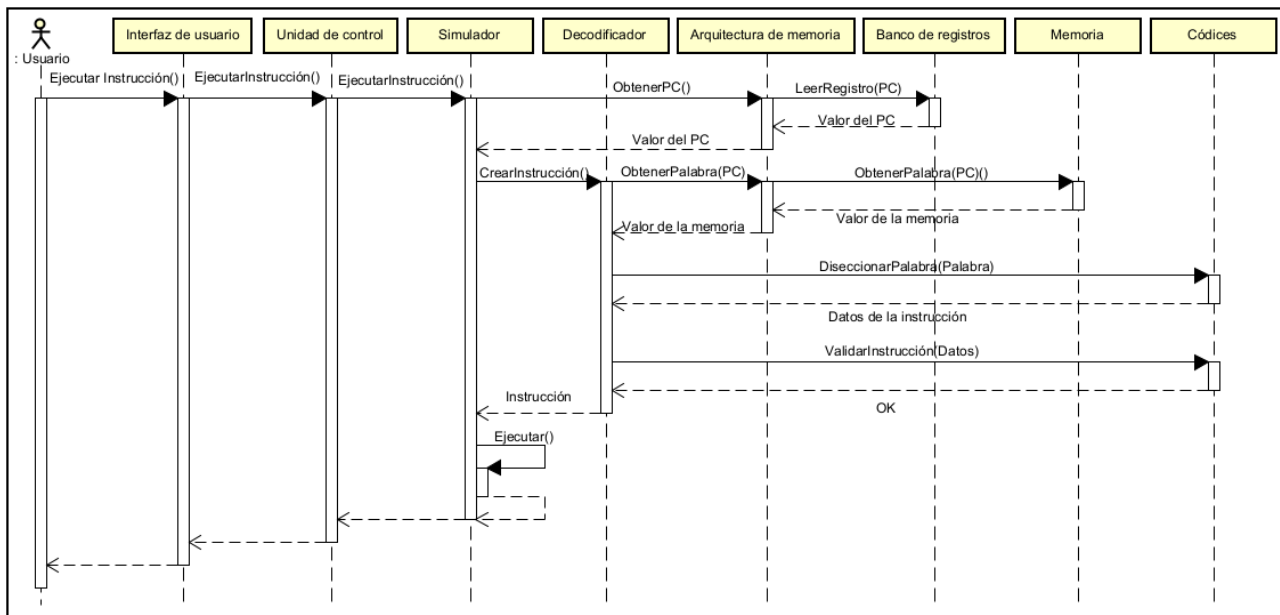


Figura 5.5: Diagrama de secuencia del caso de uso “Ejecutar instrucción”.

- **Imposibilidad de sobrecargar operadores:** Java no permite sobrecargar ni definir operadores. Por lo tanto, es necesario crear métodos en la clase que envuelve el tipo de dato para cada operación aritmética, lógica o de cualquier otro tipo de la que se le pretenda dotar.
- **Inexistencia de operadores para el tipo short:** Java no ha dotado de operadores a los tipos enteros de tamaño inferior a 32 bits, por lo que no se puede operar con ellos. Tampoco se acepta `short` como identificador válido de índice para una estructura de array, ni como argumento del método `get()` de una estructura que implemente la interfaz `List`. Debido a esto, el tamaño más pequeño que puede tener el tipo de dato asignado al atributo de valor de la clase será 32 bits, coincidente con un tipo `int`.
- **Inexistencia de enteros sin signo:** Java no contempla la existencia de enteros sin signo en sus librerías estándar. Afortunadamente, desde Java 8, sí existen métodos estáticos en las clases `Integer` y `Long` para ejecutar operaciones aritméticas sin signo, así como para generar un `String` de representación numérica sin signo. Sin embargo, el valor real contenido se considera con signo. Aunque este factor no afecta de manera directa al desarrollo del sistema debido a la restricción anterior, es conveniente tenerlo en cuenta (por ejemplo: incluso si se pudiera utilizar un tipo `short` como índice de un array, la imposición de que dicho `short` tenga signo imposibilitaría usar el rango completo de valores del `short` por ser la mitad de signo negativo).

A estas limitaciones inherentes al lenguaje hay que añadir otras restricciones autoimpuestas:

- Se debe generar una clase lo más orientada posible a la mantenibilidad y a la facilidad de refactorización. Si en un futuro se quisiera modificar el tamaño de la arquitectura (usando los 32 bits completos del `int`, por ejemplo), la cantidad de modificaciones realizadas a la clase debe ser mínima.

- Se debe buscar un comportamiento análogo al del tipo de dato con el tamaño deseado. A efectos prácticos, la clase tiene que ser vista desde fuera como si se tratase de la envoltura de un entero de 16 bits funcional, no de la envoltura de un entero de 32 que contiene uno de 16.
- Para garantizar la integridad de los datos al mover referencias instancias de la clase a lo largo y ancho del sistema, deberá permitir la generación de copias inmutables y mutables.

Con estas restricciones en mente, surgen la clase **RWData** (*Readable/Writable Data*) y su interfaz **RData** (*Readable Data*).

5.3.2.1. La clase RWData

El núcleo de **RWData** es un **int** (entero de 32 bits) que almacena el valor de palabra, junto con una serie de constantes que determinan los valores máximos y mínimos representables (tanto con signo como sin signo), así como una serie de máscaras que aplicar a su contenido para mantener la consistencia interna.

Se detallan a continuación los usos de estas constantes:

- **DATA_SIZE** y **HALF_SIZE**: Especifican el tamaño total de palabra y el tamaño de media palabra (en este caso, 16 y 8, respectivamente).
- **HALF_WORD_MAX_UNSIGNED**: Especifica el valor máximo sin signo almacenable en media palabra (en este caso, 255).
- **MAX_UNSIGNED_VALUE**: Especifica el valor máximo sin signo almacenable en la palabra completa (en este caso, 65535).
- **MIN_VALUE** y **MAX_VALUE**: Especifican los valores mínimo y máximo con signo, respectivamente, almacenables en la palabra completa (en este caso, -32768 y 32767, respectivamente).
- **MAX_OFFSET_POSITIVE**: Especifica el valor positivo máximo que se puede almacenar cuando se usa como valor de desplazamiento (en este caso, 127).
- **OFFSET_SIGNCHANGE_VALUE**: Especifica el valor empleado para cambiar el signo cuando se usa como valor de desplazamiento (en este caso, 256).
- **OFFSET_MASK**: Máscara para aislar el valor de desplazamiento cuando se usa para ese fin (en este caso, 0xFF).
- **MASK**: Máscara general para mantener la integridad del tamaño de palabra (en este caso, 0xFFFF).

Esta última constante se aplica tras cada operación con el valor interno para mantenerlo ajustado al tamaño de palabra empleado. De esta manera, siempre quedan los 16 bits más significativos a 0 y se evitan comportamientos no definidos. Si se modificase el simulador para usar

un tamaño de palabra de 32 bits, por ejemplo, bastaría con modificar el tamaño de la máscara a `0xFFFFFFFF`, y todo seguiría funcionando exactamente igual.

Esta máscara se usa en conjunto con un método privado interno, `int fitSize(Number n)`, que recibe cualquier tipo primitivo de dato numérico (o clase envoltura) que herede de la clase `Number`, llama al método `intValue()` de dicha interfaz, le aplica un AND bit a bit de la constante `MASK` y devuelve el resultado. La máscara se aplica también al final de todos los métodos relacionados con operaciones aritméticas. Con ello, se consigue de manera efectiva mantener una representación interna constante de 16 bits sin signo, quedando la interpretación del contenido como valor con signo o sin signo al contexto en el que se encuentre, y haciendo la mayoría de métodos internos agnósticos al tamaño de palabra que maneja realmente la clase.

El resto de métodos de `RWData` se describen a continuación:

- **Constructores:** Además de mediante un `Number` y de otros objetos que implementen `RData`, la clase `RWData` se puede construir también usando un `String`, que procesa para detectar automáticamente si es decimal o hexadecimal, y un carácter, del que obtiene su `codePoint`, o valor en la tabla de codificación.
- **Operador de asignación:** Existen tres versiones del método `assign()`. Una de ellas acepta como argumento un `String`, otra un `Number` y otra un `RData`. Su comportamiento es el mismo: asignar al valor interno el valor extraído del argumento.
- **Operadores aritméticos y de bit a bit:** Cada uno de estos métodos tiene dos versiones, una que admite como argumento un `Number` y otra que admite como argumento un `RData`. En todos ellos se aplica la máscara `MASK` al resultado final. Los operadores aritméticos (a excepción de la división y el módulo) se realizan sobre el `int` subyacente. Dado que los `int` de Java son siempre con signo, el resultado de la operación se ajusta posteriormente al rango de 16 bits mediante la aplicación de la máscara `MASK`, simulando el comportamiento de un entero sin signo de 16 bits para el rango de valores positivo. La interpretación de este valor como positivo o negativo queda a cargo del contexto de uso. No es necesario detallar qué hacen, pues tienen nombre autoexplicativo.
- **Métodos `divide(...)` y `modulo(...)`:** Conviene mencionar aparte estos dos métodos. La mayoría de operaciones aritméticas son agnósticas al signo, no así para las operaciones que impliquen un cociente. Por ejemplo, no es lo mismo dividir -2 (`0xFFFE`) entre -1 (`0xFFFF`) con signo, cuyo resultado es 1 (resultado esperado) que hacerlo sin signo, cuyo resultado es 0. Por ello, se comprueba internamente si el valor sin signo excede el valor máximo con signo (en cuyo caso es negativo) y se realiza una extensión temporal de signo para los cálculos, revirtiéndola al terminar.
- **Método `urShift(...)`:** Desplazamiento lógico (sin signo) a la derecha. Desplaza hacia la derecha el valor interno tantos bits como indique el valor del argumento, introduciendo ceros por la izquierda.
- **Método `rShift(...)`:** Desplazamiento aritmético (con signo) a la derecha. Desplaza hacia la derecha el valor interno tantos bits como indique el valor del argumento, introduciendo el valor de su bit más significativo. Para ello, se realiza antes una comprobación de extensión de signo.

- **Método `lShift(...)`:** Desplazamiento lógico a la izquierda. Desplaza hacia la izquierda el valor interno tantos bits como indique el valor del argumento, introduciendo ceros por la derecha.
- **Operadores de comparación:** Al igual que los operadores aritméticos y de bit, estos métodos también tienen una versión que acepta un `Number` y otra que acepta un `RData` salvo que se especifique lo contrario. También poseen nombres autoexplicativos.
- **Método `getValue()`:** Devuelve el valor interno.
- **Método `getMutableCopy()`:** Devuelve una nueva instancia de `RWData` con valor interno igual al de la instancia desde la que se ha invocado el método.
- **Método `getImmutableCopy()`:** Devuelve una nueva instancia de `RWData` con valor interno igual al de la instancia desde la que se ha invocado el método, enmascarada con la interfaz `RData` para hacerla inmutable.
- **Método `getChar()`:** Devuelve un dato tipo `char` cuyo valor es igual al valor interno.
- **Métodos `writeToStream()` y `readFromStream()`:** Escriben en y leen de, respectivamente, un flujo de datos binario el valor a almacenar internamente, escrito en *Big Endian*.
- **Métodos de representación:** Generan una cadena con el valor interno en distintas bases de representación: decimal con y sin signo, hexadecimal, hexadecimal “corto” (solamente dos cifras)...

Como se ha mencionado anteriormente, gracias al método `fitSize()` la mayoría de operaciones son agnósticas al tipo de dato empleado para el valor interno. Sin embargo, el método `getValue()` devuelve explícitamente un `int`, y los métodos `parseString()` y `bitCount()`, así como los métodos de representación, emplean métodos estáticos de las clases `Integer`, `Short` y `Byte`, por lo que una hipotética refactorización que afectase al tipo de dato interno requeriría modificar estos métodos junto con las constantes y el método `fitSize()`. Sin embargo, se considera cumplido el objetivo de reducir al mínimo la cantidad de cambios necesarios a la hora de refactorizar.

5.3.2.2. La interfaz `RData`

Esta interfaz no es más que una envoltura a la clase `RWData` que contiene únicamente los métodos no “destructivos” (aquellos que no modifican el contenido). Esencialmente, esto engloba todos los métodos públicos menos los constructores, los operadores de asignación y los operadores aritméticos y de bit a bit. También se incluye el método de escritura en fichero, pero no el de lectura.

La finalidad de esta interfaz es ejercer de contrapeso al hecho de que se mueven referencias a instancias por los distintos métodos del programa. Para evitar una llamada accidental a un método de asignación que pudiera modificar el valor en origen, se transfiere una interfaz inmutable de la instancia. En el destino se evalúa el valor contenido mediante los métodos disponibles o, si se requiere una copia local, se genera mediante un constructor que use de argumento la instancia recibida o se invoca su método `getMutableCopy()`.

De manera similar, para evitar que una modificación del valor en el origen pueda ocasionar una cascada de cambios inesperados en otros puntos de la aplicación, idealmente las estructuras que utilizan `RWData` internamente (el banco de registros, la memoria...) devolverán un `RData` mediante `getImmutableCopy()`, garantizando así la integridad en el destino.

5.3.2.3. Diferencias respecto a ENS2001

ENS2001 asignaba a su clase `centero16b` los valores de los distintos biestables (salvo el H), y calculaba internamente los valores de los mismos como resultado de las distintas operaciones aritmético-lógicas, implementando métodos para obtener dichos valores. La forma de obtener estos valores (en especial los de acarreo y desbordamiento) dependía estrictamente de usar un tipo de dato mayor: se produce desbordamiento si el resultado de la operación con signo escapa al rango de valores con signo permitido, y se produce carry si el resultado de la operación sin signo excede el valor máximo sin signo permitido.

En este caso, se ha optado por no seguir esta línea de diseño. La decisión se cimienta en los siguientes motivos:

- No todas las operaciones internas de `RWData` generan cambios en los biestables, por lo que se trata de una responsabilidad parcial (a veces se tiene, a veces no). A mayores, no siempre que se llama a un método relacionado con una operación aritmética son necesarios los biestables (cuando se aplica un incremento de un índice o un desplazamiento, por ejemplo).
- Se depende de un tipo de datos de tamaño mayor al que se use de forma inherente.
- Se vulnera el principio de responsabilidad única. `RWData` no es más que una envoltura de datos y la implementación de sus operadores y métodos de utilidad, utilizable en este proyecto o en cualquier otro con necesidad de un tipo de dato de características similares. La funcionalidad de llevar un registro de biestables interno es demasiado específica (generar valores concretos para un módulo específico) y no encaja con lo que se espera de la clase (operaciones de carácter general sobre un tipo de dato).

5.3.3. Módulo de tipos enumerados

Si hubiera que resumir la lógica del sistema en una palabra, esa palabra sería “identificadores”. El proceso de interpretación, ensamblado, codificación/decodificación y ejecución se resume en convertir información en identificadores, validar si conjuntos de distintos tipos de identificadores son compatibles entre sí, extraer o almacenar información en lugares determinados por identificadores...

Por ello, los identificadores de instrucción, de registro, de biestable y de modo de direccionamiento conforman uno de los pilares principales del sistema. El rigor y la profundidad con la que sean definidos determinará en gran medida aspectos clave como la flexibilidad y la mantenibilidad del código, así que limitarse a diseñarlos e implementarlos como simples colecciones de valores, en vez de aprovechar los recursos de Java al respecto, sería un error.

En esta sección se detallarán los cuatro tipos enumerados antes mencionados, que tienen influencia en gran parte del código. Otros tipos enumerados con influencia más localizada (como el identificador de los comandos de la aplicación de consola) o con vínculos conceptuales más cercanos a una clase concreta (como el modo de representación numérica y la clase **Settings**) se analizarán en profundidad más adelante.

Para no alargar innecesariamente la sección listando todos los valores de cada tipo (ya que en la sección sobre el diseño heredado del sistema ya se detallan estos valores), se omitirán en la descripción, y se proporcionará un ejemplo en cada uno para facilitar la comprensión de su estructura.

5.3.3.1. El tipo enumerado Flag

Este tipo enumerado, representado en la tabla 5.12, identifica los distintos biestables almacenados en el banco de registros.

«enumeration» Flag
- name:String
- mask:RData
- Flag(i:Number, name:String)
+ toString():String

Tabla 5.12: Tipo enumerado Flag.

Cada valor del tipo lleva asociados un número en binario que representa la posición que ocupa en el SR (el atributo **mask**), y un **String** con el nombre del biestable (el atributo **name**, que contiene la letra representativa del biestable).

Sea, por ejemplo, el valor **CARRY**:

CARRY(0b00_000010, "C")

El primer valor indica que ocupa el segundo bit menos significativo en el registro SR, y la C es su letra identificadora.

No es necesario incidir en los métodos, pues sus nombres son autoexplicativos.

5.3.3.2. El tipo enumerado RegisterID

Este tipo enumerado, representado en la tabla 5.13, identifica los distintos registros almacenados en el banco de registros.

Cada valor del tipo lleva asociados un **String** con el nombre del registro (el atributo **name**) y un **int** con el valor numérico del registro (el atributo **id**).

Sea, por ejemplo, el valor **R9**:

«enumeration»
RegisterID
- name:String - id:int - ID_MAP:Map<Integer, RegisterID>{final} - NAME_MAP:Map<String, RegisterID>{final}
- RegisterID(String:name, int:id) + toString():String + getId():int + getFromId(int:id):RegisterID + getFromName(String:name):RegisterID

Tabla 5.13: Tipo enumerado RegisterID.

R9("R9", 9)

El primer valor es el nombre que usará en las interacciones con el usuario (el nombre que se le muestra y el que el usuario empleará para buscar el registro), y el segundo es el código numérico usado en la codificación y decodificación de instrucciones.

`RegisterID` posee también dos mapas donde se empareja de forma inversa cada constante (es decir, cada valor posible de `RegisterID`) con su `String` de nombre y su identificador numérico, respectivamente, y que utilizan los métodos `getFromName()` y `getFromId()`.

5.3.3.3. El tipo enumerado AddressingMode

Este tipo enumerado, representado en la tabla 5.14, identifica los distintos modos de direccionamiento disponibles para un operando.

«enumeration»
AddressingMode
- id:int - mode:String - ID_MAP:Map<Integer, RegisterID>{final}
- AddressingMode(id:int, mode:String) + getId():int + toString():String + getFromId(int:id):AddressingMode

Tabla 5.14: Tipo enumerado AddressingMode.

Cada valor del tipo lleva asociados un `int` con el valor numérico del modo (el atributo `id`) y un `String` con el nombre del modo de direccionamiento (el atributo `mode`).

Sea, por ejemplo, el valor `NONE`:

`NONE(0, "SIN OPERANDO")`

El primer valor es el código del modo de direccionamiento empleado en la codificación y decodificación de instrucciones, y el segundo es el **String** mostrado en las interacciones con el usuario (como la construcción del **String** de la instrucción desensamblada y los mensajes de error de direccionamiento incompatible con instrucción durante el ensamblado).

AddressingMode posee también un mapa donde se empareja de forma inversa cada constante (es decir, cada valor posible de **AddressingMode**) con su identificador numérico, y que utiliza el método **getFromId()**.

5.3.3.4. El tipo enumerado **InstructionType**

Este tipo, representado en la tabla 5.15, identifica las distintas instrucciones del sistema, y contiene grupos de identificación para determinar si se trata de instrucciones sin operandos, de un operando, de un operando con entrada/salida o de dos operandos. Además, asocia cada identificador de instrucción con el constructor de clase de esa instrucción concreta, facilitando enormemente la implementación del proceso de decodificación y construcción de instrucciones ejecutables para el simulador.

«enumeration»	
InstructionType	
-	name:String
-	id:int
-	constructor:Function<InstructionData, ? extends Instruction>
-	ID_MAP:Map<Integer, InstructionType>{final}
-	ID_MAP:Map<String, InstructionType>{final}
-	NO_OP_SUBSET:Set<InstructionType>{final}
-	ONE_OP_SUBSET:Set<InstructionType>{final}
-	ONE_OP_IO_SUBSET:Set<InstructionType>{final}
-	TWO_OP_SUBSET:Set<InstructionType>{final}
-	InstructionType(name:String, id:int, constructor:Function<InstructionData, ? extends Instruction>)
+	getId():int
+	toString():String
+	getConstructor():Function<InstructionData, ? extends Instruction>
+	isNoOpInstruction(it:InstructionType):boolean
+	isOneOpInstruction(it:InstructionType):boolean
+	isOneOpIOInstruction(it:InstructionType):boolean
+	isTwoOpInstruction(it:InstructionType):boolean
+	getFromId(id:int):InstructionType
+	getFromName(name:String):InstructionType

Tabla 5.15: Tipo enumerado **InstructionType**.

Cada valor del tipo lleva asociados un **String** con el nombre de la instrucción (atributo **name**), un **int** con el valor numérico de la instrucción y, lo más importante, un método que implementa la interfaz funcional **Function**, recibiendo un objeto **InstructionData** (sobre el que se profundizará en el apartado sobre el módulo de instrucciones) y devolviendo una instancia de un objeto que herede de la clase abstracta **Instruction**.

Esencialmente, esto significa que cada constante del tipo enumerado que representa a una instrucción queda emparejada con su constructor, permitiendo crear nuevas instancias simplemente llamando al método `getConstructor()` asociado a ese valor del enumerado. La aplicación más directa de esto es que el decodificador, una vez recabe toda la información que debe transferirse a la instrucción, simplemente obtiene el constructor, lo ejecuta con el `InstructionData` pertinente, y genera una nueva instrucción lista para enviarse al simulador y ser ejecutada.

Sea, por ejemplo, el valor `PUSH`:

```
PUSH("PUSH", 3, PUSH::new)
```

El primer valor es el `String` utilizado en las interacciones con el usuario (como mostrar las instrucciones desensambladas o mensajes de error relacionados con las instrucciones) y en el proceso de ensamblado (más concretamente, la interpretación del código, pudiendo asociar directamente el mnemónico leído con su instrucción correspondiente), el segundo valor es el código de la instrucción (empleado en los procesos de codificación y decodificación) y el tercero es la referencia al método asociado que cumple con la interfaz funcional `Function` (en este caso, el constructor de clase, siendo igual para todos los valores del enumerado).

`InstructionType` contiene además dos mapas donde se empareja de forma inversa cada constante con su `String` de nombre y su identificador numérico, respectivamente, y que utilizan los métodos `getFromName()` y `getFromId()`.

Esto, combinado con el mapa de constructores, simplifica el código en varias partes del proyecto. Aunque se detallará en profundidad en cada sección correspondiente, merece la pena mencionar algunos ejemplos para subrayar la importancia de esta estructura, que reduce a una simple llamada operaciones como:

- La comprobación de la existencia del mnemónico de instrucción leído en la interpretación de ficheros por parte del módulo ensamblador.
- La codificación de la instrucción a lenguaje máquina por el módulo ensamblador.
- La decodificación de la instrucción leída y validación del código como existente.
- El desensamblado de una instrucción en una cadena de texto legible por el usuario.

A mayores, esta estructura (junto a otras que se verán más adelante, como las contenidas en el módulo de códigos) garantiza una mantenibilidad casi absoluta de este aspecto del sistema, facilitando enormemente la adición de nuevas instrucciones, ya que el código para operar con ellas en todos los ámbitos del sistema no depende de condicionales ni clasificadores.

Por último, `InstructionType` cuenta con varios conjuntos de pertenencia que agrupan las diferentes constantes del tipo enumerado en función de la categoría de instrucción y operando a la que pertenecen. Los atributos `XXX_OP_SUBSET`, y sus respectivos métodos `isXXXOpInstruction()`, contribuyen a simplificar el trabajo de interpretación del fichero de código para su ensamblado, ya que ayudan a determinar si, tras leer un mnemónico de instrucción válido, se debe esperar un final de línea, un operando o dos.

5.3.3.5. Diferencias respecto a ENS2001

C++ no posee tipos enumerados tan flexibles y versátiles como los de Java, por lo que no existe nada parecido en la versión anterior del sistema. Si bien es cierto que se pueden implementar clases que simulen el comportamiento, hacerlo se convierte en una decisión de diseño consciente, que requiere analizar la utilidad de las mismas frente a otras posibilidades, así como de un diseño e implementación meticulosos. En cambio, la facilidad de uso y las funcionalidades extendidas de los tipos enumerados en Java no los convierten en una opción a considerar para esta parte del código, sino prácticamente en una necesidad.

5.3.4. Módulo de la interfaz de entrada/salida

Un punto crucial a la hora de mantener desacoplados los *frontends* de la lógica interna es la cuestión de cómo resolver la entrada y salida de datos. Cuando un método solicita una entrada de datos al usuario, o cuando se genera un mensaje de error o información, debe poder hacerlo de forma que no necesite conocer el funcionamiento de las diferentes aplicaciones que se conectan a la lógica interna, ni de cuál de ellas en cuestión se trata.

Es cierto que JavaFX tiene la posibilidad de “secuestrar” flujos de datos (incluidos `System.out`, `System.err` y `System.in`) a un elemento con capacidad de mostrar texto. Esto permitiría despreocuparse de este apartado y permitir que los métodos generen su salida y entrada de datos por las vías estándar. Sin embargo, es una aproximación mucho más robusta designar una interfaz con los métodos adecuados para establecer la comunicación entrada/salida y dejar la implementación a cargo de cada *frontend*. Si entonces se decide redirigir los flujos de datos a un componente gráfico en vez de implementar otro sistema, será una decisión deliberada sin perjuicio de otras posibilidades, y no la única posible frente a la alternativa de tener que refactorizar cientos de llamadas a `System.out`. Por este motivo existe la interfaz `IOInterface`.

5.3.4.1. La interfaz `IOInterface`

Aunque es bastante sencilla, su importancia es primordial para establecer una comunicación entre la lógica interna y las distintas aplicaciones construidas en torno a ella, desplazando hacia estas últimas la responsabilidad de implementar los métodos para mostrar o introducir los datos según las especificaciones concretas de cada una. En los apartados dedicados a cada aplicación se profundizará en cómo se ha implementado cada método de la interfaz. La interfaz queda definida en la tabla 5.16.

Aunque los métodos tienen nombres autoexplicativos, sí conviene diferenciar entre sus propósitos: `printLog()` y `printlnLog()` deberían implementarse para manejar los mensajes de error y sistema, y el resto para la entrada y salida estándar.

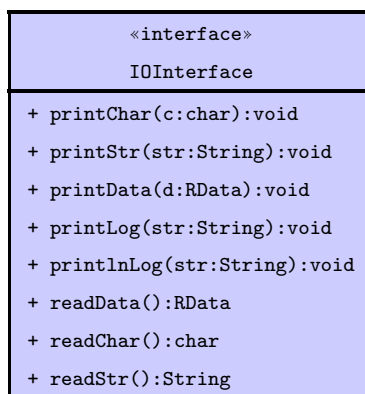


Tabla 5.16: Interfaz IOInterface.

5.3.5. Módulo de configuración

El comportamiento de ENS2025 se ve influido por parámetros de configuración definidos por el usuario y sujetos a cambio según la coyuntura. Desde necesidades más orientadas a la estética o la comodidad a necesidades puramente funcionales u orientadas a apoyar la depuración del código más allá de los puntos de ruptura, estos parámetros afectan varios aspectos del sistema. Por tanto, es necesario centralizarlos de manera que todos sean fácilmente accesibles, tanto para consulta de sus valores en un momento dado como para su modificación. Para ello se implementa el módulo de configuración.

Este módulo, centrado en la clase **Settings**, centraliza estos parámetros, incluyendo tipos enumerados como **StackGrowth** y **NumericalRepresentation** para aquellos que no se adecúen a representaciones binarias **true/false** o valores numéricos, como es el caso del crecimiento de pila y la representación numérica. Sus valores se almacenan de forma persistente entre ejecuciones mediante un fichero, cuyo formato de contenido se detalla en el Apéndice A de esta memoria.

5.3.5.1. El tipo enumerado **StackGrowth**

Este tipo enumerado, definido en la tabla 5.17, representa las dos posibles direcciones del crecimiento de la pila: positiva cuando crece de direcciones menores a mayores, y negativa cuando sucede al revés (también se usan los términos “pila creciente” y “pila decreciente” de manera intercambiable). Aunque dicha dirección de crecimiento es un conjunto binario, un enumerado proporciona mayor claridad semántica y seguridad de tipo que un **boolean**. Un valor **POSITIVE** es más legible y menos probable a inducir a error que un **true**. Combinado con la flexibilidad de los tipos enumerados en Java, esto lo convierte en una elección clara.

5.3.5.2. El tipo enumerado **NumericalRepresentation**

Este tipo enumerado, definido en la tabla 5.18, representa las tres posibles representaciones que pueden tomar los valores enteros del sistema: decimal con signo, decimal sin signo y hexadecimal (sin signo). Esta forma de representación solamente se tiene en cuenta cuando los valores se

«enumeration» StackGrowth
- name:String - description:String - NAME_MAP:Map<String, StackGrowth> - DESCRIPTION_MAP:Map<String, StackGrowth>
- StackGrowth(name:String, description:String) + toString():String + getDescription():String + getFromName(name:String):StackGrowth + getFromDescription(description:String):StackGrowth

Tabla 5.17: Tipo enumerado StackGrowth.

muestran al usuario; la representación interna se mantiene homogénea.

«enumeration» NumericalRepresentation
- name:String - description:String - NAME_MAP:Map<String, StackGrowth> - DESCRIPTION_MAP:Map<String, StackGrowth>
- NumericalRepresentation(name:String, description:String) + toString():String + getDescription():String + getFromName(name:String):NumericalRepresentation + getFromDescription(description:String):NumericalRepresentation

Tabla 5.18: Tipo enumerado NumericalRepresentation.

5.3.5.3. La clase Settings

Esta clase, definida en la tabla 5.19, almacena internamente los parámetros de configuración del sistema, gestiona su acceso y modificación, su almacenamiento persistente y los valores por defecto, almacenados en forma de constantes.

Dada la naturaleza de esta clase en tanto que almacén de parámetros de configuración, cada uno con su constante de valor por defecto, atributo y métodos get/set (y, dado que los nombres aportan suficiente información sobre a qué hace referencia cada uno), tiene más sentido detallar en qué consiste cada parámetro que una descripción secuencial típica en la que primero se detallan los atributos y después los métodos. La lista de parámetros de configuración del sistema se compone de los siguientes elementos:

- **Comprobación de invasión de pila:** Indica si, cada vez que cambie el valor del PC, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de pila.

Settings	
- <u>STACKGROWTH_DEFAULT:StackGrowth</u> = <u>NEGATIVE{final}</u> - <u>NUMREP_DEFAULT:NumericalRepresentation</u> = <u>DEC {final}</u> - <u>CODEINVASION_DEFAULT:boolean = false {final}</u> - <u>STACKINVASION_DEFAULT:boolean = false {final}</u> - <u>RESETREGISTERS_DEFAULT:boolean = true {final}</u> - <u>CODESTART_DEFAULT:RData = new RData(0) {final}</u> - <u>CODEEND_DEFAULT:RData = new RData(0) {final}</u> - <u>STACKSTART_DEFAULT:RData</u> = <u>new RData(RData.MAX_UNSIGNED_VALUE) {final}</u> - <u>OVERFLOW_DEFAULT:boolean = false {final}</u> - <u>UNDERFLOW_DEFAULT:boolean = false {final}</u> - <u>DEFAULTFILENAME:String = "ens2025.cfg" {final}</u> - <u>stackGrowth:StackGrowth</u> - <u>checkPCInvadingStack:boolean</u> - <u>checkSPInvadingCode:boolean</u> - <u>resetRegistersWithNewRun:boolean</u> - <u>checkOverflow:boolean</u> - <u>checkUnderflow:boolean</u> - <u>numRep:NumericalRepresentation</u> - <u>codeStart:RData</u> - <u>codeEnd:RData</u> - <u>stackStart:RData</u>	+ <u>setStackGrowth(sg:StackGrowth):void</u> + <u>setCheckPCInvadingStack(set:boolean):void</u> + <u>setCheckSPInvadingCode(set:boolean):void</u> + <u>setCheckOverflow(set:boolean):void</u> + <u>setCheckUnderflow(set:boolean):void</u> + <u>setResetRegistersWithNewRun(set:boolean):void</u> + <u>setNumericalRepresentation</u> <u>(nr:NumericalRepresentation):void</u> + <u>getStackGrowth():StackGrowth</u> + <u>getCheckPCInvadingStack():boolean</u> + <u>getCheckSPInvadingCode():boolean</u> + <u>checkOverflow():boolean</u> + <u>getCheckUnderflow():boolean</u> + <u>getResetRegistersWithNewRun():boolean</u> + <u>getNumericalRepresentation():NumericalRepresentation</u> + <u>setCodeStart(d:RData):void</u> + <u>setCodeEnd(d:RData):void</u> + <u>setStackStart(d:RData):void</u> + <u>getCodeStart():RData</u> + <u>getCodeEnd():RData</u> + <u>getStackStart():RData</u> + <u>loadSettingsSequence():void</u> + <u>loadSettingsFromFile(name:String):void</u> <u>throws IOException</u> - <u>parseTrueFalseOption(optString:String,</u> <u>defaultValue:boolean):boolean</u> + <u>saveSettingsToFile():void</u> - <u>stringifyTrueFalseOption(value:boolean):String</u>
+ Settings() + setToDefault():void + recalcStackStart():void	

Tabla 5.19: Clase Settings (dividida por motivos de espacio).

- **Comprobación de invasión de código:** Indica si, al modificar el SP, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de código.
- **Comprobación de reinicio de registros:** Indica si, al usar alguno de los métodos de la clase `Simulator` para ejecutar código, se debe comprobar si el biestable H está en alto a fin de reiniciar primero el banco de registros.
- **Comprobación de overflow:** Indica si, al incrementar la pila, se debe comprobar que el SP no pase de 65535 a 0 (si es creciente) o de 65535 a 0 (si es decreciente).
- **Comprobación de underflow:** Indica si, al modificar el SP, se debe comprobar que no termine en una dirección anterior a la de inicio de pila (es decir, más baja si la pila crece positivamente y más alta si la pila crece negativamente).
- **Inicio y fin del código:** Indica las direcciones de memoria donde empieza y termina la zona de código.
- **Inicio de la pila:** Indica la dirección de memoria donde empieza la zona de pila.
- **Crecimiento de pila:** Funcionalidad ya descrita en el apartado del tipo enumerado `StackGrowth`.
- **Representación numérica:** Funcionalidad ya descrita en el apartado del tipo enumerado `NumericalRepresentation`.

Por otro lado, existen otros métodos aparte de los accesorios y modificadores de los atributos, y es conveniente conocer algunos en concreto:

- **Método `recalcStackStart()`:** Recalcula el valor de la dirección de inicio de la zona de pila. Para ello, sigue el siguiente algoritmo:
 - Se calculan y almacenan temporalmente los tamaños de las zonas de memoria no marcadas como pertenecientes a la zona de código antes y después de dicha zona. El tamaño de la zona anterior al código es igual al valor de la dirección de inicio del código, y el de la posterior es igual al tamaño total de la memoria menos la dirección de fin del código.
 - Se comprueba la dirección de crecimiento de la pila.
 - Si es negativo, la dirección inicial de pila será la dirección de mayor valor de la zona de mayor tamaño. Se prioriza la zona anterior al código en caso de igual tamaño que la posterior, para minimizar el riesgo de invasión de zona de código por el SP.
 - Si es positivo, la dirección inicial de pila será la dirección de menor valor de la zona de mayor tamaño. Se prioriza la zona posterior al código en caso de igual tamaño por el mismo motivo que en el caso anterior.
- **Método `loadSettingsSequence()`:** Intenta abrir y cargar el fichero de configuración cuyo nombre se especifica en la constante `DEFAULTFILENAME`, con valor `ens2025.cfg`. Para ello, llama al método `loadSettingsFromFile()` usando dicha constante como argumento. Si dicho método fracasa (por ejemplo, porque no existe el fichero), llama a `setDefault()` e informa de que se carga la configuración por defecto.

- **Método `loadSettingsFromFile()`:** Carga un fichero de texto con el nombre suministrado e inicializa los diferentes atributos con los valores leídos. Para ello, divide el fichero en líneas, extrae de cada una el identificador del parámetro que se quiere inicializar y el valor que se quiere asignar. Dicho valor se interpreta del texto mediante los métodos `getFromName()` de `StackGrowth` y `NumericalRepresentation`, o bien, si se trata de un parámetro representado por `boolean`, llamando al método `parseTrueFalseOption()`. Antes de iniciar el proceso de lectura se inicia la configuración a los valores por defecto, por si al fichero le faltase alguna opción, y el proceso de lectura en sí sigue una aproximación benévola: se permiten parámetros repetidos (en cuyo caso se queda con el último valor leído) y las líneas erróneas se descartan sin interrumpir el proceso.

5.3.5.4. Diferencias respecto a ENS2001

El diseño e implementación de ENS2001 en este aspecto es muy similar, con la salvedad de que responsabiliza al banco de registros de la gestión de las direcciones de inicio/fin del código y la pila en vez de asignar esas propiedades y funciones a la configuración. Además, la configuración contiene referencias a todos los componentes y funciona como una especie de controlador omnipresente (antipatrón de objeto todopoderoso o *God object*) cuya referencia está presente como atributo en prácticamente todas las clases del sistema, recibiendo peticiones de acceso a componentes y devolviendo directamente la referencia a los mismos, lo que lo convierte en un punto de acoplamiento central.

ENS2025 define el módulo de configuración exclusivamente como el almacenamiento de los parámetros del sistema, como su nombre indica. Solamente forma parte de los atributos de una clase cuando ésta necesita consultar la configuración del sistema para ciertas operaciones, o cuando uno de sus cometidos es modificar dicha configuración. Por otro lado, se han añadido las comprobaciones de overflow y underflow como ayuda adicional, tanto para depurar el código como para ayudar a generar código más robusto que no incurra en comportamientos inesperados. También se ha añadido el modo de representación numérica decimal sin signo, antes inexistente.

5.3.6. Módulo de excepciones

Las excepciones personalizadas son una herramienta valiosa de cualquier software mínimamente complejo. Su mera existencia ayuda a establecer causas concretas de errores que las excepciones de propósito general no cubren, aportando claridad semántica sobre la naturaleza del error que las origina mediante el nombre con el que son definidas.

En el caso de ENS2025, además, son necesarias porque la información adicional que deben transportar no siempre está disponible en el contexto en el que se produce. Por ejemplo, un error de división entre cero durante la ejecución del simulador no dispone in situ de la dirección en la que se encuentra la instrucción que lo ha originado, ya que se lanza desde un método de `RWData`, que es quien realiza la operación aritmética. Como solución a esta necesidad se crea la clase abstracta `CustomException`, de la que heredan las distintas excepciones personalizadas del sistema.

5.3.6.1. La clase `CustomException`

La primera decisión que se debe tomar es si `CustomException` hereda de `Exception` (excepción controlada) o de `RuntimeException` (excepción no controlada). Robert C. Martin se decanta por el uso de estas últimas de forma tajante: «El precio de las excepciones controladas es una violación del principio abierto/cerrado; un cambio en un nivel inferior del software puede forzar cambios de firma en varios niveles superiores» [18].

Además, las excepciones estándar de Java carecen de métodos públicos para enriquecer la información de error a posteriori (por ejemplo, añadiendo detalles contextuales en capas superiores de la aplicación). Esta limitación, sumada a la necesidad de un formato de mensaje flexible y de transportar la ubicación del error, refuerza la necesidad de implementar una jerarquía de excepciones personalizadas. Por lo tanto, estas características deben también estar presentes en `CustomException`, definida en la tabla 5.20.

<i>CustomException</i>
line:Integer
message:String
+ CustomException()
+ CustomException(message:String)
+ getMessage():String
+ setLine(d:RData):void
+ setLine(i:Integer):void
+ setMessage(message:String):void

Tabla 5.20: Clase `CustomException`.

Los atributos `line` y `message` son, respectivamente, la línea/dirección en la que sucede el error y el mensaje con información adicional. Su visibilidad protegida es necesaria para que las clases que heredan de `CustomException` puedan usarlos para construir su mensaje.

Las clases que heredan de `CustomException` pueden ser instanciadas tanto con mensaje como sin él. El método `getMessage()`, originario de las excepciones estándar, es redeclarado como método abstracto, forzando a cada excepción heredada a implementar un método que construya el mensaje en base a la información contextual disponible.

Los métodos `setLine()` y `setMessage()` permiten añadir la línea y el mensaje en un momento diferente al de la instanciación. De esta manera, una excepción puede ser lanzada en un nivel bajo, capturada momentáneamente en un nivel intermedio donde se le añade la información pertinente y propagada hasta alcanzar el nivel donde se gestiona adecuadamente.

La estructura del método `getMessage()` que debe implementar cada clase que herede de `CustomException` sigue a grandes rasgos plantilla de la figura 5.6.

Esta estructura flexibiliza la construcción y garantiza que no sea estrictamente obligatorio definir línea y mensaje para crear un mensaje de causa con sentido.

La lista de excepciones que heredan de `CustomException` es la siguiente:

- Clase `BreakpointEncounteredException`: Generada cuando se simula el código en modo

```
1      @Override
2      public String getMessage() {
3          StringBuilder sb = new StringBuilder("");
4
5          if (line != null)
6              sb.append("Error en línea " + line + ": ");
7          else
8              sb.append("Error: ");
9
10         if (message != null)
11             sb.append(message);
12         else
13             sb.append("Mensaje de error generico para esta excepcion");
14
15         return sb.toString();
16     }
17
```

Figura 5.6: Plantilla de implementación del método `getMessage()` en una subclase de `CustomException`. El mensaje de error genérico mencionado varía según la excepción.

depuración y se encuentra un punto de ruptura. En esta excepción concreta, el mensaje generado no habla de error, sino de que se ha detenido la ejecución por el punto de ruptura. Pese a no representar un error de ejecución como tal, su existencia como excepción se justifica mediante su rol como mecanismo de control de flujo excepcional y transversal. Se genera como consecuencia de una interrupción deliberada y esperada del flujo de simulación para facilitar la depuración, y su mensaje refleja su finalidad como evento de control, no de error.

- **Clase `DataParsingException`:** Generada cuando ha sucedido un error en la conversión de un `String` a un valor numérico.
- **Clase `DivideByZeroException`:** Generada como consecuencia de intentar dividir un número entre cero.
- **Clase `EmptyFileException`:** Generada cuando el fichero buscado está vacío.
- **Clase `ExpressionIntegrityException`:** Generada durante el procesamiento de un `String` que contiene una expresión aritmética que no cumple con las reglas gramaticales especificadas.
- **Clase `FileManagementException`:** Generada por errores varios relacionados con ficheros (fichero no encontrado, error en la lectura...).
- **Clase `InstructionArchitectureException`:** Generada durante el proceso de decodificación por una incongruencia en los datos de una instrucción (código de registro o de instrucción no válidos).
- **Clase `InstructionNotImplementedException`:** Generada durante el proceso de decodificación de una instrucción ante la imposibilidad de transformar el lenguaje máquina en una instrucción conocida.

- **Clase `InvadingPointerException`:** Generada cuando el PC invade la zona de pila y/o el SP invade la de código mientras los parámetros pertinentes para comprobarlo están activos en la configuración del sistema.
- **Clase `InvalidCharacterException`:** Generada cuando se encuentra un carácter inesperado en el proceso de interpretación del fichero de código.
- **Clase `InvalidRegisterException`:** Generada durante el proceso de ensamblado cuando el mnemónico de un registro no es válido.
- **Clase `InvalidValueException`:** Generada cuando se encuentra un problema en la gestión de enteros (por ejemplo, cuando un valor no se encuentra dentro del rango permitido).
- **Clase `LabelException`:** Generada por errores relacionados con etiquetas (etiqueta duplicada, inexistente...).
- **Clase `MemoryIndexOutOfBoundsException`:** Generada cuando se excede el tamaño máximo de la memoria (intentar avanzar el PC más allá de la última posición, intentar insertar un String demasiado grande...).
- **Clase `StackException`:** Generada cuando se produce un stack overflow/underflow mientras los parámetros pertinentes para comprobarlo están activos en la configuración del sistema.

5.3.7. Módulo de códigos

Uno de los desafíos en el diseño y la implementación del simulador a la hora de mantener cierto nivel de abstracción es la necesidad inherente que tienen varios componentes del mismo de conocer el tamaño interno de una palabra, así como de conocer la distribución de la información almacenada en ella para su correcta codificación y decodificación.

Por otro lado, la necesidad de verificación de la información (combinación correcta de tipo de instrucción, número de operandos, modos de direccionamiento y rangos de valor) a codificar y decodificar surge en varios puntos del código, por lo que su centralización es obligatoria si se pretende mantener un alto estándar de accesibilidad y mantenibilidad.

En respuesta a esto surge el módulo de códigos, compuesto por las clases `ISACodex` (*Instruction Set Architecture Codex*) y `VerificationCodex`.

Estas clases son clases estáticas de utilidad, lo que quiere decir que no poseen un constructor definido y todos sus atributos y métodos son estáticos.

5.3.7.1. La clase `ISACodex`

Esta clase, definida en la tabla 5.21, contiene un conjunto de constantes que definen las distintas máscaras y valores de desplazamiento que hay que aplicar a una palabra para aislar los diversos identificadores que pueda contener (instrucción, operando...), así como métodos para obtener directamente esos identificadores, haciendo transparente el proceso.

ISACodex
<pre> - INSTRUCTION_TYPE_MASK:RData = new RWDData(0xFFC0) {final} + INSTRUCTION_CODE_SHIFT:RData = new RWDData(6) {final} - FIRST_ADDRESS_MASK:RData = new RWDData(0x0038) {final} - SECOND_ADDRESS_MASK:RData = new RWDData(0x0007) {final} + FIRST_ADDRESS_CODE_SHIFT:RData = new RWDData(3) {final} + SECOND_ADDRESS_CODE_SHIFT:RData = new RWDData(0) {final} + FIRST_OPERAND_MASK:RData = new RWDData(0xFF00) {final} + SECOND_OPERAND_MASK:RData = new RWDData(0x00FF) {final} + FIRST_OPERAND_SHIFT:RData = new RWDData(RWDData.HALF_SIZE) {final} + SECOND_OPERAND_SHIFT:RData = new RWDData(0) {final} - RELATIVE_MAX_UNSIGNED_VALUE:RData = new RWDData(0xFF) {final} - RELATIVE_MIN_VALUE:RData = new RWDData(0) {final} + getInstructionType(word:RData):InstructionType {final} + getFirstAddressingMode(word:RData):AddressingMode {final} + getSecondAddressingMode(word:RData):AddressingMode {final} - getAddressingMode(word:RData, mask:RData, shift:RData):AddressingMode {final} + getFirstOperand(word:RData, addr:AddressingMode, Settings:set):Operand {final} + getSecondOperand(word:RData, addr:AddressingMode, Settings:set):Operand {final} - getOperand(word:RData, mask:RData, shift:RData, addr:AddressingMode, Settings:set):Operand {final} + isOffsetValid(value:RData):boolean {final} </pre>

Tabla 5.21: Clase ISACodex.

Las constantes almacenadas en la clase se pueden dividir en las siguientes categorías:

- Constantes dedicadas a obtener el código de instrucción:
 - INSTRUCTION_TYPE_MASK: Máscara de aislamiento de bits del código de instrucción.
 - INSTRUCTION_CODE_SHIFT: Valor del desplazamiento necesario para alinear el LSB del código de la instrucción con el LSB de la palabra.
- Constantes dedicadas a obtener el tipo de direccionamiento de cada operando:
 - XXXX_ADDRESS_MASK: Máscaras de aislamiento de bits de cada modo de direccionamiento.
 - XXXX_ADDRESS_CODE_SHIFT: Valores del desplazamiento necesario para alinear el LSB de cada código de direccionamiento con el LSB de la palabra.
- Constantes dedicadas a obtener el valor de un operando:
 - XXXX_OPERAND_MASK: Máscaras de aislamiento de bits para los valores de cada operando.
 - XXXX_OPERAND_SHIFT: Valores del desplazamiento necesario para alinear el LSB del valor de cada operando con el LSB de la palabra.
- Constantes de restricción de tamaño empleadas para determinar la pertenencia de un valor de desplazamiento relativo al rango permitido:

- `RELATIVE_MIN_VALUE` y `RELATIVE_MAX_UNSIGNED_VALUE`: Valores mínimos con signo y máximo sin signo válidos.

En cuanto a los métodos, merece la pena comentar no tanto qué hacen, pues su función está implícita en su nombre, sino las interacciones entre algunos de ellos. Específicamente, `getFirstAddressingMode()` y `getSecondAddressingMode()` son esencialmente envolturas de `getAddressingMode()` que utilizan de manera directa los respectivos valores de máscara y desplazamiento del operando al que hacen referencia. Sucede lo mismo con `getFirstOperand()`, `getSecondOperand()` y `getOperand()`.

5.3.7.2. La clase `VerificationCodex`

Si `ISACodex` gestionaba la coherencia interna de cada elemento dentro de una instrucción, `VerificationCodex` gestiona la coherencia en la combinación de dichos elementos. Esta clase, definida en la tabla 5.22, almacena una gran cantidad de conjuntos de pertenencia, asociados a las distintas instrucciones mediante mapas. Dada la cantidad y similitud estructural de todas ellas, no tiene sentido reproducir la totalidad de la clase en el diagrama.

VerificationCodex
- <code>FULL_WORD_ADDRESSING:Set<AddressingMode> {final}</code>
- <code>FIRST_OP_MAP:Map<InstructionType, Set<AddressingMode>> {final}</code>
- <code>SECOND_OP_MAP:Map<InstructionType, Set<AddressingMode>> {final}</code>
- <code>X_Y_OP_SET:Set<AddressingMode> {final}</code>
+ <code>validateInstruction(it:InstructionType, addr1:AddressingMode, addr2:AddressingMode): void</code>
+ <code>isFullWord(addr:AddressingMode):boolean</code>

Tabla 5.22: Clase `VerificationCodex`.

En los conjuntos de pertenencia, cuya estructura de nombre es `X_Y_OP_SET`, `X` es el mnemónico de la instrucción, e `Y` es el modificador `FIRST` o `SECOND`, indicando que se trata del conjunto de modos de direccionamiento válidos para el primer o segundo operando de esa instrucción concreta. Finalmente, `FIRST_OP_MAP` y `SECOND_OP_MAP` son los mapas generales que emparejan cada instrucción con su conjunto de modos de direccionamiento permitidos para cada operando.

De este modo, cuando el método `validateInstruction()` es llamado, simplemente tiene que recuperar de cada mapa los conjuntos de `AddressingMode` para el `InstructionType` suministrado, y luego comprobar si dichos conjuntos contienen o no en cada `AddressingMode` suministrado. Cabe aclarar que `validateInstruction()` no devuelve un boolean, sino que directamente genera una excepción que deberá ser gestionada por el método llamante si las comprobaciones de pertenencia no son satisfactorias; una incompatibilidad de instrucción y direccionamientos es un error fatal en la mayoría de contextos.

Finalmente, `FULL_WORD_ADDRESSING` es un conjunto de pertenencia que agrupa los modos de direccionamiento que requieren una palabra completa para definirse. En esta implementación, esos modos son el inmediato y el directo a memoria. El método `isFullWord()`, consecuentemente, comprueba si el direccionamiento suministrado pertenece a este conjunto.

5.3.8. Módulo de instrucciones

En esta sección se profundizará sobre el modelo de instrucciones empleado en la aplicación. Para comenzar por definir a grandes rasgos cómo están diseñadas en este sistema, lo mejor será empezar por el concepto básico subyacente (la clase abstracta **Instruction**), seguido de la clase **Operand** para entender cómo funcionan los operandos y la clase **InstructionData** para entender cómo una instrucción consume los datos de los operandos para construirse y ejecutarse. Una vez estén asentadas esas bases, se podrá proceder a definir las diferentes subclases abstractas de **Instruction**, así como las instrucciones que heredan a su vez de cada subclase. Dada la abundancia de las mismas, la similitud estructural entre ellas, y que en el apartado *diseño heredado del lenguaje* ya se ha descrito la funcionalidad de cada instrucción, simplemente se listarán las instrucciones en cuestión.

La única excepción a esto serán las instrucciones aritméticas, que necesitan implementar cada una su propio algoritmo de cálculo de los biestables de acarreo y desbordamiento (como ya se vio en el módulo de datos).

5.3.8.1. La clase **Instruction**

Esta clase abstracta, definida en la tabla 5.23, proporciona el marco estructural necesario para definir posteriormente las categorías de instrucciones, así como cada una de las clases individuales de cada instrucción.

<i>Instruction</i>
+ Instruction (inst:InstructionData) + run (arc:Architecture):void + checkParity (d:RData, arc:Architecture):void + checkSign (d:RData, arc:Architecture):void + checkZero (d:RData, arc:Architecture):void

Tabla 5.23: Clase **Instruction**.

Instruction implementa un constructor vacío, cuya única finalidad es forzar a las clases que hereden de ella a implementar un constructor con este argumento. Idealmente, las implementaciones de este constructor extraen del argumento toda la información requerida para funcionar correctamente después.

Como se puede ver, **Instruction** contiene el método abstracto **run()**, a través del cual cada instrucción interactúa con los métodos del módulo de arquitectura para llevar a cabo el propósito para el que fue diseñada.

Por otro lado, los métodos estáticos **checkParity()**, **checkSign()** y **checkZero()** proporcionan una funcionalidad común para que aquellas instrucciones que necesitan comprobar si el resultado final de su actuación activa ciertos biestables puedan hacerlo sin tener que realizar las mismas comparaciones cada vez. Afectan a los biestables de paridad, signo y cero solamente, ya que los biestables de desbordamiento y acarreo requieren algoritmos específicos para cada caso.

Aunque las instrucciones se agrupan conceptualmente en varias categorías (instrucciones de transferencia de datos, de salto, aritméticas, de pila...), a nivel de diseño e implementación se usa una clasificación diferente y más relacionada con la estructura que con la funcionalidad: instrucciones sin operandos, instrucciones de un operando (y un subgrupo de instrucciones de un operando para entrada/salida) e instrucciones de dos operandos. Todas estas categorías son clases abstractas que se verán más adelante en esta sección, con las instrucciones propiamente dichas heredando de alguna de estas categorías. El esquema de herencia de las subclases de *Instruction* se puede ver con más detalle en la figura 5.7.

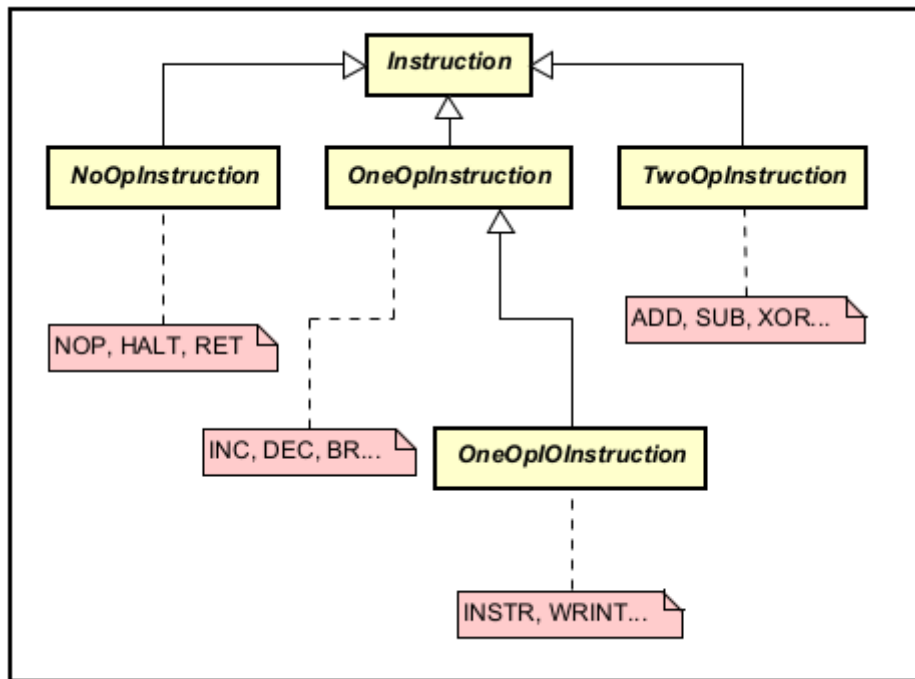


Figura 5.7: Jerarquía de herencia de las clases abstractas de instrucciones. Por concisión, solamente se muestran algunas de las instrucciones que heredan de cada subclase, del total de 37 implementadas.

Las instrucciones en sí mismas, sin embargo, son clases finales. No hay motivo para que otras clases hereden de instrucciones concretas. Si bien es cierto que eso puede decirse de la mayoría de clases del sistema y no por ello se designan como finales, también puede decirse en todos esos casos que no se puede demostrar de manera definitiva que no pueda aparecer una justificación para crear hijos para esas clases. Sin embargo, sí ocurre así con las instrucciones: su funcionalidad es atómica y bien definida. Si en algún momento se desea introducir una serie de instrucciones con un comportamiento idéntico al de una ya existente pero con añadidos, lo que se debe hacer es crear las clases pertinentes para esas funciones, no hacer que hereden de otra.

La manera más sencilla de explicarlo es mediante las instrucciones *SUB* y *CMP*. Internamente son casi idénticas: dos instrucciones de dos operandos que restan el segundo operando al primero y activan los biestables correspondientes, siendo la única diferencia que *SUB* almacena el resultado de esa resta en el registro acumulador y *CMP* no. La regla que rige una relación de herencia es que la clase que subclase “es un” subtipo de la superclase. Una resta no es una comparación, aunque casualmente funcionen igual por dentro. Por lo tanto, ninguna instrucción debe heredar de otra que no sea una categoría abstracta, y todas deben ser finales.

5.3.8.2. La clase `Operand`

Esta clase, definida en la tabla 5.24, contiene todo lo necesario para definir un operando. Su atributo principal es el modo de direccionamiento, que determinará la manera en que se construye y la información que almacena.

Operand
- <code>addr:AddressingMode</code> - <code>reg:RegisterID</code> - <code>data:RData</code> - <code>set:Settings</code>
+ <code>Operand(addr:AddressingMode, data:RData, set:Settings)</code> + <code>Operand(addr:AddressingMode, reg:RegisterID, set:Settings)</code> + <code>Operand(addr:AddressingMode, reg:RegisterID, data:RData, set:Settings)</code> + <code>getAddressingMode():AddressingMode</code> + <code>getRegister():RegisterID</code> + <code>getData():RData</code> + <code>toString():String</code> - <code>dataString():String</code> - <code>offsetString():String</code>

Tabla 5.24: Clase `Operand`.

Los atributos aportan distintos datos sobre el operando, y no siempre serán utilizados todos. Aquellos constructores que no reciben un argumento para cada atributo de la clase inicializan el atributo a `null`.

Se utiliza un constructor u otro, esencialmente, en función del tipo de direccionamiento del operando que se quiere crear:

- **Constructor `Operand(AddressingMode, RData, Settings)`:** Para operandos con direccionamiento inmediato o directo a memoria (contiene únicamente un valor y ninguna información de registro).
- **Constructor `Operand(AddressingMode, RegisterID, Settings)`:** Para operandos con direccionamiento directo a registro o indirecto (contiene únicamente un identificador de registro y ningún valor).
- **Constructor `Operand(AddressingMode, RegisterID, RData, Settings)`:** Para operandos con desplazamiento relativo a un registro índice (contiene tanto identificador de registro como valor).

No es tarea de la clase garantizar la validez de los datos introducidos (por ejemplo, un identificador de registro no válido para direccionamiento relativo, o un valor de desplazamiento muy grande), sino del contexto que genere las instancias de `Operand` y/o del que las reciba.

Sobrecargar el método `toString()` de la clase `Object` es una práctica habitual para poder generar un `String` que represente de forma coherente la estructura interna. Sin embargo, es interesante observar cómo lo hace `Operand`, ya que la aproximación no es mostrar de manera accesible

los valores de los atributos, sino construir la representación en lenguaje ensamblador del operando que representa (por ejemplo, "#3445", "/-2", ".R1", "#-23[.IX]" o "\$4"). La construcción de esta representación se apoya en los métodos internos `dataString()` y `offsetString()`, y varía según el valor del modo de direccionamiento almacenado.

5.3.8.3. La clase `InstructionData`

La clase `InstructionData`, definida en la tabla 5.25, puede entenderse como un paquete que recibe cada instrucción en el momento de su creación con todos los elementos necesarios para configurarse internamente de manera satisfactoria. Esto permite ceñir el constructor de cualquier instrucción, independientemente de su número de operandos, a la interfaz funcional `Function` empleada por el tipo enumerado `InstructionType`, como ya se vio en su sección. La consecuencia directa de esto es la homogeneización de sus firmas, permitiendo su clasificación por el método `getConstructor()` asociado a cada valor de dicho tipo enumerado. Pese a su importancia, es bastante sencilla y no requiere explicación sobre sus atributos y métodos.

InstructionData
- io:IOInterface
- opList:List<Operand>
+ InstructionData(opList:List<Operand>, io:IOInterface)
+ getOp(index:int):Operand
+ getIO():IOInterface

Tabla 5.25: Clase `InstructionData`.

5.3.8.4. La clase `NoOpInstruction`

Esta subclase abstracta, definida en la tabla 5.26, engloba a las instrucciones que no utilizan operandos. Por tanto, no extrae nada del paquete `InstructionData` que recibe en su construcción.

<i>NoOpInstruction</i> extends <code>Instruction</code>
+ <code>NoOpInstruction</code> (inst:InstructionData)

Tabla 5.26: Clase `NoOpInstruction`.

Las clases de instrucción que heredan de `NoOpInstruction` son `NOP`, `RET` y `HALT`.

5.3.8.5. La clase `OneOpInstruction`

Esta subclase abstracta, definida en la tabla 5.27, engloba a las instrucciones que utilizan un operando (aunque, como se verá en el apartado siguiente, existe una subcategoría para las que interactúan con la interfaz entrada/salida).

<i>OneOpInstruction</i> extends <i>Instruction</i>
op:Operand
+ OneOpInstruction(inst:InstructionData)

Tabla 5.27: Clase OneOpInstruction.

El único atributo de esta clase es el operando que se extrae del *InstructionData* recibido en el constructor. Las clases de instrucción que heredan de *OneOpInstruction* son BR, BZ, BNZ, BC, BNC, BV, BNV, BN, BP, BO, BE, CALL, INC, DEC, NOT, NEG, PUSH y POP.

5.3.8.6. La clase OneOpIOInstruction

Esta subclase abstracta, definida en la tabla 5.28, engloba a las instrucciones de un operando que interactúan con el módulo de entrada/salida de la aplicación a través de la interfaz *IOInterface*.

<i>OneOpIOInstruction</i> extends <i>OneOpInstruction</i>
io:IOInterface
+ OneOpInstruction(inst:InstructionData)

Tabla 5.28: Clase OneOpIOInstruction.

Además del atributo *op* heredado de la clase *OneOpInstruction*, esta clase añade su propio atributo, *io*, que es la referencia a un objeto que implemente la interfaz *IOInterface*. Dicha referencia es extraída del argumento de tipo *InstructionData* recibido en el constructor. Las clases de instrucción que heredan de *OneOpIOInstruction* son INCHAR, ININT, INSTR, WRCHAR, WRINT y WRSTR.

5.3.8.7. La clase TwoOpInstruction

Esta subclase abstracta, definida en la tabla 5.29, engloba a las instrucciones de dos operandos.

<i>TwoOpInstruction</i> extends <i>Instruction</i>
op1:Operand
op2:Operand
+ TwoOpInstruction(inst:InstructionData)

Tabla 5.29: Clase TwoOpInstruction.

Los dos atributos de esta clase son los dos operandos extraídos del *InstructionData* suministrado como argumento al constructor. Las clases de instrucción que heredan de *TwoOpInstruction* son MOVE, AND, OR, XOR, ADD, SUB, CMP, MUL, DIV y MOD.

5.3.8.8. Las operaciones aritméticas. Desbordamiento y acarreo

Dado que se busca no recurrir a tamaños de dato superiores para simplificar los cálculos, es necesario implementar algoritmos específicos para cada operación aritmética. Se debe recordar que en este apartado solamente se incide en la influencia sobre los biestables de desbordamiento y acarreo, sin perjuicio del resto de biestables, que seguirán necesitando una comprobación mediante los métodos implementados en **Instruction**.

5.3.8.8.1 Instrucciones INC y DEC

Por su naturaleza opuesta y la sencillez de la casuística, se pueden describir ambas instrucciones a la vez.

Para el desbordamiento:

- **INC**: Se produce si el resultado tras el incremento es igual a la constante de valor mínimo con signo de **RWData**. Es decir, si el valor pasa de positivo a negativo.
- **DEC**: Se produce si el resultado tras el decremento es igual a la constante de valor máximo con signo de **RWData**. Es decir, si el valor pasa de negativo a positivo.

Para el acarreo:

- **INC**: Se produce si el resultado tras el incremento es igual a 0.
- **DEC**: Se produce si el resultado tras el decremento es igual a la constante de valor máximo sin signo de **RWData**.

5.3.8.8.2 Instrucción NEG

La instrucción **NEG** tiene un caso límite que se produce cuando se intenta negar el valor mínimo con signo. El resultado sigue siendo el valor mínimo con signo:

$$\begin{array}{r}
 00000000 \ 00000000 \\
 \underline{-10000000 \ 00000000} \\
 (1) \ 10000000 \ 00000000
 \end{array}$$

Como se verá en el apartado de **SUB**, esto significa que se ha producido acarreo (sustraendo sin signo mayor que minuendo sin signo) y desbordamiento (se ha restado un negativo a un positivo o 0 y ha dado negativo).

Para cualquier otro caso, **NEG** siempre activa el biestable de acarreo (de nuevo: sustraendo sin signo mayor que minuendo sin signo) y nunca el de desbordamiento.

5.3.8.8.3 Instrucción ADD

Existen dos escenarios en los que la instrucción **ADD** genera desbordamiento:

- Si ambos operandos son positivos y el resultado es negativo.
- Si ambos operandos son negativos y el resultado es positivo o 0.

Aunque teóricamente en el caso de la suma de positivos también se produciría desbordamiento si el resultado fuera 0, el uso del MSB como bit de signo no permite un escenario en el que la suma de dos enteros pueda dar 0, como sí lo permite en el caso de negativos.

Por otro lado, existen también dos escenarios en los que **ADD** genera acarreo:

- Ambos operandos son negativos.
- Uno de los dos operandos es negativo y el resultado es mayor o igual que 0.

Esto se resume, a un nivel más bajo, en que uno o ambos operandos tienen un 1 como MSB y el resultado tiene 0 como MSB.

5.3.8.8.4 Instrucciones SUB y CMP

Como ambas instrucciones son idénticas salvo por el almacenamiento del resultado, se aplica el mismo algoritmo de biestables a ambas.

Se puede generar desbordamiento por dos motivos:

- El minuendo es mayor o igual que 0 y el sustraendo y la diferencia son negativos.
- El minuendo es negativo y el sustraendo y la diferencia son positivos.

El acarreo también tiene dos posibles causas:

- El minuendo es mayor o igual que 0 y el sustraendo es o negativo o mayor que el minuendo.
- El minuendo y el sustraendo son negativos y el sustraendo es mayor que el minuendo

5.3.8.8.5 Instrucción MUL

Existe un caso límite en el que, si uno de los operandos tiene el valor mínimo con signo y el otro es -1, el resultado es el valor mínimo con signo, y se producen tanto desbordamiento (el producto de dos negativos ha dado un resultado negativo) como acarreo:

```

      10000000 00000000
      x11111111 11111111
11111111 11111111 10000000 00000000

```

Para el resto de casos, el desbordamiento de **MUL** se calcula dividiendo el resultado entre uno de los dos operandos. Si el resultado de la división no es el otro operando, se ha producido desbordamiento. Evidentemente, hay que controlar que el operando que se usa para dividir no sea 0, pero también se define internamente un caso en el que uno de los operandos es 0 para evitar tener este conflicto.

En cuanto al acarreo, el algoritmo **MUL** es el más extenso y complejo de todos, debido al tamaño del resultado. En cualquier otra operación, el acarreo se controla mediante un bit adicional “imaginario” situado a la izquierda del MSB del resultado. Si la acción a realizar provocaría que ese bit valiera 1, significaría que hay acarreo. Sin embargo, la multiplicación de operandos de tamaño N tiene un tamaño de resultado de $2N$, por lo que podría darse el caso de que un bit de acarreo no aterrizase en el bit inmediatamente a la izquierda del MSB de la parte del resultado con la que nos quedamos. Por ello, hay que descomponer ambos operandos en dos mitades cada uno, y multiplicar todas las posibles combinaciones entre miembros de operandos distintos.

El resultado final, que ocuparía dos palabras estándar de 16 bits para el caso de esta implementación, se calcularía mediante la siguiente fórmula: $A1 \cdot B1$ desplazados 16 (o sea, N) bits a la izquierda + $(A1 \cdot B2 + A2 \cdot B1)$ desplazados 8 ($N/2$) bits + $A2 \cdot B2$. Puesto que solamente interesa de esas dos palabras la menos significativa para el resultado, esto quiere decir que cualquier bit de la palabra más significativa que tenga valor 1 implica que se ha producido acarreo.

Partiendo de esta premisa, se pueden realizar las operaciones de comprobación en cierto orden y, si en algún momento el acarreo es positivo, se puede dejar de calcular. El orden es el siguiente:

1. En primer lugar, se calcula $A1 \cdot B1$. Como la totalidad del resultado está desplazado 16 bits y ocupa la palabra más significativa del resultado completo, cualquier valor distinto de 0 en este producto es un indicio inequívoco de acarreo.
2. Si no se ha detectado, se procede a calcular $A1 \cdot B2$ y $A2 \cdot B1$. Si cualquiera de ellos tiene un bit a 1 en su mitad más significativa, al desplazar 8 bits a la izquierda ese bit quedaría en la mitad más significativa del resultado completo, lo que es indicio inequívoco de acarreo.
3. Después se procede a sumar los resultados anteriores. De nuevo, si el resultado de la suma tiene un bit a 1 en su mitad más significativa, es indicio inequívoco de acarreo por el razonamiento anterior.
4. Finalmente, si tras desplazar el resultado anterior 8 bits a la izquierda y sumarle $A2 \cdot B2$ se produce acarreo (para determinar esto se emplea el mismo algoritmo que en la instrucción **ADD**), entonces, evidentemente, la operación ha producido acarreo.
5. Si no se ha detectado acarreo llegado a este punto, la operación no ha generado acarreo.

Sea, por ejemplo, el producto de $0x00AB$ (00000000 10101011) y $0xCDEF$ (11001101 11101111). El producto mediante calculadora nos indica un resultado de $0x898EA5$ (00000000 10001001 10001110 10100101). Se comprueba el algoritmo:

A1 · B1 =	00000000	00000000		
A1 · B2 =		00000000	00000000	
A2 · B1 =		10001000	11101111	
A2 · B2 =	+		10011111	10100101
TOTAL =		00000000	10001001	10001110 10100101

Se puede comprobar que el algoritmo se podría haber detenido en el segundo paso, ya que A2·B1 ya tiene un 1 en su mitad más significativa, y esto implica acarreo.

5.3.8.8.6 Instrucción DIV

En el caso límite de que se intente dividir el valor mínimo con signo entre -1, el resultado seguirá siendo el valor mínimo con signo, y se genera tanto desbordamiento como acarreo. En el resto de casos, la división nunca produce ni desbordamiento ni acarreo.

5.3.8.8.7 Instrucción MOD

Una operación de módulo nunca genera ni desbordamiento ni acarreo.

5.3.8.9. Diferencias respecto a ENS2001

La línea de diseño inicial en ENS2001 es la misma que se ha seguido aquí: una clase abstracta que sirve de plantilla para cada instrucción, dotada de un método de ejecución que se llama desde el simulador. Si bien la clase de ENS2001 implementa directamente los valores y modos de direccionamiento como atributos en vez de pasar un único objeto con los atributos empaquetados al constructor, el resultado es el mismo. De manera análoga a los métodos `getValue()`, `getJumpValue()`, `getStringPointer()` y `setValue()` del módulo de arquitectura, tiene métodos de entrada y salida donde, a partir de los modos de direccionamiento y valores de sus operandos, la instrucción obtiene los valores con los que tiene que operar.

Sin embargo, la ausencia de capas de abstracción genera vulneraciones del principio de responsabilidad única: las instrucciones relacionadas con la pila cargan con la responsabilidad adicional de determinar mediante la configuración si se debe aumentar o disminuir el valor del SP en función de la dirección de crecimiento, y los métodos de entrada y salida de las instrucciones se ven obligados a implementar selectores de modo de direccionamiento para determinar qué se debe hacer con los valores disponibles.

Esto, a su vez, genera redundancia de código. Por ejemplo, el método `Entrada2Operandos` de ENS2001 implementa dos veces el mismo selector de modo de direccionamiento, uno por cada operando, y buena parte de los métodos de entrada y salida son idénticos (los métodos de entrada de `MOVE` y `PUSH`, por ejemplo).

Finalmente, la lógica de las instrucciones aritméticas está centralizada en un solo método que implementa un selector para operar con los valores de la instrucción en función de la operación

seleccionada, lo que vulnera el principio abierto-cerrado (y, como ya se mencionó en la sección del tipo interno de datos, atribuye al tipo de datos la responsabilidad de gestionar los biestables como resultado de las operaciones).

En contraste, ENS2025 relega la lógica adyacente al módulo de arquitectura, como se verá a continuación, generando un proceso transparente en el que las instrucciones reciben e insertan valores y resultados en la arquitectura con una llamada a un método del que no son responsables. Esto centra la responsabilidad de la instrucción en listar las peticiones que debe realizar la arquitectura y el orden en que deben hacerse para desempeñar correctamente su función.

Además, como ya se mencionó en el apartado sobre el enumerado `InstructionType`, el diseño modular de las instrucciones y su relación con dicho enumerado y con el módulo de códigos permiten la adición de nuevas instrucciones con gran facilidad. Basta con crear la clase, implementar su comportamiento, añadir en el enumerado de `InstructionType` sus características y crear en `VerificationCodex` los conjuntos de pertenencia de los modos de direccionamiento que admite para cada operando.

Por otro lado, se carga sobre las instrucciones aritméticas el peso de calcular los valores de los biestables. Esto tiene dos motivos:

- Se percibe como responsabilidad de las instrucciones. Si es responsabilidad suya generar un resultado y es responsabilidad suya actualizar los biestables, tiene sentido pensar que es responsabilidad suya implementar el proceso que analiza el resultado para determinar qué biestables se actualizan; o, al menos, es más responsabilidad suya que de una clase que representa únicamente un tipo de dato y sus operaciones.
- Eliminan la necesidad de recurrir a un tamaño mayor para almacenar los resultados puros y comparar con los valores máximos permitidos. Si bien algunos de los algoritmos son más complejos, como la multiplicación, lo que se obtiene a cambio es un cálculo de biestables ceñido estrictamente al tamaño de palabra empleado, sea de 16 bits o de 2048.

5.3.9. Módulo de arquitectura

La arquitectura de memoria (referida como “arquitectura” para acortar) de ENS2025 se compone de tres elementos: el banco de registros, definido por la clase `RegisterBank`, la memoria, definida por la clase `Memory`, y el gestor de puntos de ruptura, definido por `BreakpointManager`. Todos ellos están encapsulados en la clase envoltura `Architecture`, que contiene métodos de mediación para interactuar con sus submódulos. Esta estructura es visible en la figura 5.8.

En esta sección se profundizará primero sobre las clases que constituyen el banco de registros, la memoria y el gestor de puntos de ruptura, para luego incidir sobre la propia clase `Architecture`.

5.3.9.1. La clase `RegisterBank`

El banco de registros es el encargado de gestionar los diferentes registros del procesador, así como los biestables de estado, a lo largo de la simulación. Constituye una estructura centralizada

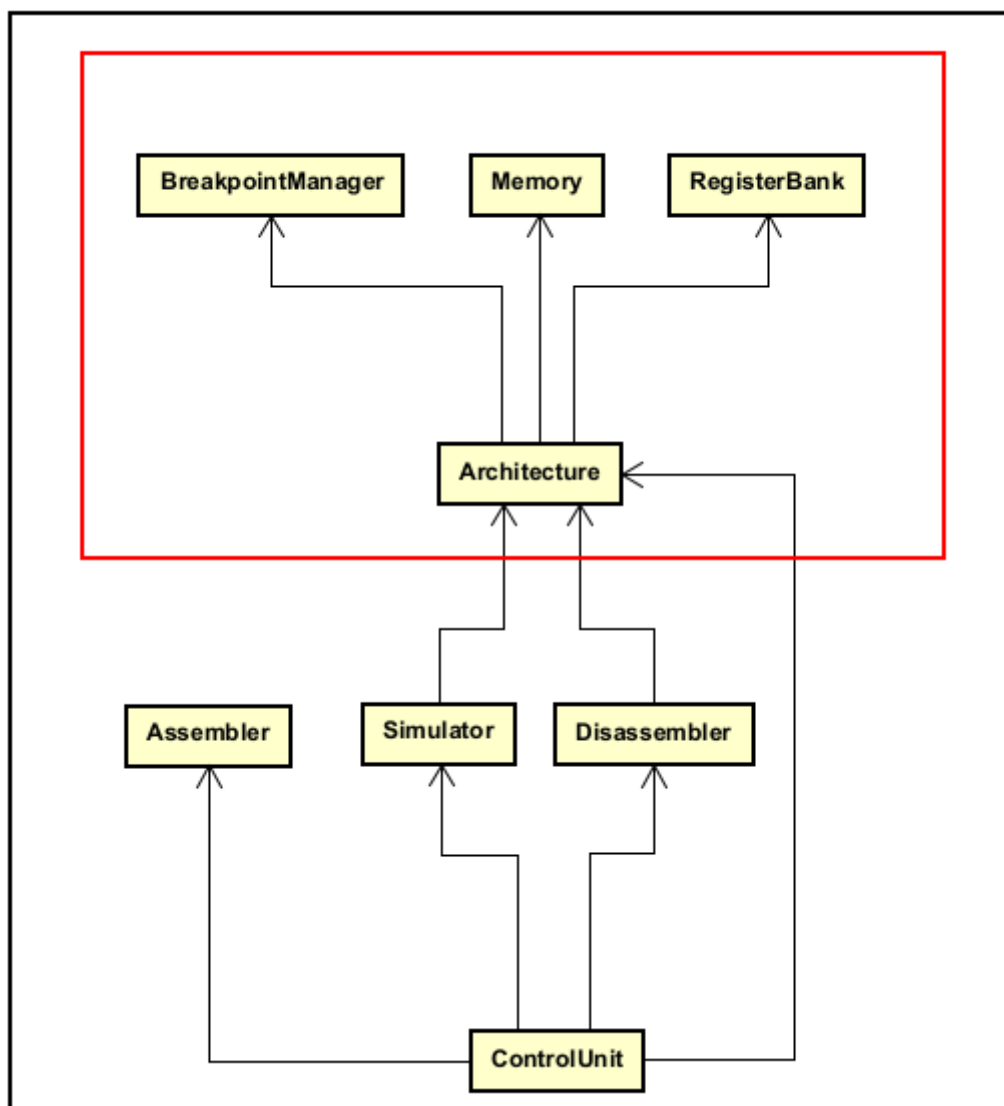


Figura 5.8: Vista de la unidad de control con la arquitectura expandida.

para el acceso y la modificación a estos datos. Esta estructura se ve representada en la clase `RegisterBank`, definida en la tabla 5.30.

Su estructura está diseñada para contener y operar sobre los 16 registros de la aplicación, así como los 6 biestables asociados al registro de estado. Estos últimos están representados por valores `boolean`, por lo que se suelen usar de forma intercambiable los términos “en alto”, “1” y “`true`” para referirse al hecho de que el biestable está activado, y “en bajo”, “0” y “`false`” para referirse a que está desactivado.

Ambos grupos están almacenados en sendos mapas, uno emparejando el identificador de cada registro con la instancia de `RWDData` que contiene su valor y el otro emparejando el identificador de cada biestable (expresado mediante el tipo enumerado `Flag`) con su estado.

Aunque las funciones primarias de los distintos métodos son fácilmente deducibles de sus nombres, merece la pena resaltar el mecanismo de consistencia de los biestables:

- Cada vez que se llama a `writeRegister()`, éste comprueba si el registro a escribir es el

RegisterBank
- registers:Map<RegisterID, RWDData>
- flags:Map<Flag, Boolean>
+ RegisterBank()
+ readRegister(reg:RegisterID):RData
+ writeRegister(reg:RegisterID, value:RData):void
+ setFlag(f:Flag, status:boolean):void
+ getFlag(f:Flag):boolean
- updateFlags():void

Tabla 5.30: Clase RegisterBank.

registro de estado. Si es así, al terminar de escribir llama a `updateFlags()` para mantener los valores actualizados.

- Cada vez que se llama a `setFlag()`, éste actualiza su bit correspondiente en el registro de estado extrayendo su valor, aplicando una máscara y volviendo a escribirlo (de manera directa, no a través de `writeRegister()`).

Estas máscaras de actualización (usadas también por `updateFlags()` para actualizar en lote todos los biestables) se obtienen directamente de cada valor del enumerado `Flag`, y se aplican de la siguiente manera:

- Cuando se pretende escribir un 1 en una posición determinada del SR, se toma la máscara de ese valor de `Flag` y se aplica directamente mediante una operación OR lógica.
- Cuando se pretende escribir un 0 en una posición determinada del SR, se toma la máscara, se invierte bit a bit, y se aplica al valor del SR mediante una operación AND lógica.

5.3.9.2. La clase Memory

La memoria es la encargada de almacenar el código de los programas una vez son ensamblados, así como las cadenas definidas y los datos que se quieran almacenar en tiempo de simulación. También almacena la pila de forma conceptual (es decir, no hay una estructura de pila definida como tal; simplemente se llama de esa manera al grupo de posiciones de memoria comprendidas entre la posición inicial de pila y el valor actual del SP). La memoria se implementa mediante la clase `RegisterBank`, definida en la tabla 5.31.

Desgraciadamente, es necesario que `Memory` llame al método `getValue()` de `RData/RWDData`, lo que otorga a la clase cierto nivel de “conocimiento” sobre la estructura interna del tipo de dato empleado en el sistema. Este mal necesario tiene su razón en que los arrays solamente admiten tipos `int` (y su envoltura `Integer`) como índices. Por lo tanto, no queda más remedio que asumir desde `Memory` que se conoce el tipo de dato usado en `RWDData`.

Por otro lado, una refactorización del tamaño de palabra tendría un efecto profundo sobre la estructura de la clase `Memory`, hasta el punto de que conocer el tamaño usado por `RWDData` es el menor de los problemas. Si, por ejemplo, se decidiera ampliar el tamaño de palabra a 32

Memory
- mem:RWData[]
+ Memory() + fetchAt(index:RData):RData + fetchAt(index:int):RData + writeAt(index:RData, value:RData):void + writeAt(index:int, value:RData):void + wipe():void + loadMemory(newMem:Memory):void + dumpMemory():Memory

Tabla 5.31: Clase Memory.

bits y, en consecuencia, el número de posiciones direccionables a 2^{32} , sería necesario utilizar un `int` completo. Ante la inexistencia de números negativos en Java, ya no se podría usar una estructura de array para almacenar las celdas de memoria. A priori, se podría pensar que un `HashMap` puede cumplir con el cometido; pero la idea se descarta rápidamente al evaluar los costes de memoria: solamente el coste de los 2^{32} emparejamientos `<RWData, RWData>` ya ascendería a los 32GB, sin contar con el *overhead* generado por cada emparejamiento. Además, en Java el `HashMap` utiliza un array para sus punteros, por lo que no sería una implementación viable (y, de serlo por usar otro tipo de estructura, generaría otro coste de memoria adicional cuantioso). Finalmente, el tiempo de acceso se vería drásticamente aumentado respecto su valor promedio de $O(1)$ por la gran cantidad de colisiones. Lo más fácil, en ese caso, sería establecer una lista de dos arrays de 2^{31} posiciones, simulando un array de 2^{32} . Esto requeriría un algoritmo para interpretar el valor del índice y correlacionarlo de manera efectiva con el índice correcto del array correcto. Otra opción sería el uso de un sistema de ficheros de paginación.

Todo esto simplemente para usar el `int` completo; una hipotética refactorización a `long` acarrearía una estructura aún más compleja. Por lo tanto, el impacto de un cambio de retorno en método que devuelve un índice no es una preocupación que deba considerarse, ya que hay que descartarlo todo y reconstruirlo desde cero igualmente.

5.3.9.3. La clase BreakpointManager

Esta clase, definida en la tabla 5.32, guarda constancia de las posiciones de memoria en las que se ha activado un punto de ruptura, cuyo uso es detener la simulación si se ha iniciado en modo de depuración.

BreakpointManager
- points:boolean[]
+ BreakpointManager() + hasBreakpoint(index:RData):boolean + setBreakpoint(index:RData, bp:RData):void

Tabla 5.32: Clase BreakpointManager.

La clase `BreakpointManager` adolece de los mismos problemas estructurales que `Memory`, por lo que es también inherentemente vulnerable a la refactorización y difícilmente mantenible.

Esta similitud estructural es la razón por la que se incluye **BreakpointManager** en el módulo de arquitectura en vez de en el de la unidad de control, situado en una capa superior, pese a que a priori pueda suponer una vulneración del principio de responsabilidad única. Una hipotética refactorización a 32 o 64 bits conllevaría la necesidad de reescribir desde cero ambas clases. Por lo tanto, para minimizar el número de módulos y capas de abstracción afectados, y contener en el nivel más profundo posible el impacto de los cambios, se optó por esta solución. Inicialmente, se consideró por este mismo motivo la posibilidad de unificar el gestor de puntos de ruptura y la memoria, creando un único array con celdas de objetos **MemoryPosition** que contuvieran por un lado el **RWData** con el valor almacenado y por otro el **boolean** con el estado del punto de ruptura; sin embargo, se consideró que la vulneración del principio de responsabilidad única en este caso era excesiva y se optó por esta solución intermedia.

5.3.9.4. La clase **Architecture**

Ahora que se ha profundizado sobre los elementos que componen el módulo de arquitectura, se puede pasar a describir de manera apropiada el módulo en sí, así como la clase que lo implementa, definida en la tabla 5.33.

El módulo de arquitectura centraliza toda la lógica relacionada con los diferentes tipos de memoria del sistema (la memoria propiamente dicha, el banco de registros y el gestor de puntos de ruptura), y actúa como capa de abstracción traduciendo peticiones más complejas que puedan llegar de otros módulos en conjuntos de llamadas a métodos básicos de los atributos locales, haciendo todo el proceso transparente al resto del sistema y permitiendo que las modificaciones futuras tengan un impacto mínimo y contenido.

Por ejemplo, se podría decidir implementar un retardo de un milisegundo cada vez que se ejecuta una operación de lectura/escritura en un registro, y uno de cinco milisegundos cada vez que se ejecuta una de lectura/escritura en la memoria, para representar la diferencia de eficiencia entre ambos y con el fin de alentar a los alumnos a considerar cómo escriben el código en vez de permitir la utilización de registros y memoria de forma intercambiable. Esta modificación, claramente, no atañe de manera estricta ni al banco de registros ni a la memoria, pero sí guarda relación con ellos. Gracias al módulo de arquitectura, esa lógica adyacente tiene un lugar donde implementarse de manera transparente para el resto del sistema.

La mayoría de los métodos son envolturas de otros de igual funcionalidad pertenecientes a las clases de los atributos, pero sí conviene detallar algunos de ellos:

- **Método `increaseProgramCounter()`:** En este método se incluye la lógica de comprobación de final de memoria alcanzado.
- **Métodos `increaseStackPointer()` y `decreaseStackPointer()`:** Contienen la lógica de consulta la dirección de crecimiento de la configuración del sistema, y el consecuente efecto en la dirección de movimiento del puntero de pila. También contienen la lógica de control de overflow, pero no la de underflow. Se considera que se ha producido overflow de la pila cuando el valor del SP cambia de la última dirección a la (primera si la pila es creciente) o de la primera a la última (si es decreciente).
- **Método `getStringPointer()`:** Interpreta el contenido de un objeto **Operand** en el con-

Architecture
- rb:RegisterBank - mem:Memory - bpm:BreakpointManager - set:Settings
+ Architecture(set:Settings) + hasBreakpoint(index:RData):boolean + setBreakpoint(index:RData, value:boolean):void + resetRegisterBank():void + increaseProgramCounter():void + increaseStackPointer():void + decreaseStackPointer():void + setFlag(f:Flag, status:boolean):void + getFlag(f:Flag):boolean + getStringPointer(op:Operand):RData + getJumpValue(op:Operand):RData + getValue(op:Operand):RData + setValue(op:Operand, value:RData):void + getStackPointer():RData + getProgramCounter():RData + writeAccumulator(value:RData):void + setProgramCounter(value:RData):void + readRegister(reg:RegisterID):RData + writeRegister(reg:RegisterID, value:RData):void + readMemory(index:RData):RData + writeMemory(index:RData, value:RData):void + loadMemory(newMem:Memory):void + dumpMemory():Memory + wipeMemory():void

Tabla 5.33: Clase Architecture.

texto de la recuperación de un puntero de cadena. En función del modo de direccionamiento almacenado se procesará el resto de contenidos:

- **Directo a memoria:** Se trata como un literal (se devuelve el valor de la dirección contenido en el operando, no lo que hay almacenado en dicha dirección).
 - **Indirecto:** Se utiliza el valor `RegisterID` contenido en el operando para identificar en qué registro se almacena la dirección a devolver.
 - **Relativo a IX o IY:** Se utiliza el valor contenido en el operando como desplazamiento. Se utiliza el valor `RegisterID` contenido en el operando para determinar el registro al que afecta; del registro identificado se extrae la dirección inicial a desplazar. Se llama al método `convertToOffset()` de `RWData` para convertir el valor absoluto en un desplazamiento, que se aplica a la dirección extraída del registro. Se devuelve el resultado final como puntero de la cadena.
- **Método `getJumpValue()`:** Interpreta el contenido de un objeto `Operand` en el contexto de la recuperación de una dirección de salto. En función del modo de direccionamiento almacenado se procesará el resto de contenidos:

- **Directo a memoria:** Funciona igual que en `getStringPointer()`.
 - **Indirecto:** Se utiliza el valor `RegisterID` contenido en el operando para extraer del registro pertinente una dirección de memoria. El valor contenido en dicha dirección es el valor a devolver como dirección de salto.
 - **Relativo a PC:** Funciona igual que en `getStringPointer()`. La limitación de usar el registro PC viene impuesta por diseño; implementarlo de tal forma que siga solicitando el identificador de registro al operando facilita una hipotética refactorización si se permitiera el uso de otros registros en un futuro.
- **Método `getValue()`:** Interpreta el contenido de un objeto `Operand` en el contexto de la recuperación de un valor de algún punto de la arquitectura. En función del modo de direccionamiento almacenado se procesará el resto de contenidos:
- **Inmediato:** Se devuelve directamente el valor almacenado en el operando.
 - **Directo a memoria:** Se devuelve el valor almacenado en la posición de memoria facilitada en el operando.
 - **Directo a registro:** Se utiliza el valor `RegisterID` contenido en el operando para identificar en qué registro se almacena el valor a devolver.
 - **Indirecto:** Funciona igual que en `getJumpValue()`.
 - **Relativo a IX o IY:** Se realiza el mismo proceso que en `getStringPointer()` y `getJumpValue()`, pero el valor final obtenido se emplea para identificar la posición de memoria de la que se extrae el valor a devolver.
- **Método `setValue()`:** Interpreta el contenido de un objeto `Operand` en el contexto de la inserción en algún punto de la arquitectura de un valor suministrado. En función del modo de direccionamiento almacenado se procesará el resto de contenidos:
- **Directo a memoria:** Se inserta el valor en la dirección de memoria contenida en el operando.
 - **Directo a registro:** Se utiliza el valor `RegisterID` contenido en el operando para identificar en qué registro se almacena el valor a insertar.
 - **Indirecto:** Se utiliza el valor `RegisterID` contenido en el operando para extraer del registro pertinente una dirección de memoria, en la que se almacenará el valor a insertar.
 - **Relativo a IX o IY:** Se obtiene del operando el valor de desplazamiento respecto al registro (calculado como en los métodos anteriores) y el identificador del registro a usar como referencia. Se extrae el valor de dicho registro, se le aplica el desplazamiento y se usa el resultado como dirección de memoria en la que se inserta el valor.
- **Método `writeRegister()`:** Además de ser una envoltura de su homónimo de la clase `RegisterBank`, este método centraliza la lógica de comprobación de invasión de zonas de código y pila, así como de underflow. Estas comprobaciones dependen de que el parámetro correspondiente esté activo en la configuración del sistema.
- **Invasión de zona de pila:** Si se va a insertar un valor en el PC, se generará un error si el nuevo valor del PC está comprendido entre la dirección actual del SP y la dirección de inicio de la pila.

- **Invasión de zona de código:** Si se va a insertar un valor en el SP, se generará un error si el nuevo valor del SP está comprendido entre las direcciones inicial y final de la zona de código.
- **Stack underflow:** Si se va a insertar un valor de SP, se generará un error si el valor a insertar es “anterior” a la dirección de inicio de la pila (direcciones de valor inferior en caso de pila con crecimiento positivo o de valor superior en caso de pila con crecimiento negativo). A Esta comprobación tiene menor prioridad que la de overflow, por lo que un error de este tipo solamente puede suceder en caso de desbordamiento de memoria durante una inserción si la comprobación de overflow está desactivada.

5.3.9.5. Diferencias respecto a ENS2001

Si bien el diseño y la implementación de ENS2025 son similares a los de ENS2001 en lo tocante a la estructura y uso de los módulos de gestor de puntos de ruptura y la memoria, sí existen diferencias con la implementación del banco de registros y al concepto de la arquitectura de memoria en general.

La aproximación de ENS2001 al banco de registros mediante la clase `cbancoregistros` se basa en:

- Empleo de arrays de tamaños iguales al total de registros y biestables, respectivamente, además de otro array para almacenar los nombres de los registros.
- Almacenamiento y gestión de los límites de las zonas de código y pila.
- Centralización de la lógica satélite del banco de registros, asignando a la clase las responsabilidades de comprobar desbordamientos al avanzar el PC, determinar si el PC o el SP invaden las zonas de pila y código respectivamente.

Los motivos por los que ENS2025 difiere de esa implementación, sustituyéndola por la mencionada en la clase `RegisterBank`, son los siguientes:

- **Principio de responsabilidad única:** El banco de registros no debería responsabilizarse de gestionar lógica relacionada con el sistema en el que es creado (control de desbordamientos e invasiones), ni almacenar datos ajenos a los que conceptualmente le corresponden (límites de las zonas de código y pila). Idealmente, el banco solamente debe gestionar el almacenamiento, lectura y escritura de los registros y biestables, y mantener la coherencia interna entre estos últimos con el SR. El resto debe ser manejado por fuera; en este caso, los límites de las zonas de código y pila forman parte de la clase `Settings`, y la lógica satélite mencionada es llevada a cabo por la clase `Architecture`.
- **Abstracción y orientación a la mantenibilidad:** Los objetos `Map`, combinados con los tipos enumerados `RegisterID` y `Flag` y sus propiedades internas (nombre, identificador, valor de máscara...) conforman un alto grado de resistencia a la refactorización. Para añadir, por ejemplo, otros 16 registros y otros 6 biestables al banco, basta con modificar directamente los tipos enumerados con los datos pertinentes, y la clase `RegisterBank` seguirá funcionando exactamente igual sin necesidad de añadir modificaciones.

Por otro lado, ENS2001 sufre de ciertos problemas a la hora de mantener la consistencia interna entre el registro SR y los biestables, que han sido corregidos para ENS2025:

- Al cargar y ensamblar un archivo, el reinicio rutinario del banco de registros no reinicia a 0 los valores de los biestables, por lo que el nuevo programa cargado podría tener un flujo de ejecución inesperado debido a esto. Esto también sucede si se asigna manualmente (es decir, mediante aplicación y no mediante una instrucción simulada) valor 0 al registro SR o se reinicia el banco de registros bajo demanda.
- Aunque el valor del SR sea 0, si los biestables no están todos reiniciados, almacenar en alguna parte el valor del SR resulta en el almacenamiento de los valores de los biestables también. Por ejemplo: sea el caso en el que, tras reiniciar el banco de registros, sigue quedando el biestable de paridad activo pese a que el valor del SR es 0. La siguiente instrucción a ejecutar es `MOVE .SR, .R2`. Tras su ejecución, el valor del registro R2 debería ser 0x0000, pero en su lugar es 0x0008.
- Las instrucciones que modifican el estado de los biestables sobrescriben el segmento del SR no dedicado al almacenamiento de biestables. Es decir, una instrucción `MOVE #-1, .SR` seguida de una instrucción `ADD #0, #0` deja el estado final del SR como 0x0000 en vez de 0xFFC0.

Respecto a la clase **Architecture** como fachada de este módulo, ENS2001 no solamente responsabiliza a módulos como el banco de registros de lógica inherentemente externa a ellos, sino que utiliza la clase **CConfiguración** como repositorio de referencias a los diferentes componentes del sistema, pero sin actuar como fachada. Por ejemplo, si una instrucción necesita escribir en un registro, solicitará a la configuración que le facilite el objeto del banco de registros para llamar directamente a su método de escritura.

5.3.10. Módulo del decodificador

La decodificación es el proceso mediante el cual el sistema extrae de la memoria una o más palabras de lenguaje máquina puro y genera, en función del contenido, una instrucción ejecutable para el simulador. Gracias a las decisiones de diseño tomadas en otros módulos (códices, enumerado de instrucciones..., este proceso es relativamente sencillo y directo.

Además, gracias también a la existencia de los códigos, la arquitectura interna de una instrucción es relativamente transparente al decodificador, que solamente necesita partir de las siguientes premisas:

- La primera palabra siempre contiene el código de instrucción y los códigos de direccionamiento.
- Las palabras adicionales, de haberlas, contienen los valores de los operandos.
- Una instrucción sin operandos no requiere ninguna palabra adicional, una instrucción de un operando requiere una palabra adicional y una instrucción de dos operandos requiere dos palabras adicionales. Esta propiedad se cumple en ambas direcciones: una instrucción

con código de primer operando nulo es una instrucción sin operandos. De entre las que no tienen código de primer operando nulo, aquellas que tienen código de segundo operando nulo son instrucciones de un operando, y el resto, por fuerza, son de dos operandos. Una aproximación similar es la que se usará para simplificar el flujo de ejecución del decodificador.

- En caso de tener una instrucción de dos operandos, si ninguno de los modos de direccionamiento necesita una palabra completa para codificarse, la instrucción solamente requiere una palabra adicional en vez de dos.

Aunque esto no hace al decodificador resistente a refactorizaciones de gran magnitud, sí lo dota de cierto nivel de desacople con respecto al nivel más bajo de la arquitectura de instrucciones, minimizando el impacto de dichas refactorizaciones.

El algoritmo de decodificación se puede describir paso a paso de la siguiente manera:

1. Se extrae el valor actual del PC y se guarda como “rollback” para volver a él si surgiera alguna contingencia durante la decodificación, evitando así que el siguiente intento de decodificar una instrucción no empezase con la palabra correcta.
2. Se genera una lista vacía de operandos.
3. Se extrae de la posición de memoria apuntada por el PC una palabra y se incrementa el PC.
4. Se disecciona dicha palabra para obtener el código de instrucción y los códigos de direccionamiento.
5. Se verifica que la combinación de instrucción y direccionamientos es válida.
6. Si el segundo direccionamiento es nulo (es decir, la instrucción no tiene segundo operando), se trata de una instrucción sin operandos o con uno solo.
 - Si el primer direccionamiento también es nulo, se trata de una instrucción sin operandos y se procede directamente al paso final sin introducir operandos en la lista.
 - Si el primer direccionamiento no es nulo, se trata de una instrucción de un operando. Se extrae una palabra adicional, se incrementa el PC, se procesa la palabra para extraer el primer operando y éste se inserta en la lista.
7. Si el segundo direccionamiento no es nulo, se trata de una instrucción de dos operandos.
 - Si uno o ambos modos de direccionamiento son de palabra completa, cada operando necesitará una palabra. Por tanto, se extraen dos palabras adicionales de la memoria (con sus consiguientes incrementos del PC) y se extraen el primer y segundo operando de ellas, para después añadirlos a la lista.
 - Si ninguno de los modos de direccionamiento requiere una palabra completa, ambos operandos compartirán una sola palabra. Se extrae una única palabra adicional, se incrementa el PC y se extrae de cada mitad de la palabra el operando correspondiente, para después introducir ambos en la lista.

8. Para terminar, se crea una nueva instrucción ejecutable usando el código de instrucción y la lista de operandos.

Si en algún momento del proceso se produce un error (instrucción no válida, límite de memoria alcanzado...), se devuelve el PC al valor inicial de “rollback” y después se propaga el error, debiendo éste ser manejado en la capa de abstracción superior a la que ejecuta este algoritmo.

Las decisiones de diseño comentadas permiten que, mientras no se modifique de manera significativa la arquitectura de instrucciones (por ejemplo, añadiendo instrucciones de tres operandos, o trasladando a otra palabra alguno de los códigos contenidos en la primera), la implementación de este algoritmo de decodificación funcione sin necesidad de cambios. Se podría, por ejemplo, cambiar el orden en el que se almacenan los datos en la primera palabra (primero los modos de direccionamiento y luego el tipo de instrucción) o incluso redistribuir la cantidad de bits asignados (eliminando dos bits de los códigos de instrucción para añadir un bit a cada modo de direccionamiento para poder implementar hasta 8 más), y no sería necesario modificar el comportamiento de esta clase.

5.3.10.1. La clase Decoder

Esta clase, definida en la tabla 5.34, implementa el algoritmo de decodificación analizado.

Decoder
- arc:Architecture - io:IOInterface - set:Settings
+ Decoder(arc:Architecture, io:IOInterface, set:Settings) + getInstruction(ignoreBreakpoint:boolean):Instruction - fetchWord(rollback:RData, ignoreBreakpoint:boolean):RData - createInstruction(iType:InstructionType, opList:List<Operand>):Instruction

Tabla 5.34: Clase Decoder.

El método `getInstruction()` implementa la lógica de decodificación detallada al comienzo de la sección. Se apoya en `fetchWord()` para la extracción de palabras de la memoria y en `createInstruction()` para crear la `Instruction` que luego se devuelve.

El método `fetchWord()`, como su nombre indica, extrae una palabra de la memoria donde esté apuntando el contador de programa, incrementa el contador y devuelve la palabra leída. Entre medias, controla si existe un punto de ruptura en la posición de la palabra a extraer y, si tiene orden de no ignorarlos, asigna al PC el valor del argumento `rollback` (la dirección de la primera palabra de la instrucción cuya decodificación está en curso), abortando el proceso de decodificación.

El método `createInstruction()` genera internamente una instancia de `InstructionData` con los datos suministrados, obtiene el constructor asociado al valor de `InstructionType` recibido y lo ejecuta, devolviendo el objeto resultante.

5.3.10.2. Diferencias respecto a ENS2001

Si bien ENS2001 mantiene una línea de diseño similar a la hora de implementar un decodificador agnóstico a la instrucción en sí, responsabiliza a la clase `CProcesador` (lo que aquí equivaldría a la clase `Simulator`) de las tareas de decodificación de instrucciones y operandos, así como de su ejecución. Esto vulnera el principio de responsabilidad única, razón por la cual ENS2025 separa la decodificación de la abstracción.

Además, ENS2001 implementa la decodificación directamente a bajo nivel, interpretando directamente los bits de las palabras leídas. Esto entra en conflicto con el principio de dependencia inversa, ya que un módulo de alto nivel (el procesador/simulador) depende directamente de detalles de bajo nivel. Este conflicto es precisamente el motivo por el que ENS2025 implementa el módulo de códigos: para proporcionar una capa de abstracción que invierte esta dependencia, permitiendo que el decodificador interactúe con abstracciones de alto nivel en lugar de forzarlo a la manipulación directa de bits.

5.3.11. Módulo del simulador

La función del simulador es relativamente sencilla: su papel consiste en solicitar instrucciones ejecutables al decodificador y pedir a dichas instrucciones que se ejecuten. El desafío de este módulo viene de la concurrencia: aunque durante la ejecución de la simulación no se deben poder modificar otros aspectos del sistema, la ejecución no debe bloquearlo (es decir, es responsabilidad de las distintas aplicaciones restringir el acceso al resto de elementos de las mismas). De lo contrario, un programa ensamblador mal diseñado que incurriera en un bucle infinito bloquearía indefinidamente el sistema.

Sin embargo, no es responsabilidad del simulador gestionar esas contingencias, sino de las distintas aplicaciones. Lo que sí es responsabilidad del simulador es aportar los mecanismos para permitir ser ejecutado concurrentemente al hilo principal, dejando a cada aplicación la implementación de dicha concurrencia.

El simulador tiene tres modos de ejecución:

- **Continúa:** Ejecuta instrucciones hasta que el biestable H esté en alto, hasta que lo detenga el usuario o hasta que se produzca una excepción en la simulación (límite de memoria alcanzado, división entre 0...).
- **Depuración:** Ejecuta instrucciones hasta detenerse bajo las mismas premisas que el modo de ejecución continua o hasta que se encuentre un punto de ruptura.
- **Paso a paso:** Ejecuta una única instrucción.

El módulo del simulador se encuentra definido enteramente por la clase `Simulator`, y además contiene al módulo del decodificador descrito en el apartado anterior como módulo auxiliar para separar las tareas de decodificación de las de ejecución, como se puede ver en el diagrama ampliado de la figura 5.9.

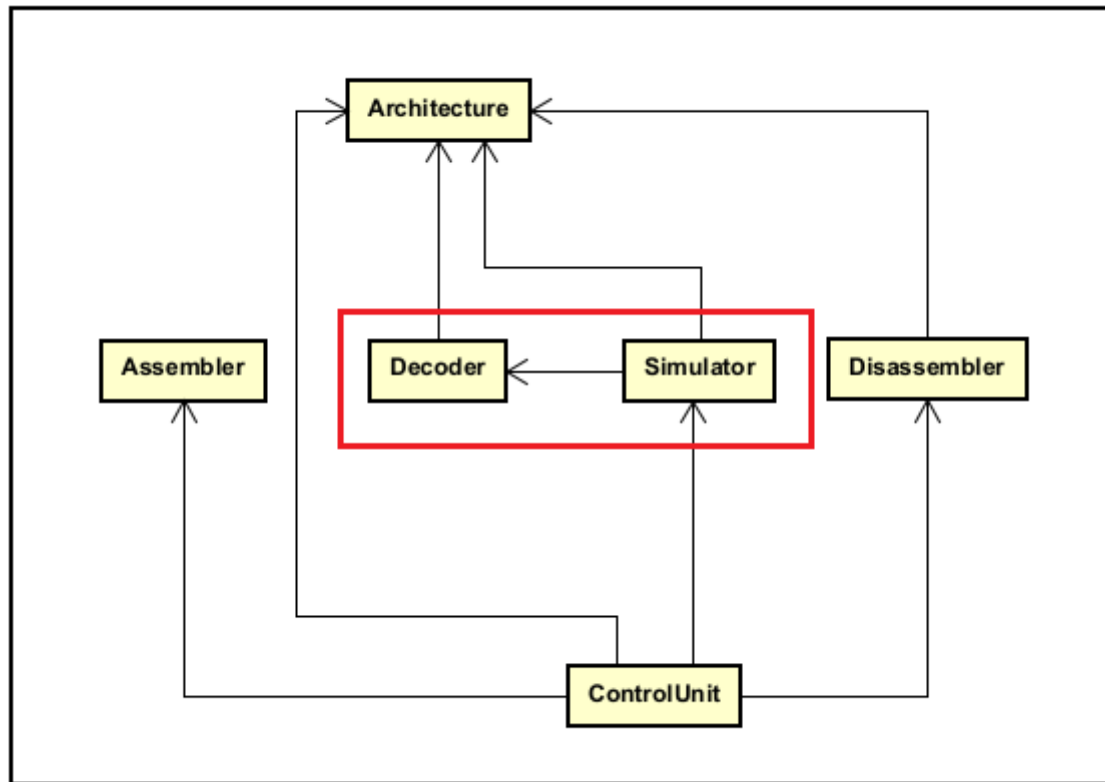


Figura 5.9: Vista de la unidad de control con el simulador expandido.

5.3.11.1. La clase Simulator

Esta clase, definida en la tabla 5.35, gestiona exclusivamente los modos de simulación existentes, solicitando las tareas de decodificación al **Decoder** incluido en la misma.

Simulator
- set:Settings - arc:Architecture - decoder:Decoder - io:IOInterface - stopSimulation:AtomicBoolean
+ Simulator(set:Settings, arc:Architecture, decoder:Decoder, io:IOInterface) + runSimulation():void + debugSimulation():void + runSimulationOnce():void + checkHalt():void + checkResetRegisters():void + getStopFlag():AtomicBoolean + setStopFlag(value:Boolean):void

Tabla 5.35: Clase Simulator.

Los atributos de la clase son, en su mayoría, simplemente referencias a otros componentes necesarios en la ejecución: la configuración del sistema, la arquitectura de memoria, el decodificador y la interfaz de entrada/salida que conecta con la aplicación. El quinto atributo, un

AtomicBoolean, es lo que permitirá a las aplicaciones implementar la concurrencia, modificando el valor de **stopSimulation** desde el hilo principal para detener forzosamente la ejecución del simulador a petición del usuario. Un **AtomicBoolean** garantiza operaciones atómicas y visibles entre hilos, evitando problemas de concurrencia que podrían surgir con una variable **boolean** simple en un entorno multi-hilo.

El método **checkHalt()** se sitúa justo al final de cada método de ejecución, y comprueba si el biestable H está en alto. De ser así, muestra un mensaje indicando que se ha encontrado el final del programa.

El método **checkResetRegisters()**, por el contrario, se sitúa al principio de cada método de ejecución, antes de entrar al bucle. Este método comprueba primero en la configuración del sistema si se deben reiniciar los registros cuando se inicia una nueva ejecución mientras el biestable H está en alto. Si es así y el biestable H está en alto en ese momento, solicita al módulo de arquitectura para reiniciar todos los registros (incluido el PC, lo que significa que vuelve a empezar a ejecutar instrucciones desde el principio). Si no es así, reinicia a **false** el biestable H manualmente para que no fuerce la detención en cada nueva ejecución.

5.3.12. Módulo del desensamblador

El desensamblado es el proceso mediante el cual la información de una instrucción ensamblada en lenguaje máquina se revierte a su estado original de cadena de texto en lenguaje ensamblador legible para humanos. Esta funcionalidad es crucial para navegar el código una vez cargado en el sistema.

Este módulo existe para suplir esa necesidad. Al igual que sucede con el módulo del decodificador, el alto nivel de abstracción en su diseño hace de esta tarea algo relativamente sencillo.

El módulo consta de dos clases, **Disassembler** y **DisassembledInstruction**. El primero lee el código máquina de la memoria y genera instancias del segundo. Se puede entender como un proceso análogo a la decodificación, con la que comparte bastantes similitudes estructurales y de flujo de ejecución. Sin embargo, su objetivo difiere: en vez de crear una instrucción ejecutable que interactúa con la memoria y el banco de registros, el desensamblador construye un contenedor de información relativa a una instrucción. La finalidad de este contenedor es generar una cadena de texto casi idéntica a la que interpretó el ensamblador para producir el código máquina asociado (si bien se pierden detalles no esenciales como el espaciado, la etiqueta, la tabulación o los comentarios).

El proceso de desensamblado a partir de una dirección dada sigue el algoritmo descrito a continuación. Nótese la gran similitud con el algoritmo de decodificación:

1. Se crea una copia de la dirección suministrada, que servirá como puntero de lectura, dejando la original como punto de retorno si se encuentra un error.
2. Se crea una lista vacía de operandos, un indicador de puntos de ruptura y un contador para almacenar el tamaño en palabras de la instrucción, empezando en 1.
3. Se extrae una palabra de la memoria en la posición marcada por el puntero. Se comprueba

- si existe un punto de ruptura en esa dirección y activa el indicador de puntos de ruptura si es así. Se incrementa el valor del puntero.
4. Se disecciona dicha palabra para obtener el código de instrucción y los códigos de direccionamiento.
 5. Se verifica que la combinación de instrucción y direccionamiento es válida.
 6. Si el segundo direccionamiento es nulo (es decir, la instrucción no tiene segundo operando), se trata de una instrucción sin operandos o con uno solo.
 - Si el primer direccionamiento también es nulo, se trata de una instrucción sin operandos y se procede al paso final sin introducir operandos en la lista.
 - Si el primer direccionamiento no es nulo, se trata de una instrucción de un operando. Se extrae una palabra adicional (siguiendo el mismo proceso que en el paso 3), que se procesa para extraer de ella el primer operando e insertarlo en la lista. Se incrementa en 1 el contador de palabras de la instrucción.
 7. Si el segundo direccionamiento no es nulo, se trata de una instrucción de dos operandos.
 - Si uno o ambos modos de direccionamiento son de palabra completa, cada operando necesitará una palabra. Por tanto se extraen dos palabras adicionales de la memoria (es decir, dos ejecuciones completas del paso 3) y se extrae de cada una su correspondiente operando, para después añadir ambos a la lista. Se incrementa en 2 el contador de palabras de la instrucción.
 - Si ninguno de los modos de direccionamiento es de palabra completa, ambos operandos comparten una sola palabra. Se extrae una palabra adicional (de nuevo, siguiendo el paso 3) y se extrae de cada mitad el operando correspondiente, para después añadir ambos operandos a la lista. Se incrementa en 1 el contador de palabras de la instrucción.
 8. Si en algún punto del proceso anterior se ha producido un error (combinación no válida de instrucción y direccionamientos, límite de memoria alcanzado...), no se propaga. En su lugar, se crea una instrucción desensamblada consistente en una cadena de error, se restaura el puntero a su valor original (la dirección inicial) y se procede directamente al paso 10. De lo contrario, se procede al paso 9 con normalidad.
 9. Se crea una nueva instrucción desensamblada con todos los datos recabados (tipo de instrucción, indicador de punto de ruptura, cantidad de palabras, operandos...).
 10. Se asigna a la dirección original el valor actual del puntero y se incrementa en 1 para que apunte a la primera palabra de la siguiente instrucción.
 11. Se devuelve la instrucción desensamblada creada.

5.3.12.1. La clase `DisassembledInstruction`

Del mismo modo que el proceso de desensamblado es similar al de decodificación, la clase `DisassembledInstruction`, definida en la tabla 5.36, guarda cierta similitud con la clase

DisassembledInstruction
- address():RData - set:Settings - op:List<Operand> - size:int - iType:InstructionType - breakpoint:boolean - isFaulty:boolean - WRONG_INSTRUCTION:String = "INSTRUCCIÓN NO VÁLIDA" {final}
+ DisassembledInstruction(address():RData, iType:InstructionType, op:List<Operand>, size:int, set:Settings, hasBreakpoint:boolean) + DisassembledInstruction(address():RData, hasBreakpoint:boolean) + toString():String + getInstructionString():String + getAddressString():String + getAddress():RData + hasBreakpoint():boolean + getSize():int

Tabla 5.36: Clase DisassembledInstruction.

InstructionData, en tanto que ambas son paquetes que guardan información relativa a una instrucción.

Como se puede observar, `DisassembledInstruction` tiene dos constructores. El primero, con más argumentos, es el que se utiliza cuando el proceso de desensamblado no ha encontrado errores y se ha podido aportar toda la información relativa a la instrucción. El segundo se utiliza cuando ha surgido algún error en el proceso de desensamblado y se debe generar una cadena que indique al usuario que en esa posición de memoria no hay codificada una instrucción reconocible.

Por lo demás, es una clase sencilla, en la que los roles del resto atributos y métodos son evidentes en sí mismos.

5.3.12.2. La clase Disassembler

Esta clase, definida en la tabla 5.37, implementa el algoritmo de desensamblado detallado al inicio del módulo.

El único método visible de esta clase, `disassembleInstructions()`, no genera una única instrucción, sino una lista de ellas. El proceso de desensamblado individual según el algoritmo mostrado se lleva a cabo en el método `disassemble()`. Éste, a su vez, se apoya en `incrementIndexClone()` para el aumento del puntero de lectura y su consiguiente comprobación de punto de ruptura, y en `wrongInstruction()` para generar la `DisassembledInstruction` fallida con el mensaje de error.

Disassembler
- arc:Architecture
- set:Settings
+ Disassembler(arc:Architecture, set:Settings)
+ disassembleInstructions(index:RData, n:int):List<DisassembledInstruction>
- disassemble(index:RWData):DisassembledInstruction
- incrementIndexClone(indexClone:RWData):RData
- wrongInstruction(index:RWData, breakpoint:boolean):DisassembledInstruction

Tabla 5.37: Clase Disassembler.

5.3.12.3. Diferencias respecto a ENS2001

Al igual que sucedía con el decodificador, ENS2001 opera directamente a nivel de bit sobre las palabras leídas de la memoria en vez de implementar una capa de abstracción intermedia. Por otro lado, en este caso no carga al simulador con la responsabilidad del desensamblado, como sí hacía con la del decodificado (mayormente porque el desensamblado no es un proceso llevado a cabo en tiempo de simulación); pero sí se accede a este proceso directamente desde la aplicación.

ENS2025 encapsula el desensamblador, igual que el resto de módulos de la lógica principal, en el módulo de la unidad de control, haciendo su funcionamiento interno totalmente transparente a la aplicación que lo manipula.

5.3.13. Módulo del ensamblador

Este módulo se encarga de transformar un fichero con sentencias en lenguaje ensamblador en un conjunto de palabras en código máquina ejecutable por el simulador en forma de instrucciones. Aunque técnicamente es un módulo contenido dentro de la unidad de control, por su tamaño y complejidad sería catalogable como un subsistema, ya que tiene su propio subconjunto de módulos, cada uno con distintas funciones e integrado por varias clases. Esta división inicial se puede ver en la figura 5.10.

Como ya se ha visto al inicio de este capítulo en el apartado de “Diseño heredado de ENS2001”, el formato de la gran mayoría de las sentencias del lenguaje ensamblador del estándar IEEE 694 sigue una estructura gramatical lineal y autocontenida en una sola línea, con un número finito y reducido de elementos. Las dos únicas excepciones a esta estructura son la directiva de ensamblado DATA, que pueden contener un número variable de elementos (tanto enteros como cadenas de texto), y las expresiones aritméticas usadas para calcular el valor numérico que se da a las directivas ORG, EQU y RES, que tienen una estructura recursiva.

Además, la existencia de etiquetas y la posibilidad de que se haga referencia en una sentencia a etiquetas aún no definidas hace imposible resolver las referencias según se van encontrando. Por tanto, es necesaria la existencia de una tabla de símbolos para almacenar las etiquetas que se van encontrando y su posición, y una tabla de referencias para poder resolverlas, convirtiendo el ensamblador en un compilador de dos pasadas, en el que durante la primera pasada se procesa el código y se llena la tabla de referencias con las instrucciones con referencia a etiqueta, y

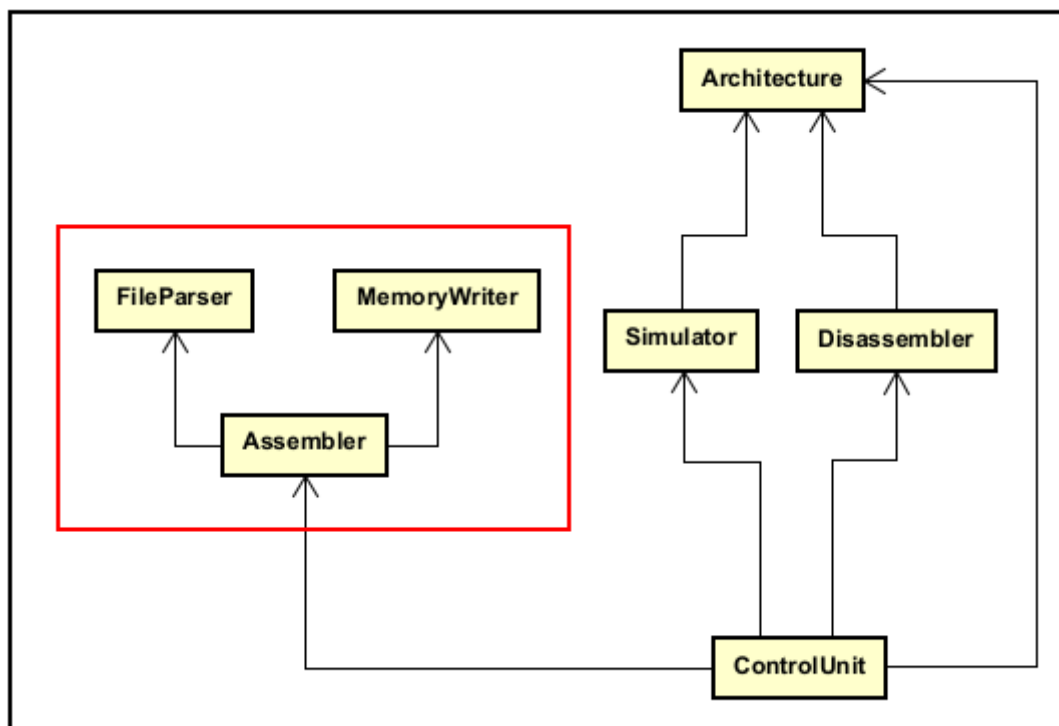


Figura 5.10: Vista de la unidad de control con el ensamblador expandido. El módulo de utilidad no representa un componente real, por lo que no se incluye.

durante la segunda pasada se recorre la tabla de referencias y se usa la tabla de símbolos para completar las instrucciones que quedaron incompletas en la primera.

Cabe recordar que, como excepción a la regla de sentencias contenidas en una línea, el lenguaje ensamblador permite declarar etiquetas seguidas de saltos de línea, por lo que es posible procesar una etiqueta y que ésta quede “en espera” de que termine la sentencia de la que forma parte. Por lo tanto, hay que tener en cuenta este elemento a la hora de procesar una sentencia.

Partiendo de estas premisas, el comportamiento esperado por este módulo al procesar una sentencia durante la primera pasada está definido por el siguiente algoritmo:

1. Se obtiene un mnemónico del texto de la sentencia y se valida su formato.
2. Se comprueba si se trata de una directiva de ensamblador. Si lo es:
 - Se comprueba si hay una etiqueta “en espera”. Si es así, se inserta en la tabla de símbolos con el valor actual del puntero de escritura antes de ejecutar la directiva.
 - Se siguen las instrucciones de la directiva (reservar memoria, insertar datos...).
3. Si no, se comprueba si se trata de una instrucción. Si lo es:
 - Se comprueba si hay una etiqueta “en espera”. Si es así, se inserta en la tabla de símbolos con el valor actual del puntero de escritura para que apunte a la primera palabra de la instrucción.
 - Se analiza el mnemónico obtenido y se clasifica por número de operandos.
 - Se obtienen la o las cadenas de texto pertenecientes a los operandos.

- Se extrae el modo de direccionamiento del operando y el mnemónico de la etiqueta si el operando contiene una referencia. Si es este último caso, se crea una referencia en la tabla pertinente.
 - Se valida la instrucción (combinación permitida de operación y modos de direccionamiento).
 - Se codifica la instrucción y se escribe en la memoria. Si la instrucción contenía una referencia, la sección de la palabra de la memoria en la que se escribiría queda temporalmente inicializada a cero.
4. Si no es ni directiva ni instrucción, se comprueba si se trata de una etiqueta válida:
- Se comprueba que el mnemónico va seguido de dos puntos (':').
 - Se comprueba que no exista dicho mnemónico ya en la tabla de símbolos.
 - Se comprueba que no exista ya una etiqueta “en espera”. El etiquetado múltiple (varias etiquetas en una misma dirección) no está permitido.
 - Si todas las comprobaciones son exitosas, la etiqueta queda “en espera”. Se registrará en la tabla de símbolos con el valor actual del puntero de escritura cuando se procese la siguiente directiva o instrucción (ya sea en la misma línea o en una posterior). En caso contrario, se genera un error.
5. Si tampoco es una etiqueta válida, se produce un error.

Este proceso se repite hasta haber leído todo el fichero, o hasta haber procesado la directiva `END`.

Como se puede ver, en cada lectura de sentencia se llevan a cabo casi por completo ambas etapas generales del proceso de compilación (análisis y síntesis) [2], dejando la última parte de síntesis para la “media pasada” final, donde se gestiona el contenido de la tabla de referencias.

Entrando más en detalle, se puede observar que, efectivamente, en cada sentencia se realizan el análisis léxico (comprobación de los elementos o tokens que componen la cadena) y sintáctico (comprobación de una estructuración concreta de los tokens) a medida que progresa la lectura de la cadena y, al terminar de leer la sentencia, tienen lugar el análisis semántico (validación de la instrucción) y parte de la generación de código (primera pasada de escritura, a falta de las referencias), ignorando la generación de código intermedio y la optimización, que no suceden en el ensamblador.

Podría argumentarse, si se considera código generado únicamente el código final tras todas las adiciones, que en la primera pasada tiene lugar la generación de código intermedio y en la media pasada final la generación de código propiamente dicha. Esta aproximación no sería del todo correcta, ya que el código generado en la primera pasada no es un código intermedio, sino lenguaje máquina en el que no se ha escrito aún la parte concerniente a las referencias. Dicho código es ejecutable por el simulador, simplemente los operandos que manejasen una referencia usarían un valor de salto o desplazamiento igual a cero. Por lo tanto, la primera aproximación (fase de generación de código dividida en dos pasadas) es la más correcta para definir el flujo de compilación del módulo ensamblador.

El proceso de implementación habitual en estos casos es definir una gramática y construir un autómata finito determinista (AFD) con sus analizadores léxico, sintáctico y semántico (generalmente mediante herramientas como JFlex y JCup). Sin embargo, la sencillez de la estructura del lenguaje ensamblador permite como alternativa elaborar un sistema de tokenización y procesamiento a medida sin mediar el diseño formal de la máquina de estados de un AFD. Esto otorga un control más directo sobre el código y su flujo, y mantiene la estructura de código del sistema dependiente en exclusiva de librerías oficiales. Se evita también la generación de código boilerplate y el uso de archivos .jar externos, reduciendo la fricción con el control de versiones de dichas herramientas y el propio sistema. El precio a pagar por este enfoque es el traslado del tiempo y el esfuerzo dedicados a la especificación formal de la gramática y su autómata asociado al diseño e implementación directa de los algoritmos de tokenización y parsing.

Se pueden distinguir los siguientes elementos necesarios para el funcionamiento de este módulo:

- **Procesador de texto:** Encargado de consumir el fichero de texto, separándolo en líneas que contendrán las sentencias a procesar. Se encarga también de proporcionar al ensamblador los tokens que solicite. Contiene un puntero de línea y otro de carácter, que le permiten procesar los caracteres del texto uno a uno e identificar caracteres inesperados, lo que proporciona granularidad a los errores. La estructura gramaticalmente recursiva de las expresiones difiere de la estructura lineal del resto del código, y requiere un subsistema de análisis aparte. Genera estructuras que contienen toda la información pertinente para la codificación de una instrucción.
- **Sistema de codificación y escritura:** Encargado de recibir las estructuras generadas por el procesador de texto, codificarlas en lenguaje máquina y escribirlas en la memoria. Contiene un puntero de memoria para mantener la referencia de escritura. Al final de la primera pasada, recibe las tablas de símbolos y referencias y se encarga de insertar las referencias en el código.
- **Gestor del ensamblado:** Dirige el proceso de ensamblado, gestionando los elementos anteriores y actuando de puente de comunicación entre uno y otro. Gestiona las tablas de símbolos y referencias.

Estos elementos se diseñan e implementan en los submódulos que se detallan a continuación.

5.3.13.1. Submódulo de utilidad

Este módulo contiene las clases que implementan los elementos comunes al resto de submódulos. Esto es, las referencias y los “planos” para codificar en lenguaje máquina operandos e instrucciones en base a los atributos contenidos. Por lo tanto, no representa un componente propiamente dicho del módulo ensamblador.

5.3.13.1.1 La clase Reference

Esta clase, definida en la tabla 5.38, contiene la información necesaria para describir una referencia, con vistas a su futuro procesamiento por el módulo escritor en la segunda pasada. Las

referencias pueden ser absolutas, en las que el valor contenido es la dirección de la etiqueta, o relativas a PC, donde el valor contenido es el desplazamiento que debe aplicarse al PC para situarse en la dirección de la etiqueta. También pueden ocupar una palabra entera o un segmento de la misma, con un atributo que indica en cuál de las dos mitades de la palabra está ubicada.

Reference
- memPos:RData - label:String - isRelativeToPC:boolean - opSegment:int
+ Reference(memPos:RData, label:String, isRelativeToPC:boolean) + Reference(memPos:RData, label:String, isRelativeToPC:boolean, opSegment:int) + getPointer():RData + getLabel():String + isRelativeToPC():boolean + getOpSegment():int

Tabla 5.38: Clase Reference.

La clase contiene dos constructores. El primero, o “abreviado”, es el que se usa habitualmente (por la prevalencia de referencias de palabra completa), e inicia el atributo `opSegment` a `-1`. El segundo se encarga de inicializar `opSegment` a un valor correspondiente a la mitad de palabra que aloja la referencia.

5.3.13.1.2 La clase `OperandBlueprint`

Esta clase, definida en la tabla 5.39, contiene la información necesaria para que el módulo escritor pueda codificar correctamente un operando. Funciona de manera similar a la clase `Operand` del módulo de instrucciones, con diferentes constructores según la configuración del contenido, y siendo el atributo relacionado con el modo de direccionamiento el que marca qué información se almacena y cómo se procesa. Incluye, además, un atributo para la etiqueta de la referencia, si la contiene, y un método para insertar dicha referencia.

5.3.13.1.3 La clase `InstructionBlueprint`

Esta clase, definida en la tabla 5.40, contiene la información necesaria para que el módulo escritor pueda codificar correctamente una instrucción. Si `OperandBlueprint` era el análogo de `Operand` en el módulo ensamblador, `InstructionBlueprint` es el análogo de `InstructionData`.

5.3.13.2. Submódulo del procesador de texto

Este módulo se encarga del procesado carácter a carácter cada línea procesada del fichero, agrupando grupos de caracteres en tokens que representan los mnemónicos de las diferentes instrucciones, operandos y etiquetas. También gestiona la instanciación de clases que sirven de

OperandBlueprint
- addr:AddressingMode - reg:RegisterID - data:RData - reference:String
+ OperandBlueprint(addr:AddressingMode, data:RData) + OperandBlueprint(addr:AddressingMode, reg:RegisterID) + OperandBlueprint(addr:AddressingMode, reg:RegisterID, data:RData) + void(rName:String):void + getAddressingMode():AddressingMode + getRegister():RegisterID + getData():RData + getReference():String

Tabla 5.39: Clase OperandBlueprint.

InstructionBlueprint
- it:InstructionType - opList:List<OperandBlueprint>
+ InstructionBlueprint(it:InstructionType, opList:List<OperandBlueprint>) + getInstructionType():InstructionType + getOps():List<OperandBlueprint>

Tabla 5.40: Clase InstructionBlueprint.

bloques de construcción para que el módulo de escritura los transforme directamente en código máquina. Los componentes de este módulo se pueden observar en la figura 5.11

5.3.13.2.1 La clase FileData

Esta clase, definida en la tabla 5.41, almacena las líneas del fichero de código y funciona a modo de gestor del puntero de lectura para las distintas clases del procesador de texto, ofreciendo métodos de consulta y tokenización. Su tiempo de vida es dependiente del fichero: cada instancia de **FileData** se crea para gestionar un único fichero, por lo que la capa de abstracción superior encargada de manejar esta clase deberá crear una nueva instancia y abandonar la actual si desea cambiar de fichero, o recrear otra instancia con el mismo fichero si cierra la actual.

Los atributos **lines** y **currentLine** almacenan todas las líneas leídas del fichero y la línea de lectura actual, respectivamente. El atributo **listIndex** indica el número de línea actual, y **stringIndex** la posición sobre la que está el puntero de lectura dentro de la línea actual. La constante **HEXLETTERS** contiene las letras de la A a la F, ambas incluidas, tanto en mayúscula como en minúscula.

Dado que los comentarios son monolínea y abarcan desde que se designa su comienzo hasta el final de la línea en la que se encuentran, su eliminación es trivial: tanto la primera línea del fichero cargada por el constructor como cada nueva línea cargada mediante **newLine()** son preprocesadas para descartar cualquier carácter a partir del que inicia un comentario.

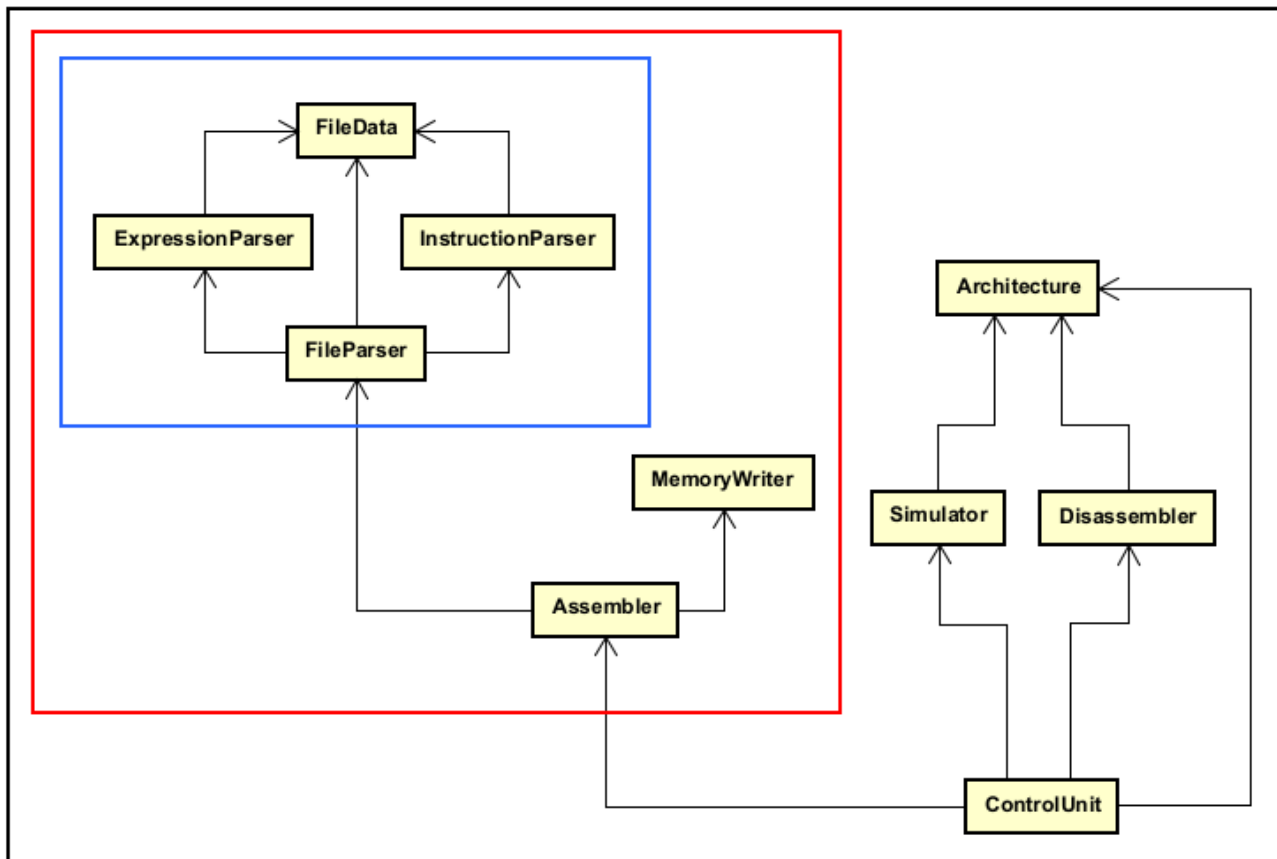


Figura 5.11: Vista de la unidad de control con el procesador de texto expandido dentro del ensamblador.

Cuenta con una serie de métodos dedicados a navegar el texto de cada línea. Varios de estos métodos aceptan un atributo `boolean` que les indica si deben desplazarse ignorando los caracteres en blanco hasta encontrar uno válido, o si deben tomar el primer carácter que se encuentren, en blanco o no.

- **Método `currentChar()`:** Devuelve el carácter apuntado por el índice de cadena, o `'\0'` si se ha llegado al fin de la cadena.
- **Método `peekChar()`:** Consulta un carácter sin avanzar el puntero de lectura. Esencialmente, una envoltura de `currentChar()` que incluye la opción de consumir caracteres en blanco.
- **Método `consumeChar()`:** Consulta un carácter y avanzar el puntero de lectura, consumiéndolo. Funciona llamando primero a `peekChar()`. Si el resultado es distinto de `'\0'`, incrementa el índice de cadena. Devuelve el resultado obtenido.
- **Método `seekChar()`:** Busca una coincidencia de carácter específico. Llama a `consumeChar()` y compara el resultado con el `char` recibido como primer argumento.
- **Método `seekEOS()`:** Valida que no hay más caracteres (aparte de posibles caracteres en blanco) hasta el final de la cadena actual. Desplaza el índice de cadena ignorando caracteres de espacio hasta llegar al final o encontrarse un carácter.

FileData
<pre> - lines:List<String> - currentLine:String - stringIndex:int - listIndex:int - HEX_LETTERS:Set<Character> </pre>
<pre> + FileData(fname:String) + isEmpty():boolean + newLine():boolean + increaseIndex():void + getData(ignoreSpaces:boolean):RData + processDecimal(ignoreSpaces:boolean):String + processHex(ignoreSpaces:boolean):String + isHexChar(c:char):boolean + getMnemonic(ignoreSpaces:boolean):String + seekEOS():boolean + seekChar(c:char, ignoreSpaces:boolean):boolean + consumeChar(ignoreSpaces:boolean):char + peekChar(ignoreSpaces:boolean):char - currentChar():char + getLineNumber():int + isEOS():boolean </pre>

Tabla 5.41: Clase FileData.

Por último, los dos métodos más importantes son `getMnemonic()`, que captura y devuelve un mnemónico hasta su último carácter válido, y `getData()`, que hace lo propio con valores numéricos enteros (decimales con y sin signo y hexadecimales sin signo).

5.3.13.2.2 La clase InstructionParser

Esta clase, definida en la tabla 5.42, se ocupa de procesar la lectura de una instrucción en una sentencia de ensamblador, con vistas a convertirla en un `InstructionBlueprint` que pueda ser utilizado por el módulo escritor.

De entre todos los métodos que posee, los más relevantes son los de procesamiento de operandos, que conviene detallar para entender el proceso detrás de cada uno.

- **Método `parseOperand()`:** Analiza el primer carácter de la cadena de texto de un operando para determinar a qué estructura de modo de direccionamiento pertenece, llamando al método correspondiente para interpretarlo correctamente, o generando un error si se encuentra un carácter no válido.
- **Método `parseMemory()`:** Procesa un operando con direccionamiento directo a memoria. Puesto que este modo de direccionamiento admite tanto literales numéricos como etiquetas, se comprueba con el primer carácter a cuál de los dos tipos pertenece, creando el `OperandBlueprint` y marcándolo como referencia si procede.

InstructionParser
<pre> - fd:FileData + InstructionParser() - loadFileData(fd:FileData):void - close():void + parseInstruction(it:InstructionType,):InstructionBlueprint - validateInstruction(it:InstructionType, ops:List<OperandBlueprint>):void - parseOperand():OperandBlueprint - parseMemory():OperandBlueprint - parseLiteralOrRelative():OperandBlueprint - parseRelativeToIndex(val:RData):OperandBlueprint - parseRegister():OperandBlueprint - parseIndirect():OperandBlueprint - parseRelativeToPC():OperandBlueprint </pre>

Tabla 5.42: Clase InstructionParser.

- **Método parseLiteralOrRelative():** Procesa un operando con direccionamiento inmediato o relativo a índice, puesto que ambos direccionamientos comienzan con el carácter ‘#’. Para distinguirlos, se comprueba si, tras el valor numérico o etiqueta después de ‘#’, el primer carácter leído es ‘[’. En ese caso, se trata de direccionamiento relativo y se llama a `parseRelativeToIndex()`; de lo contrario, es simplemente direccionamiento inmediato y se procesa en este mismo método.
- **Método parseRelativeToIndex():** Procesa un operando con direccionamiento relativo a índice. Comprueba no solamente que se cumple el formato ‘#’ + desplazamiento + “[.IX|Y]”, sino que dicho valor de desplazamiento está dentro del rango de valores permitido. Se comprueba también que el mnemónico de registro es exclusivamente “IX” o “IY”. Genera un error si se incumple alguna de estas condiciones.
- **Método parseRegister():** Procesa un operando con direccionamiento directo a registro. Genera un error si el mnemónico de registro no es válido.
- **Método parseIndirect():** Procesa un operando con direccionamiento indirecto. Genera un error si el mnemónico de registro no es válido.
- **Método parseRelativeToPC():** Procesa un operando con direccionamiento relativo a PC. Al igual que `parseRelativeToIndex()`, comprueba si el valor de desplazamiento es válido, y genera un error en caso contrario.

5.3.13.2.3 El procesador de expresiones aritméticas y la clase ExpressionParser

La estructura gramaticalmente recursiva de las expresiones no permite la aproximación lineal y finita que se puede aplicar en el resto del código. Para resolver esto, las expresiones tienen su propio procesador. Este procesador aplica en primer lugar el algoritmo shunting yard [19], que, a medida que va leyendo la expresión, la convierte de notación infija a notación postfija, y después realiza los cálculos pertinentes mediante una calculadora de pila.

El procesador de expresiones requiere los siguientes elementos:

- Distinción entre tokens de valor y tokens de operador.
- Pila de operadores y pila de valores.
- Selector de la operación asociada a cada carácter de operador
- Sistema de precedencia entre los diferentes operadores.
- Variable temporal para almacenar el último tipo de token leído, con valor nulo para indicar que se está leyendo el primer token.
- Un contador de paréntesis abiertos. No es imprescindible, pero permite detección temprana de incongruencias.

Además, para este caso concreto, es necesario tener en cuenta las siguientes reglas y restricciones:

- Los valores pueden ser decimales con signo y hexadecimales.
- Los hexadecimales van precedidos de la cadena "0x".
- Los operadores que se deben implementar son suma, resta, multiplicación, división, módulo y el operador unario de cambio de signo.
- El cambio de signo comparte carácter con el de resta ('-'), por lo que es necesario diferenciar uno de otro según contexto.
- El operador de cambio de signo no puede aplicarse a valores hexadecimales, por lo que los tokens de valor deben diferenciar entre valores decimales y hexadecimales.
- El resultado de una operación siempre se considera un valor decimal aunque ambos operandos sean hexadecimales.
- No se permite la concatenación de un cierre y una apertura de paréntesis, de un valor y una apertura de paréntesis o de un cierre de paréntesis y un valor. Esto es, no existe operador de multiplicación implícito.
- El orden de precedencia es el siguiente: Contenido de paréntesis > cambio de signo > multiplicación, división y módulo > suma y resta.
- Entre operadores de igual precedencia, se prioriza la resolución por la izquierda.
- El primer token de la cadena solamente puede ser una apertura de paréntesis, un valor o un operador de cambio de signo.
- No se admiten los paréntesis sin valor. Es decir, no se admite una estructura "()".
- No se puede concatenar la lectura de dos operadores, con independencia del número de espacios que haya en medio. Se aplica lo mismo para los valores.

Estos conjuntos de elementos y reglas permiten modelar la clase que gestionará todo el proceso de parsing de las expresiones.

Es necesario también definir el comportamiento que debe seguir la clase para procesar las expresiones. En este caso, se sigue el algoritmo descrito a continuación:

1. Se lee la cadena carácter a carácter, agrupándolos para formar tokens de valor o tomándolos individualmente para formar tokens de operador según corresponda.
2. Cuando se lee un token de valor, y el último token leído es nulo (principio de la cadena) o es cualquier token menos un cierre de paréntesis, un valor decimal o un valor hexadecimal, se clasifica el token actual en decimal o hexadecimal y se introduce en la pila de valores. Cualquier otro caso genera un error.
3. Cuando se lee un token de operando, el modo de actuar dependerá del token que se acaba de leer y del último token leído antes que éste. Si el último token leído no tiene el valor esperado, se producirá un error. La relación de tokens leídos y su comportamiento se puede apreciar en la tabla 5.43.
4. Una vez procesada la cadena, se comprueba que el contador de paréntesis abiertos valga cero y que el último token leído sea un paréntesis cerrado o un valor decimal o hexadecimal. En caso de que no se cumpla alguna de estas condiciones, se produce un error.
5. Tras esta comprobación, se procede a vaciar la pila de operadores si aún quedasen elementos dentro, llamando repetidamente al procedimiento de ejecutar una operación hasta que la pila quede vacía.
6. Finalmente, debería quedar un único token de valor en su pila correspondiente. Se extrae y se usa para crear el objeto de datos que se envía de vuelta.

Por otro lado, el proceso de ejecución de una operación aritmética sigue este algoritmo:

1. Se comprueba que ninguna de las pilas está vacía.
2. Se extrae un elemento de la pila de operadores. En función del operador extraído, se actúa de distinta forma:
 - **Apertura de paréntesis:** Se descarta el operador leído.
 - **Cambio de signo:** Se extrae un token de la pila de valores. Si es hexadecimal, se producirá un error. En caso contrario, se resta a 0 el valor del token.
 - **Otros operadores:** Se extrae un token de la pila de valores. Se comprueba que la pila no ha quedado vacía (en cuyo caso se produce un error) y se extrae un segundo valor. Se aplica a ambos valores la operación asociada al operador.
3. Se comprueba que la operación aplicada sobre el o los operandos no produce un valor que exceda el rango de valores permitido.
4. Se crea un nuevo token de valor numérico (de tipo decimal) con el resultado de la operación y se almacena en la pila de valores.

Para implementar este algoritmo, el procesador de expresiones hace uso de las clases Token, ValueToken e InstructionParser, así como del tipo enumerado TokenType.

Nuevo token leído	Último token leído	Condición	Acción
'-'	NULL, ('	n/a	Insertar NEGACIÓN en la pila de operadores
'-'	'), VALOR	n/a	Insertar RESTA en la pila de operadores
VALOR	NULL, cualquiera excepto) o VALOR	n/a	Insertar VALOR en la pila de operadores
'('	NULL, cualquiera excepto) o VALOR	n/a	Insertar (en la pila de operadores, aumentar contador de aperturas
')'	'), VALOR	Contador de aperturas a cero	Error
')'	'), VALOR	Cima de la pila de operadores distinta de (	Bucle de ejecución de operación hasta que la cima de la pila de operadores sea (, disminuir contador de aperturas tras el bucle
Otro operador	'), VALOR	Pila de operadores vacía o cima de la pila con precedencia menor	Insertar el operador en la pila de operadores
Otro operador	'), VALOR	Cima de la pila de operadores con precedencia mayor o igual	Bucle de ejecución de operación hasta que la pila de operadores quede vacía o su cima tenga precedencia menor, insertar el operador en la pila tras el bucle

Tabla 5.43: Acciones a emprender por el procesador de expresiones en función del token actual y el último token leído.

El tipo enumerado TokenType

A diferencia del resto de tipos enumerados empleados en este sistema, que hacen un uso extenso de las funcionalidades que Java otorga a estos objetos, **TokenType** no necesita funcionalidades adicionales, y se limita a ser una lista de constantes asociadas a cada uno de los tokens que intervienen en el algoritmo de procesado. Por lo tanto, no hace falta adjuntar su diagrama y basta con enumerar sus valores:

- **L_PAR:** Paréntesis izquierdo (abierto).
- **R_PAR:** Paréntesis derecho (cerrado).
- **ADD:** Suma.
- **SUB:** Resta.
- **NEG:** Cambio de signo (operador unario).
- **MUL:** Multiplicación.
- **DIV:** División.
- **MOD:** Módulo.
- **DEC:** Valor numérico decimal.
- **HEX:** Valor numérico hexadecimal.

Las clases **Token** y **ValueToken**

La clase **Token**, definida en la tabla 5.44a, encapsula en su interior un atributo de tipo **TokenType**, con un constructor para su inicialización y un método getter. **ValueToken** sirve para representar específicamente a los tokens de valor numérico. Para ello, hereda de **Token**, incluyendo un atributo para almacenar su valor, un constructor para inicializar ese nuevo atributo, y un getter adicional para obtenerlo. Así, el procesador utilizará **Token** para almacenar los operadores (pues no requieren información más allá de qué tipo son) y **ValueToken** para los valores numéricos (pudiendo así distinguir si además son decimales o hexadecimales).

Token	ValueToken extends Token
- type:TokenType	- value:int
+ Token(type:TokenType)	+ ValueToken(type:TokenType, value:int)
+ getType():TokenType	+ getValue():int

(a) Clase Token. (b) Clase ValueToken.

Tabla 5.44: Clases **Token** y **ValueToken**.

La clase **ExpressionParser**

Esta clase, definida en la tabla 5.45, centraliza la lógica del procesador de expresiones. Mantiene una referencia al puntero de lectura de texto para tokenizar la cadena de la expresión, así como las pilas de valores y operadores. Implementa dos constantes consistentes en conjuntos de caracteres, **OP_CHARS** (operadores) y **PAR_CHARS** (paréntesis), que contienen los caracteres válidos en la expresión.

ExpressionParser
- fd:FileData
- opStack:Stack<Token>
- valStack:Stack<ValueToken>
- OP_CHARS:Set<Character> {final}
- PAR_CHARS:Set<Character> {final}
+ ExpressionParser()
+ loadFileData(fd:FileData):void
+ close():void
- isValidCharacter(c:char):boolean
- isOperator(c:char):boolean
- getOpFromChar(c:char):TokenType
+ parseExpression():RData
- applyOp():void
- validateOpStack():void
- validateValStack():void
- precedence(t:TokenType):int
- parseValueToken():ValueToken

Tabla 5.45: Clase **ExpressionParser**.

A continuación, se describen los métodos más relevantes de la clase:

- **Método `parseExpression()`:** Implementa la lógica del procesador de expresiones descrita con anterioridad. Se apoya en el método `parseValueToken()` para obtener un token de valor cuando el carácter obtenido es un dígito, y en `applyOp()` para externalizar la lógica de la ejecución de operación.
- **Método `applyOp()`:** Implementa la lógica de ejecución de una operación, descrita tras el algoritmo de procesamiento de expresiones. Además de controlar que los valores del resultado no excedan el rango de representación permitido, controla eventos como la división o módulo entre cero y el desbordamiento (multiplicar o dividir el valor mínimo por -1 , por ejemplo).
- **Método `parseValueToken()`:** Procesa un valor numérico. A diferencia del método `getData()` de `FileData`, este método devuelve un `ValueToken`; además, solamente procesa el valor sin signo (incluso en base decimal), ya que el operador de cambio de signo se gestiona de manera separada en `parseExpression()`.
- **Métodos `validateOpStack()` y `validateValStack()`:** Comprueban si las respectivas pilas están vacías y, de estarlo, generan un error. Su propósito es simplemente reducir boilerplate, ya que estas comprobaciones suceden en varios puntos de `applyOp()` y casi todas tienen la misma consecuencia.

5.3.13.2.4 La clase `FileParser`

Esta clase, definida en la tabla 5.46, actúa de fachada (encapsulamiento y capa de abstracción) de todo el submódulo procesador de texto.

FileParser
- fd:FileData - ip:InstructionParser - xp:ExpressionParser
+ FileParser() + loadFile(fileName:String):void + close():void + parseInstruction(it:InstructionType):InstructionBlueprint + getData(ignoreSpaces:boolean):RData + getMnemonic(ignoreSpaces:boolean):String + parseExpression():RData + seekEOS():boolean + seekChar(c:char, ignoreSpaces:boolean):boolean + peekChar(ignoreSpaces:boolean):char + consumeChar(ignoreSpaces:boolean):char + newLine():boolean + getLineNumber():int + checkForFileLoaded():void

Tabla 5.46: Clase `FileParser`.

La clase gestiona, además, las instancias de `FileData`: se encarga de crear nuevas instancias con cada fichero cargado en el ensamblador, así como de desvincular la referencia a las mismas e

indicar a las otras clases internas que hagan lo mismo una vez que el ensamblado ha terminado, “cerrando” así el procesador para ese fichero concreto.

5.3.13.3. Submódulo de escritura

Este módulo es el responsable de recibir las instancias de `InstructionBlueprint`, convertirlas en palabras de ensamblador legibles durante el proceso de codificación e insertarlas en la memoria. Del mismo modo que el módulo del procesador de texto tenía su propio puntero de lectura, este módulo tiene su propio puntero de escritura que indica dónde se introducirá el código. También maneja el codificador y el escritor, cuyas funciones van implícitas en el nombre. Los componentes de este módulo se pueden ver en la figura 5.12.

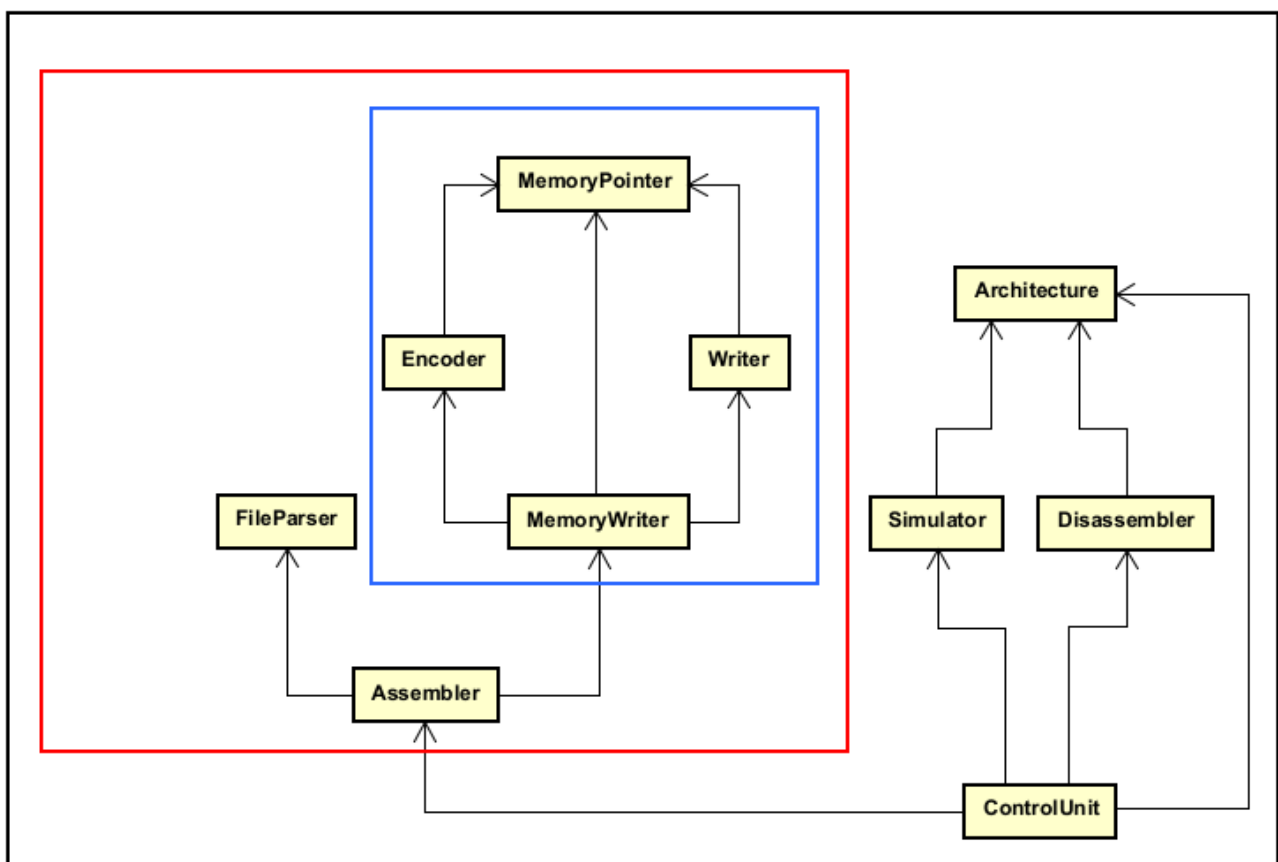


Figura 5.12: Vista de la unidad de control con el codificador-escriptor expandido dentro del ensamblador.

5.3.13.3.1 La clase `MemoryPointer`

Esta clase, definida en la tabla 5.47, gestiona el puntero de escritura. Su importancia no se limita a indicar en qué parte de la memoria se van a insertar las instrucciones una vez codificadas, sino que se usa para establecer las referencias que se resolverán en la segunda pasada del ensamblador. Además del indicador de la dirección, contiene un atributo que indica si el puntero ha sobrepasado los límites de la memoria.

MemoryPointer
- pointer:RData - overflow:boolean
+ MemoryPointer() + getPointer():RData + setPointer(ind:RData):void + offsetPointer(offset:RData):void + increasePointer():void + getOverflow():boolean + resetOverflow():void + resetPointer():void

Tabla 5.47: Clase MemoryPointer.

5.3.13.3.2 La clase EncodedInstruction

Las instancias de esta clase, definida en la tabla 5.48, son generadas durante el proceso de decodificación y contienen los datos puros a insertar en memoria durante el proceso de escritura. Contiene un atributo llamado memBlock, cuyo contenido conjunto, aunque esté organizado en forma de lista de palabras, es esencialmente un bloque de memoria de tamaño variable cargado con código máquina.

EncodedInstruction
- memBlock:List<RData>
+ EncodedInstruction() + addWord(word:RData):void + getBlockSize():int + getWord(index:int):RData

Tabla 5.48: Clase EncodedInstruction.

5.3.13.3.3 La clase Encoder

El proceso de codificación de instrucciones y asignación de referencias en la primera pasada y el de resolución de referencias en la segunda pasada son los componentes más importantes del proceso de ensamblado. El objetivo de la clase **Encoder** es el primero de los dos, que implica convertir los “bloques de construcción” suministrados en forma de un objeto **InstructionBlueprint** en un bloque de código máquina listo para insertar en la memoria.

A continuación, se describe el algoritmo de codificación empleado:

1. Se crea una máscara con el código de instrucción en la posición adecuada y se aplica un OR a la primera palabra para insertarlo.
2. Se analiza el tamaño de la lista de operandos.
3. Si no tiene operandos:

- (I) Se añade al bloque de memoria la primera (y única) palabra con el código de instrucción y se termina el proceso.

4. Si tiene un operando:

- (I) Se aplica mediante OR otra máscara a la primera palabra, esta vez conteniendo el código de direccionamiento del primer operando en su posición correspondiente.
- (II) Se añade la primera palabra al bloque de memoria.
- (III) Se comprueba si el primer operando contiene una referencia a etiqueta. Si es así, se incluye la referencia en la tabla de referencias y se deja la segunda palabra temporalmente vacía.
- (IV) En caso contrario, se codifica el valor del primer operando en la segunda palabra.
- (V) Sea cual sea el resultado, se añade al bloque de memoria la segunda palabra y se termina el proceso.

5. Si tiene dos operandos:

- (I) Se aplican mediante OR dos máscaras a la primera palabra, cada una con el código de direccionamiento de uno de los operandos en su posición correspondiente.
- (II) Se añade la primera palabra al bloque de memoria.
- (III) Se comprueba si cada operando utiliza una palabra o si ambos operandos comparten una.
- (IV) Si se usa una palabra para cada uno, cada operando será codificado en una palabra si no posee referencia a etiqueta. De poseerla, se incluirá en la tabla de referencias, dejando su respectiva palabra temporalmente vacía. Se añaden ambas palabras al bloque de memoria.
- (V) Si se usa una palabra para ambos, cada operando será codificado en una mitad de la palabra si no posee referencia a etiqueta. De poseerla, se incluirá en la tabla de referencias, dejando su mitad respectiva temporalmente vacía. Se añade la palabra al bloque de memoria.

Una vez detallado el algoritmo, se puede describir fácilmente la clase mediante la tabla 5.49.

Encoder
- mp:MemoryPointer
+ Encoder(mp:MemoryPointer)
+ encodeInstruction(ib:InstructionBlueprint, refList:List<Reference>):EncodedInstruction
- checkReference(op:OperandBlueprint, opNumber:int):Reference
- checkUnifiedReference(op:OperandBlueprint, opNumber:int):Reference
- encodeOperand(op:OperandBlueprint, shift:RData, opMask:RData):RData

Tabla 5.49: Clase Encoder.

Como se indica en el algoritmo, durante el proceso de codificación tiene también lugar la asignación inicial de referencias. La clase **Encoder** emplea para ello el método `checkReference()`, utilizado en la mayoría de situaciones, y el método `checkUnifiedReference()`, utilizado cuando se la referencia está situada en una instrucción de dos operandos que comparten una única palabra.

5.3.13.3.4 La clase Writer

Esta clase, además de la escritura en memoria de las instrucciones codificadas, lleva a cabo el proceso de resolución de referencias mencionado en el apartado anterior. Puesto que el decodificador deja vacía la parte de la palabra donde debe ir la referencia, este proceso consiste en construir una máscara de bits para integrar la información de salto o desplazamiento en la palabra sin detrimento de su contenido. El algoritmo consiste en el siguiente proceso:

1. Se extrae una referencia de la tabla de referencias, y de ella se extrae el mnemónico de su etiqueta.
2. Se comprueba si el mnemónico está en la tabla de símbolos. Si no es así, se produce un error.
3. Se obtiene de la tabla de símbolos la dirección en la que se declara la etiqueta.
4. Se comprueba si la referencia ocupa solamente la mitad de la palabra. Si es así, se trata de un valor de salto relativo a índice que usa la dirección de la etiqueta como salto.
 - (I) Se valida que dicho salto está dentro del rango permitido, produciéndose un error si no lo está.
 - (II) Se aplica un desplazamiento y una máscara a la dirección de la etiqueta de acuerdo a la mitad en la que se encuentra, obteniendo el valor final de la máscara de bits buscada.
5. Se obtiene de la referencia extraída de la tabla la dirección en la que se encuentra la palabra que la contiene.
6. Se comprueba si la referencia es relativa al PC. Si lo es, la dirección de la referencia contendrá el valor de la primera posición del PC tras ejecutar la instrucción que la contiene, y no la dirección del operando.
 - (I) Se resta a la dirección de la etiqueta el valor de la dirección de referencia para obtener el valor de salto, y se valida que esté en el rango permitido, produciéndose un error si no lo está.
 - (II) Se reduce en 1 el valor de la dirección de la referencia para que apunte correctamente al operando que la contiene.
 - (III) Se aplica al valor de salto el desplazamiento y la máscara necesarios para situarlo en la primera mitad de la palabra, como corresponde a un primer operando con este modo de direccionamiento. Se obtiene así el valor final de la máscara de bits buscada.
7. Si la referencia no ocupa solamente media palabra ni se trata de una referencia relativa al PC, la máscara de bits a aplicar al final es directamente el valor de la dirección de la etiqueta.
8. Se extrae de la memoria la palabra apuntada por la dirección de la referencia.
9. Se aplica mediante OR la máscara de bits obtenida durante el proceso.

10. Se inserta de nuevo en la memoria la palabra, ahora cargada correctamente con la referencia.

El proceso se repite hasta que se termina de recorrer la tabla de referencias.

Esta clase se define mediante el diagrama de la tabla 5.50.

Writer
- mem:Memory - mp:MemoryPointer - tempCodeStart:RData - tempCodeEnd:RData - set:Settings
+ Writer(mp:MemoryPointer, set:Settings) + writeInstruction(ei:EncodedInstruction):void + writeReferences(refList:List<Reference>, labelMap:Map<String, RData>):void + writeData(data:RData):void + setReservedMemory(res:RData):void + newMemory():void + getMemory():Memory + close():void + commitZoneBoundaries():void

Tabla 5.50: Clase Writer.

Además del `MemoryPointer` para gestionar el puntero de escritura, esta clase contiene un objeto `Memory` y dos `RData` para escribir de manera provisional las instrucciones que se van procesando, así como para guardar simulaciones de las direcciones de inicio y fin del código, que van expandiéndose en función de la cantidad de instrucciones introducidas. De esta manera, si surge un error durante el proceso de ensamblado, no se sobrescribe la memoria real del sistema, quedando a salvo el último programa cargado hasta que se procese entero uno nuevo de forma satisfactoria. En ese momento, el módulo ensamblador solicita que se apliquen los cambios a las direcciones de código y pila (`commitZoneBoundaries()`) y que se devuelva el objeto `Memory` cargado con el programa codificado (`getMemory()`).

Contiene además los métodos `writeData()` y `setReservedMemory()`, que llevan a cabo las tareas designadas por las directivas `DATA` (inserción de un dato en el punteo de escritura) y `RES` (reserva de memoria sin inicializar; esencialmente, desplazar el puntero una cantidad determinada).

5.3.13.3.5 La clase MemoryWriter

Del mismo modo que `FileParser` actúa de fachada para las clases del módulo procesador de texto, `MemoryWriter`, definida en la tabla 5.51, actúa de fachada para las del módulo de codificación y escritura.

MemoryWriter
- mp:MemoryPointer - writer:Writer - encoder:Encoder
+ MemoryWriter(set:Settings) + reset():void + resetPointer():void + getPointer():MemoryPointer + setPointer(ind:RData):void + resetPointerOverflow():void + getPointerData():RData + encodeAndWrite(ib:InstructionBlueprint, refList:List<Reference>):void + writeData(data:RData):void + setReservedMemory(data:RData):void + commitZoneBoundaries():void + getMemory():Memory + writeReferences(refList:List<Reference>, labelMap:Map<String, RData>):void + close():void

Tabla 5.51: Clase MemoryWriter.

5.3.13.4. Submódulo de gestión del ensamblado

La gestión del ensamblado coordina los procesos descritos en los submódulos anteriores, encargándose también de atender a las directivas del ensamblador.

5.3.13.4.1 El tipo enumerado DirectiveType

La función de este tipo, definido en la tabla 5.52, es proveer de un modo rápido de relacionar los mnemónicos leídos del fichero con el identificador de su directiva correspondiente.

«enumeration» DirectiveType
- name:String {final} - NAME_MAP:Map<String, DirectiveType> {final}
- DirectiveType(name:String) + getFromName(name:String):DirectiveType

Tabla 5.52: Tipo enumerado DirectiveType.

5.3.13.4.2 La clase Assembler

Tras conocer todos y cada uno de los algoritmos y entidades empleados en el módulo ensamblador, es momento de definir la clase que coordina todo el proceso. Sus atributos y métodos se pueden ver en la tabla ??.

Assembler
- waitingLabel:String - labelMap:Map<String, RData> - refList:List<Reference> - endProcessing:boolean - fp:FileParser - mw:MemoryWriter - io:IOInterface - ESCAPE_MAP:Map<Character, Character>{final}
+ Assembler(set:Settings, io:IOInterface) - loadFile(fileName:String):void - close():void + processFile(fileName:String):Memory - manageLabel(value:RData):void - processDirective(dir:DirectiveType):void - processDirEND():void - processDirEQU():void - processDirORG():void - processDirRES():void - processDirDATA():void - processDataString():void - processLabel(label:String):boolean - processInstruction(it:InstructionType):void - isAllCase(s:String):boolean

Tabla 5.53: Clase Assembler

Los atributos `labelMap` y `refList` corresponden a las tablas de símbolos y referencias, respectivamente. `ESCAPE_MAP` es un mapa que contiene la relación de los caracteres escapables en las cadenas de la directiva `DATA` con el carácter que representan (por ejemplo, el carácter ‘n’ está relacionado con el carácter ‘\n’), y `waitingLabel` almacena la etiqueta actualmente “en espera”. De este modo, cuando se está procesando una cadena de texto mediante la directiva `DATA`, si se encuentra un carácter ‘\’ se comprueba el siguiente en este mapa (por ejemplo, ‘n’), y se devuelve el carácter escapado completo para ser insertado en lugar de ambos por separado (‘\n’ en lugar de ‘\’ + ‘n’).

Respecto a los métodos, conviene mencionar, aunque pueda resultar obvio, que `processFile()` tiene el conjunto de las responsabilidades detalladas hasta ahora: implementa el algoritmo de ensamblado detallado al inicio del módulo, seguido de la escritura de referencias, la confirmación de las direcciones de inicio y fin de las zonas de código y pila, y el cierre del ciclo de vida de los punteros abiertos, para finalmente devolver el objeto `Memory` cargado con el código máquina.

5.3.13.5. Diferencias respecto a ENS2001

Dada la aproximación totalmente divergente en diseño e implementación, no tiene sentido tratar de establecer las diferencias inherentes entre ambas, pues carecen de puntos en común que comparar.

Sin embargo, sí se menciona que, al estar el módulo ensamblador generado en C por las herramientas de generación automática de código empleadas, se determinó la necesidad de almacenar temporalmente el código máquina generado en un fichero binario con el contenido de todas las posiciones de memoria, que luego es leído por el sistema principal.

Si bien en ENS2025 no existe ese problema porque se genera un objeto **Memory** y se devuelve directamente, el problema que sufre ENS2001 no sería replicable en esta ocasión, ya que JFlex y JCup generan el código directamente en Java, y además son compatibles desde Java 8 en adelante. Sin embargo, sí pone el foco en la necesidad de depender de herramientas externas para resolver

Sin embargo, sí se han detectado ciertos errores en el proceso de ensamblado de ficheros de código de ENS2001. Estos errores, ya provengan de la definición de la gramática o de cómo el código que la rodea gestiona los datos una vez obtenidos, han sido subsanados en ENS2025.

El direccionamiento relativo a IX/IY/PC mediante etiqueta generaba un error de desbordamiento si la etiqueta estaba dentro del rango de desplazamiento pero para alcanzarla era necesario cruzar el umbral entre la última posición de memoria y la primera, y viceversa. Este comportamiento no era congruente con el mismo tipo de desplazamiento pero aplicado directamente con literales numéricos, que sí permitía cruzar el umbral. Puesto que la documentación original no menciona que deba ser una acción no permitida, se ha optado por igualar la conducta del desplazamiento mediante etiqueta a la del desplazamiento mediante valores enteros.

El direccionamiento relativo a PC mediante etiqueta generaba un error de valor no permitido si el valor asignado a la etiqueta generaba un desplazamiento superior al valor máximo con signo (por ejemplo, `label: EQU 200` y `BR $label`). Por definición, los valores de salto comprendidos entre el valor máximo con signo y el valor máximo sin signo se convierten internamente en desplazamiento negativo, como sí ocurre si se aplica ese desplazamiento mediante valor numérico directo (por ejemplo, `BR $200`). Dado que, además, esta restricción no se aplicaba al direccionamiento relativo a IX/IY, claramente se trataba de un comportamiento no deseado.

La directiva **RES** parece inicializar a 0 la memoria en ciertos casos, pese a la indicación tanto de la memoria del proyecto [1] como en la documentación del estándar IEEE 694 [4]. ENS2001 no solamente inicializa a cero una única posición reservada y deja el resto intactas, sino que tampoco lo hace de manera consistente consigo mismo. Si se aplica **RES 0** tras una instrucción, en vez de no reservar memoria, inicializa a 0 el bloque inmediatamente anterior al puntero de escritura cuando se llamó a la directiva; en cambio, si se aplica **RES 0** cuando el puntero de escritura está en 0x0000, no inicializa a 0 la posición 0xFFFF.

La directiva **RES** también presentaba inconsistencias a la hora de marcar las zonas de código. Si se encadenan dos directivas **RES** al inicio del fichero de código, la primera no se considera zona de código (y tampoco se reserva la totalidad del tamaño solicitado), pero la segunda sí. Puesto que se está reservando espacio para su uso y la finalidad de la zona de código es establecer una frontera arbitraria para determinar si la pila está invadiendo dicha frontera, se opta por determinar que todo el espacio reservado mediante **RES** es zona de código, aunque se declare al inicio.

ENS2001 ensambla exitosamente la cadena `DATA ""` (así como `DATA """"` y similares; también `DATA " a secas`), lo que provoca un comportamiento inesperado: se definen como zona de código

todas las posiciones de memoria desde que se declara la cadena hasta el final de la memoria. La cadena `DATA ""`, 2, por ejemplo, no escribe nunca su segundo elemento, y tampoco da un error de límite de memoria sobrepasado. También marca toda la memoria como zona de código al intentar procesar una cadena aparentemente bien construida: la cadena vacía `""`.

El uso de la barra invertida (`'\'`) como carácter individual de la cadena (es decir, la cadena `"\"`) genera una cadena con un carácter de valor 0 y otro más de fin de cadena. Sin embargo, la cadena `"\"` almacena un carácter de valor 92 (la propia barra), otro de valor 0, y otro 0 de fin de cadena. Sucesivas adiciones de barras invertidas a la cadena añaden más caracteres de valor 92, siempre cambiando el último por uno de valor 0.

Si se usa la barra invertida para intentar escapar caracteres no pensados para ello (por ejemplo, la cadena `"\a"`), almacena por duplicado el carácter, seguido del 0 de final de cadena (en el caso anterior, sería como almacenar la cadena `"aa"`).

Para solucionar estos problemas con la directiva `DATA`, se han añadido tanto la comilla doble como la barra invertida a los caracteres escapables. De esta forma, si se quiere insertar un carácter de barra invertida en la cadena, deberá ir precedido de otra barra invertida; ídem para la comilla doble. Por otro lado, se ha optado por generar un error cuando se intenta procesar una cadena que contenga una secuencia de escape no válida en vez de optar por insertar la barra invertida y el siguiente carácter por separado; esta decisión deriva de la anterior: ahora hay un mecanismo para insertar correctamente las barras invertidas en una cadena, por lo que no es necesario interpretar benévolamente las secuencias de escape erróneas.

5.3.14. Módulo de la unidad de control

Este módulo constituye la capa más alta de abstracción de la lógica interna de ENS2025. Cabe aclarar que el nombre de este módulo no se corresponde con el concepto de unidad de control de Von Neumann, sino que recibe esta denominación por tratarse del elemento de coordinación principal. Las interfaces de nivel de usuario se comunican únicamente con este módulo, que se encarga de trasladar sus peticiones al componente pertinente y devolver el resultado correspondiente, si procede.

5.3.14.1. La clase `ControlUnit`

Esta clase, definido en la tabla 5.54, es la implementación del módulo que se acaba de describir. Sus métodos son envolturas de sus equivalentes de los módulos contenidos, y ofrecen una interacción sencilla y funcional no solamente a las dos aplicaciones creadas a tal efecto, sino a otras que pudieran surgir en un futuro. A excepción del módulo de configuración y de la interfaz E/S, todos los módulos ven su clase central instanciada en el constructor de `ControlUnit`. También coordina las tareas de volcado y recuperación de los datos del sistema mediante ficheros binarios.

ControlUnit
- architecture:Architecture - disassembler:Disassembler - sim:Simulator - assembler:Assembler - set:Settings
+ ControlUnit(set:Settings, io:IOInterface) + assembleFile(fileName:String):void + runSimulation():void + debugSimulation():void + runSimulationOnce():void + disassembleInstructions(index:RData, n:int):List<DisassembledInstruction> + hasBreakpoint(index:RData):boolean + setBreakpoint(index:RData, value:boolean):void + readMemory(index:RData):RData + writeMemory(index:RData, value:RData):void + wipeMemory():void + getStackPointer():RData + getProgramCounter():RData + readRegister(reg:RegisterID):RData + writeRegister(reg:RegisterID, value:RData):void + getFlag(f:Flag):boolean + setFlag(f:Flag, status:boolean):void + resetRegisters():void + writeMemoryToFile(name:String):void - writeMemoryToStream(stream:OutputStream):void throws IOException - writeRegisterBankToStream(stream:OutputStream):void throws IOException - writeZoneBoundariesToStream(stream:OutputStream):void throws IOException + loadMemoryFromFile(name:String):void - loadMemoryFromStream(stream:InputStream):void throws IOException - loadRegisterBankFromStream(stream:InputStream):void throws IOException - loadZoneBoundariesFromStream(stream:InputStream):void throws IOException + getStopFlag():AtomicBoolean + setStopFlag(value:Boolean):void

Tabla 5.54: Clase ControlUnit.

5.4. Diseño e implementación de las interfaces de nivel de usuario

En esta sección se tratarán los dos programas implementados para manejar la unidad de control de ENS2025. Además de la propia clase `ControlUnit`, estos programas contienen clases que implementan la interfaz `IOInterface`, esencial para poder mostrar al usuario los mensajes del sistema y gestionar la comunicación entrada/salida durante la simulación.

A diferencia de la aproximación seguida hasta ahora, donde se ha detallado el sistema clase a clase por la importancia que tiene cada componente del mismo, con las interfaces de usuario

se seguirá un método más descriptivo, incidiendo más en cómo se comporta la interfaz y cuál es el alcance de sus funcionalidades. Esto es especialmente necesario para la interfaz gráfica, donde muchas de las clases son simplemente agrupaciones de componentes en contenedores para facilitar su manejo.

Por supuesto, las clases más importantes de cada una, como las respectivas implementaciones de `IOWriter` o el adaptador de la unidad de control en la GUI, sí serán debidamente detalladas.

5.4.1. Módulo de la interfaz de línea de comandos

La interfaz de línea de comandos es una aplicación sencilla que utiliza la consola del sistema como medio para su ejecución y comunicación con el usuario. Consiste en un *prompt* que el usuario utiliza para introducir diversos comandos. Estos comandos serán interpretados por el módulo, que llamará a los métodos pertinentes de la unidad de control para cumplir con el cometido esperado de cada uno de ellos.

5.4.1.1. Inicio del programa y parámetros de invocación

Al estar compilado en un objeto JAR, la aplicación se inicia introduciendo en la línea de comandos la siguiente invocación:

```
java -jar ens2025.jar [parámetros]
```

Si se invoca sin ningún parámetro, la secuencia de inicio es la siguiente:

1. Se intenta cargar el fichero de configuración.
2. Si no se encuentra, se carga la configuración estándar.
3. Se muestra el estado de la configuración del sistema.
4. Se muestra la ayuda simplificada de los comandos.
5. Se muestra el *prompt* y se devuelve el control al usuario.

Por otro lado, la aplicación puede ser invocada mediante los siguientes parámetros:

- **-a <nombre>**: Inicia la ejecución cargando y ensamblando un fichero de código ensamblador.
- **-m <nombre>**: Inicia la ejecución cargando directamente un fichero binario de imagen de memoria.
- **-rec**: Inicia la ejecución cargando el fichero de recuperación generado al interrumpir la ejecución del programa mientras el proceso de simulación está activo.
- **-r <nombre>**: Carga un fichero ensamblador, lo ensambla, ejecuta el código resultante y finaliza el programa.

- **-rm <nombre>**: Carga una imagen de memoria, la ejecuta automáticamente y finaliza el programa.
- **-d <nombre>**: Si se ha seleccionado un argumento de ejecución (**-r** o **-rm**), este parámetro especifica un nombre de fichero binario de imagen de memoria en el que se volcará el estado del sistema al finalizar la simulación antes de cerrar el programa.
- **-s <nombre>**: Permite cargar un fichero de configuración específico para esta ejecución del programa. La configuración generada por dicho fichero no será permanente: al cerrar el programa e invocarlo sin este parámetro, se cargará con normalidad el fichero de configuración estándar. La configuración durante la ejecución del programa con este parámetro es temporal, y los cambios realizados en la misma no se guardarán ni en el fichero cargado mediante parámetro ni en el fichero estándar.

La finalidad de dichos parámetros es otorgar flexibilidad y fluidez a la ejecución, en especial la ejecución por lotes. De esta manera, tanto los alumnos como el equipo docente pueden hacer comprobaciones de ejecución secuenciales en ciclos de menor duración. La opción de cargar un fichero de configuración permite, además, hacer comprobaciones de un mismo fichero de código con distintas configuraciones (por ejemplo, una ejecución normal sin guardas, otra con las guardas de invasión de punteros y desbordamiento de pila activas, otra con la pila creciendo en la dirección opuesta...) sin necesidad de andar cambiando el fichero para cada ejecución.

Los parámetros pueden invocarse en cualquier orden, pero atendiendo a ciertas restricciones:

- No se puede usar dos veces el mismo parámetro.
- Solamente se puede usar un único parámetro de carga o de ejecución. Es decir, **-a**, **-m**, **-rec**, **-r** y **-rm** son incompatibles entre sí.
- Solamente se puede usar el parámetro **-d** (volcado) si se usa un parámetro de ejecución (**-r** o **-rm**) en la misma invocación.
- El parámetro **-s** es totalmente independiente y no tiene más restricción que la de no ser duplicado.

5.4.1.2. Comandos

Los comandos que acepta la aplicación pueden ir, en algunos casos, acompañados de parámetros, tanto <obligatorios> como [opcionales]. La lista completa de comandos, con sus posibles parámetros y funcionalidades, es la siguiente:

- **ENS <f>**: Ensambla el fichero de nombre ‘f’ y carga en la memoria el contenido de las instrucciones ensambladas.
- **DES <dir><num>**: Desensambla ‘num’ instrucciones a partir de la dirección de memoria ‘dir’.
- **RUN**: Simula el código con normalidad.

- PASO: Muestra la instrucción apuntada por el PC y la ejecuta.
- DEBUG: Simula el código hasta encontrar un punto de ruptura.
- BP <dir>[val]: Si se invoca sin segundo parámetro, añade un punto de ruptura en la dirección 'dir' si no había uno, y lo elimina si ya existía. Si se invoca con segundo argumento, fuerza ese estado. Los valores pueden ser:
 - SI: Introduce el punto de ruptura.
 - NO: Elimina el punto de ruptura.
- LEERMEM <dir><num>: Muestra 'num' posiciones de memoria a partir de la dirección de memoria 'dir'. Si el número de posiciones a mostrar es mayor que el número de posiciones restantes hasta el final de la memoria, se mostrarán solamente las posiciones disponibles.
- ESCMEM <dir><val>: Escribe el valor 'val' en la dirección de memoria 'dir'.
- RESMEM: Restablece a 0 el valor almacenado en todas las posiciones de memoria.
- LEERPILA <num>: Muestra 'num' posiciones de memoria a partir de la dirección de memoria del puntero de pila, empezando por la cima. Si el número de posiciones a mostrar es mayor que el número de elementos restantes hasta el final de la pila, o si se produce desbordamiento, se mostrarán solamente las posiciones disponibles.
- CARGAR <f>: Carga el estado del procesador (memoria, registros, zonas de código y pila) contenida en el fichero de nombre 'f'. Compatible con ficheros de memoria generados por ENS2001, pero solamente contienen la memoria.
- GUARDAR <f>: Guarda el estado actual del procesador (memoria, registros, zonas de código y pila) en el fichero de nombre 'f'. Estos ficheros pueden ser cargados por ENS2001, pero solamente cargará el estado de la memoria.
- BANCO: Imprime por pantalla los valores de todos los registros.
- REGESC <reg><val>: Inserta el valor 'val' en el registro identificado por 'reg'.
- H [com]: Si se invoca sin parámetros, muestra la ayuda simplificada de todos los comandos. Si se invoca usando el comando 'com' como parámetro, muestra la ayuda avanzada para ese comando específico.
- CONF: Imprime por pantalla los valores actuales de configuración del sistema.
- INVPILA [val]: Modifica la comprobación de invasión de la zona de pila por parte del PC. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - SI: Activa la comprobación.
 - NO: Desactiva la comprobación.
- INVCODE [val]: Modifica la comprobación de invasión de la zona de código por parte del SP. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:

- SI: Activa la comprobación.
- NO: Desactiva la comprobación.
- DIRPILA [val]: Modifica la dirección hacia la que se mueve el SP cuando se introduce un elemento en la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - POS: Crecimiento positivo (“hacia abajo”).
 - NEG: Crecimiento negativo (“hacia arriba”).
- OVERFLOW [val]: Modifica la comprobación de desbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - SI: Activa la comprobación.
 - NO: Desactiva la comprobación.
- UNDERFLOW [val]: Modifica la comprobación de subdesbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - SI: Activa la comprobación.
 - NO: Desactiva la comprobación.
- SETINIC <dir>: Cambia el valor de la dirección inicial de la zona de código por la suministrada en ‘dir’.
- SETFINC <dir>: Cambia el valor de la dirección final de la zona de código por la suministrada en ‘dir’.
- SETINIP <dir>: Cambia el valor de la dirección inicial de la zona de pila por la suministrada en ‘dir’.
- CONFREG [val]: Modifica la comprobación de reinicio del banco de registros al recibir una orden de simulación mientras el biestable H está en alto. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - SI: Activa la comprobación.
 - NO: Desactiva la comprobación.
- BASE <val>: Modifica la base numérica en la que se representan los valores. Algunos elementos, como las direcciones de memoria, mostrarán su valor decimal sin signo incluso si la representación es decimal con signo. Los valores pueden ser:
 - HEX: Hexadecimal.
 - DEC: Decimal con signo.
 - USDEC: Decimal sin signo.
- X: Guarda el estado actual de la configuración del sistema en el fichero pertinente y termina la ejecución.

5.4.1.3. Implementación de la interfaz entrada/salida

Las interfaces de nivel de usuario necesitan crear una clase que implemente los métodos de `IOInterface` para poder establecer el flujo de comunicación con el usuario, como ya se anticipó en el módulo pertinente. En el caso de la aplicación por línea de comandos, la implementación es trivial: los métodos `printLog()` y `printlnLog()` muestran los mensajes recibidos por la salida de `System.err`, mientras que el resto de métodos de salida imprimen el contenido por `System.out`, y los de entrada emplean `System.in`. Aunque a ojos del usuario no hay diferencia entre `System.err` y `System.out` por usarse una misma ventana de consola para todo, esto permite la redirección de los flujos de salida a distintos ficheros durante la ejecución por lotes si se desea, separando así los mensajes generados por el proceso de simulación de los generados por el sistema.

5.4.1.4. El problema de la gestión de bucles infinitos

En ocasiones, un usuario puede diseñar un código fuente que no funcione de la forma esperada, repitiendo en un ciclo perpetuo el mismo conjunto de instrucciones y usurpando el control al usuario. Como se mencionó en el módulo del simulador, el proceso de simulación debe ejecutarse de forma concurrente al del programa principal para no interrumpirlo. Este requisito está más orientado a permitir que el hilo de la GUI no quede bloqueado mientras tiene lugar una simulación, permitiendo el uso del botón de parada.

En teoría, esto también permitiría al hilo de la aplicación de consola tomar medidas para la gestión de estos incidentes: el hilo principal quedaría a la espera de que el hilo de simulación mandase una señal de finalización, o bien el hilo principal recibiría una señal por parte del usuario para devolverle el control y finalizar el hilo de simulación.

Sin embargo, replicar este comportamiento en Java no es sencillo. O, mejor dicho, no es factible si se pretende mantener la base de código dentro de los límites de las librerías estándar. Java no está preparado para capturar eventos de pulsación de teclas (por ejemplo, pulsar la tecla ESC) en aplicaciones de línea de comandos, requiriendo la pulsación específica de la tecla ENTER para ello.

Esto, de por sí, no constituye necesariamente un problema: el hilo principal podría quedar a la espera y, de recibir una pulsación de ENTER, o un mensaje completo como “PARAR”, finalizaría el hilo de simulación y devolvería el control al usuario. El problema surge cuando esta limitación de Java se combina con la existencia de las instrucciones de entrada/salida del simulador. El resultado es que ambos hilos compiten por el mismo recurso, lo que puede causar comportamientos no deseados, como que uno bloquee a otro o que al pulsar ENTER para confirmar la entrada de datos para el hilo de simulación se active el protocolo de finalización de emergencia del simulador implementado en el hilo principal.

Por lo tanto, es necesaria una solución de compromiso. Java permite la inclusión de un mecanismo de salvaguarda ante la recepción de una señal de finalización del programa (recibida, generalmente, mediante la combinación de teclas CONTROL+C), que le permite “poner en orden sus asuntos” antes de devolver el control a la consola del sistema: el `ShutdownHook`. Éste consiste en un hilo que se encarga de realizar una serie de tareas cuando se detecta la recepción

de una señal de interrupción. Desgraciadamente, dicho evento no puede configurarse para ser recuperable: desde el momento en el que se recibe la señal, la JVM comienza su proceso de apagado, por lo que el `ShutdownHook` es simplemente una forma de minimizar el impacto de ese suceso.

En este caso, el `ShutdownHook` se usa para generar una imagen de recuperación. Cuando se produce un bucle infinito (o una simulación está tardando más de lo esperado), el usuario puede pulsar `CONTROL+C` para salir del programa. Como resultado, el programa guardará una imagen del estado del simulador (exactamente igual que las que se generan al solicitar un volcado) en un fichero binario llamado `recovery.dump`. Este fichero puede ser cargado manualmente, o cargado durante la invocación del programa con el parámetro `-rec`.

Esto permite al usuario observar el estado del simulador en el momento de su terminación para tratar de entender qué pudo ocasionar el bucle.

5.4.2. Módulo de la interfaz gráfica de usuario

La interfaz gráfica está pensada para facilitar al usuario el acceso a las mismas funcionalidades que la versión por línea de comandos, pero en un entorno donde todos los elementos estén visualmente presentes, reduciendo la carga cognitiva necesaria para entender en todo momento el estado de la simulación, así como para ejecutar las acciones deseadas. La aplicación está dividida en cinco elementos principales, como se puede ver en la figura 5.13: la barra de herramientas (1), el banco de registros y biestables (2), el visor de memoria (3), el visor de código (4) y las zonas de texto para los mensajes del sistema y la consola E/S de la simulación (5). A lo largo de esta sección se analizarán detalladamente estos elementos y sus componentes, así como algunos aspectos importantes a nivel lógico de esta implementación.

5.4.2.1. La barra de herramientas

A diferencia de ENS2001, la parte superior de ENS2025 carece de menús. En su lugar, se ha optado por incluir todos los elementos de configuración y ejecución en una misma barra de herramientas, ya que el espacio lo permite. Esto facilita la interacción del usuario con el software, según el principio de “reconocimiento en vez de recuerdo” [20]. El perfil de usuario está formado por alumnos que interactuarán durante un corto período de tiempo con el software, por lo que se requiere un aprendizaje rápido para poder hacer uso adecuado de todas sus opciones. Así, separar en distintos botones los modos de ejecución en lugar de tener que abrir una ventana de configuración para determinar si se ejecuta paso a paso y/o en modo depuración; o poder cambiar los modos de representación numérica o las comprobaciones de invasión de punteros con un clic en vez de tener que navegar por menús, facilitan que el usuario comprenda de un vistazo el abanico de funcionalidades a su disposición.

La barra de herramientas está dividida en tres secciones:

- **Gestión de ficheros:** Contiene los elementos dedicados a la carga y guardado de ficheros, como se puede ver en la figura 5.14a. Los elementos son, en orden, los siguientes:

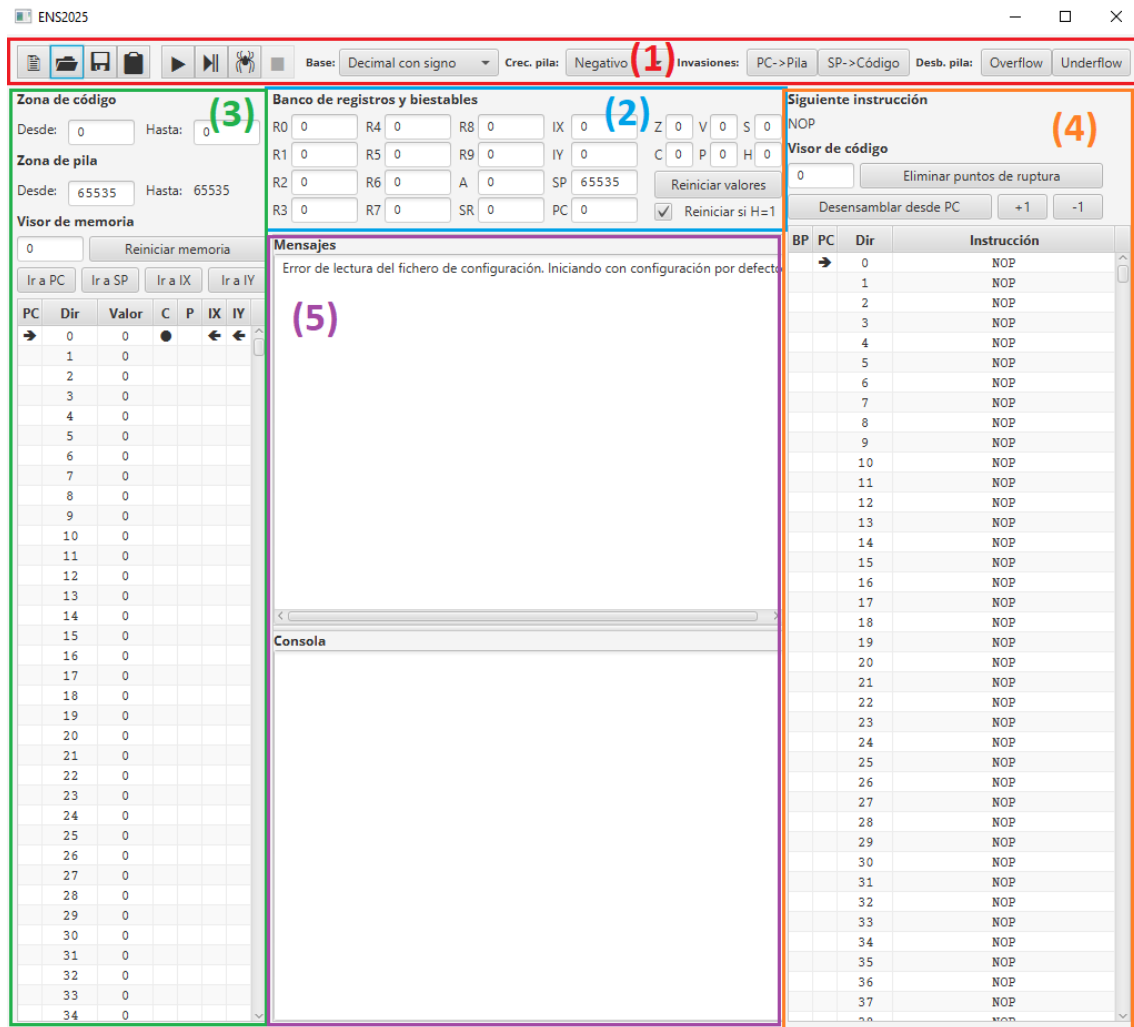
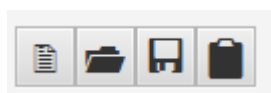


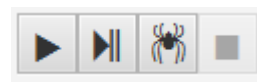
Figura 5.13: Vista general de la ventana de interfaz gráfica.

- **Botón de ensamblado de fichero:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.ens`, con la finalidad de seleccionar un fichero de código fuente ensamblador. El fichero seleccionado será ensamblado y cargado en la memoria.
- **Botón de carga de imagen:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.mem`, con la finalidad de seleccionar un fichero binario de imagen de memoria. El fichero seleccionado se cargará en la memoria.
- **Botón de guardado de imagen:** Abre una ventana de exploración para designar la localización y nombre del fichero binario a escribir (o sobrescribir) con la imagen de memoria de sistema.
- **Botón de guardado de mensajes:** Abre una ventana de exploración para designar la localización y nombre del fichero de texto a escribir (o sobrescribir) con un volcado del área de texto de mensajes, seguido de un volcado del área de texto de la consola de entrada/salida.
- **Gestión de simulación:** Contiene los elementos dedicados al inicio y parada del simulador, como se puede ver en la figura 5.14b. Los elementos son, en orden, los siguientes:

- **Botón de ejecución estándar:** Inicia la ejecución estándar del simulador (de manera continua y sin detenerse ante puntos de ruptura).
 - **Botón de ejecución paso a paso:** Ejecuta una sola instrucción.
 - **Botón de depuración:** Inicia la simulación con la condición de detenerse ante un punto de ruptura. Si la depuración se inicia mientras el PC apunta a una instrucción que contiene un punto de ruptura, dicho punto de ruptura se ignora para evitar que haya que desactivarlo para poder iniciar la simulación.
 - **Botón de parada:** Normalmente deshabilitado, se habilita cuando uno de los modos de ejecución está activo, dando la opción de detenerlo prematuramente.
- **Gestión de configuración:** Contiene los elementos dedicados a la visualización y modificación de los valores de configuración del sistema, como se puede ver en la figura 5.14c. Los elementos son, en orden, los siguientes:
- **Desplegable de base numérica:** Permite al usuario modificar el formato de representación de los valores.
 - **Desplegable del crecimiento de pila:** Permite al usuario modificar la dirección de crecimiento de la pila.
 - **Botón de alternancia de invasión de pila:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de pila por el PC. En la figura 5.14c se puede ver que este botón está activado, por lo que la comprobación de invasión de pila también lo está.
 - **Botón de alternancia de invasión de código:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de código por el SP. En la figura 5.14c se puede ver que este botón está desactivado, por lo que la comprobación de invasión de código tampoco lo está.
 - **Botón de alternancia de comprobación de desbordamiento:** Permite al usuario activar o desactivar la comprobación de desbordamiento de la pila. En la figura 5.14c se puede ver que este botón está desactivado, por lo que la comprobación de desbordamiento tampoco lo está.
 - **Botón de alternancia de comprobación de subdesbordamiento:** Permite al usuario activar o desactivar la comprobación de subdesbordamiento de la pila. En la figura 5.14c se puede ver que este botón está activado, por lo que la comprobación de subdesbordamiento también lo está.



(a) Herramientas de archivos.



(b) Herramientas de simulación.



(c) Herramientas de configuración.

Figura 5.14: Divisiones de la barra de herramientas.

5.4.2.2. El banco de registros y biestables

Este componente, que se puede ver con más detalle en la figura 5.15 contiene los siguientes elementos:

- Campos de texto para visualizar y editar los 16 registros.
- Campos de texto para visualizar y editar los 6 biestables.
- Botón para reiniciar los valores de todos los campos.
- Opción para reiniciar los valores del banco automáticamente si se inicia una simulación mientras el biestable H está en alto.

Banco de registros y biestables													
R0	0	R4	0	R8	0	IX	0	Z	0	V	0	S	0
R1	0	R5	0	R9	0	IV	0	C	0	P	0	H	0
R2	0	R6	0	A	0	SP	65535	Reiniciar valores ☒ Reiniciar si H=1					
R3	0	R7	0	SR	0	PC	0						

Figura 5.15: Banco de registros y biestables. El registro A tiene el foco.

Los campos no se pueden editar directamente, sino que requieren hacer doble clic sobre ellos, o pulsar la tecla ENTER mientras el campo deseado tiene el foco (borde azul). En ambos casos, se abre una pequeña ventana de diálogo para elegir el nuevo valor: para los campos de los registros, solicita introducir el valor nuevo (respetando siempre los formatos permitidos y rangos de valores), mientras que para los campos de biestables se pide seleccionar un botón radial para poner el valor a 0 o 1. Ambas variaciones se pueden ver en la figura 5.16. El funcionamiento de los campos de texto de los registros se aplica a todos los campos de texto similares de la aplicación. Mientras el cuadro de diálogo de modificación de un campo está activo, no se puede interactuar con el resto de la aplicación.

(a) Edición de campos de valor generales.

(b) Edición de biestables.

Figura 5.16: Cuadros de diálogo de edición de valores.

El botón de reiniciar valores, como su nombre indica, restablece a 0 todos los registros y biestables, con excepción del asociado con el SP, que se restablecerá al valor de la dirección inicial de la pila. El **CheckBox** activa o desactiva la comprobación del valor del biestable H antes de cada ejecución.

5.4.2.3. El visor de memoria

Este componente, que se puede ver con más detalle en la figura 5.17, se encarga de la gestión de la memoria por parte de la aplicación. Permite explorar la memoria y modificar su contenido, así como visualizar y modificar las zonas de código y pila. Por último mantiene, información visual sobre las posiciones de los distintos punteros del sistema y permite saltar hacia ellos para ver el estado de la memoria en sus inmediaciones. Agrupa los siguientes elementos:

- Módulo de visualización y edición de las zonas de código y pila.
- Barra de herramientas del visor.
- El propio visor de memoria.

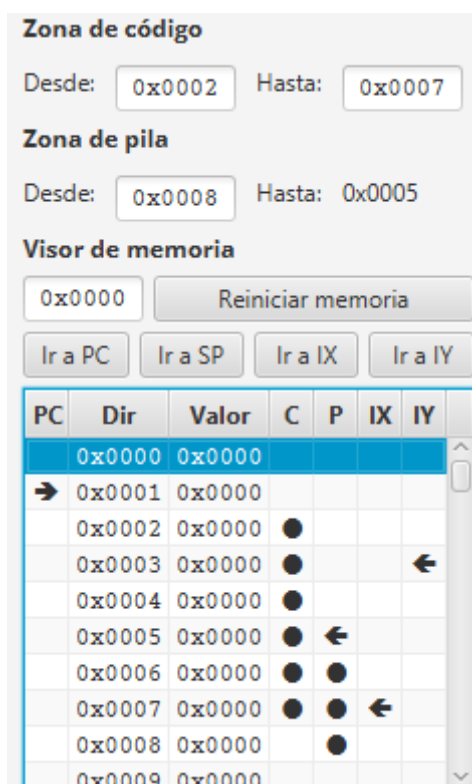


Figura 5.17: Visor de memoria con su barra de herramientas.

El editor de zona de código contiene dos campos donde se reflejan automáticamente los valores de las direcciones inicial y final del código desde el momento en que se carga y ensambla un fichero. Al igual que con los campos de valores de los registros, hacer doble clic o pulsar ENTER con el foco en uno de estos campos permite su modificación, cambiando el tamaño designado de la zona de código. Por otro lado, el editor de la zona de pila, si bien muestra las direcciones de inicio y fin de la misma, solamente permite la edición de la dirección de inicio, ya que la dirección de fin es el propio SP, y el valor mostrado se actualiza automáticamente con los cambios del mismo.

La barra de herramientas del visor de memoria contiene, en la primera línea, un campo de texto para escribir una dirección de memoria y un botón para reiniciar la memoria; y, en la segunda

línea, botones de navegación al PC, SP, IX e IY. Al confirmar la entrada de datos en el campo de texto, el foco se sitúa en esa dirección de memoria específica dentro del visor de memoria y, al pulsar el botón de reinicio, todos los valores almacenados en la memoria se restablecen a 0. Los cuatro botones de navegación sitúan el foco en la posición respectiva de cada uno de esos punteros en el visor de memoria.

El visor en sí es una tabla con las siguientes columnas:

- **PC:** Indica si la dirección de memoria de dicha celda está contenida en el PC en ese momento. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.
- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (no tiene sentido pasar de direcciones positivas a negativas en mitad de la memoria).
- **Valor:** El valor almacenado dentro de la memoria para esa dirección. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, permite modificar esta celda de la misma, mediante la misma ventana de diálogo que aparece cuando se editan los registros o las direcciones de código y pila.
- **C:** Indicador de pertenencia a la zona de código. Si la dirección asociada a la celda está contenida en la zona de código, aparecerá en la celda un punto negro. De lo contrario, la celda quedará vacía.
- **P:** Indicador de pertenencia a la pila. Si la dirección asociada a la celda está contenida entre el SP (no incluido) y el inicio de la pila, aparecerá en la celda un punto negro. Si el SP apunta a esa dirección, la celda contendrá una flecha orientada a la izquierda. En cualquier otro caso, la celda quedará vacía.
- **IX e IY:** Al igual que la columna del PC, indican si la dirección de memoria está contenida, respectivamente, en los registros IX e IY. Cuando se produzca una coincidencia, la celda contendrá una flecha apuntando a la izquierda, y estará vacía en caso contrario.

Éste es uno de los elementos de la GUI que más ampliaciones ha sufrido respecto a ENS2001. Si bien ENS2001 contenía visores de memoria separados para la memoria en sí y la pila, con botones de navegación y salto para los punteros respectivos de cada uno, como se puede ver en la tabla 5.18, esto era innecesariamente complejo y, pese a mostrar el SP, no mostraba el PC. Además, las zonas de pila y código se mezclaban en una sola celda, no quedaban alineadas y la visualización del puntero de pila dependía de mantener la correspondiente ventana abierta. Por último, los botones y el campo de dirección quedaban recortados, y solamente se podía saltar a una dirección pulsando ENTER en el campo donde se escribía, pues el botón no era funcional.

Aunque la mayoría de problemas eran meramente estéticos, en conjunto eran suficientes como para requerir un rediseño del concepto tras la funcionalidad. Finalmente, se optó por la solución que se ha descrito: un solo visor, con las columnas de ambas zonas claramente diferenciadas (así como el propio puntero de pila), con un sistema de navegación funcional e intuitivo, y con la adición de los punteros PC, IX e IY, lo que permite visualizar en un instante el efecto que tendrá una instrucción con direccionamiento relativo a dichos punteros.

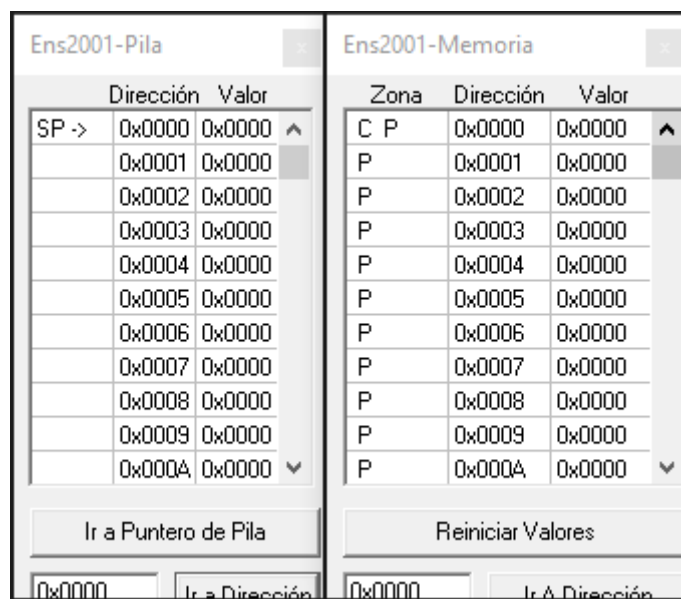


Figura 5.18: Los visores de pila y código de ENS2001.

5.4.2.4. El visor de código

Este componente, que se puede ver con más detalle en la figura 5.19, se encarga de aportar información sobre el código contenido en la memoria y gestiona los puntos de ruptura. Agrupa los siguientes elementos:

- Módulo de visualización de siguiente instrucción a ejecutar.
- Barra de herramientas del visor.
- El propio visor de código.

El visualizador de la siguiente instrucción permite mantener en todo momento constancia de la misma, aunque el visor esté desplazado o incluso desfasado. La barra de herramientas, en dos filas como la del visor de memoria, tiene en la primera de ellas un campo de texto para desensamblar el código a partir de una dirección introducida, así como un botón para eliminar todos los puntos de memoria; la segunda contiene un botón que devuelve el foco a la dirección actual del PC y desensambla a partir del mismo, así como dos botones para ‘desfasar’ una posición hacia atrás o hacia delante, desensamblando a partir de la misma.

El sistema de desensamblado de ENS2025 sigue estas reglas:

- Al cargar un programa, desensambla todo el código desde la posición inicial de la memoria.
- Es posible navegar hacia adelante cuanto se desee sin que eso implique regenerar la lista de instrucciones desensambladas (a diferencia de ENS2001), pero no es posible navegar más atrás de la dirección de desensamblado actual (también a diferencia de ENS2001).
- Los botones ‘+1’ y ‘-1’ desplazan hacia delante y hacia atrás, respectivamente, la dirección de desensamblado y, en consecuencia, regeneran la lista de instrucciones.



Figura 5.19: Visor de código con su barra de herramientas.

- El campo de navegación establece la dirección introducida como nueva dirección de desensamblado, regenerando la lista de instrucciones.
- Un cambio en el modo de representación numérica regenerará la lista de instrucciones.
- La lista de instrucciones se regenerará tras cualquier ejecución. Se tomará como dirección de desensamblado el valor actual del PC.

Por otro lado, el visor de código en sí contiene una tabla, similar a la del visor de memoria, con las siguientes columnas:

- **BP:** Indicador de punto de ruptura. Cuando la instrucción tiene asociado un punto de ruptura, en esta celda aparece un círculo negro, quedando vacía en caso contrario. Si alguna de las posiciones de memoria que contienen las palabras que conforman esta instrucción tiene un punto de ruptura asociado, se considera que toda la instrucción lo tiene. Eliminar el punto de ruptura de la instrucción lo elimina de todas las palabras que lo contengan. Crear un punto de ruptura en la instrucción lo establece únicamente en la primera palabra de la misma. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, creará un punto de ruptura o eliminará el existente para esa instrucción.
- **PC:** Indica si el contador de programa apunta actualmente a la primera palabra de la instrucción contenida en la fila. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.

- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (igual que en el visor de memoria). El visor no muestra necesariamente todas las direcciones de memoria existentes, solamente las filas cuya dirección sea la primera de una instrucción válida, o bien aquellas que apunten a una dirección inválida.
- **Instrucción:** Contiene la cadena de texto de la instrucción desensamblada a código fuente legible para el usuario.

5.4.2.5. Las zonas de texto

Este componente separa los flujos de mensajes del sistema en dos áreas de texto distintas, situadas una encima de la otra: la de arriba se encarga de mostrar todos los mensajes del sistema (avisos de ensamblado exitoso, mensajes de error...), mientras que la segunda gestiona el flujo de entrada y salida de la simulación en curso. Ambas áreas de texto están separadas por un elemento deslizante, que permite redistribuir el espacio vertical asignado a ambas, y crecen horizontalmente con la redimensión de la aplicación.

El área de texto de mensajes del sistema no permite escritura, solamente lectura; y el área de texto de la consola de simulación solamente permite escritura cuando se ejecuta una instrucción `ININT`, `INCHAR` o `INSTR`, que solicita explícitamente una entrada. Además, incluso cuando se permite escribir en la consola, no se permite al usuario editar (borrar, modificar, insertar) el texto ya existente, solamente puede escribir a partir del último carácter del texto previo, y solamente se le permite borrar hasta ese punto.

Además, ambas áreas de texto tienen asociadas ciertas acciones, tanto a través de menú contextual (clic derecho) como mediante atajos de teclado:

- Seleccionar todo (`CONTROL+A`).
- Copiar selección (`CONTROL+C`).
- Vaciar de texto la zona (`CONTROL+W`).

Para evitar comportamientos inesperados, esta última opción está desactivada para el área de la consola mientras está activo el modo de espera de entrada del usuario.

5.4.2.6. Implementación lógica

JavaFX es un sistema de creación de interfaces gráficas para Java que emplea el paradigma de programación dirigida por eventos para establecer la comunicación entre los componentes de sus interfaces. Para ello, implementa el patrón Observer mediante un sistema de propiedades (observables) y oyentes (observadores) que se adjuntan a las mismas; cada vez que se modifica una propiedad, se notifica del cambio de forma secuencial a todos los oyentes adjuntos a ella, lo que permite que cada uno realice una serie de acciones por su parte como respuesta, lo que

a su vez puede desencadenar nuevas notificaciones y cambios. Otro aspecto de este paradigma es la captura de eventos de pulsación de teclas o clicado de botones dentro de un componente, que permite configurar acciones de respuesta similares a las de las propiedades y los oyentes.

ENS2025 implementa un sistema de adaptadores para gestionar las propiedades de los principales componentes de la aplicación. Los visores de código y memoria, así como el banco de registros de la GUI o el módulo de configuración, tienen sus propios adaptadores, que intercambian información entre sí. Para coordinarlos a todos y llevar a cabo las acciones más importantes existe una especie de *“adaptador maestro”*: la clase **ControlUnitAdapter**. Esta clase funciona como espejo de la unidad de control para la lógica interna del programa, y gestiona las relaciones de otros adaptadores entre sí y con la propia unidad de control.

Así, por ejemplo, este adaptador se encarga de asignar un oyente a la propiedad asociada al registro PC (y también al SP, así como a IX e IY) en el módulo visual del banco de registros. Cuando se genera un cambio en este valor, no solamente se consolida escribiéndolo en la unidad de control real, sino que se notifica al oyente en **ControlUnitAdapter**, que da la orden al adaptador del visor de memoria de refrescarse. Mediante este proceso, todas las filas de la tabla del visor renuevan su contenido, incluyendo cuál de ellas contiene una dirección igual al nuevo valor del PC y cuáles no, cambiando la celda en la que se muestra la flecha que indica la posición de memoria señalada por el PC. Cuando se altera el valor contenido en una dirección de memoria, se da orden al visor de código de comprobar si dicha dirección es igual o posterior a la dirección de desensamblado actual, y de regenerarse en caso afirmativo.

En contextos muy grandes, esto podría desembocar en un antipatrón God Object, algo que podría evitarse mediante una implementación por capas del patrón Mediator, creando un grupo de controladores dedicados exclusivamente a gestionar el tráfico de eventos entre los componentes y entre sí, y dejando a **ControlUnitAdapter** solamente la función de hacer de envoltura del resto de adaptadores y sus métodos. Sin embargo, el número reducido de componentes y propiedades hace que sean fácilmente gestionables bajo el modelo actual, aunque es algo a considerar de cara al futuro.

5.4.2.7. Implementación de la interfaz entrada/salida

Al igual que en la interfaz de línea de comandos, es un requisito indispensable para la GUI implementar la interfaz **I0Interface**. Sin embargo, la complejidad en este caso es considerablemente mayor que simplemente usar **System.in**, **System.out** y **System.err**. Si bien la salida de sistema no reviste dificultad (ya que no tiene que coordinar entradas del usuario y, por funcionamiento, durante una simulación su propósito va a ser publicar un único mensaje de error antes de que ésta se detenga) la entrada y salida en el área de consola sí presentan ciertos desafíos.

5.4.2.7.1 Entrada de datos por consola

Debido a que la simulación se ejecuta de manera concurrente en un hilo distinto al de la GUI para poder garantizar acceso al botón de parada y que, al mismo tiempo, la entrada de datos se produce en un elemento de la GUI, es necesaria cierta gestión de concurrencia en la

lectura de elementos. Afortunadamente, dada la simplicidad del sistema, solamente se realiza una simulación cada vez, por lo que no hace falta un mecanismo particularmente complicado y se puede resolver con un par de instancias de `CountdownLatch`.

La clase `CountdownLatch`, esencialmente, crea un elemento bloqueante con un contador que puede ser accedido desde otros hilos. Cuando un hilo supedita la continuación de su ejecución a una instancia de `CountdownLatch`, queda en espera hasta que el contador de dicha instancia llega a cero. Así, otros hilos pueden realizar las tareas que necesiten y, a medida que las terminen, pueden indicar al contador que debe disminuir. Esto, combinado con el sistema de eventos comentado en la sección anterior, permite crear un sistema robusto para mantener la sincronía en la recepción de datos por parte del usuario:

1. En el hilo de la simulación se crean dos instancias de `CountdownLatch`, una de apertura y una de cierre. Ambas tienen un valor de contador de 1.
2. Desde el hilo de la simulación se indica al hilo de la GUI que debe activar la propiedad del área de consola que permite al usuario escribir.
3. El hilo de simulación queda a la espera del `CountdownLatch` de apertura.
4. En la GUI, el usuario introduce los datos pertinentes en la consola y finaliza con la pulsación de ENTER.
5. El cambio en el texto es detectado y se reduce el contador del `CountdownLatch` de apertura, desbloqueando el hilo de simulación.
6. En el hilo de simulación se obtiene el texto introducido y se elimina el `CountdownLatch` de apertura para su recreación en la siguiente llamada.
7. Se indica al hilo de GUI que debe retirar el permiso de escritura del área de consola y se queda a la espera del `CountdownLatch` de cierre.
8. En el hilo de GUI se limpia la entrada de texto para detectar futuros cambios, se retira el permiso de escritura en el área de consola y se reduce el contador del `CountdownLatch` de cierre, liberando el hilo de simulación de nuevo.
9. El hilo de simulación procesa el texto leído según proceda (obtención de un valor numérico, carácter o directamente la cadena leída) y lo devuelve.

5.4.2.7.2 Salida de datos por consola

La salida de datos presenta un desafío atípico y totalmente inesperado: es *demasiado* rápida. En circunstancias normales eso no supone un problema; pero existe un caso concreto donde las consecuencias de no gestionar adecuadamente esta cualidad son catastróficas. Ante la eventualidad de un bucle infinito de instrucciones de escritura por consola como `WRINT`, `WRCHAR` y `WRSTR`, el número de operaciones de escritura se dispara rápidamente.

Dado que bastan unos segundos para bloquear la aplicación, es difícil establecer unas métricas fidedignas; pero es digno de mención aunque sea de manera informal: en una situación como la descrita, ENS2001 genera en unos pocos segundos aproximadamente 500 escrituras en la

consola². En ese mismo tiempo (y tras dejar el programa bloqueado durante unos diez minutos³), ENS2025 ha escrito dos millones y medio de líneas.

Las soluciones más aparentes, como buffering mediante concurrencia o colas de mensajes, no acaban de servir para este problema. Simplemente, lo trasladan y lo diferencian. En algún momento, el programa puede ser detenido por el usuario, y en ese momento se debe tomar una decisión: intentar volcar todos los mensajes de una vez en la consola (lo cual lleva al problema inicial) o descartar todo lo que aún no se haya publicado, perdiendo posible información relevante (poco probable, pero no imposible). Aunque esta última solución al menos evita el bloqueo del programa, es preferible encontrar una alternativa no destructiva.

La solución elegida para esta implementación es imponer un *throttling* (es decir, una limitación) a la velocidad a la que la consola de la GUI puede publicar mensajes. Con una llamada `Thread.sleep(1)`, se impone un máximo de 1000 impresiones de enteros y caracteres por segundo. En el caso de la publicación de cadenas, por precaución, se opta por dinamizar el *throttling*: se divide la cadena en fragmentos de 10.000 caracteres y se publican con un milisegundo de diferencia.

El resultado sigue siendo más rápido que en ENS2001, y soluciona la saturación derivada del exceso de mensajes por segundo. Solamente afecta a la salida de datos, y solamente para la GUI; la versión de línea de comandos no se ve afectada, ya que no sufría este problema en primer lugar.

5.5. Decisiones de implementación

Aunque a lo largo de este capítulo se han ido justificando las decisiones de diseño e implementación de cada módulo del proyecto, existen algunas decisiones transversales que afectan a la base del proyecto en su conjunto, más allá de la implementación particular de cada elemento del mismo.

5.5.1. La elección de JavaFX frente a Swing

La decisión de utilizar JavaFX en lugar de Swing como tecnología principal para la interfaz de usuario del sistema nace de la búsqueda de una experiencia de usuario moderna, visualmente atractiva y con capacidad de evolución. Si bien es cierto que Swing es una biblioteca robusta y ampliamente utilizada, JavaFX ofrece una serie de ventajas significativas que se alinean mejor con las necesidades actuales y futuras de las aplicaciones de escritorio.

De entre ellas, dado que el alcance gráfico del proyecto no es particularmente amplio, destacan dos:

- **Enlaces de propiedades:** JavaFX introduce un sistema de propiedades y enlaces que

²Cifra calculada tras realizar un volcado de datos a fichero y contar las líneas del volcado de consola.

³De nuevo, esto dificulta tomar una métrica adecuada. ¿Realmente ha transcurrido el mismo tiempo que en ENS2001, o el propio bloqueo del programa ha retrasado la gestión del evento de parada?

simplifica la sincronización de datos entre la lógica de la aplicación y la interfaz de usuario. Esto reduce la cantidad de código *boilerplate* y facilita la creación de interfaces dinámicas y actualizadas automáticamente. Swing carece de este mecanismo integrado, lo que a menudo lleva a la necesidad de gestionar manualmente las actualizaciones de los componentes, mientras que JavaFX genera de manera implícita las notificaciones a los observadores pertinentes cada vez que una propiedad experimenta un cambio.

- **Estilo mediante CSS:** A diferencia de Swing, que necesita de forma explícita editar todos los atributos gráficos de un nodo mediante setters, JavaFX permite la aplicación de plantillas CSS a todo el proyecto, o incluso la aplicación de CSS “en línea” a componentes específicos, con mayor prioridad que el impuesto por las hojas de estilo. Aunque no se ha explotado en exceso esta funcionalidad, es un elemento que se ha tenido en cuenta.

5.5.2. Java 11 como versión mínima

Esta decisión atiende a una combinación de necesidades y restricciones surgidas de la decisión de empleo de JavaFX como módulo gráfico para la aplicación con GUI.

La intención inicial era usar la versión LTS⁴ más antigua posible que diera soporte a funcionalidades clave del proyecto, como las interfaces funcionales que permiten almacenar las referencias de los constructores de las instrucciones como referencias en mapas. Dicha versión, lógicamente, sería Java 8, que es con la que se añadieron esas interfaces al lenguaje.

Por otro lado, JLink, la herramienta utilizada para la generación del directorio ejecutable, se introdujo en Java 9 como parte del sistema de módulos de Java, JPMS. Además, JavaFX fue desacoplado del JDK con el lanzamiento de Java 11 para convertirse en un conjunto de módulos separados. Las implicaciones de estos factores son las siguientes:

- **JavaFX en Java 8 no es modular:** JLink funciona por módulos, y JavaFX era parte del JDK estándar, no un módulo.
- **JLink requiere descriptores de módulo:** Esto se consigue mediante el uso del fichero `module-info.java`. De nuevo, el hecho de que JavaFX no fuera un módulo reconocible en Java 8 dificulta la tarea.
- **Librerías no modulares:** Si bien JLink puede trabajar con JAR antiguos, esto requiere el uso de versiones posteriores a Java 8 igualmente; y esto no resuelve el problema de que JavaFX no es un módulo.

Dada la complejidad de la disyuntiva técnica y de las posibles soluciones, se ha optado por escoger Java 11 como versión mínima. De esta forma, el proyecto generado por JLink y listo para su distribución incluye JavaFX de forma explícita.

En cualquier caso, se ha buscado compatibilidad con compiladores de Java 8, por si un usuario tuviera un interés o necesidad especial en compilar el código en Java 8 de manera estándar en vez de usar la versión distribuida directamente o crear un nuevo directorio con JLink. Esto

⁴*Long-Term Support*, soporte a largo plazo

se traduce en el uso de métodos más antiguos, como `trim()` en vez de `strip()`, o el uso de bloques estáticos y `Collections.unmodifiableMap()` en vez de `Map.ofEntries()` para definir los mapas inmutables que contienen las distintas relaciones entre elementos.

5.5.3. El uso de JLink frente a JPackage y a un *fat JAR*

La elección de JLink frente a JPackage viene determinada por el perfil principal del usuario descrito en el documento de requisitos: estudiantes universitarios de una asignatura concreta. La principal ventaja que ofrece JLink frente a JPackage es que genera una aplicación no invasiva:

- **Portabilidad y cero instalación:** El estudiante solamente tiene que descargar y descomprimir el directorio con la aplicación para empezar a utilizarla en el momento, sin necesidad de ejecutar un instalador.
- **Impacto mínimo en el sistema:** Al no requerir instalación, no se modifican las rutas del sistema ni el registro de Windows, los ficheros de propiedades `.plist` de MacOS ni los ficheros de configuración del sistema de Linux ubicados en `/etc`. Una vez el estudiante haya cursado la asignatura, si quiere deshacerse de ENS2025 solamente tiene que eliminar el directorio que lo contiene, no pasar por un proceso de desinstalación.
- **Versión:** JPackage fue introducido como herramienta en producción con la llegada de Java 16, lo que implica una problemática de versión mínima similar a la expuesta en el apartado anterior.

Por otro lado, aunque un *fat JAR* (un JAR que contiene todas las dependencias necesarias para su ejecución, sin depender de librerías externas del sistema) podría servir como alternativa compacta, autocontenida y portable, éste sí necesita de una JVM para funcionar. Si bien esto no es necesariamente un problema (al fin y al cabo, la versión de consola está compilada en un JAR estándar), vuelve a surgir la problemática de la estructura de módulos: un *fat JAR* no es una herramienta apropiada para distribuir proyectos de Java que utilizan módulos. El proceso de creación de un JAR aplanar la estructura de módulos, cargando todas las clases en un único "módulo sin nombre". Generalmente, esto no suele tener consecuencias más allá de un *warning* al inicio de la ejecución advirtiendo de esta situación; pero, habiendo otras herramientas que permiten un despliegue sin necesidad de arriesgarse, es mejor optar por ellas. En contraposición, JLink y JPackage generan un *runtime* de Java modular autocontenido y mínimo que permite al sistema funcionar en entornos sin un JDK preinstalado o que no incluyan los módulos que requiere la aplicación (en este caso, JavaFX). Sin embargo, nada impide al usuario compilar el proyecto en un *fat JAR* para su uso si así lo prefiere bajo su cuenta y riesgo.

Por lo tanto, al igual que la elección de Java 11, la elección de JLink es una solución de compromiso que busca ampliar al máximo el abanico de compatibilidades.

Capítulo 6

Pruebas

En este capítulo se detallan las pruebas realizadas para validar el funcionamiento de ENS2025. Dado que las pruebas consisten en una gran cantidad de ficheros cortos para hacer pruebas específicas, no se incluye el contenido de los mismos en esta memoria, sino que se agruparán por temática y se comentará la finalidad y resultado esperado de cada uno.

Las baterías de pruebas están diseñadas para abarcar todo el rango de instrucciones, realizando comprobaciones variadas como la validez de los operandos, los rangos de valores permitidos según el tipo de direccionamiento e incluso pruebas específicas para subconjuntos de instrucciones, como las instrucciones de salto, las de escritura o la correcta modificación del registro de estado por las aritméticas.

Aunque el comportamiento de ENS2025 tras finalizar la fase de implementación ya era bastante cercano al deseado, estas baterías de pruebas han constituido una valiosa herramienta para validar aquellos comportamientos esperados y corregir los inesperados. No se informará en cada apartado de si ENS2025 ha pasado la batería de pruebas del mismo, ya que las ha pasado todas, ensamblando y ejecutando correctamente con el código fuente apropiado y generando los errores pertinentes cuando el código de prueba estaba diseñado con expectativas de dicha respuesta.

Los ficheros de prueba han sido simulados tanto en ENS2025 como en ENS2001, para garantizar que el comportamiento del primero replica el del segundo, excluyendo aquellos casos en los que se haya identificado que ENS2001 no estaba cumpliendo con su propio comportamiento esperado y estaba actuando de manera incorrecta, en cuyo caso se ha buscado que ENS2025 se comporte ante las pruebas del mismo modo en que ENS2001 *debería* haberse comportado. Finalmente, algunos de los ficheros suministrados junto a ENS2001 ni siquiera funcionaban bien con dicho simulador, por lo que han sido modificados para que funcionen correctamente e idéntica en ambas versiones de la herramienta. Si bien dichos ficheros han sido renombrados para encajar en la nomenclatura de los ficheros de pruebas usados, la autoría original de los mismos ha sido debidamente acreditada dentro de los comentarios del código.

Todos los ficheros de prueba se suministran con la memoria y el código.

6.1. Pruebas de operandos y direccionamientos

Esta batería de pruebas contiene un gran número de ficheros fuente, con nombres siguiendo el formato `INST_X.ens`, donde `INST` es la instrucción sobre la que hacen pruebas, y `X` el número de fichero, agrupados en carpetas según la instrucción a la que hacen referencia. Estos ficheros tienen como objetivo determinar los siguientes aspectos:

- **Generación de errores por número incorrecto de operandos:** Algunos ficheros contienen instrucciones con operandos de menos o de más, esperando generar un error.
- **Generación de errores por modos de direccionamiento incorrectos:** Algunos ficheros contienen modos de direccionamiento no aceptados por la instrucción en cuestión. Para las instrucciones de dos operandos, hay ficheros de error con un primer operando incorrecto y otros con un primer operando correcto y otro incorrecto.
- **Corrección en número de operandos y direccionamientos:** Para terminar de cubrir todos los casos, cada instrucción tiene asociado un fichero que contiene varias instancias de la misma con el número correcto de operandos y con todos los modos de direccionamiento correctos, para demostrar que son aceptados por el ensamblador. En las instrucciones de dos operandos no se comprueban todas las permutaciones posibles de direccionamientos, ya que no existen reglas de compatibilidad entre sí. solamente se prueban todos los direccionamientos posibles para un primer operando con un direccionamiento válido en el segundo y todos los direccionamientos posibles para un segundo operando con un direccionamiento válido en el primero.

Sea, por ejemplo, el código de la figura 6.1. En este caso, se pueden observar los modos de direccionamiento válidos para ambos operandos de la instrucción `ADD`.

```

1      ;Pruebas de validez del primer operando
2      ADD #0, .R0
3      ADD .R1, .R0
4      ADD /0, .R0
5      ADD [.R1], .R0
6      ADD #1[.IX], .R0
7      ADD #1[.IY], .R0
8
9      ;Pruebas de validez del segundo operando
10     ADD #0, #0
11     ADD #0, .R0
12     ADD #0, /0
13     ADD #0, [.R0]
14     ADD #0, #1[.IX]
15     ADD #0, #1[.IY]
16

```

Figura 6.1: Contenido del fichero de pruebas de validez de operandos de la instrucción `ADD`.

En cambio, en la figura 6.2 se puede observar una de las pruebas para un tipo de direccionamiento no compatible con dicha instrucción.

```

1      ;Prueba de invalidez: segundo operando con direccionamiento de PC
2      ADD #0, $10
3

```

Figura 6.2: Contenido de un fichero de prueba de invalidez de modo de direccionamiento para la instrucción `ADD`.

En la figura 6.3 se puede observar un ejemplo de prueba con un número incorrecto de operandos.

```
1      ;Fallo al ensamblar, se esperan argumentos
2      ADD #0
3
```

Figura 6.3: Contenido de un fichero de prueba de invalidez de número de operandos para la instrucción ADD.

Finalmente, en el caso de las instrucciones sin operando se unifican las pruebas en un único fichero, como es el caso de la instrucción HALT en la figura 6.4. La primera instrucción está en la forma válida, sin operandos, y la segunda tiene un operando añadido. El ensamblador deberá leer y ensamblar correctamente la primera, y generar un error en la segunda.

```
1      ;Esto ensambla correctamente
2      HALT
3
4      ;Esto no, con un error que indica que se espera fin de línea (no
5      se admiten argumentos)
6      HALT #0
```

Figura 6.4: Contenido del fichero de pruebas de validez/invalidez de operandos de la instrucción HALT.

6.2. Pruebas de dos operandos en una única palabra

Estas pruebas están compuestas por un único fichero, cuyo código se puede ver en la figura 6.5, y que contiene exclusivamente instrucciones de dos operandos. Estas instrucciones combinan los distintos modos de direccionamiento que no necesitan una palabra completa, de tal manera que, al ser ensambladas, el resultado en un visor de memoria debería mostrar instrucciones que solamente requieren dos palabras pese a ser de dos operandos, y que la información pertinente a cada operando se ha almacenado correctamente en la mitad de palabra que les corresponde.

```
1      ;Pruebas de instrucciones de dos operandos con direccionamientos
2      tales que ambos operandos caben en la segunda palabra
3      MOVE .R0, .R1
4      MOVE #23[.IX], #0xFF[.IY]
5      MOVE [.R5], [.A]
6
7      ADD .R9, #-1[.IY]
8      SUB [.R2], .R0
9      MUL #255[.IX], [.SR]
10     DIV [.A], #0x7A[.IX]
```

Figura 6.5: Contenido del fichero de pruebas de instrucciones de dos operandos que comparten una misma palabra.

6.3. Pruebas de instrucciones aritmético-lógicas

Estas pruebas, que constan de tres ficheros, se enfocan en la correcta ejecución de las instrucciones aritméticas y lógicas, así como de comprobar la correcta modificación de los biestables cuando procede. Dos de los ficheros, `alu_0.ens` y `alu_1.ens`, se corresponden con los ficheros de prueba `ENS-PRI-009.ens` y `ENS-PRI-010.ens`, respectivamente, cuyo código se encuentra en el anexo de pruebas de la memoria de trabajo de ENS2001 [1]. El primero es una prueba exhaustiva de la ejecución de instrucciones aritméticas, algunas de las cuales se pueden ver en la figura 6.6, y el segundo de las lógicas, como se puede apreciar en la figura 6.7; sin embargo, `ENS-PRI-009.ens` ha sido modificado para mostrar por consola el valor del registro de estado y el registro acumulador tras cada instrucción aritmética, para poder comprobar todos los resultados en una sola orden de ejecución sin necesidad de recurrir a puntos de ruptura o ejecución paso a paso. Además, `ENS-PRI-009.ens` incluía algunas pruebas sobre biestables. Si bien se han mantenido, se ha ampliado sobre las mismas en un tercer fichero original, `alu_2.ens`, que realiza pruebas de biestables individuales y múltiples, y del cual se puede ver un extracto en la figura 6.8.

```
1  DEC [.R6]
2  WRINT [.R6]
3  WRCHAR #10
4  WRINT .SR
5  WRSTR /0x1000
6
7  NEG #0xFF[.IX]
8  WRINT #0xFF[.IX]
9  WRCHAR #10
10 WRINT .SR
11 WRSTR /0x1000
12
```

Figura 6.6: Extracto del fichero de pruebas de instrucciones aritméticas.

```
1  AND #6,#5
2  WRINT .A
3  WRSTR /eol
4
5  OR #6,#5
6  WRINT .A
7  WRSTR /eol
8
```

Figura 6.7: Extracto del fichero de pruebas de instrucciones lógicas.

6.4. Pruebas de instrucciones de control de subrutinas

Esta batería de pruebas está integrada por dos ficheros, `subroutine_stack_positive.ens` y `subroutine_stack_negative.ens`, ambos versiones de `ENS-PRI-012`, encontrado en la memoria original de ENS2001 [1]. Esta prueba usa varias instancias de la instrucción `CALL` para

```

1      ;Biestable S:
2
3      ADD #-2, #1
4
5      ;la salida debe ser 0x0010
6      WRINT .SR
7      WRCHAR #10
8
9      ;Pruebas de biestables multiples:
10
11     ;Biestables ZC
12     ADD #65535, #1
13
14     ;la salida debe ser 0x0003
15     WRINT .SR
16     WRCHAR #10
17

```

Figura 6.8: Extracto del fichero de pruebas de biestables.

navegar por subrutinas definidas, imprimiendo el valor del puntero de pila en cada una. La diferencia entre ambos ficheros radica simplemente en que el original incluía la instrucción `MOVE` para desplazar el `SP` a las posiciones `0x7FFF` o `0xFFFF` en función de si la pila era, respectivamente, creciente o decreciente. Como se puede inferir por los nombres, se ha separado dicho comportamiento en dos ficheros por comodidad. Además, se ha corregido un error en el que una subrutina se llamaba a sí misma, generando un bucle infinito. Un extracto del código de uno de estos ficheros (no se considera necesario reproducir ambos) se puede consultar en la figura 6.9.

```

1      MOVE #0x7FFF,.SP ; Pila creciente
2      CALL /sub1
3      CALL /sub2
4      CALL /sub3
5
6      HALT
7
8      sub1: WRSTR /id1
9            WRSTR /eol
10           WRSTR /rsp
11           WRINT .SP
12           WRSTR /eol
13           CALL /sub2
14           RET
15           ;[...]
16      ORG 60000
17
18      id1:  DATA "Subrutina 1"
19      id2:  DATA "Subrutina 2"
20      id3:  DATA "Subrutina 3"
21      rsp:  DATA "Valor de SP="
22      eol:  DATA "\n"
23

```

Figura 6.9: Extracto del fichero de pruebas de instrucciones de control de subrutinas. Se han cortado fragmentos del código original para poder representar las secciones del mismo sin alargarlo demasiado.

6.5. Pruebas de instrucciones de salto

Esta batería de pruebas consta de dos ficheros, `branch_0.ens` y `branch_1.ens`. El primero, cuyo código se puede consultar en la figura 6.10, comprueba simplemente desplazamientos incondicionales usando distintos modos de direccionamiento, así como saltos mediante desplazamiento que desbordan la frontera entre el inicio y final de la memoria. El segundo está adaptado del fichero `ENS-PRI-011.ens`, contenido en la memoria de ENS2001 [1], modificado para suprimir un error de bucle infinito al final del mismo. Consiste en el uso de distintas instrucciones para modificar los biestables, ejecutar instrucciones de salto condicional en base al estado de dichos biestables y saltar a instrucciones de escritura en consola una serie de cadenas, coincidentes con el nombre de la instrucción de salto ejecutada. Un extracto de este fichero se puede ver en la figura 6.11.

```
1      ;Prueba de validez: desplazamientos validos
2
3      BR /100
4
5      ORG 200
6      BR /-50
7
8      ;Desplazamiento con "desbordamiento"
9      ORG -5
10     BR $lab1
11
12     ORG 120
13     lab1: BR $lab2
14
15     ORG -3
16     lab2: BR $3
17
18     ORG 5
19     BR $-100
20
21     ORG -100
22     BR $127
23
```

Figura 6.10: Contenido del fichero de pruebas de instrucciones de salto incondicional.

6.6. Pruebas de instrucciones entrada/salida

Esta batería de pruebas, compuesta por los ficheros `io_0.ens` y `io_1.ens`, comprueba el funcionamiento adecuado de las instrucciones de entrada y salida. El primer fichero coincide con el fichero `ENS-PRI-013.ens` incluido en la memoria de ENS2001 [1], y consiste en pruebas de lectura y escritura mediante direccionamiento directo a memoria con etiquetas, mientras que el segundo es una versión modificada de éste que emplea distintos direccionamientos. Extractos de ambos ficheros se pueden encontrar en las figuras 6.12 y 6.13, respectivamente.

```

1          BR /e_br
2
3      n_br:  CMP #0xFF,#0xFF
4            BZ  /e_bz
5
6      n_bz:  ADD #0,#1
7            BNZ /e_bnz
8
9      n_bnz:  ;[...]
10
11     e_br:   WRSTR /s_br
12           BR /n_br
13     e_bz:   WRSTR /s_bz
14           BR /n_bz
15     e_bnz:  ;[...]
16
17     s_br:   DATA "BR\n"
18     s_bz:   DATA "BZ\n"
19     s_bnz:  ;[...]
20

```

Figura 6.11: Extracto del fichero de pruebas de instrucciones de salto condicional. Se han cortado fragmentos del código original para poder representar las secciones del mismo sin alargarlo demasiado.

```

1          WRSTR /cad01
2          ININT /entero
3
4          WRSTR /cad11
5          WRINT /entero
6          WRSTR /eol
7          ;[...] se omite código similar para comprobar INCHAR/
WRCHAR e INSTR/WRSTR
8
9      cad01: DATA "Introduzca un entero"
10     cad11: DATA "El entero leído es: "
11           ;[...]
12     entero: RES 1
13     caracter: RES 1
14     cadena: RES 0xFF
15

```

Figura 6.12: Extracto del fichero de pruebas de instrucciones de entrada/salida con direccionamiento a memoria. Se han cortado fragmentos del código original para poder representar las secciones del mismo sin alargarlo demasiado.

6.7. Pruebas de excepciones en expresiones

Esta batería de pruebas, que está formada por los ficheros `expression_exception_00.ens` a `expression_exception_13.ens`, comprueba que se generan correctamente distintos errores al procesar expresiones. Dado que cada fichero contiene una única línea con una expresión, se reproduce el conjunto del código en la figura 6.14.

```

1      ;[...] se omite codigo similar para comprobar ININT/WRINT
con otros direccionamientos
2      WRSTR /cad02
3      INCHAR #-99[.IY]
4      WRSTR /cad12
5      WRCHAR /caracter
6      WRSTR [.R3]
7      ;[...] se omite codigo similar para comprobar INSTR/WRSTR
con otros direccionamientos
8
9      cad02: DATA "Introduzca un caracter: "
10     cad12: DATA "El caracter leido es: "
11           ;[...]
12           entero: RES 1
13           caracter: RES 1
14           cadena: RES 0xFF
15

```

Figura 6.13: Extracto del fichero de pruebas de instrucciones de entrada/salida con otros direccionamientos. Se han cortado fragmentos del código original para poder representar las secciones del mismo sin alargarlo demasiado.

```

1      ORG 65500 + 100
2      ORG (
3      ORG )
4      ORG -0x15
5      ORG *-1
6      ORG +1
7      ORG 1*
8      ORG 1 3
9      ORG - +
10     ORG (3)(4)
11     ORG 3(4)
12     ORG ()
13     ORG 2*-1
14     ORG 70000
15

```

Figura 6.14: Contenido de los distintos ficheros de pruebas de excepciones en expresiones.

6.8. Pruebas de etiquetas

Esta batería de pruebas, formada por los ficheros `label_0.ens` a `label_3.ens`, comprueba que se generan correctamente distintos errores al procesar etiquetas. Dado que cada fichero contiene unas pocas líneas, se reproduce el conjunto del código en la figura 6.15.

6.9. Pruebas de procesamiento de texto

Esta batería de pruebas, formada por los ficheros `parsing_0.ens` a `parsing_2.ens`, comprueba que se generan correctamente distintos errores al intentar procesar texto para construir instruc-

```
1           ;Prueba de invalidez: concatenar dos etiquetas
2 lab1: lab2: NOP
3
4           ;Prueba de invalidez: etiqueta duplicada
5 lab3: NOP
6 lab3: NOP
7
8           ;Prueba de invalidez: etiqueta no definida
9 WRINT /undefined
10
11          ;Prueba de invalidez: etiqueta con nombre de palabra
12 reservada
13 org: org 0
```

Figura 6.15: Contenido de los distintos ficheros de pruebas de etiquetas.

ciones. Dado que cada fichero contiene unas pocas líneas, se reproduce el conjunto del código en la figura 6.16.

```
1           ;Prueba de invalidez: ausencia de elemento separador entre
2 etiqueta e instruccion
3 label HALT
4
5           ;Prueba de invalidez: ausencia de elemento separador entre
6 operandos
7 label: ADD #1 #3
8
9           ;Prueba de invalidez: contenido mas alla del segundo
operando
label: ADD #1, #3, #5
```

Figura 6.16: Contenido de los distintos ficheros de pruebas de procesamiento de texto.

6.10. Pruebas de estrés

Esta batería de pruebas, formada por los ficheros `stress_wrchar.ens`, `stress_wrint.ens` y `stress_wrstr.ens`, contienen bucles infinitos de las instrucciones que dan nombre a dichos ficheros. En este caso, no se busca generar un error, sino comprobar que la ejecución es fluida y que el sistema no colapsa pese a recibir un gran número de peticiones de escritura por consola en un período muy corto de tiempo. Esta batería de pruebas se diseñó específicamente para comprobar la eficacia de la medida de *throttling* tomada para solucionar el problema de cuello de botella sufrido por la consola de la versión GUI de la herramienta. Los dos primeros ficheros siguen un formato idéntico al de la figura 6.17, sustituyendo, obviamente, `WRCHAR` por `WRINT` en el segundo. El tercer fichero declara una cadena de datos de aproximadamente 60.000 caracteres y llama en bucle a `WRSTR` de la misma forma que los dos primeros.


```
1 WRCHAR #99
2 BR /0
3
4 HALT
5
```

Figura 6.17: Código de la prueba de estrés de la instrucción WRCHAR.

6.11. Pruebas de excepciones por división entre cero

Esta batería de pruebas, formada por los ficheros `zero_division_0.ens` a `zero_division_5.ens`, comprueba que se generan correctamente distintos errores al realizar divisiones entre cero mediante distintos direccionamientos. Dado que cada fichero contiene unas pocas líneas, se reproduce el conjunto del código en la figura 6.18.

```
1 ;Direccionamiento inmediato
2 DIV #3, #0
3
4 ;Direccionamiento directo a registro
5 MOVE #0, .R0
6 DIV #30, .R0
7
8 ;Direccionamiento directo a memoria
9 MOVE #0, /1000
10 DIV #30, /1000
11
12 ;Direccionamiento indirecto
13 MOVE #1000 .R1
14 MOVE #0, [.R1]
15 DIV #10, [.R1]
16
17 ;Direccionamiento relativo a indice (positivo)
18 MOVE #1000, .IX
19 MOVE #0, #120[.IX]
20 DIV #400, #120[.IX]
21
22 ;Direccionamiento relativo a indice (negativo)
23 MOVE #1000, .IY
24 MOVE #0, #-120[.IX]
25 MOD #400, #-120[.IX]
26
```

Figura 6.18: Contenido de los distintos ficheros de pruebas de división entre cero.

6.12. Pruebas de directivas del ensamblador

Como su nombre indica, estas pruebas están orientadas a comprobar el correcto funcionamiento de las directivas definidas. Cada una de las directivas tiene asignado un fichero, salvo la directiva `DATA`, que necesita de varios para hacer todas las comprobaciones. Se definen a continuación los propósitos de los distintos ficheros:

- **Fichero equ.ens:** Asigna valores a varias etiquetas usando distintos modos de representación. Después, imprime el valor de las etiquetas. El código puede consultarse en la figura 6.19.
- **Fichero end.ens:** Verifica que el ensamblador deja de actuar tras esta instrucción, como se muestra en la figura 6.20. El fichero debe poder ensamblar sin errores.
- **Fichero res.ens:** Reserva varios tamaños de memoria en diferentes formatos de representación, unos a continuación de otros. Después, imprime los valores de las etiquetas para mostrar las direcciones de memoria en las que se han definido como prueba del desplazamiento generado por la reserva de memoria. El código puede verse en la figura 6.21.
- **Fichero org.ens:** Ensambla una instrucción en distintos puntos de la memoria y usando diferentes modos de representación. Después, imprime el contenido de las direcciones de memoria en las que se ha escrito el código (así como de las dos siguientes palabras) para verificar que, efectivamente, se ha escrito donde **ORG** indicaba. Parte del código puede verse en la figura 6.22.
- **Ficheros data.X.ens:** El primero de estos ficheros, **data_0.ens**, define dos cadenas de datos correctas, una de valores enteros y otra de cadenas de caracteres, esta última con cadenas que contienen secuencias de escape. Imprime también los contenidos de ambas secuencias. El código de este fichero se puede consultar en la figura 6.23. El resto de ficheros de esta directiva generan distintos tipos de error: secuencias de escape incorrectas, terminar una directiva con una coma, no cerrar unas comillas abiertas, escribir una cadena que excede la memoria disponible o no usar comas para separar elementos de la cadena.

```
1 lab1: EQU 0
2 lab2: EQU -1
3 lab3: EQU -32768
4 lab4: ;[...] Continua hasta la etiqueta lab9
5
6 WRINT #lab1
7 WRCHAR #10
8 WRINT #lab2
9 WRCHAR #10
10
```

Figura 6.19: Extracto del fichero de pruebas de validación de la directiva EQU.

```
1 END
2
3 Se puede escribir cualquier cosa a partir de la directiva sin
4 necesidad de hacer comentarios porque el ensamblador se detiene al
   procesarla.
```

Figura 6.20: Contenido del fichero de pruebas de validación de la directiva END.

```
1          RES 10
2  lab1:  INC .R1
3          RES 0xA
4  lab2:  INC .R2
5          RES -32000
6  lab3:  INC .R3
7          WRINT #lab1
8          WRCHAR #10
9          WRINT #lab2
10         WRCHAR #10
11         WRINT #lab3
12         WRCHAR #10
13         HALT
14
```

Figura 6.21: Contenido del fichero de pruebas de validación de la directiva RES.

6.13. Pruebas de restricciones de valores

Esta batería de pruebas, que está compuesta por los ficheros `value_constraint_0.ens` a `value_constraint_8.ens`, se divide en dos partes. Por un lado, todos los ficheros menos el último comprueban valores fuera de rango para distintos modos de direccionamiento. Al ser ficheros breves, se unifican en la figura 6.24. Por otro lado, el último fichero contiene pruebas de validación del funcionamiento de conversión implícita. Esto es, cuando un entero supera el valor máximo con signo, pero se mantiene dentro del rango de valores sin signo, se interpreta como un valor negativo. En el fichero se incluyen comentarios indicando cómo deberían leerse las instrucciones desensambladas, señalando que el direccionamiento directo a memoria emplea siempre representación sin signo. También incluye la casuística inversa: empleo de valores negativos que se verán convertidos a enteros sin signo en el direccionamiento directo a memoria por el desensamblador. El contenido de este fichero puede consultarse en la figura 6.25.

```
1      ;Establecimiento del primer bloque de codigo
2      ORG 50
3      MOVE #-1, .R3
4
5      ;Establecimiento del segundo bloque de codigo
6      ORG 100
7      MOVE #-2, .R4
8
9      ;Establecimiento del tercer bloque de codigo
10     ORG 0xFF
11     MOVE #-3, .R5
12
13     ;Establecimiento del cuarto bloque de codigo
14     ORG -32000
15     MOVE #-4, .R6
16
17     ;Impresion del primer bloque de codigo
18     WRINT /50
19     WRCHAR #10
20     WRINT /51
21     WRCHAR #10
22     WRINT /52
23     WRCHAR #10
24     WRCHAR #10
25
26     ;Impresion del segundo bloque de codigo
27     WRINT /100
28     WRCHAR #10
29     WRINT /101
30     WRCHAR #10
31     WRINT /102
32     WRCHAR #10
33     WRCHAR #10
34
35     ;Impresion del tercer bloque de codigo
36     WRINT /0xFF
37     WRCHAR #10
38     WRINT /0x100
39     WRCHAR #10
40     WRINT /0x101
41     WRCHAR #10
42     WRCHAR #10
43
44     ;Impresion del cuarto bloque de codigo
45     WRINT /-32000
46     WRCHAR #10
47     WRINT /-31999
48     WRCHAR #10
49     WRINT /-31998
50     WRCHAR #10
51     WRCHAR #10
52
53     HALT
54
```

Figura 6.22: Extracto del fichero de pruebas de validación de la directiva ORG.

```
1          ORG 100
2 lab1: DATA 1, 2, 3, 4, -1, 0x8000
3          ORG 120
4 lab2: DATA "a", "abc", "a\t", "\0", "\"", "\\", "a\0b"
5
6          WRINT /100
7          WRCHAR #10
8          WRINT /101
9          WRCHAR #10
10         WRINT /102
11         WRCHAR #10
12         WRINT /103
13         WRCHAR #10
14         WRINT /104
15         WRCHAR #10
16         WRINT /105
17         WRCHAR #10
18         WRINT /106
19         WRCHAR #10
20
21         WRSTR /120
22         WRCHAR #10
23         WRSTR /122
24         WRCHAR #10
25         WRSTR /125
26         WRCHAR #10
27         WRSTR /128
28         WRCHAR #10
29         WRSTR /130
30         WRCHAR #10
31         WRSTR /132
32         WRCHAR #10
33         WRSTR /134
34         WRCHAR #10
35
36         HALT
37
```

Figura 6.23: Contenido del fichero de pruebas de validación de la directiva DATA.

6.14. Programas de ejemplo

Además de los distintos ficheros de código creados para validar el funcionamiento de la herramienta, se adjuntan varios ficheros con programas de ejemplo. Dichos programas se heredan directamente de ENS2001 (algunos, a su vez, heredados de ENS96), y han sido debidamente modificados y corregidos para que funcionen correctamente. A continuación, se enumeran dichos ficheros y se describen brevemente:

- **Fichero `matriz.ens`:** Permite almacenar valores en una matriz 5x5 y mostrarla al usuario. Corregido un error que impedía añadir valores a la quinta fila y columna.
- **Fichero `fact.ens`:** Calcula el factorial de un valor dado.
- **Fichero `fact.rec.ens`:** Igual que el programa anterior, pero usando recursividad.

```

1      ;Prueba de invalidez: entero superando la cota maxima en
      direccionamiento inmediato
2      ADD #65536, .R0
3
4      ;Prueba de invalidez: entero superando la cota maxima en
      direccionamiento directo a memoria
5      ADD /65536, .R0
6
7      ;Prueba de invalidez: entero superando la cota minima en
      direccionamiento inmediato
8      ADD #-32769, .R0
9
10     ;Prueba de invalidez: entero superando la cota minima en
      direccionamiento directo a memoria
11     ADD /-32769, .R0
12
13     ;Prueba de invalidez: entero superando la cota minima en
      direccionamiento relativo a indice
14     ADD #-129[.IX], .R0
15
16     ;Prueba de invalidez: entero superando la cota maxima en
      direccionamiento relativo a indice
17     ADD #256[.IX], .R0
18
19     ;Prueba de invalidez: entero superando la cota minima en
      direccionamiento relativo a PC
20     BR $-129
21
22     ;Prueba de invalidez: entero superando la cota maxima en
      direccionamiento relativo a PC
23     BR $256
24

```

Figura 6.24: Contenido de los distintos ficheros de pruebas de restricciones de valores.

- **Fichero maxmin.ens:** Solicita al usuario valores enteros, tanto positivos como negativos, hasta que recibe un cero. Después, muestra los valores máximo y mínimo recibidos. Corregido un error que impedía al algoritmo procesar correctamente los valores negativos, mostrándolos en ocasiones como valores superiores y en ocasiones inferiores a los positivos.
- **Fichero cadenas.ens:** Solicita al usuario dos cadenas de caracteres. Después, las muestra concatenadas, para luego invertir la cadena resultante.

6.15. Pruebas de despliegue

Para finalizar, se ha comprobado que, efectivamente, la versión GUI de ENS2025 es compatible con los principales sistemas operativos. Para ello, se ha compilado el proyecto en Windows, MacOS y Linux mediante JLink, generando los directorios específicos de cada uno, y se ha ejecutado.

El resultado es mayormente satisfactorio: el funcionamiento del sistema, elemento primordial, es el esperado. Sin embargo, se ha encontrado que el tamaño y apariencia del contenido de los

```

1      ;Prueba de validez: valores que exceden la cota maxima con signo
      pero son menores que la cota maxima sin signo
2      ;Esto provoca que sean convertidos a representacion con signo
      automaticamente en modos distintos al direccionamiento a memoria
3
4      ADD #65535, .R0
5      ADD /65535, .R0
6      ADD #32768, .R0
7      ADD /32768, .R0
8      ADD #128[.IX], .R0
9      ADD #255[.IX], .R0
10     BR $128
11     BR $255
12
13     ;En el visor de codigo deberia leerse esto al visualizarse con
      representacion decimal con signo:
14
15     ;ADD #-1, .R0
16     ;ADD /65535, .R0
17     ;ADD #-32768, .R0
18     ;ADD /32768, .R0
19     ;ADD #-128[.IX], .R0
20     ;ADD #-1[.IX], .R0
21     ;BR $-128
22     ;BR $-1
23
24     ;Del mismo modo, el uso de valores negativos que no excedan el
      valor negativo minimo permitido se convertira a representacion sin
      signo
25     ;en el modo de direccionamiento directo a memoria, quedando igual
      en el resto
26
27     ADD #-1, .R0
28     ADD /-1, .R0
29     ADD #-32768, .R0
30     ADD /-32768, .R0
31
32     ;En el visor de codigo deberia leerse esto al visualizarse con
      representacion decimal con signo:
33
34     ;ADD #-1, .R0
35     ;ADD /65535, .R0
36     ;ADD #-32768, .R0
37     ;ADD /32768, .R0
38

```

Figura 6.25: Contenido del fichero de pruebas de validación de restricciones de valores.

botones de la barra de herramientas (gestión de ficheros y ejecución de la simulación) varía según el SO. Esto es comprensible, ya que los botones no contienen iconos, sino caracteres Unicode redimensionados. Se considera que esto es un contratiempo menor, ya que los botones siguen mostrando los tooltips correctamente y, en el peor de los casos, el manual permite identificarlos unívocamente por la posición que ocupan.

En el caso de MacOS, sin embargo, este problema se extiende a los caracteres empleados en los visores de memoria y código. Los círculos y flechas utilizados para representar las zonas de

código y pila, los puntos de ruptura y las posiciones de los punteros no renderizan correctamente, dejando un espacio en blanco. Este suceso sí se considera crítico, ya que impide el correcto uso de la herramienta por imposibilidad de procesar visualmente la información que muestra.

Debido a la imposibilidad de probar la herramienta en un número suficientemente grande de equipos y distribuciones de SO, no se puede establecer con certeza si este comportamiento se debe a alguna particularidad del equipo empleado, de la versión en particular del SO, de una combinación de ambos, o si se trata de un comportamiento generalizado bajo ciertas circunstancias no determinadas. Por lo tanto, como solución, se ha creado una versión “a prueba de fallos” del código, donde estos símbolos se han sustituido por la letra **X** para los círculos, y los símbolos de desigualdad **<** y **>** para representar las flechas. La herramienta de MacOS viene compilada por defecto usando este código.

En cualquier caso, se suministran con la presente memoria de trabajo ambas versiones del código, así como el fichero `pom.xml` de configuración del proyecto Maven para compilar las herramientas, tanto CLI como GUI. De esta forma, el usuario puede compilar el proyecto en la versión segura para su SO si lo necesita.

Capítulo 7

Conclusiones y trabajos futuros

Finalizado el proceso de construcción del simulador ENS2025, este capítulo final sirve para recoger las conclusiones extraídas del trabajo realizado. El proyecto ha consistido en el desarrollo completo de una nueva versión del simulador, partiendo de un análisis en profundidad de su predecesor, ENS2001, para replicar su comportamiento con fidelidad y subsanar sus deficiencias. El resultado es un producto software funcional, robusto y mantenible, basado en principios de diseño sólidos y en la separación de responsabilidades. No solamente solventa los errores detectados en la versión original: además, introduce nuevas funcionalidades y garantiza portabilidad y facilidad para futuras ampliaciones. A continuación, se evalúa el grado de consecución de los objetivos específicos planteados al inicio del proyecto y se proponen diversas líneas de trabajo futuras que podrían dar continuidad al sistema y expandir el alcance del mismo.

7.1. Consecución de objetivos propuestos

Finalizado el proyecto, es momento de considerar si se han cumplido los objetivos propuestos:

- 1. Replicación del comportamiento de ENS2001:** El principal objetivo de este proyecto era crear un sistema que, como mínimo, modelase el comportamiento de simulación esperado en ENS2001 de manera exacta, con el fin de considerar esta nueva versión como candidata a la sustitución de dicho software en las prácticas de la asignatura Procesadores del Lenguaje II sin que dicha sustitución tenga un impacto negativo en el desempeño de las prácticas por parte de los estudiantes. Si bien queda a discreción del Equipo Docente implantar dicha sustitución tras realizar pruebas adicionales, todas las pruebas con distintos programas e instrucciones han tenido el comportamiento de simulación esperado.
- 2. Identificación y corrección de errores:** Aunque el funcionamiento de ENS2001 es, en líneas generales, acorde al esperado según su propia documentación, ciertos errores han persistido. Como se ha podido comprobar a lo largo de esta memoria, se ha trabajado para hacer una recopilación exhaustiva de dichos errores en los distintos módulos responsables, y se han arreglado para que el comportamiento final de ENS2025 sea acorde a las especificaciones iniciales de ENS2001.
- 3. Aportación de nuevas características:** Con la adición de funcionalidades como la comprobación de overflow/underflow, la invocación con parámetros de la CLI y la visualización y navegación de punteros adicionales a la GUI, se puede considerar cumplido este objetivo.
- 4. Portabilidad:** La interfaz gráfica de ENS2001 funcionaba únicamente para Windows y MacOS; la de ENS2025 funciona también en Linux.

5. **Creación de un código más mantenible:** A lo largo de la memoria se ha podido observar cómo el código de ENS2025 contiene numerosas abstracciones diseñadas para minimizar el impacto de cambios localizados, así como de facilitar la adición de nuevos elementos y funcionalidades sin que ello suponga tener que modificar de forma extensiva la base de código. Por lo tanto, se considera cumplido este objetivo.
6. **Mejora de la interfaz gráfica:** La aplicación de principios de diseño para hacer más accesibles las opciones de la aplicación, así como la unificación de componentes en un único elemento compacto, debería suavizar la curva de aprendizaje de los usuarios y facilitar su interés por el rango completo de funcionalidades. Por último, aunque el gusto es subjetivo, ENS2001 arrastraba, por su antigüedad, una estética de la era Windows 95/98. La renovación a JavaFX ha supuesto una modernización inmediata en ese aspecto, y aporta un toolkit sobre el que es posible iterar diseños de interfaz más avanzados.

7.2. Líneas de trabajo futuras

Existen varias opciones de desarrollo para el futuro de ENS2025 (o, como quizás sería más correcto decir, la línea de simuladores ENS).

La primera posibilidad es la inclusión de funcionalidades u optimizaciones de la lógica que, bien por falta de tiempo, bien porque la idea de implementarlas llegó en las fases finales del proyecto e implicaba realizar cambios de gran calado, fueron descartadas:

- Posibilidad de insertar una instrucción en tiempo de ejecución de la interfaz de usuario, escribiéndola en ensamblador junto a la dirección en la que debe insertarse, y dejando que el sistema la ensamble, codifique y almacene en el momento, es una opción que no debería ser difícil de implementar. Esta funcionalidad facilitaría al usuario la realización de pequeños cambios en zonas concretas del código sin tener que elegir entre abandonar la atención de la aplicación, modificar el fichero original y cargarlo de nuevo o buscar la posición de memoria donde se quiere realizar el cambio y consultar las referencias con los códigos de instrucción y direccionamiento para que el cambio sea válido.
- Usar de un fichero de configuración para la interfaz gráfica que afecte al tamaño de ciertos componentes internos, otorgando cierto control sobre las relaciones de aspecto.
- Centralizar la lógica de coordinación de propiedades y observadores de la interfaz gráfica mediante patrones Mediator. Aunque la escala del proyecto y el número de propiedades y observadores no lo hace inherentemente necesario, es otra manera de facilitar la mantenibilidad y ampliabilidad del código.
- Crear iconos para los botones de las barras de herramientas de archivo y ejecución de la interfaz gráfica. Aunque la implementación actual de caracteres Unicode es funcional para una primera versión, la creación de iconos garantiza homogeneidad entre versiones de distintos SO.

Otra opción, ya detallada en las líneas de trabajo futuras de la memoria de ENS2001, sería la incorporación de un editor de texto encargado de reconocer en tiempo real la sintaxis y de señalar errores en ella, facilitando la escritura de código correcto.

A nivel de contenido del simulador, no se han incluido más instrucciones ni modos de direccionamiento porque el objetivo principal era que debía emularse el comportamiento exacto de ENS2001; pero la estructura actual de código da facilidades para añadir dichos elementos incluso por los propios usuarios sin necesidad de tener un conocimiento extenso del sistema.

Por último, quizá la línea futura más interesante y ambiciosa sería llevar al extremo el proceso de abstracción: establecer interfaces para cada módulo y elemento de los mismos, con inyección de dependencias mediante un proveedor por capas con los objetos concretos a instanciar en cada módulo. Idealmente, esto convertiría ENS en un *framework* de componentes intercambiables con diferentes funcionalidades (memorias de mayor tamaño con mecanismos de paginación, cachés, ciclos, tiempos de acceso...), permitiendo su uso, una vez creados los componentes adecuados, para simular diferentes arquitecturas de instrucción o incluso diferentes arquitecturas de procesador.

Bibliografía

- [1] F. J. Álvarez Valero, “*ENS2001. Ensamblador y simulador del lenguaje ensamblador IEEE 694*,” Trabajo de Fin de Carrera, Universidad Politécnica de Madrid, 2003.
- [2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers. Principles, Techniques, & Tools*, 2nd ed. Pearson Education, Inc, 2007.
- [3] J. von Neumann, *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania, 1945.
- [4] I. of Electrical and E. Engineers, *IEEE Std 694-1985, IEEE Standard for Microprocessor Assembly Language*. Institute of Electrical and Electronic Engineers, 1985.
- [5] J. Larus. *SPIM: A MIPS32 Simulator*. Acceso: 2025-05-15. [Online]. Available: <https://spimsimulator.sourceforge.net>
- [6] P. L. Higginson. *AQA Assembly Language Simulator V1.3*. Acceso: 2025-05-23. [Online]. Available: <https://www.peterhigginson.co.uk/AQA>
- [7] AQA (Assessment and Qualifications Alliance). (2018) *AQA Assembly Language Instruction Set*. Acceso: 2025-05-18. [Online]. Available: <https://filestore.aqa.org.uk/resources/computing/AQA-75162-75172-ALI.PDF>
- [8] ——. *Sitio web oficial de AQA*. Acceso: 2025-05-18. [Online]. Available: <https://www.aqa.org.uk>
- [9] I. Branović, R. Giorgi, and E. Martinelli, “Webmips: a new web-based mips simulation environment for computer architecture education,” in *Proceedings of the 2004 workshop on Computer architecture education: held in conjunction with the 31st International Symposium on Computer Architecture*, ser. WCAE '04. ACM, 2004, la aplicación descrita en este artículo está disponible online en: <https://webmips.diism.unisi.it>. Acceso a ambos recursos: 2025-05-25. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/1275571.1275596>
- [10] M. A. Lerma, F. Malpartida Candel, M. A. Vicente Puente, and D. Marín Carreño. (1992) *ASS*. Autor original: Miguel A. Lerma (Enero 1992). Actualizaciones significativas por: Francisco Malpartida Candel (Mayo 1995), Miguel A. Vicente Puente (Mayo 1996), David Marín Carreño (Mayo 2002). Obtenido de Internet Archive (Snapshot: 16-07-2013). Acceso: 2025-05-12. [Online]. Available: <https://web.archive.org/web/20130716000000/http://www-lt.ls.fi.upm.es/compiladores/Software/ass13.zip>
- [11] I. of Electrical and E. Engineers, *IEEE/ISO/IEC 29148-2018, ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering*. Institute of Electrical and Electronic Engineers, 2018.
- [12] R. Rodríguez. (1996) *ENS96*. Práctica de asignatura. Acceso: 2025-05-12. [Online]. Available: dlsiis.fi.upm.es/traductores/Software/ens96201.zip

- [13] Zilog Corporation, *Z80 CPU User Manual*. Zilog Corporation, 2016, versión revisada de 2016. Acceso: 2025-04-10. [Online]. Available: <https://www.zilog.com/docs/z80/um0080.pdf>
- [14] R. C. Martin. (2000) Design principles and design patterns. Acceso: 2025-04-08. [Online]. Available: https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf
- [15] Oracle Corporation. *Naming Standards*. Acceso: 2025-04-24. [Online]. Available: https://docs.oracle.com/cd/E18727_01/doc.121/e12897/T302934T458266.htm
- [16] Agencia Estatal del Boletín Oficial del Estado, “*Resolución de 4 de abril de 2025, de la Dirección General de Trabajo, por la que se registra y publica el XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública.*” Boletín Oficial del Estado (BOE), abr 2025, número de referencia: BOE-A-2025-7766. Sección: III. Otras disposiciones. Fecha de disposición: 04-04-2025. Acceso: 04-10-2025. [Online]. Available: <https://www.boe.es/boe/dias/2025/04/16/pdfs/BOE-A-2025-7766.pdf>
- [17] P. de Miguel Anasagasti, *Fundamentos de los Computadores*, Quinta ed. Editorial Paraninfo, 1996.
- [18] R. C. Martin, *Clean Code. A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2009.
- [19] E. W. Dijkstra, *Algol 60 translation: An Algol 60 translator for the X1 and making a translator for Algol 60*. Stichting Mathematisch Centrum, 1961.
- [20] J. Nielsen, “Heuristic Evaluation,” in *Usability Inspection Methods*, J. Nielsen and R. L. Mack, Eds. John Wiley & Sons, 1994.

Apéndice A

Formato del fichero de configuración

El fichero de configuración del sistema, de nombre `ens2025.cfg`, almacena los parámetros de la configuración, así como el valor de cada uno, mediante un formato como el mostrado en la figura A.1.

```
1  CrecimientoPila=Negativo
2  BaseNumerica=DEC
3  InvasionCodigo=NO
4  InvasionPila=NO
5  ReiniciarRegistros=SI
6  Overflow=NO
7  Underflow=NO
8
```

Figura A.1: Contenidos del fichero de configuración.

Como ya se indicó en el apartado pertinente, solamente se almacena un conjunto parámetro-valor por cada línea. Las líneas pueden repetirse, en cuyo caso se almacena el último valor leído para cada parámetro determinado; no se generará error en las líneas incorrectas, y aquellos parámetros que estén ausentes serán asignados su valor por defecto. La tabla A.1 muestra de manera completa los parámetros, así como los posibles valores y valor por defecto de cada uno.

Parámetro	Valores	Valor por defecto
CrecimientoPila	Negativo Positivo	Negativo
BaseNumerica	DEC USDEC HEX	DEC
InvasionCodigo	SI NO	NO
InvasionPila	SI NO	NO
ReiniciarRegistros	SI NO	SI
Overflow	SI NO	NO
Underflow	SI NO	NO

Tabla A.1: Tabla de parámetros de configuración

Apéndice B

Manual de usuario

B.1. Introducción

ENS2025 es una herramienta dedicada al ensamblado y simulación de código ensamblador del estándar IEEE 694. Se trata de una mejora de herramientas ya existentes como ENS96 y ENS2001; en especial de esta última, que ya ofrecía una arquitectura de máquina virtual versátil y una interfaz de usuario, tanto por línea de comandos como de manera gráfica, superiores a sus predecesores. ENS2025 busca mejorar esta experiencia de usuario, añadiendo funcionalidades y facilitando el uso de las ya existentes.

El propósito de este manual es guiar al usuario de manera rápida y sencilla en el uso de la herramienta, dejando para la memoria de desarrollo de la misma los fundamentos técnicos y teóricos.

Además, este manual contiene tanto la guía de uso de la interfaz de línea de comandos como la interfaz gráfica, ya que ambas comparten el mismo sistema subyacente.

La herramienta se distribuye en tres directorios: uno para Windows, otro para MacOS y otro para Linux. Cada directorio contiene un fichero `.jar` con la versión de interfaz de línea de comandos, una carpeta con la versión de interfaz gráfica, y un acceso directo al script de lanzamiento de la versión de interfaz gráfica. Se incluye también el manual de la herramienta y el código fuente de la misma.

La versión de interfaz de línea de comandos requiere Java 11 instalado¹. La interfaz gráfica, por el contrario, no requiere la presencia de Java en el equipo. En ningún caso se requiere un proceso de instalación o desinstalación propio para la herramienta.

B.2. Arquitectura de la máquina virtual

El sistema simula una arquitectura estructurada de la siguiente forma:

- Procesador con tamaño de palabra de 16 bits.
- Memoria de 2^{16} (65536) palabras de 16 bits.
- Banco de 16 registros, con un código de identificación de 4 bits indicado por la tabla B.1:

¹En teoría, está programado para ser compatible con Java 8; pero solamente se garantiza a partir de Java 11.

- **R0 a R9:** Registros de propósito general. Aunque el sistema da libertad al usuario para modificar el contenido de los registros como vea conveniente, el contenido de estos diez registros no varía salvo que el usuario lo decida, mientras que los otros seis son empleados de manera implícita por el sistema y su contenido puede ser manipulado en cualquier momento.
 - **A:** Registro acumulador. Almacena el resultado de las operaciones aritmético-lógicas.
 - **SR:** Registro de estado. Almacena el conjunto de biestables del sistema en los seis bits menos significativos. El resto de bits del registro no sufren cambios cuando se modifica un biestable.
 - **IX, IY:** Índices para el direccionamiento relativo.
 - **SP:** Puntero de pila. Indica la primera posición **libre** por encima de la pila.
 - **PC:** Contador de programa. Indica la posición de memoria de la primera palabra de la siguiente instrucción a ejecutar.
- Biestables de estado contenidos en los 6 bits menos significativos del registro de estado, asignados según la tabla B.2:
 - **Z:** Cero. Se activa si el resultado de una operación es 0, se desactiva en caso contrario.
 - **C:** Acarreo. Se activa si el resultado excede los 16 bits de longitud, se desactiva en caso contrario.
 - **V:** Desbordamiento. Se activa si el resultado excede el rango de representación de 16 bits en complemento a 2; es decir, de -32768 a 32767. Se desactiva en caso contrario.
 - **P:** Paridad. Se activa si el resultado tiene un número impar de bits a 1, se desactiva en caso contrario.
 - **S:** Signo. Se activa si el resultado es negativo (si bit más significativo es 1), se desactiva en caso contrario.
 - **H:** Fin de ejecución. Se activa tras leer una instrucción **HALT**.
 - Entrada y salida por consola de números enteros de 16 bits y texto, tanto caracteres individuales como cadenas de ellos.
 - Capacidad de operar sobre enteros con signo de 16 bits.
 - Generación de excepciones durante la ejecución:
 - **Instrucción no implementada:** Se produce cuando se intenta decodificar un código de operación que no corresponde con ninguna instrucción definida, o bien cuando la instrucción tiene uno o dos operandos y se ve asociada con un modo de direccionamiento incompatible.
 - **División entre cero:** Se produce cuando el segundo operando de instrucción de división (**DIV**) o módulo (**MOD**) es cero.
 - **Final de memoria alcanzado:** Se produce cuando, tras ejecutar una instrucción (o durante la decodificación de una), el registro **PC** desborda el límite de memoria y vuelve a valer 0.
 - **Punto de ruptura encontrado:** Se produce cuando, durante la ejecución en modo de depuración, se intenta decodificar y ejecutar una instrucción que contiene un punto de ruptura en alguna de sus palabras.

- **Contador de programa invade la zona de pila (opcional):** Se produce cuando el PC toma un valor comprendido entre la dirección inicial de la pila y el valor actual del SP. Se puede activar o desactivar en la configuración.
- **Puntero de pila invade la zona de código (opcional):** Se produce cuando el SP toma un valor comprendido entre las direcciones inicial y final del código. Se puede activar o desactivar en la configuración.
- **Desbordamiento de pila (opcional):** Se produce cuando, como resultado de una inserción, el SP cruza el umbral entre la última dirección de memoria y la primera. Se puede activar o desactivar en la configuración.
- **Subdesbordamiento de pila (opcional):** Se produce cuando, como resultado de una operación (extracción, asignación de valor...), el SP tiene un valor “anterior” a la dirección inicial de la pila (mayor si la pila crece negativamente, menor si crece positivamente). Se puede activar o desactivar en la configuración.

Registro	Código	Bits	Registro	Código	Bits
R0	0	0000	R8	8	1000
R1	1	0001	R9	9	1001
R2	2	0010	A	10	1010
R3	3	0011	SR	11	1011
R4	4	0100	IX	12	1100
R5	5	0101	IY	13	1101
R6	6	0110	SP	14	1110
R7	7	0111	PC	15	1111

Tabla B.1: Tabla de códigos de registro.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	H	S	P	V	C	Z

Tabla B.2: Tabla de posicionamiento de biestables en el registro SR.

B.3. Configuración del sistema

ENS2025 tiene una serie de parámetros de configuración que modifican su comportamiento bajo ciertas circunstancias. Dichos parámetros son los siguientes:

- **Comprobación de invasión de pila:** Indica si, cada vez que cambie el valor del PC, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de pila.
- **Comprobación de invasión de código:** Indica si, al modificar el SP, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de código.

- **Comprobación de reinicio de registros:** Indica si, al usar alguno de los métodos de la clase `Simulator` para ejecutar código, se debe comprobar si el biestable H está en alto a fin de reiniciar primero el banco de registros.
- **Comprobación de overflow:** Indica si, al incrementar la pila, se debe comprobar que el SP no pase de 65535 a 0 (si es creciente) o de 65535 a 0 (si es decreciente).
- **Comprobación de underflow:** Indica si, al modificar el SP, se debe comprobar que no termine en una dirección anterior a la de inicio de pila (es decir, más baja si la pila crece positivamente y más alta si la pila crece negativamente).
- **Inicio y fin del código:** Indica las direcciones de memoria donde empieza y termina la zona de código.
- **Inicio de la pila:** Indica la dirección de memoria donde empieza la zona de pila.
- **Crecimiento de pila:** Indica la dirección actual de crecimiento de la pila. El crecimiento es positivo cuando crece de direcciones menores a mayores, y negativo cuando sucede al revés (también se usan los términos “pila creciente” y “pila decreciente” de manera intercambiable).
- **Representación numérica:** Indica el formato actual de representación numérica. Las tres posibles representaciones que pueden tomar los valores enteros del sistema son decimal con signo, decimal sin signo y hexadecimal (sin signo). Esta forma de representación solamente se tiene en cuenta cuando los valores se muestran al usuario; la representación interna se mantiene homogénea.

Al ensamblar y cargar en la memoria un nuevo programa, se calculan las direcciones de inicio y fin de las zonas de pila y código mencionadas más arriba. La dirección inicial de la zona de código queda definida como la dirección de menor valor en la que se haya codificado una instrucción, insertado un dato o reservado memoria, al igual que la dirección más alta en la que sucede esto queda definida como la dirección final. Por otra parte, la zona inicial de la pila será asignada, en función de su dirección de crecimiento, a la menor o mayor posición del bloque de memoria vacío más grande disponible, mientras que la posición final cambiará dinámicamente con el valor del SP. Las direcciones de ambas zonas pueden ser modificadas por el usuario.

La zona inicial de pila se calcula en función de dónde esté ubicado el código en la memoria; es decir, si deja un espacio vacío antes de empezar la zona de código, y cómo de grande es dicho espacio en comparación al sector de memoria vacío después de la zona de código. También se tiene en cuenta la dirección de crecimiento de la pila. El criterio a seguir es el siguiente:

- Si el crecimiento es negativo, la dirección inicial de pila será la dirección de mayor valor de la zona de mayor tamaño. Se prioriza la zona anterior al código en caso de igual tamaño que la posterior, para minimizar el riesgo de invasión de zona de código por el SP.
- Si el crecimiento es positivo, la dirección inicial de pila será la dirección de menor valor de la zona de mayor tamaño. Se prioriza la zona posterior al código en caso de igual tamaño por el mismo motivo que en el caso anterior.

El propósito de la existencia de ambas zonas es facilitar la depuración mediante la activación de los parámetros de comprobación de invasión. Estos parámetros permiten que se genere un error si en algún momento el PC entra dentro de la zona de pila o si el SP entra dentro de la zona de código, deteniendo la ejecución.

B.4. Estructura del código ensamblador

El código fuente es el texto plano que será interpretado por el sistema para su posterior ensamblado en instrucciones ejecutables. Los ficheros de código fuente siguen una estructura sencilla compuesta por una sucesión de líneas. No existe un número límite de caracteres por línea, por lo que la única limitación para la longitud de las etiquetas y los comentarios dependerá del sistema operativo o la memoria disponible.

El fichero debe estar codificado en UTF-8 sin BOM (Byte Order Mark). Si se detecta otra codificación, se instará al usuario a recodificar el fichero o, en su defecto, a eliminar los caracteres fuera del ASCII estándar (como letras con tilde) para tratar de hacerlo compatible². En caso de usar UTF con BOM, es posible que el fichero sea aceptado inicialmente, pero el propio BOM será leído como un carácter no válido para el procesador de código y se generará un error.

Por otro lado, se permite la inserción de tantas líneas en blanco y/o con comentarios entre la declaración de una etiqueta y la instrucción o directiva como se desee, lo que deja una estructura general con el siguiente formato:

```
[[Etiqueta ':' ] [\n [';' Comentario]]* Instrucción|Directiva [ ';' Comentario]
```

Dicho de otro modo:

- La declaración de una etiqueta siempre debe ir seguida de ':', y siempre en la misma línea que la etiqueta.
- La declaración de una instrucción o directiva debe suceder por completo dentro de la misma línea.
- Una etiqueta solamente puede ir seguida de una instrucción o directiva, nunca de otra etiqueta.
- Se ignorará todo texto situado a la derecha de ';', incluido dicho carácter, hasta el final de línea.

Algunos ejemplos correctos e incorrectos se pueden observar, respectivamente, en las figuras B.1 y B.2:

El formato que comparten las instrucciones es el siguiente:

```
Mnemónico [Operando[',' Operando]]
```

²Esta solución no garantiza la compatibilidad, y se recomienda recurrir simplemente a guardar de nuevo el fichero usando la codificación UTF-8 sin BOM.

```

1      ADD #32, /24 ; Esto es un comentario
2      etiq1: BR \ $40
3      etiq2:
4          ; Otro comentario
5      HALT
6

```

Figura B.1: Ejemplos válidos de instrucciones y comentarios.

```

1      etiq1 ADD #32, /24 ; ERROR: No se ha usado el separador ':' tras
      la etiqueta.
2
3      etiq2: etiq3: BR \ $40 ; ERROR: No se permite doble etiquetado
4
5      etiq4: ADD #32, ; Comentario ; /24 ; ERROR: los comentarios no
      pueden abrirse y cerrarse
6      HALT
7

```

Figura B.2: Ejemplos inválidos de instrucciones y comentarios.

Sin embargo, como se describirá más adelante, las operaciones están segmentadas según su número de operandos, por lo que el formato mencionado es una manera general de establecer su estructura común. En realidad, la estructura es fija para cada tipo, como se puede ver en la tabla B.3.

N. Operandos	Estructura
0	Mnemónico
1	Mnemónico Operando
2	Mnemónico Operando ',' Operando

Tabla B.3: Estructura de una instrucción según sus operandos

Donde el mnemónico es, esencialmente, el identificador de la instrucción. Como se ha mencionado, existen también las directivas del ensamblador, que se verán en su propia sección más adelante.

Por último, cabe destacar que el lenguaje es *case-sensitive*, esto es, distingue entre mayúsculas y minúsculas. No es lo mismo definir la etiqueta `etiq1` que la etiqueta `ETIQ1`, por ejemplo. Por otro lado, las instrucciones y directivas se aceptan únicamente en un mismo formato de capitalización (todo en mayúscula o todo en minúscula). Por ejemplo:

- `HALT`: Instrucción correcta.
- `halt`: Instrucción correcta.
- `Halt`: Instrucción incorrecta (puede usarse como etiqueta).

B.5. Modos de direccionamiento

Los modos de direccionamiento indican al simulador dónde y cómo debe buscar y almacenar los datos requeridos y generados por una instrucción. ENS2025 permite un total de seis modos de direccionamiento. Puesto que se representan con 3 bits (lo que da un máximo de 8 modos representables), uno de los modos se desdobra, como se verá a continuación:

- **Direccionamiento inmediato.**

En este modo de direccionamiento, el valor final del operando está explícito en el mismo. En la arquitectura de este sistema, eso significa que se emplean los 16 bits de palabra completos para definir dicho valor en complemento a 2.

Este modo de direccionamiento va indicado por un carácter de almohadilla ('#') precediendo al valor del operando.

Los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos #100 y #0x64 son equivalentes entre sí, y del mismo modo lo son entre sí #-1, #65535 y #0xFFFF.

Por ejemplo, la instrucción `CMP #23, .R1` compara el valor 23 con el que esté almacenado en el registro R1.

- **Direccionamiento directo a registro.**

En este modo, el operando contiene un mnemónico con el nombre del registro donde se almacena el valor. Dado que el banco de registros del sistema propuesto contiene 16 registros, serán necesarios 4 bits para representar el identificador del registro escogido.

Este modo de direccionamiento va indicado por un punto ('.') precediendo al mnemónico del registro. Al igual que sucede con los mnemónicos de instrucción, se puede representar el mnemónico del registro mediante mayúsculas o minúsculas, siendo equivalentes los operandos `.r0` y `.R0`.

Por ejemplo, la instrucción `INC .R0` incrementa en 1 el valor almacenado en el registro R0.

- **Direccionamiento directo a memoria.**

En este modo de direccionamiento, el operando expresa la dirección absoluta donde se encuentra el valor del mismo. Al igual que sucede con el direccionamiento inmediato, se requiere una palabra completa de 16 bits para expresar este direccionamiento para poder representar todas las posiciones de memoria disponibles.

Este modo de direccionamiento va indicado por un carácter de barra inclinada ('/') precediendo al valor del operando.

Al igual que en el direccionamiento inmediato, los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos `/100` y `/0x64` son equivalentes entre sí (representando la dirección de memoria 100), y del mismo modo lo son entre sí `/-1`, `/65535` y `/0xFFFF` (representando la dirección de memoria 65535).

Por ejemplo, la instrucción `INC /0x0500` incrementa en 1 el valor almacenado en la posición de memoria 0x0500.

- **Direccionamiento indirecto.**

En este modo de direccionamiento, el operando contiene un mnemónico con el nombre de un registro, y en dicho registro se almacena la dirección donde se encuentra el valor. Al igual que en el modo directo a registro, se necesitan 4 bits para representar el identificador del registro escogido.

Este modo de direccionamiento se representa de manera similar a la del directo a registro (el mnemónico precedido de un punto), pero insertando dicha forma de representación entre corchetes (por ejemplo, `[.R0]`).

Por ejemplo, sea R2 con un valor almacenado de 100. La instrucción `DEC [.R2]` decrementará en 1 el valor almacenado en la posición de memoria 100 (es decir, equivale a la instrucción `DEC /100`).

- **Direccionamiento relativo a registro índice.**

En este modo de direccionamiento, el operando expresa un desplazamiento sobre la dirección marcada por un puntero. El rango de desplazamiento respecto al puntero es de 8 bits en complemento a 2, lo que otorga un rango de desplazamiento de -128 a 127 posiciones de memoria respecto a la posición actual del puntero.

En esta implementación del sistema, los únicos punteros permitidos son los registros de índice IX e IY (por lo tanto, para acortar, es preferible referirse a este direccionamiento como *relativo a IX* o *relativo a IY*). A cada uno de ellos les corresponde un código de direccionamiento distinto, por lo que solamente son necesarios los 8 bits del desplazamiento para representar cada direccionamiento.

Este modo de direccionamiento se representa mediante una combinación del direccionamiento inmediato y el indirecto: el valor numérico va precedido por la almohadilla ('#') y seguido por el registro en cuestión (IX o IY) en notación indirecta (precedido por un punto y entre corchetes).

Al margen de la diferencia de tamaño, se aplican las mismas reglas de interpretación numérica que con los direccionamientos inmediato y directo a memoria: las instrucciones `INC #64[.IX]` y `#0x40[.IX]` incrementan en 1 el valor almacenado en la dirección situada 40 posiciones *después* de la almacenada en IX, mientras que las instrucciones `INC #FF[.IY]`, `INC #-1[.IY]` y `INC #255[.IY]` incrementan en 1 el valor almacenado en la dirección situada 1 posición *antes* de la almacenada en IY.

Por ejemplo, sea IX con un valor almacenado de 100. La instrucción `MOVE #200, #25[.IX]` almacenará el valor 200 en la posición de memoria 125 (es decir, sería lo mismo que la instrucción `MOVE #200, /125`), y la instrucción `MOVE #200, #-10[.IX]` almacenará el valor 200 en la posición de memoria 90.

- **Direccionamiento relativo a contador de programa.**

Similar al relativo a índice, este modo de direccionamiento requiere 8 bits en complemento a 2 para representar un desplazamiento de -128 a 127 posiciones respecto al puntero. La diferencia aquí es que el puntero es el valor del contador de programa *después* de ejecutar la instrucción.

Este modo de direccionamiento se representa mediante el carácter del símbolo de dólar ('\$'), seguido del valor del desplazamiento.

Al igual que en el relativo a índice, en este direccionamiento también se aplican las mismas reglas de representación numérica: las instrucciones BR \$0xFC, BR \$-4 y BR \$251 son equivalentes.

Por ejemplo, sea la instrucción BR \$20 situada en la dirección 0x0000. Cuando termine de ser leída, el valor del PC antes de ejecutar la instrucción será de 0x0002, ya que es una instrucción que ocupa dos palabras. Por lo tanto, el valor final del PC una vez se ejecute la instrucción será de 0x0022.

Sin embargo, este comportamiento no es universal. Existen ciertas diferencias de comportamiento para algunos modos de direccionamiento en función del tipo de instrucción; concretamente, variaciones para las instrucciones de gestión de cadenas de caracteres (INSTR y WRSTR) y las instrucciones de salto. Por lo tanto, aunque es en el apartado siguiente donde se clasifican con propiedad los tipos de instrucciones y se define la función a realizar por cada una, así como los modos de direccionamiento que acepta, es necesario profundizar en esta situación.

En primer lugar, se detalla el comportamiento de los modos de direccionamiento para las instrucciones de salto:

- **Direccionamiento directo a memoria.**

Se trata como un direccionamiento inmediato, ya que este último no se emplea en este tipo de instrucciones. Se usa el valor literal de la dirección, no el contenido de la misma. Por ejemplo, la instrucción BR /1000 establece para el PC el valor 1000, no el valor contenido en la dirección 1000. Es decir, equivale a MOVE #1000, .PC, no a MOVE /1000, .PC.

- **Direccionamiento indirecto.**

El direccionamiento indirecto funciona de manera idéntica a la estándar.

- **Direccionamiento relativo a contador de programa.**

El direccionamiento relativo a contador de programa funciona de manera idéntica a la estándar.

Finalmente, se detalla el comportamiento de los modos de direccionamiento empleados por INSTR y WRSTR para determinar la posición donde se va a comenzar a leer o escribir la cadena:

- **Direccionamiento directo a memoria.**

Funciona igual que con las instrucciones de salto: se interpreta el operando como si fuera un direccionamiento inmediato. Es decir, WRSTR /100 equivaldría a WRSTR #100 si se tuviera que definir su comportamiento en base a las reglas generales de direccionamiento.

- **Direccionamiento indirecto.**

Se trata como un direccionamiento a registro. Es decir, en vez de usar el contenido del registro como dirección de memoria que apunta al primer carácter de la cadena, el valor contenido en el registro *es* la dirección del primer carácter de la cadena. Por lo tanto, WRSTR [.R0] equivaldría a WRSTR .R0 si su comportamiento se tuviera que definir usando las reglas para los casos generales.

■ Direccionamiento relativo a registro índice.

En este caso, el desplazamiento sobre el registro IX o IY apunta al primer carácter de la cadena, no al valor de la dirección de dicho carácter. Así, para un valor de 100 almacenado en IX, si la instrucción `INC #25[.IX]` equivale a `INC /125`, bajo el comportamiento general la instrucción `WRSTR #25[.IX]` tal y como funciona actualmente equivaldría en realidad a `WRSTR #125`.

Los identificadores de los modos de direccionamiento pueden consultarse en la tabla B.4.

Código	Bits	Modo de direccionamiento	Ancho de representación
0	000	Sin operandos	N/A
1	001	Inmediato	16 bits
2	010	Registro	4 bits
3	011	Memoria	16 bits
4	100	Indirecto	4 bits
5	101	Relativo a IX	8 bits
6	110	Relativo a IY	8 bits
7	111	Relativo a PC	8 bits

Tabla B.4: Tabla de códigos de direccionamiento.

B.6. Juego de instrucciones

En esta sección se listan todas las instrucciones soportadas por ENS2025. Dichas instrucciones están agrupadas de manera conceptual en los siguientes conjuntos:

- **Instrucciones de transferencia de datos:** `MOVE`, `PUSH`, `POP`.
- **Instrucciones aritméticas:** `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `INC`, `DEC`, `NEG`.
- **Instrucciones de comparación:** `CMP`.
- **Instrucciones lógicas:** `AND`, `OR`, `XOR`, `NOT`.
- **Instrucciones de salto (bifurcación):** `BR`, `BZ`, `BNZ`, `BP`, `BN`, `BV`, `BNV`, `BC`, `BNC`, `BE`, `BO`.
- **Instrucciones de manejo de subrutinas:** `CALL`, `RET`.
- **Instrucciones de entrada/salida:** `INCHAR`, `ININT`, `INSTR`, `WRCHAR`, `WRINT`, `WRSTR`.
- **Otras instrucciones:** `NOP`, `HALT`.

A continuación, se listan individualmente, indicando su mnemónico, su función, el número de operandos que contiene y los modos de direccionamiento que admite para cada uno, y su comportamiento:

- **Instrucción NOP:**
 - **Descripción:** No operación.
 - **Número de operandos:** 0.
 - **Direccionamientos:** Ninguno.
 - **Comportamiento:** Ninguno.
 - **Modifica biestables:** No.
- **Instrucción HALT:**
 - **Descripción:** Detener ejecución.
 - **Número de operandos:** 0.
 - **Direccionamientos:** Ninguno.
 - **Comportamiento:** Activa el biestable H.
 - **Modifica biestables:** No (el resto de biestables mantienen su último valor, no se reinician).
- **Instrucción MOVE:**
 - **Descripción:** Copia de valores entre operandos.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Copia el contenido de OP1 en OP2.
 - **Modifica biestables:** No
- **Instrucción PUSH:**
 - **Descripción:** Inserción en pila.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
 - **Comportamiento:** Almacena el valor contenido en el operador en la dirección de memoria apuntada por el puntero de pila (SP) y luego desplaza el puntero de pila una posición en la dirección de crecimiento de la misma.
 - **Modifica biestables:** No.
- **Instrucción POP:**
 - **Descripción:** Extracción de la pila.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo.
 - **Comportamiento:** Desplaza el puntero de pila (SP) una posición en dirección opuesta al crecimiento de la misma y luego almacena el valor contenido en la nueva dirección apuntada por el SP en el operando.

- **Modifica biestables:** No.

■ **Instrucción ADD:**

- **Descripción:** Suma de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Suma los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí

■ **Instrucción SUB:**

- **Descripción:** Resta de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Resta los contenidos de OP1 y OP2 (OP1-OP2) y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí.

■ **Instrucción MUL:**

- **Descripción:** Multiplicación de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Multiplica los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí.

■ **Instrucción DIV:**

- **Descripción:** División de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Divide OP1 entre OP2 y almacena el resultado en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.

- **Modifica biestables:** Sí.
- **Instrucción MOD:**
 - **Descripción:** Resto de la división de enteros.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Resta los contenidos de OP1 y OP2 y almacena el resto de dicha división en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.
 - **Modifica biestables:** Sí.
- **Instrucción INC:**
 - **Descripción:** Incremento unitario.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Incrementa en 1 el contenido de OP1.
 - **Modifica biestables:** Sí.
- **Instrucción DEC:**
 - **Descripción:** Decremento unitario.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Decrementa en 1 el contenido de OP1.
 - **Modifica biestables:** Sí.
- **Instrucción NEG:**
 - **Descripción:** Cambio de signo.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Cambia de signo el operando mediante la operación 0-OP1.
 - **Modifica biestables:** Sí.
- **Instrucción CMP:**
 - **Descripción:** Comparación.
 - **Número de operandos:** 2
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **Comportamiento:** El mismo que en la instrucción SUB, pero sin almacenar el resultado de la operación en ningún sitio.
- **Modifica biestables:** Sí.

■ **Instrucción AND:**

- **Descripción:** AND lógico.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Efectúa la operación lógica AND entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** No.

■ **Instrucción OR:**

- **Descripción:** OR lógico.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Efectúa la operación lógica OR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si el bit de uno o ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** No.

■ **Instrucción XOR:**

- **Descripción:** XOR lógico.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Efectúa la operación lógica XOR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen distintos valores (uno 0 y el otro 1). Almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** No.

■ **Instrucción NOT:**

- **Descripción:** Negación lógica.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Invierte bit a bit el valor del operando (cambia los 0 por 1 y viceversa).
 - **Modifica biestables:** No.
- **Instrucción BR:**
- **Descripción:** Bifurcación incondicional.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 .
 - **Modifica biestables:** No.
- **Instrucción BZ:**
- **Descripción:** Bifurcación si Z=1.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 1.
 - **Modifica biestables:** No.
- **Instrucción BNZ:**
- **Descripción:** Bifurcación si Z=0.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 0.
 - **Modifica biestables:** No.
- **Instrucción BP:**
- **Descripción:** Bifurcación si S=0.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 0.
 - **Modifica biestables:** No.
- **Instrucción BN:**
- **Descripción:** Bifurcación si S=1.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.

- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 1.
- **Modifica biestables:** No.
- **Instrucción BV:**
 - **Descripción:** Bifurcación si $V=1$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 1.
 - **Modifica biestables:** No.
- **Instrucción BNV:**
 - **Descripción:** Bifurcación si $V=0$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 0.
 - **Modifica biestables:** No.
- **Instrucción BC:**
 - **Descripción:** Bifurcación si $C=1$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 1.
 - **Modifica biestables:** No.
- **Instrucción BNC:**
 - **Descripción:** Bifurcación si $C=0$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 0.
 - **Modifica biestables:** No.
- **Instrucción BE:**
 - **Descripción:** Bifurcación si $P=0$.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.

- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 0.
- **Modifica biestables:** No.
- **Instrucción B0:**
 - **Descripción:** Bifurcación si P=1.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 1.
 - **Modifica biestables:** No.
- **Instrucción CALL:**
 - **Descripción:** Llamada a subrutina.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
 - **Comportamiento:** Almacena el valor actual del PC en la dirección de memoria apuntada por el SP, desplaza el SP una posición en la dirección de crecimiento de la pila y carga en el PC el valor almacenado en el operando (es decir, **CALL <OP>** equivale a **PUSH .PC** seguido de **BR <OP>**, siendo <OP> un operando genérico con direccionamiento permitido).
 - **Modifica biestables:** No.
- **Instrucción RET:**
 - **Descripción:** Retorno de subrutina.
 - **Número de operandos:** 0.
 - **Direccionamientos:** Ninguno.
 - **Comportamiento:** Desplaza el SP una posición en la dirección opuesta al crecimiento de la pila y carga en el PC el valor almacenado en la nueva dirección del SP (es decir, **RET** equivale a **POP .PC**)
 - **Modifica biestables:** No.
- **Instrucción INCHAR:**
 - **Descripción:** Lectura de carácter.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo.
 - **Comportamiento:** Lee un carácter de la consola y lo codifica en ASCII, dejando a cero los 8 bits superiores de la palabra de 16 bits y almacenando el carácter leído en el operando.
 - **Modifica biestables:** No.
- **Instrucción ININT:**

- **Descripción:** Lectura de entero.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo.
 - **Comportamiento:** Lee un entero de la consola. Si la entrada no puede ser convertida en un entero (formato incorrecto, valor fuera de rango, entrada vacía), se considerará que se ha introducido un 0. Los enteros se pueden escribir en formato decimal (con y sin signo) y hexadecimal (sin signo y precedido por "0x").
 - **Modifica biestables:** No.
- **Instrucción INSTR:**
- **Descripción:** Lectura de cadena.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, indirecto, relativo.
 - **Comportamiento:** Lee una cadena de caracteres de la consola y la almacena en posiciones consecutivas de memoria a partir de la dirección indicada por el operando, acabada con el carácter '\0'. No se comprueba previamente si hay espacio para almacenar la cadena, por lo que se produce un error si se sobrepasa el límite superior de la memoria.
 - **Modifica biestables:** No.
- **Instrucción WRCHAR:**
- **Descripción:** Escritura de carácter.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
 - **Comportamiento:** Escribe en la consola el carácter cuyo valor ASCII se corresponde con los 8 bits inferiores del valor del operando.
 - **Modifica biestables:** No.
- **Instrucción WRINT:**
- **Descripción:** Escritura de entero.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
 - **Comportamiento:** Escribe en la consola el valor del operando. El formato de representación numérica (hexadecimal, numeral con/sin signo) dependerá de la configuración de ese parámetro en el sistema.
 - **Modifica biestables:** No.
- **Instrucción WRSTR:**
- **Descripción:** Escritura de cadena.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Memoria, indirecto, relativo.

- **Comportamiento:** Escribe en la consola una cadena de caracteres, que estará almacenada en memoria a partir de la dirección indicada por el operando. Se escribirán caracteres hasta que se llegue al carácter ‘\0’. Por tanto, puede ocurrir que durante este proceso se genere un error si no se encuentra el carácter de fin de cadena antes de llegar al límite superior de la memoria.
- **Modifica biestables:** No.

B.7. Etiquetas

Las etiquetas son mnemónicos definidos por el usuario que sirven como nombres simbólicos. Comienzan por un carácter alfabético, seguido o no de cualquier combinación de caracteres alfanuméricos y de barra baja (‘_’). Solamente pueden ser definidos precediendo a una instrucción o directiva, y siempre irán finalizados por un carácter de dos puntos (‘:’) a modo de separador (es decir, no es parte del mnemónico como tal). Toman el valor que tenga el puntero de escritura cuando son declaradas, antes del procesamiento de una instrucción o la ejecución de una directiva, salvo si se trata de la directiva EQU, en cuyo caso toman como valor el resultado de la expresión aritmética asociada a la directiva.

Las etiquetas pueden usarse como sustitución de cualquier literal entero que forme parte de un operando, como por ejemplo WRCHAR #VALOR, BR \$ini, WRSTR /cadena o ADD #BASE, #DESP[.IX]. La etiqueta se sustituye por su valor pertinente durante el ensamblado.

B.8. Directivas del ensamblador

Las directivas del ensamblador son órdenes dirigidas al programa ensamblador con el fin de organizar el proceso de ensamblado; pero no forman parte del código y, por tanto, no se codifican en lenguaje máquina para su posterior simulación.

Las directivas definidas para este sistema (así como sus comportamientos), son las siguientes:

- **Directiva ORG:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará la dirección de memoria a la que se moverá el puntero de escritura. **IMPORTANTE:** No se realiza ninguna comprobación implícita de sobrescritura. Por ejemplo, en el código presentado en la figura B.3, la instrucción ADD invade el código escrito más arriba. Si no se incluye directiva ORG en el código, éste se ensamblará a partir de la primera dirección (0).
- **Directiva EQU:** Tiene como argumento una expresión aritmética. Asigna el resultado de dicha expresión a la etiqueta asociada. Si no se indica ninguna etiqueta, la expresión será evaluada igualmente, pero no se asignará su resultado a ningún elemento.
- **Directiva END:** No tiene argumentos. Señala el final del código fuente. No es obligatoria, ya que el proceso de ensamblado finaliza sin errores igualmente una vez que se termina de leer el fichero de código. El ensamblador ignorará todo lo que haya escrito a partir de esta directiva.

```
1      ORG 4
2      INC /23
3      HALT
4
5      ORG 0
6      BR /599
7      PUSH #33
8      ADD #200, [.R3]
9
```

Figura B.3: Ejemplo de directiva ORG provocando sobrescritura de código.

- **Directiva RES:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará cuántas posiciones de memoria se reservan (sin inicializar su valor). En caso de ir precedida de una etiqueta, ésta indicará la primera posición reservada.
- **Directiva DATA:** Tiene como argumento una sucesión tan larga como desee el usuario (siempre que la memoria lo permita) de valores numéricos y cadenas de caracteres (que deberán ir entrecomilladas) separados por comas. Las cadenas se almacenarán en memoria indicando su final mediante el carácter ‘\0’ como añadido final. Pueden estar compuestas de cualquier carácter imprimible, y pueden contener los caracteres especiales ‘\0’, ‘\n’ y ‘\t’, así como las propias comillas y la barra invertida usada como escapador de caracteres, siempre que vayan ambos precedidos de otra barra invertida (‘\'’ y ‘\\’).

B.9. Detección de errores en el código fuente

Además de los errores en tiempo de ejecución mencionados, pueden producirse errores durante la lectura de código. Estos errores aportan información sobre su categoría, su causa y, si procede, la línea en la que se han producido. Por ejemplo, si se intenta ensamblar este código:

```
ADD #-2 #4
```

Se generará este error:

Error en línea 1: Se esperaba el carácter ‘,’ tras el primer operando.

B.10. Interfaz de línea de comandos

La interfaz de línea de comandos es una aplicación sencilla que utiliza la consola del sistema como medio para su ejecución y comunicación con el usuario. Consiste en un *prompt* que el usuario utiliza para introducir diversos comandos. Estos comandos serán interpretados por el módulo, que llamará a los métodos pertinentes de la unidad de control para cumplir con el cometido esperado de cada uno de ellos.

B.10.1. Inicio del programa y parámetros de invocación

Al estar compilado en un objeto JAR, la aplicación se inicia introduciendo en la línea de comandos la siguiente invocación:

```
java -jar ens2025.jar [parámetros]
```

Si se invoca sin ningún parámetro, la secuencia de inicio es la siguiente:

1. Se intenta cargar el fichero de configuración.
2. Si no se encuentra, se carga la configuración estándar.
3. Se muestra el estado de la configuración del sistema.
4. Se muestra la ayuda simplificada de los comandos.
5. Se muestra el *prompt* y se devuelve el control al usuario.

Por otro lado, la aplicación puede ser invocada mediante los siguientes parámetros:

- **-a <nombre>**: Inicia la ejecución cargando y ensamblando un fichero de código ensamblador.
- **-m <nombre>**: Inicia la ejecución cargando directamente un fichero binario de imagen de memoria.
- **-rec**: Inicia la ejecución cargando el fichero de recuperación generado al interrumpir la ejecución del programa mientras el proceso de simulación está activo.
- **-r <nombre>**: Carga un fichero ensamblador, lo ensambla, ejecuta el código resultante y finaliza el programa.
- **-rm <nombre>**: Carga una imagen de memoria, la ejecuta automáticamente y finaliza el programa.
- **-d <nombre>**: Si se ha seleccionado un argumento de ejecución (**-r** o **-rm**), este parámetro especifica un nombre de fichero binario de imagen de memoria en el que se volcará el estado del sistema al finalizar la simulación antes de cerrar el programa.
- **-s <nombre>**: Permite cargar un fichero de configuración específico para esta ejecución del programa. La configuración generada por dicho fichero no será permanente: al cerrar el programa e invocarlo sin este parámetro, se cargará con normalidad el fichero de configuración estándar. La configuración durante la ejecución del programa con este parámetro es temporal, y los cambios realizados en la misma no se guardarán ni en el fichero cargado mediante parámetro ni en el fichero estándar.

La finalidad de dichos parámetros es otorgar flexibilidad y fluidez a la ejecución, en especial la ejecución por lotes. De esta manera, tanto los alumnos como el equipo docente pueden hacer comprobaciones de ejecución secuenciales en ciclos de menor duración. La opción de cargar

un fichero de configuración permite, además, hacer comprobaciones de un mismo fichero de código con distintas configuraciones (por ejemplo, una ejecución normal sin guardas, otra con las guardas de invasión de punteros y desbordamiento de pila activas, otra con la pila creciendo en la dirección opuesta...) sin necesidad de andar cambiando el fichero para cada ejecución.

Los parámetros pueden invocarse en cualquier orden, pero atendiendo a ciertas restricciones:

- No se puede usar dos veces el mismo parámetro.
- Solamente se puede usar un único parámetro de carga o de ejecución. Es decir, `-a`, `-m`, `-rec`, `-r` y `-rm` son incompatibles entre sí.
- Solamente se puede usar el parámetro `-d` (volcado) si se usa un parámetro de ejecución (`-r` o `-rm`) en la misma invocación.
- El parámetro `-s` es totalmente independiente y no tiene más restricción que la de no ser duplicado.

Por ejemplo, las siguientes invocaciones son correctas:

```
java -jar ens2025.jar -a "fichero de prueba.ens"
```

```
java -jar ens2025.jar -r c:/Users/Me/Documents/test18.ens
```

```
java -jar ens2025.jar -r c:/Users/Me/Documents/test18.ens -d volcado.mem
```

Mientras que las siguientes invocaciones no lo son:

```
java -jar ens2025.jar -a test18.ens -a test19.ens
```

```
java -jar ens2025.jar -a test18.ens -d volcado.mem
```

```
java -jar ens2025.jar -a test18.ens -r test19.ens
```

B.10.2. Comandos

Los comandos que acepta la aplicación pueden ir, en algunos casos, acompañados de parámetros, tanto <obligatorios> como [opcionales]. La lista completa de comandos, con sus posibles parámetros y funcionalidades, es la siguiente:

- **ENS <f>**: Ensambla el fichero de nombre ‘f’ y carga en la memoria el contenido de las instrucciones ensambladas.
- **DES <dir><num>**: Desensambla ‘num’ instrucciones a partir de la dirección de memoria ‘dir’. Un ejemplo de instrucción desensamblada es el siguiente:

```
[ ] ADD /23, #3599
```

Donde [] indica que esa instrucción no contiene un punto de ruptura en ninguna de las palabras que la forman. De tenerlo, se mostraría con un asterisco dentro ([*]).

- **RUN**: Simula el código con normalidad.
- **PASO**: Muestra la instrucción apuntada por el PC y la ejecuta.
- **DEBUG**: Simula el código hasta encontrar un punto de ruptura.
- **BP <dir>[val]**: Si se invoca sin segundo parámetro, añade un punto de ruptura en la dirección ‘dir’ si no había uno, y lo elimina si ya existía. Si se invoca con segundo argumento, fuerza ese estado. Los valores pueden ser:
 - **SI**: Introduce el punto de ruptura.
 - **NO**: Elimina el punto de ruptura.

- **LEERMEM <dir><num>**: Muestra ‘num’ posiciones de memoria a partir de la dirección de memoria ‘dir’. Si el número de posiciones a mostrar es mayor que el número de posiciones restantes hasta el final de la memoria, se mostrarán solamente las posiciones disponibles. Un ejemplo de posición de memoria es el siguiente:

[][][] 0x0F38: 0xCB9F

Donde el primer [], al igual que en el desensamblado, indica si hay un punto de ruptura en esa posición, y se marca con un asterisco ([*]); el segundo [] indica la pertenencia a la zona de código y se marca con una C si se cumple ([C]), y el tercer [] indica la pertenencia a la zona de pila, marcándose con una P si pertenece a dicha zona ([P]), o con un ¿si el SP apunta justo a esa dirección ([>]).

- **ESCMEM <dir><val>**: Escribe el valor ‘val’ en la dirección de memoria ‘dir’.
- **RESMEM**: Restablece a 0 el valor almacenado en todas las posiciones de memoria.
- **LEERPILA <num>**: Muestra ‘num’ posiciones de memoria a partir de la dirección de memoria del puntero de pila, empezando por la cima. Si el número de posiciones a mostrar es mayor que el número de elementos restantes hasta el final de la pila, o si se produce desbordamiento, se mostrarán solamente las posiciones disponibles.
- **CARGAR <f>**: Carga el estado del procesador (memoria, registros, zonas de código y pila) contenida en el fichero de nombre ‘f’. Compatible con ficheros de memoria generados por ENS2001, pero solamente contienen la memoria.
- **GUARDAR <f>**: Guarda el estado actual del procesador (memoria, registros, zonas de código y pila) en el fichero de nombre ‘f’. Estos ficheros pueden ser cargados por ENS2001, pero solamente cargará el estado de la memoria.
- **BANCO**: Imprime por pantalla los valores de todos los registros.
- **REGESC <reg><val>**: Inserta el valor ‘val’ en el registro identificado por ‘reg’.
- **H [com]**: Si se invoca sin parámetros, muestra la ayuda simplificada de todos los comandos. Si se invoca usando el comando ‘com’ como parámetro, muestra la ayuda avanzada para ese comando específico.
- **CONF**: Imprime por pantalla los valores actuales de configuración del sistema.

- **INVPILA [val]**: Modifica la comprobación de invasión de la zona de pila por parte del PC. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **INVCODE [val]**: Modifica la comprobación de invasión de la zona de código por parte del SP. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **DIRPILA [val]**: Modifica la dirección hacia la que se mueve el SP cuando se introduce un elemento en la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **POS**: Crecimiento positivo (“hacia abajo”).
 - **NEG**: Crecimiento negativo (“hacia arriba”).
- **OVERFLOW [val]**: Modifica la comprobación de desbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **UNDERFLOW [val]**: Modifica la comprobación de subdesbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **SETINIC <dir>**: Cambia el valor de la dirección inicial de la zona de código por la suministrada en ‘dir’.
- **SETFINC <dir>**: Cambia el valor de la dirección final de la zona de código por la suministrada en ‘dir’.
- **SETINIP <dir>**: Cambia el valor de la dirección inicial de la zona de pila por la suministrada en ‘dir’.
- **CONFREG [val]**: Modifica la comprobación de reinicio del banco de registros al recibir una orden de simulación mientras el biestable H está en alto. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.

- **BASE <val>**: Modifica la base numérica en la que se representan los valores. Algunos elementos, como las direcciones de memoria, mostrarán su valor decimal sin signo incluso si la representación es decimal con signo. Los valores pueden ser:
 - **HEX**: Hexadecimal.
 - **DEC**: Decimal con signo.
 - **USDEC**: Decimal sin signo.
- **X**: Guarda el estado actual de la configuración del sistema en el fichero pertinente y termina la ejecución.

Si durante la ejecución se genera un bucle infinito, se deberá pulsar las teclas **CONTROL+C** para detenerlo. Si bien esto termina el programa forzosamente, antes se generará un volcado del estado del simulador en un fichero llamado **recovery.dump** que, como ya se ha dicho, se puede cargar invocando el parámetro **-rec** al iniciar el programa.

B.11. Interfaz Gráfica de Usuario

La interfaz gráfica está pensada para facilitar al usuario el acceso a las mismas funcionalidades que la versión por línea de comandos, en un entorno donde todos los elementos estén visualmente presentes, facilitando su uso. La aplicación está dividida en cinco elementos principales, como se puede ver en la figura B.4: la barra de herramientas (1), el banco de registros y biestables (2), el visor de memoria (3), el visor de código(4) y las zonas de texto para los mensajes del sistema y la consola E/S de la simulación (5).

Antes de profundizar en los distintos elementos de la GUI, es conveniente explicar brevemente cómo lanzar esta versión de ENS2025.

B.11.1. Ejecución de la GUI

El precio a pagar por un ejecutable portable de Java que no requiera pasar por un proceso de instalación y desinstalación es la necesidad de mantener un directorio con los elementos necesarios para la ejecución. Dicho directorio se suministra en forma de una carpeta llamada ENS2025-GUI. En función del sistema operativo desde el que se trabaje, el sistema de ejecución varía ligeramente:

- **Windows.**

Junto a la carpeta ENS2025-GUI se suministra un fichero **.bat**, que consiste simplemente en el comando **./ENS2025-GUI/bin/ens2025.bat**. Para lanzar ENS2025, basta con hacer doble clic en este fichero. El **.bat** interno es el verdadero lanzador de la aplicación, que no puede ser movido de la carpeta.

- **Linux.**

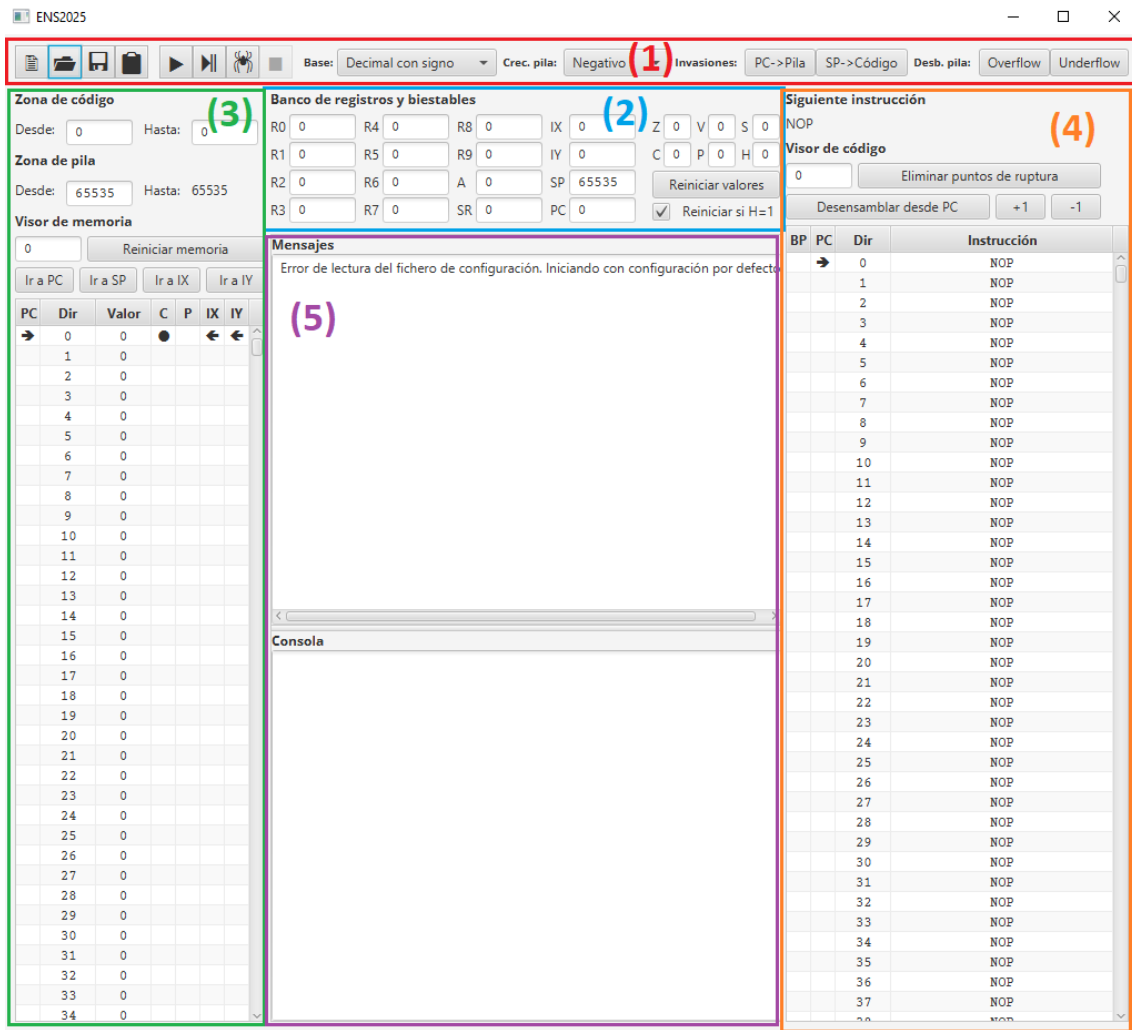


Figura B.4: Vista general de la ventana de interfaz gráfica.

De manera similar, se suministra junto a la carpeta ENS2025-GUI un fichero que contiene el comando `./ENS2025-GUI/bin/ens2025`. Para lanzar ENS2025, se puede ejecutar este fichero desde la consola, o hacer clic derecho y seleccionar "Ejecutar como programa".

■ MacOS.

Se suministra junto a la carpeta ENS2025-GUI un alias que dirige al script de ejecución. Para lanzar ENS2025, basta con hacer doble clic en este alias. También se puede hacer doble clic en el script situado en `ENS2025-GUI/bin/ens2025`.

B.11.2. La barra de herramientas

La parte superior de la GUI incluye todos los elementos de configuración y ejecución en una misma barra de herramientas, visualmente dividida en tres secciones:

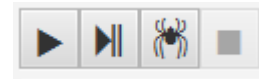
- **Gestión de ficheros:** Contiene los elementos dedicados a la carga y guardado de ficheros, como se puede ver en la figura B.5a. Los elementos son, en orden, los siguientes:

- **Botón de ensamblado de fichero:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.ens`, con la finalidad de seleccionar un fichero de código fuente ensamblador. El fichero seleccionado será ensamblado y cargado en la memoria.
 - **Botón de carga de imagen:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.mem`, con la finalidad de seleccionar un fichero binario de imagen de memoria. El fichero seleccionado se cargará en la memoria.
 - **Botón de guardado de imagen:** Abre una ventana de exploración para designar la localización y nombre del fichero binario a escribir (o sobrescribir) con la imagen de memoria de sistema.
 - **Botón de guardado de mensajes:** Abre una ventana de exploración para designar la localización y nombre del fichero de texto a escribir (o sobrescribir) con un volcado del área de texto de mensajes, seguido de un volcado del área de texto de la consola de entrada/salida.
- **Gestión de simulación:** Contiene los elementos dedicados al inicio y parada del simulador, como se puede ver en la figura B.5b. Los elementos son, en orden, los siguientes:
- **Botón de ejecución estándar:** Inicia la ejecución estándar del simulador (de manera continua y sin detenerse ante puntos de ruptura).
 - **Botón de ejecución paso a paso:** Ejecuta una sola instrucción.
 - **Botón de depuración:** Inicia la simulación con la condición de detenerse ante un punto de ruptura. Si la depuración se inicia mientras el PC apunta a una instrucción que contiene un punto de ruptura, dicho punto de ruptura se ignora para evitar que haya que desactivarlo para poder iniciar la simulación.
 - **Botón de parada:** Normalmente deshabilitado, se habilita cuando uno de los modos de ejecución está activo, dando la opción de detenerlo prematuramente.
- **Gestión de configuración:** Contiene los elementos dedicados a la visualización y modificación de los valores de configuración del sistema, como se puede ver en la figura B.5c. Los elementos son, en orden, los siguientes:
- **Desplegable de base numérica:** Permite al usuario modificar el formato de representación de los valores.
 - **Desplegable del crecimiento de pila:** Permite al usuario modificar la dirección de crecimiento de la pila.
 - **Botón de alternancia de invasión de pila:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de pila por el PC. En la figura B.5c se puede ver que este botón está activado, por lo que la comprobación de invasión de pila también lo está.
 - **Botón de alternancia de invasión de código:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de código por el SP. En la figura B.5c se puede ver que este botón está desactivado, por lo que la comprobación de invasión de código tampoco lo está.

- **Botón de alternancia de comprobación de desbordamiento:** Permite al usuario activar o desactivar la comprobación de desbordamiento de la pila. En la figura B.5c se puede ver que este botón está desactivado, por lo que la comprobación de desbordamiento tampoco lo está.
- **Botón de alternancia de comprobación de subdesbordamiento:** Permite al usuario activar o desactivar la comprobación de subdesbordamiento de la pila. En la figura B.5c se puede ver que este botón está activado, por lo que la comprobación de subdesbordamiento también lo está.



(a) Herramientas de archivos.



(b) Herramientas de simulación.



(c) Herramientas de configuración.

Figura B.5: Divisiones de la barra de herramientas.

B.11.3. El banco de registros y biestables

Este componente, que se puede ver con más detalle en la figura B.6 contiene los siguientes elementos:

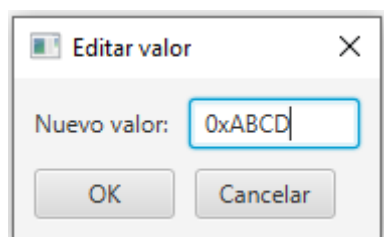
- Campos de texto para visualizar y editar los 16 registros.
- Campos de texto para visualizar y editar los 6 biestables.
- Botón para reiniciar los valores de todos los campos.
- Opción para reiniciar los valores del banco automáticamente si se inicia una simulación mientras el biestable H está en alto.

Banco de registros y biestables													
R0	0	R4	0	R8	0	IX	0	Z	0	V	0	S	0
R1	0	R5	0	R9	0	IV	0	C	0	P	0	H	0
R2	0	R6	0	A	0	SP	65535	Reiniciar valores					
R3	0	R7	0	SR	0	PC	0						
								☒	Reiniciar si H=1				

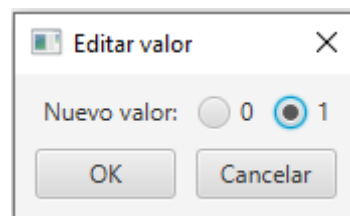
Figura B.6: Banco de registros y biestables. El registro A tiene el foco.

Los campos no se pueden editar directamente, sino que requieren hacer doble clic sobre ellos, o pulsar la tecla ENTER mientras el campo deseado tiene el foco (borde azul). En ambos casos, se abre una pequeña ventana de diálogo para elegir el nuevo valor: para los campos de los registros, solicita introducir el valor nuevo (respetando siempre los formatos permitidos y rangos

de valores), mientras que para los campos de biestables se pide seleccionar un botón radial para poner el valor a 0 o 1. Ambas variaciones se pueden ver en la figura B.7. El funcionamiento de los campos de texto de los registros se aplica a todos los campos de texto similares de la aplicación. Mientras el cuadro de diálogo de modificación de un campo está activo, no se puede interactuar con el resto de la aplicación.



(a) Edición de campos de valor generales.



(b) Edición de biestables.

Figura B.7: Cuadros de diálogo de edición de valores.

El botón de reiniciar valores, como su nombre indica, restablece a 0 todos los registros y biestables, con excepción del asociado con el SP, que se restablecerá al valor de la dirección inicial de la pila. El **CheckBox** activa o desactiva la comprobación del valor del biestable H antes de cada ejecución.

B.11.4. El visor de memoria

Este componente, que se puede ver con más detalle en la figura B.8, se encarga de la gestión de la memoria por parte de la aplicación. Permite explorar la memoria y modificar su contenido, así como visualizar y modificar las zonas de código y pila. Por último, mantiene información visual sobre las posiciones de los distintos punteros del sistema y permite saltar hacia ellos para ver el estado de la memoria en sus inmediaciones. Agrupa los siguientes elementos:

- Módulo de visualización y edición de las zonas de código y pila.
- Barra de herramientas del visor.
- El propio visor de memoria.

El editor de zona de código contiene dos campos donde se reflejan automáticamente los valores de las direcciones inicial y final del código desde el momento en que se carga y ensambla un fichero. Al igual que con los campos de valores de los registros, hacer doble clic o pulsar ENTER con el foco en uno de estos campos permite su modificación, cambiando el tamaño designado de la zona de código. Por otro lado, el editor de la zona de pila, si bien muestra las direcciones de inicio y fin de la misma, solamente permite la edición de la dirección de inicio, ya que la dirección de fin es el propio SP, y el valor mostrado se actualiza automáticamente con los cambios del mismo.

La barra de herramientas del visor de memoria contiene, en la primera línea, un campo de texto para escribir una dirección de memoria y un botón para reiniciar la memoria; y, en la segunda línea, botones de navegación al PC, SP, IX e IY. Al confirmar la entrada de datos en el campo

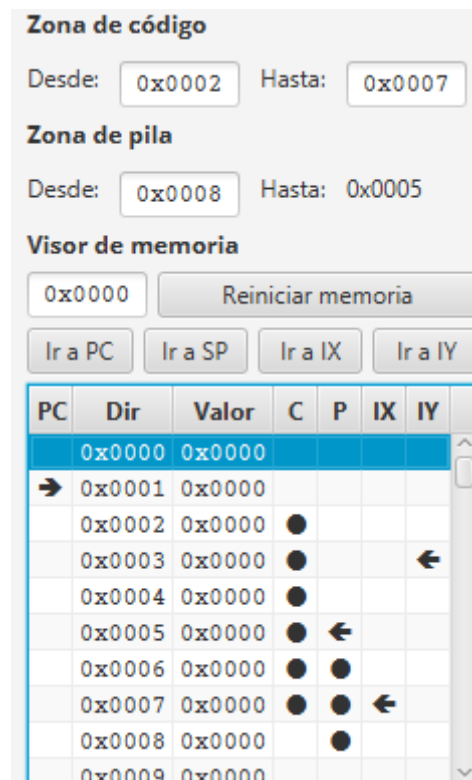


Figura B.8: Visor de memoria con su barra de herramientas.

de texto, el foco se sitúa en esa dirección de memoria específica dentro del visor de memoria y, al pulsar el botón de reinicio, todos los valores almacenados en la memoria se restablecen a 0. Los cuatro botones de navegación sitúan el foco en la posición respectiva de cada uno de esos punteros en el visor de memoria.

El visor en sí es una tabla con las siguientes columnas:

- **PC:** Indica si la dirección de memoria de dicha celda está contenida en el PC en ese momento. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.
- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (no tiene sentido pasar de direcciones positivas a negativas en mitad de la memoria).
- **Valor:** El valor almacenado dentro de la memoria para esa dirección. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, permite modificar esta celda de la misma, mediante la misma ventana de diálogo que aparece cuando se editan los registros o las direcciones de código y pila.
- **C:** Indicador de pertenencia a la zona de código. Si la dirección asociada a la celda está contenida en la zona de código, aparecerá en la celda un punto negro. De lo contrario, la celda quedará vacía.
- **P:** Indicador de pertenencia a la pila. Si la dirección asociada a la celda está contenida entre el SP (no incluido) y el inicio de la pila, aparecerá en la celda un punto negro. Si

el SP apunta a esa dirección, la celda contendrá una flecha orientada a la izquierda. En cualquier otro caso, la celda quedará vacía.

- **IX e IY:** Al igual que la columna del PC, indican si la dirección de memoria está contenida, respectivamente, en los registros IX e IY. Cuando se produzca una coincidencia, la celda contendrá una flecha apuntando a la izquierda, y estará vacía en caso contrario.

B.11.5. El visor de código

Este componente, que se puede ver con más detalle en la figura B.9, se encarga de aportar información sobre el código contenido en la memoria y gestiona los puntos de ruptura. Agrupa los siguientes elementos:

- Módulo de visualización de siguiente instrucción a ejecutar.
- Barra de herramientas del visor.
- El propio visor de código.



Figura B.9: Visor de código con su barra de herramientas.

El visualizador de la siguiente instrucción permite mantener en todo momento constancia de la misma, aunque el visor esté desplazado o incluso desfasado. La barra de herramientas, en dos filas como la del visor de memoria, tiene en la primera de ellas un campo de texto para desensamblar el código a partir de una dirección introducida, así como un botón para eliminar todos los puntos de memoria; la segunda contiene un botón que devuelve el foco a la dirección

actual del PC y desensambla a partir del mismo, así como dos botones para ‘desfasar’ una posición hacia atrás o hacia delante, desensamblando a partir de la misma.

El sistema de desensamblado sigue estas reglas:

- Al cargar un programa, desensambla todo el código desde la posición inicial de la memoria.
- Es posible navegar mediante scroll hacia adelante cuanto se desee, pero no es posible navegar más atrás de la dirección de desensamblado actual.
- Los botones ‘+1’ y ‘-1’ desplazan hacia delante y hacia atrás, respectivamente, la dirección de desensamblado y, en consecuencia, regeneran la lista de instrucciones.
- El campo de navegación establece la dirección introducida como nueva dirección de desensamblado, regenerando la lista de instrucciones.
- Un cambio en el modo de representación numérica regenerará la lista de instrucciones.
- La lista de instrucciones se regenerará tras cualquier ejecución. Se tomará como dirección de desensamblado el valor actual del PC.

Por otro lado, el visor de código en sí contiene una tabla, similar a la del visor de memoria, con las siguientes columnas:

- **BP:** Indicador de punto de ruptura. Cuando la instrucción tiene asociado un punto de ruptura, en esta celda aparece un círculo negro, quedando vacía en caso contrario. Si alguna de las posiciones de memoria que contienen las palabras que conforman esta instrucción tiene un punto de ruptura asociado, se considera que toda la instrucción lo tiene. Eliminar el punto de ruptura de la instrucción lo elimina de todas las palabras que lo contengan. Crear un punto de ruptura en la instrucción lo establece únicamente en la primera palabra de la misma. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, creará un punto de ruptura o eliminará el existente para esa instrucción.
- **PC:** Indica si el contador de programa apunta actualmente a la primera palabra de la instrucción contenida en la fila. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.
- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (igual que en el visor de memoria). El visor no muestra necesariamente todas las direcciones de memoria existentes, solamente las filas cuya dirección sea la primera de una instrucción válida, o bien aquellas que apunten a una dirección inválida.
- **Instrucción:** Contiene la cadena de texto de la instrucción desensamblada a código fuente legible para el usuario.

B.11.6. Las zonas de texto

Este componente separa los flujos de mensajes del sistema en dos áreas de texto distintas, situadas una encima de la otra: la de arriba se encarga de mostrar todos los mensajes del sistema (avisos de ensamblado exitoso, mensajes de error...), mientras que la segunda gestiona el flujo de entrada y salida de la simulación en curso. Ambas áreas de texto están separadas por un elemento deslizante, que permite redistribuir el espacio vertical asignado a ambas, y crecen horizontalmente con la redimensión de la aplicación.

El área de texto de mensajes del sistema no permite escritura, solamente lectura; y el área de texto de la consola de simulación solamente permite escritura cuando se ejecuta una instrucción `ININT`, `INCHAR` o `INSTR`, que solicita explícitamente una entrada. Además, incluso cuando se permite escribir en la consola, no se permite al usuario editar (borrar, modificar, insertar) el texto ya existente, solamente puede escribir a partir del último carácter del texto previo, y solamente se le permite borrar hasta ese punto.

Además, ambas áreas de texto tienen asociadas ciertas acciones, tanto a través de menú contextual (clic derecho) como mediante atajos de teclado:

- Seleccionar todo (`CONTROL+A`).
- Copiar selección (`CONTROL+C`).
- Vaciar de texto la zona (`CONTROL+W`).

Para evitar comportamientos inesperados, esta última opción está desactivada para el área de la consola mientras está activo el modo de espera de entrada del usuario.

B.12. Tablas de referencia

B.12.1. Tablas de formato de instrucciones

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Código de operación										Dir. OP1		Dir. OP2			

(a) Formato de la primera palabra.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP															

(b) Formato de operando de 16 bits.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	Valor OP2							

(c) Formatos de operando de 8 bits para el primer operando (arriba) y el segundo (abajo).

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	-	-	Valor OP2			

(d) Formatos de operando de 4 bits para el primer operando (arriba) y el segundo (abajo).

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								Valor OP2							
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	Valor OP2			
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				Valor OP2							
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	Valor OP2			

(e) Combinaciones de dos operandos en una misma palabra.

Tabla B.5: Formato de instrucciones.

B.12.2. Tabla de modos de direccionamiento, ancho y codificación

Código	Bits	Modo de direccionamiento	Ancho de representación
0	000	Sin operandos	N/A
1	001	Inmediato	16 bits
2	010	Registro	4 bits
3	011	Memoria	16 bits
4	100	Indirecto	4 bits
5	101	Relativo a IX	8 bits
6	110	Relativo a IY	8 bits
7	111	Relativo a PC	8 bits

Tabla B.6: Tabla de códigos de direccionamiento.

B.12.3. Tabla de códigos de registros

Registro	Código	Bits	Registro	Código	Bits
R0	0	0000	R8	8	1000
R1	1	0001	R9	9	1001
R2	2	0010	A	10	1010
R3	3	0011	SR	11	1011
R4	4	0100	IX	12	1100
R5	5	0101	IY	13	1101
R6	6	0110	SP	14	1110
R7	7	0111	PC	15	1111

Tabla B.7: Tabla de códigos de registro.

B.12.4. Tabla de posicionamiento de biestables en el registro de estado

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	H	S	P	V	C	Z

Tabla B.8: Tabla de posicionamiento de biestables en el registro SR.

B.12.5. Tabla de compatibilidad instrucción-direccionamientos

Identificadores		Direccionamiento OP1						Direccionamiento OP2					
Cód.	Inst.	#	.	/	[]	# []	\$	#	.	/	[]	# []	\$
0	NOP	X	X	X	X	X	X	X	X	X	X	X	X
1	HALT	X	X	X	X	X	X	X	X	X	X	X	X
2	MOVE	✓	✓	✓	✓	✓	X	X	✓	✓	✓	✓	X
3	PUSH	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
4	POP	X	✓	✓	✓	✓	X	X	X	X	X	X	X
5	ADD	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
6	SUB	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
7	MUL	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
8	DIV	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
9	MOD	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
10	INC	X	✓	✓	✓	✓	X	X	X	X	X	X	X
11	DEC	X	✓	✓	✓	✓	X	X	X	X	X	X	X
12	NEG	X	✓	✓	✓	✓	X	X	X	X	X	X	X
13	CMP	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
14	AND	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
15	OR	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
16	XOR	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
17	NOT	X	✓	✓	✓	✓	X	X	X	X	X	X	X
18	BR	X	X	✓	✓	X	✓	X	X	X	X	X	X
19	BZ	X	X	✓	✓	X	✓	X	X	X	X	X	X
20	BNZ	X	X	✓	✓	X	✓	X	X	X	X	X	X
21	BP	X	X	✓	✓	X	✓	X	X	X	X	X	X
22	BN	X	X	✓	✓	X	✓	X	X	X	X	X	X
23	BV	X	X	✓	✓	X	✓	X	X	X	X	X	X
24	BNV	X	X	✓	✓	X	✓	X	X	X	X	X	X
25	BC	X	X	✓	✓	X	✓	X	X	X	X	X	X
26	BNC	X	X	✓	✓	X	✓	X	X	X	X	X	X
27	BE	X	X	✓	✓	X	✓	X	X	X	X	X	X
28	BO	X	X	✓	✓	X	✓	X	X	X	X	X	X
29	CALL	X	X	✓	✓	X	✓	X	X	X	X	X	X
30	RET	X	X	X	X	X	X	X	X	X	X	X	X
31	INCHAR	X	✓	✓	✓	✓	X	X	X	X	X	X	X
32	ININT	X	✓	✓	✓	✓	X	X	X	X	X	X	X
33	INSTR	X	X	✓	✓	✓	X	X	X	X	X	X	X
34	WRCHAR	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
35	WRINT	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
36	WRSTR	X	X	✓	✓	✓	X	X	X	X	X	X	X

Tabla B.9: Tabla de modos de direccionamiento aceptados en cada operando por cada instrucción.

Apéndice C

Guía para la inclusión de nuevas instrucciones

Dado que uno de los aspectos notables de ENS2025 es la facilidad para la inclusión de nuevas instrucciones, en este apéndice se detalla el proceso a llevar a cabo para dicha inclusión.

1. Diseño de la instrucción.

En primer lugar, es necesario definir el comportamiento de la instrucción. En este caso, se opta por un ejemplo sencillo con las siguientes propiedades:

- **Mnemónico:** SHL.
- **Descripción:** Desplazamiento lógico hacia la izquierda.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo a índice.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo a índice.
- **Comportamiento:** Desplaza los bits del primer operando una cantidad igual al valor del segundo operando, rellenando con ceros. Almacena el resultado en el registro acumulador.
- **Modifica biestables:** No.

2. Implementación de la instrucción.

Con el comportamiento diseñado, ahora se debe crear el código para implementar ese comportamiento. Para ello, se creará una clase heredada de la familia `Instruction` (concretamente, de `TwoOpInstruction` en este caso), y cuyo código puede consultarse en la figura C.1.

3. Inclusión de la instrucción en el enumerado `InstructionType`.

En primer lugar, se debe comprobar que el mnemónico escogido no coincida con alguno de los ya definidos para evitar comportamientos imprevistos. En segundo lugar, se elige un código de identificación de instrucción siguiendo el mismo criterio (lo más sencillo es usar el número siguiente al último definido). Por lo tanto, en este caso se añadirá esta constante y atributos al enumerado:

```
SHL("SHL", 37, SHL::new)
```

Después, se debe añadir la constante al conjunto de instrucciones que le corresponda por número de operandos, lo que en este caso implica añadir `SHL` a `TWO_OP_SUBSET`, quedando el conjunto como se indica en la figura C.2.

```

1 //Declaracion de paquete e imports pertinentes.
2
3 public final class SHL extends TwoOpInstruction {
4
5     public SHL(InstructionData inst) {
6         super(inst);
7     }
8
9     @Override
10    public void run(Architecture arc) {
11        //Se emplea getValue() al no tratarse de una instruccion de
12        //salto o que opera con cadenas de caracteres.
13        RData value1 = arc.getValue(op1);
14        RData value2 = arc.getValue(op2);
15        RWData result = value1.getMutableCopy();
16
17        result.lShift(value2);
18
19        arc.writeAccumulator(result);
20    }
21 }

```

Figura C.1: Código completo de la clase que implementa la instrucción SHL.

```

1 private static final Set<InstructionType> TWO_OP_SUBSET
2     = new HashSet<>(Arrays.asList(
3         MOVE, ADD, SUB, MUL,
4         DIV, MOD, CMP, AND,
5         OR, XOR, SHL));
6

```

Figura C.2: Contenido del conjunto de instrucciones de dos operandos tras la adición de SHL.

4. Adición de la instrucción al código VerificationCodex.

En esta última fase se crean los conjuntos de asociación de modos de direccionamiento para cada operando de la instrucción y se añaden al mapa correspondiente.

Dados los direccionamientos del primer operando, su conjunto de asociación queda como se ve en la figura C.3.

```

1 private static final Set<AddressingMode> INCHAR_FIRST_OP_SET
2     = new HashSet<>(Arrays.asList(
3         AddressingMode.LITERAL, AddressingMode.REGISTER,
4         AddressingMode.MEMORY, AddressingMode.INDIRECT,
5         AddressingMode.RELATIVE_IX, AddressingMode.RELATIVE_IY));
6

```

Figura C.3: Contenido del conjunto de asociación de los modos de direccionamiento del primer operando de la nueva instrucción SHL.

Puesto que el segundo operando usa los mismos modos de direccionamiento, el conjunto SHL_SECOND_OP_SET será idéntico.

Finalmente, se añaden los conjuntos a sus respectivos mapas. La figura C.4 ilustra el

ejemplo para `FIRST_OP_MAP`, y el proceso es idéntico para el segundo mapa, insertando `SHL_SECOND_OP_SET` en `SECOND_OP_MAP` en el segundo bloque estático.

```
1 //Inicializacion del mapa de primer operando.
2 static {
3     Map<InstructionType, Set<AddressingMode>> tempMap
4         = new HashMap<>();
5
6     //[...] Otras asignaciones de conjuntos de asociacion en el mapa.
7     //A continuacion, la linea de codigo a insertar:
8     tempMap.put(InstructionType.SHL, SHL_FIRST_OP_SET);
9
10    //Finalmente, se crea el mapa inmutable.
11    FIRST_OP_MAP = Collections.unmodifiableMap(tempMap);
12 }
13
```

Figura C.4: Código abreviado del mapa de conjuntos de asociación de modos de direccionamiento para el primer operando de cada instrucción, con la adición de `SHL`.

Tras llevar a cabo estos pasos, la instrucción forma parte del sistema y será debidamente interpretada y ejecutada. Nótese la ausencia de secciones de ensamblado, codificación y decodificación en este apéndice, lo que simplifica considerablemente este proceso.